



Bài giảng KIẾN TRÚC MÁY TÍNH 2022 final

kiến trúc máy tính (Học viện Công nghệ Bưu chính Viễn thông)



Scan to open on Studeersnel

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

**NGUYỄN THỊ THU HIÊN
ĐẶNG TRẦN LÊ ANH
BÙI TRƯỜNG SƠN**



BÀI GIẢNG
KIẾN TRÚC MÁY TÍNH
Dành cho sinh viên ngành Điện tử Viễn thông

HÀ NỘI 2022

MỤC LỤC

LỜI NÓI ĐẦU	4
DANH MỤC CÁC HÌNH	5
DANH MỤC CÁC BẢNG	8
THUẬT NGỮ VIẾT TẮT	9
CHƯƠNG 1. TỔNG QUAN VỀ KIẾN TRÚC MÁY TÍNH	11
1.1. KHÁI NIỆM VỀ TỔ CHỨC VÀ KIẾN TRÚC MÁY TÍNH.....	11
1.2. PHÂN LOẠI MÁY TÍNH.....	12
1.3. SỰ PHÁT TRIỂN CỦA CÔNG NGHỆ MÁY TÍNH	13
1.3.1. Thế hệ đầu tiên: đèn ống chân không.....	13
1.3.2. Thế hệ thứ hai: transistor.....	19
1.3.3. Thế hệ thứ ba: mạch tích hợp.....	19
1.3.4. Các thế hệ tiếp theo.....	24
1.4. HIỆU SUẤT CỦA MÁY TÍNH.....	26
1.5. HỆ ĐẾM VÀ TỔ CHỨC LƯU TRỮ DỮ LIỆU TRONG HỆ THỐNG MÁY TÍNH.....	29
1.5.1. Hệ thập phân	29
1.5.2. Hệ đếm có vị trí.....	30
1.5.3. Hệ nhị phân	31
1.5.4. Chuyển đổi giữa nhị phân và thập phân.....	31
1.5.5. Ký hiệu thập lục phân	34
1.5.6. Biểu diễn số nguyên	35
1.5.7. Biểu diễn dấu chấm động.....	39
1.6. CÁC THÀNH PHẦN CỦA HỆ THỐNG MÁY TÍNH.....	46
1.6.1. Các thành phần máy tính.....	46
1.6.2. Chức năng của máy tính.....	48
1.6.3. Cấu trúc kết nối	57
1.6.4. Kết nối bus	58
1.6.5. Kết nối điểm-điểm	60
CHƯƠNG 2. KIẾN TRÚC TẬP LỆNH VÀ BỘ XỬ LÝ	64
2.1. GIỚI THIỆU CHUNG.....	64
2.2. LỆNH MÁY VÀ TOÁN HẠNG.....	64
2.2.1. Biểu diễn lệnh	64
2.2.2. Phân loại lệnh.....	65
2.2.3. Số lượng địa chỉ trong lệnh.....	66
2.2.4. Các kiểu toán hạng.....	68
2.2.5. Các loại hành động.....	69

2.3. MÃ HOÁ LỆNH VÀ CÁC CHẾ ĐỘ ĐỊA CHỈ.....	78
2.3.1. Mã hoá lệnh.....	78
2.3.2. Các chế độ địa chỉ	80
2.4. CẤU TRÚC CPU.....	87
2.4.1. Tổ chức của CPU	87
2.4.2. Tổ chức thanh ghi.....	88
2.4.3. Ví dụ tổ chức thanh ghi CPU	90
2.5. CHU KỲ LỆNH.....	91
2.5.1. Chu kỳ gián tiếp	91
2.5.2. Luồng dữ liệu	92
2.6. KỸ THUẬT ỨNG LỆNH	94
2.6.1. Chiến lược ứng lệnh	94
2.6.2. Xung đột trong ứng lệnh	98
2.6.3. Tránh xung đột rẽ nhánh	100
2.7. CISC VÀ RISC	104
CHƯƠNG 3. HỆ THỐNG BỘ NHỚ.....	108
3.1. TỔNG QUAN HỆ THỐNG NHỚ.....	108
3.1.1. Các đặc tính của hệ thống bộ nhớ	108
3.1.2. Phân cấp bộ nhớ	110
3.2. BỘ NHỚ CHÍNH	112
3.2.1. Tổ chức	112
3.2.2. DRAM và SRAM.....	112
3.2.3. Các loại ROM – Bộ nhớ chỉ đọc	114
3.2.4. Chip Logic.....	115
3.2.5. Đóng gói chip.....	117
3.2.6. Tổ chức module nhớ	118
3.2.7. Tổ chức bộ nhớ đan xen.....	119
3.2.8. Tổ chức dram mở rộng.....	119
3.3. BỘ NHỚ ĐỆM.....	123
3.3.1. Nguyên lý bộ nhớ cache.....	123
3.3.2. Các yếu tố thiết kế cache.....	126
3.3.3. Hàm ánh xạ	128
3.3.4. Thuật toán thay thế.....	133
3.3.5. Chính sách ghi.....	133
3.3.6. Kích thước line.....	134
3.3.7. Số lượng cache.....	134
3.4. BỘ NHỚ NGOÀI	136
3.4.1. Đĩa từ.....	136
3.4.2. Ổ đĩa bán dẫn SSD	141

3.4.3. Một số biện pháp an toàn dữ liệu trong việc lưu trữ thông tin trong đĩa từ	144
3.5. BỘ NHỚ ẢO	147
3.6. CÁC CÔNG NGHỆ LƯU TRỮ DỮ LIỆU	150
CHƯƠNG 4. XỬ LÝ SONG SONG VÀ ĐA LỖI	156
4.1. MÔ HÌNH TỔ CHỨC ĐA CPU	156
4.2. ĐA LUỒNG ĐỒNG THỜI	158
4.3. ĐA XỬ LÝ ĐỐI XỨNG (SMP) VÀ CỤM (CLUSTER).....	160
4.3.1. Đa xử lý đối xứng	160
4.3.2. Cụm máy tính.....	163
4.4. MÔ HÌNH TỔ CHỨC HỆ THỐNG MÁY TÍNH ĐA LỖI.....	164
4.4.1. Tổ chức đa lỗi.....	164
4.4.2. Tổ chức đa lỗi không đồng nhất.....	166
CHƯƠNG 5. HỆ THỐNG VÀO RA	169
5.1. TỔNG QUAN VỀ HỆ THỐNG VÀO RA	169
5.1.1. Thiết bị ngoại vi	170
5.1.2. Các module I/O	171
5.2. CÁC PHƯƠNG PHÁP ĐIỀU KHIỂN VÀO RA	173
5.2.1. I/O chương trình.....	173
5.2.2. I/O điều khiển ngắt.....	176
5.2.3. Cơ chế truy cập bộ nhớ trực tiếp.....	180
5.3. NỐI GHÉP THIẾT BỊ I/O	182
5.3.1. Kết nối có dây	182
5.3.2. Kết nối không dây	184
TÀI LIỆU THAM KHẢO	186

LỜI NÓI ĐẦU

Cuốn bài giảng Kiến trúc máy tính được biên soạn dựa trên nội dung đề cương đã được duyệt để giảng dạy cho sinh viên Khoa Viễn thông, Học viện công nghệ Bưu chính Viễn thông.

Với thời lượng giảng dạy trong 30 tiết, cuốn bài giảng chỉ đề cập được một số vấn đề cơ bản trong lĩnh vực “Kiến trúc máy tính” – một môn học có ý nghĩa quan trọng trong hành trang kiến thức của sinh viên Khoa Viễn thông.

Nội dung cuốn bài giảng bao gồm năm chương:

Chương 1. Tổng quan về kiến trúc máy tính

Chương 2. Kiến trúc tập lệnh và bộ xử lý

Chương 3. Hệ thống bộ nhớ

Chương 4. Xử lý song song và đa lõi

Chương 5. Hệ thống vào ra

Cuốn bài giảng được biên soạn lần đầu nên chắc chắn sẽ còn rất nhiều thiếu sót. Tác giả rất mong nhận được ý kiến đóng góp của độc giả để có thể sửa chữa và bổ sung.

Mọi góp ý xin gửi về địa chỉ: hiennt@ptit.edu.vn.

Xin chân thành cảm ơn!

Tác giả

DANH MỤC CÁC HÌNH

Hình 1.1. Ví dụ minh họa về máy tính cụm	12
Hình 1.2. Cấu trúc máy tính IAS	14
Hình 1.3. Định dạng bộ nhớ IAS	15
Hình 1.4. Cấu trúc mở rộng của máy IAS	15
Hình 1.5. Một phần lưu đồ hoạt động của IAS	16
Hình 1.6. Các linh kiện máy tính cơ bản	20
Hình 1.7. Mối quan hệ giữa tấm khuôn Wafer, Chip và Cổng logic	21
Hình 1.8. Mức tăng số lượng transistor trên mạch tích hợp	21
Hình 1.9. Cấu trúc bus PDP-8	24
Hình 1.10. Ví dụ chuyển đổi từ thập phân sang nhị phân đối với phần nguyên	32
Hình 1.11. Ví dụ chuyển đổi từ thập phân sang nhị phân đối với phần phân số	33
Hình 1.12. Sử dụng hộp giá trị để chuyển đổi thập phân – nhị phân	38
Hình 1.13. Định dạng biểu diễn dấu chấm động 32-bit	40
Hình 1.14. Các số có thể biểu diễn với định dạng 32-bit	41
Hình 1.15. Mật độ các số dấu chấm động	42
Hình 1.16. Các định dạng IEEE 754	42
Hình 1.17. Lập trình phần cứng và phần mềm	46
Hình 1.18. Các thành phần máy tính	47
Hình 1.19. Chu kỳ lệnh cơ bản	48
Hình 1.20. Đặc tính của máy giả thiết	49
Hình 1.21. Ví dụ về thực thi chương trình (dữ liệu bộ nhớ và thanh ghi biểu diễn dạng HEX) ..	50
Hình 1.22. Sơ đồ trạng thái chu kỳ lệnh	50
Hình 1.23. Tiến trình của chương trình không có ngắt và có ngắt	52
Hình 1.24. Luồng điều khiển trong trường hợp có ngắt	52
Hình 1.25. Chu kỳ lệnh có ngắt	53
Hình 1.26. Tiến trình thời gian của chương trình: thời gian đợi hoạt động Vào/ra ngắn	53
Hình 1.27. Tiến trình thời gian của chương trình	54
Hình 1.28. Sơ đồ trạng thái chu kỳ lệnh có ngắt	55
Hình 1.29. Truyền điều khiển trong trường hợp đa ngắt	55
Hình 1.30. Ngắt trong hệ thống hệ thống với ba thiết bị vào/ra	56
Hình 1.31. Các module máy tính	57
Hình 1.32. Sơ đồ kết nối bus	58
Hình 1.33. Cấu hình bus	60
Hình 1.34. Cấu hình đa lỗi sử dụng QPI	61
Hình 1.35. Các lớp QPI	62
Hình 2.1. Một định dạng lệnh đơn giản	65
Hình 2.2. Chương trình tính biểu thức $Y = A - BC + (DxE)$	67
Hình 2.3. Phép dịch và xoay vòng	73
Hình 2.4. Lệnh rẽ nhánh	76
Hình 2.5. Thủ tục lồng nhau	77
Hình 2.6. Sử dụng ngăn xếp để thực thi các chương trình con lồng nhau ở hình 2.5.	78
Hình 2.7. Liệt kê tất cả các giá trị OPX và các lệnh tương ứng.	79
Hình 2.8. Địa chỉ tức thì	81
Hình 2.9. Địa chỉ trực tiếp	82
Hình 2.10. Địa chỉ gián tiếp	82
Hình 2.11. Địa chỉ thanh ghi	83

Hình 2.12. Địa chỉ gián tiếp thanh ghi.....	84
Hình 2.13. Địa chỉ dịch chuyển.....	84
Hình 2.14. Địa chỉ ngăn xếp	86
Hình 2.15. CPU và bus hệ thống.....	87
Hình 2.16. Cấu trúc bên trong CPU	87
Hình 2.17. Ví dụ tổ chức thanh ghi vi xử lý.....	90
Hình 2.18. Chu kỳ gián tiếp.....	92
Hình 2.19. Sơ đồ trạng thái chu kỳ lệnh gián tiếp.....	92
Hình 2.20. Luồng dữ liệu, chu kỳ truy xuất.....	93
Hình 2.21. Luồng dữ liệu, chu kỳ gián tiếp.....	93
Hình 2.22. Luồng dữ liệu, chu kỳ ngắt.....	94
Hình 2.23. Ống lệnh hai giai đoạn.....	95
Hình 2.24. Biểu đồ thời gian của hành động ống lệnh	96
Hình 2.25. Ảnh hưởng của nhánh có điều kiện tới hoạt động của pipeline lệnh	96
Hình 2.26. Ống lệnh sáu giai đoạn	97
Hình 2.27. Một cách mô tả khác về ống lệnh.....	98
Hình 2.28. Ví dụ về xung đột tài nguyên	99
Hình 2.29. Ví dụ về xung đột dữ liệu.....	100
Hình 2.30. Bộ đệm vòng lặp.....	101
Hình 2.31. Lưu đồ dự đoán nhánh	103
Hình 2.32. Lưu đồ trạng thái dự đoán nhánh	103
Hình 2.33. Cơ chế lưu trữ	104
Hình 3.1. Phân cấp bộ nhớ.....	111
Hình 3.2. Các hoạt động của ô nhớ	112
Hình 3.3. Cấu trúc ô nhớ bộ nhớ thông thường.....	113
Hình 3.4. Tổ chức bộ nhớ DRAM 16Mb (4M × 4)	116
Hình 3.5. Sơ đồ chân và tín hiệu của các chip bộ nhớ.....	117
Hình 3.6. Tổ chức bộ nhớ 256Kbyte	118
Hình 3.7. Tổ chức bộ nhớ 1Mbyte.....	119
Hình 3.8. DRAM đồng bộ (SDRAM).....	120
Hình 3.9. Tiến trình thời gian hoạt động Đọc SDRAM (kích thước burst = 4, độ trễ CSA = 2).....	121
Hình 3.10. Cấu trúc RDRAM.....	122
Hình 3.11. Đồ thị thời gian DDR SDRAM.....	123
Hình 3.12. Cache và bộ nhớ chính.....	124
Hình 3.13. Cấu trúc Cache/Bộ nhớ chính.....	124
Hình 3.14. Hành động đọc cache.....	125
Hình 3.15. Tổ chức cache điển hình.....	125
Hình 3.16. Cache vật lý và cache logic.....	127
Hình 3.17. Ánh xạ từ Bộ nhớ chính sang Cache: Trực tiếp và Kết hợp.....	128
Hình 3.18. Tổ chức cache ánh xạ trực tiếp	129
Hình 3.19. Tổ chức cache kết hợp toàn phần.....	130
Hình 3.20. Ánh xạ từ Bộ nhớ chính sang Cache: Tập kết hợp k line.....	131
Hình 3.21. Tổ chức cache tập kết hợp k line.....	132
Hình 3.22. Sự biến thiên của mức độ kết hợp theo kích thước cache	133
Hình 3.23. Tỷ lệ truy cập thành công tổng cộng (L1 và L2) với L1 8 Kbyte và 16 Kbyte	135
Hình 3.24. Đầu đọc điện từ/ghi điện cảm	137
Hình 3.25. Bố trí dữ liệu trên đĩa.....	138
Hình 3.26. So sánh các phương pháp bố trí đĩa.....	139

Hình 3.27. Các thành phần của ổ đĩa cứng	140
Hình 3.28. Track và cylinder.....	140
Hình 3.29. Thời gian truyền vào/ra của đĩa.....	141
Hình 3.30. Hoạt động của bộ nhớ flash	142
Hình 3.31. Kiến trúc ổ đĩa SSD.....	144
Hình 3.32. Minh họa chương trình gồm 4 trang A,B,C,D trong đó trang D nằm trong ổ đĩa... 147	
Hình 3.33. Ảnh xạ các trang nhớ của bộ nhớ ảo vào bộ nhớ vật lý.....	148
Hình 3.34. Minh họa sự ánh xạ địa chỉ giữa bộ nhớ ảo và bộ nhớ vật lý trong định vị trang . 149	
Hình 3.35. Ảnh xạ địa chỉ giữa bộ nhớ ảo và bộ nhớ vật lý trong cách định vị đoạn	149
Hình 3.36. Giải pháp lưu trữ dữ liệu DAS, NAS, SAN.....	150
Hình 4.1. Một số mô hình tổ chức đa CPU.....	157
Hình 4.2. Các kiến trúc phân chia các khe cấp phát: (a) siêu vô hướng; (b) siêu vô hướng đa luồng; (c) SMT.	159
Hình 4.3. Speedup với tải làm việc đa chương trình (a) và tải làm việc song song (b).....	160
Hình 4.4. Đa chương trình và đa xử lý	161
Hình 4.5. Sơ đồ khối của bộ đa xử lý	162
Hình 4.6. Tổ chức bộ đa xử lý đối xứng.....	162
Hình 4.7. Các cấu hình cụm máy tính.....	164
Hình 4.8. Các giải pháp tổ chức đa lõi	165
Hình 4.9. Các thành phần chip đa lõi không đồng nhất	167
Hình 4.10. Các thành phần chip big.Little.....	167
Hình 5.1. Mô hình cơ bản của các module vào/ra	169
Hình 5.2. Sơ đồ khối của các thiết bị ngoài	170
Hình 5.3. Sơ đồ khối Module I/O	172
Hình 5.4. Ba kỹ thuật Đọc một khối dữ liệu vào	174
Hình 5.5. So sánh I/O ánh xạ bộ nhớ và I/O riêng biệt.....	175
Hình 5.6. Quá trình xử lý ngắt đơn giản	177
Hình 5.7. Sự thay đổi bộ nhớ và thanh ghi khi có ngắt.....	178
Hình 5.8. Kỹ thuật nhiều đường ngắt.....	179
Hình 5.9. Kỹ thuật thăm dò phần mềm.....	179
Hình 5.10. Kỹ thuật chuỗi Daisy.....	180
Hình 5.11. Sơ đồ khối DMA	181
Hình 5.12. Điểm ngắt của DMA và của ngắt trong chu kỳ lệnh	181
Hình 5.13. Các cấu hình DMA.....	182

DANH MỤC CÁC BẢNG

<i>Bảng 1.1. Tập lệnh trong IAS.....</i>	<i>17</i>
<i>Bảng 1.2. Các thế hệ máy tính</i>	<i>19</i>
<i>Bảng 1.3. Các đặc điểm chính của họ System/360.....</i>	<i>22</i>
<i>Bảng 1.4. Sự phát triển của PDP-8.....</i>	<i>23</i>
<i>Bảng 1.5. Quá trình phát triển của vi xử lý Intel</i>	<i>25</i>
<i>Bảng 1.6. Vị trí của một số thập phân.....</i>	<i>30</i>
<i>Bảng 1.7. Vị trí của một số trong hệ cơ số 7.....</i>	<i>30</i>
<i>Bảng 1.8. Thập phân, nhị phân và thập lục phân</i>	<i>34</i>
<i>Bảng 1.9. Các đặc điểm chính của dạng biểu diễn bù hai.....</i>	<i>36</i>
<i>Bảng 1.10. Các dạng biểu diễn khác nhau đối với số nguyên 4 – bit</i>	<i>37</i>
<i>Bảng 1.11. Quy định trong chuẩn IEEE 754.....</i>	<i>43</i>
<i>Bảng 1.12. Các định dạng IEEE</i>	<i>44</i>
<i>Bảng 1.13. Các quy ước biểu diễn số nhị phân dấu chấm động trong IEEE 754.....</i>	<i>45</i>
<i>Bảng 1.14. Các loại ngắt.....</i>	<i>51</i>
<i>Bảng 2.1. Cách sử dụng địa chỉ lệnh (Lệnh không rẽ nhánh).....</i>	<i>68</i>
<i>Bảng 2.2. Các hành động của tập lệnh</i>	<i>70</i>
<i>Bảng 2.3. Hoạt động của bộ xử lý tương ứng với từng loại hành động.....</i>	<i>71</i>
<i>Bảng 2.4. Các phép toán logic cơ bản</i>	<i>72</i>
<i>Bảng 2.5. Ví dụ cho phép dịch và xoay vòng</i>	<i>74</i>
<i>Bảng 2.6. Mã hoá OPX cho các lệnh định dạng R.....</i>	<i>79</i>
<i>Bảng 2.7. Các chế độ địa chỉ cơ bản.....</i>	<i>81</i>
<i>Bảng 3.1. Đặc tính quan trọng của hệ thống bộ nhớ máy tính</i>	<i>108</i>
<i>Bảng 3.2. Các loại bộ nhớ bán dẫn.....</i>	<i>113</i>
<i>Bảng 3.3. So sánh hiệu suất của các chip DRAM thay thế</i>	<i>120</i>
<i>Bảng 3.4. Các chân SDRAM</i>	<i>121</i>
<i>Bảng 3.5. Các yếu tố thiết kế cache</i>	<i>126</i>
<i>Bảng 3.6. Kích thước cache của một số bộ xử lý</i>	<i>127</i>
<i>Bảng 3.7. Đặc tính vật lý của hệ thống đĩa</i>	<i>139</i>
<i>Bảng 3.8. Các thông số ổ đĩa điển hình</i>	<i>140</i>
<i>Bảng 3.9. So sánh ổ đĩa bán dẫn SSD và ổ cứng HDD</i>	<i>143</i>
<i>Bảng 3.10. So sánh các cấp RAID</i>	<i>146</i>
<i>Bảng 3.11. Một số tham số điển hình cho bộ nhớ cache và bộ nhớ ảo.....</i>	<i>148</i>
<i>Bảng 4.1. Bảng phân loại một số mô hình tổ chức đa CPU</i>	<i>157</i>
<i>Bảng 4.2. Các thông số hoạt động của Bộ xử lý đa lõi không đồng nhất AMD 5100K.....</i>	<i>167</i>
<i>Bảng 5.1. Các phương pháp vào/ra</i>	<i>173</i>

THUẬT NGỮ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Tiếng Việt
AC	ACcumulator	Thanh chứa
ACK	ACKnowledgement	Tín hiệu xác nhận
ALU	Arithmetic Logic Unit	Khối số học và logic
CAS	Column Address Selection	Chọn địa chỉ cột
CAV	Constant Angular Velocity	Vận tốc góc không đổi
CE	Chip Enable	Cho phép chip
CO	Calculate Operands	Tính toán các toán hạng
COM	Communication port	Cổng nối tiếp kết nối các thiết bị ngoại vi với máy tính
CISC	Complex Instruction Set Architecture	Kiến trúc tập lệnh phức tạp
CPU	Central Processing Unit	Khối xử lý trung tâm
CPI	Cycles Per Instruction	Số xung nhịp trung bình cho mỗi lệnh
CU	Control Unit	Khối điều khiển
DAS	Direct Attached Storage	Lưu trữ dữ liệu qua các thiết bị gắn trực tiếp
DDR-DRAM	Double Data Rate Synchronous Dynamic Random Access Memory	Bộ nhớ truy cập ngẫu nhiên động đồng bộ tốc độ dữ liệu gấp đôi
DMA	Direct Memory Access	Truy cập bộ nhớ trực tiếp
DI	Decode Instruction	Giải mã lệnh
DRAM	Dynamic Random Access Memory	Bộ nhớ truy cập ngẫu nhiên động
EA	Effective Address	Địa chỉ hiệu dụng
EEPROM	Electrically Erasable Programmable Read-Only Memory	Bộ nhớ chỉ đọc, xóa được bằng điện
EI	Execute Instruction	Thực thi lệnh
ENIAC	Electronic Numerical Integrator And Computer	Máy tích hợp điện tử và máy tính
EPROM	Erasable Programmable Read-Only Memory	Bộ nhớ chỉ đọc, Lập trình được, Xóa được
ET	Execution Time	Thời gian thực thi
FI	Fetch Instruction	Truy xuất lệnh
FIFO	First-In-First-Out	Vào trước ra trước
FO	Fetch Operands	Truy xuất toán hạng
GCC	GNU C Compiler	Trình biên dịch C của GNU
GPS	Global Positioning System	Hệ thống định vị toàn cầu
GPU	Graphics Processing Unit	Đơn vị xử lý đồ họa
IBR	Instruction Buffer Register	Thanh ghi đệm chứa tập lệnh
IEEE	Institute of Electrical and Electronics Engineers	Hội Kỹ sư Điện và Điện tử
ILP	Instruction Level Parallelism	Song song mức lệnh
I/O	Input/Output	Vào/ra
IR	Instruction Register	Thanh ghi tập lệnh
ISR	Interrupt Service Routine	Trình phục vụ ngắt
LCD	Liquid Crystal Display	Màn hình tinh thể lỏng
LFU	Least Frequently Used	Ít được sử dụng nhất
LLC	Lowest Level Cache	Bộ nhớ đệm cấp cuối

LRU	Least Recently Used	Ít được sử dụng gần đây nhất
LPT	Line Printer Terminal	Thiết bị đầu cuối in
LSI	Large Scale Integration	Mạch tích hợp cỡ lớn
MAR	Memory Address Register	Thanh ghi địa chỉ bộ nhớ
MBR	Memory Buffer Register	Thanh ghi đệm bộ nhớ
MIMD	Multiple Instruction, Multiple Data Stream	Đa dòng lệnh, đa dòng dữ liệu
MISD	Multiple Instruction, Single Data Stream	Đa dòng lệnh, đơn dòng dữ liệu
MMU	Memory Management Unit	Đơn vị quản lý bộ nhớ
NAS	Network Attached Storage	Lưu trữ dữ liệu vào thiết bị lưu trữ thông qua mạng
NFC	Near Field Communication	Giao tiếp trường gần
NUMA	Non-Uniform Memory Access	Truy nhập bộ nhớ không đồng nhất
OLED	Organic Light Emitting Diodes	Điốt phát quang hữu cơ
PC	Personal Computer	Máy tính cá nhân
PMD	Personal Mobility Device	Thiết bị di động cá nhân
PC	Program Counter	Bộ đếm chương trình
PROM	Programmable Read Only Memory	ROM có thể lập trình được
PS/2	PlayStation 2	Loại cổng kết nối đặc biệt cho chuột và bàn phím
RAID	Redundant Array of Independent Disks	Hệ thống đĩa dự phòng
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
RAS	Row Address Selection	Chọn địa chỉ hàng
RAW	Read After Write	Đọc sau ghi
RE	Read Enable	Cho phép đọc
RDRAM	Rambus Dynamic Random Access Memory	RAM động Rambus
RISC	Reduced Set Instruction Set Architecture	Kiến trúc tập lệnh rút gọn
ROM	Read Only Memory	Bộ nhớ chỉ đọc
QPI	QuickPath Interconnect	Liên kết đường nhanh
SAN	Storage Area Network	Lưu trữ dữ liệu qua mạng lưu trữ chuyên dụng riêng
SDRAM	Synchronous Dynamic Random Access Memory	DRAM đồng bộ
SIMD	Single Instruction, Multiple Data Stream	Đơn dòng lệnh, đa dòng dữ liệu
SISD	Single Instruction, Single Data Stream	Đơn dòng lệnh, đơn dòng dữ liệu
SMP	Symmetric Multiprocessing	Đa xử lý đối xứng
SMT	Simultaneous MultiThreading	Đa luồng đồng thời
SRAM	Static Random Access Memory	Bộ nhớ truy cập ngẫu nhiên tĩnh
SSD	Solid State Drive	Ổ đĩa bán dẫn
TLP	Task Level Parallism	Song song mức luồng
ULSI	Ultra Large Scale Integration	Mạch tích hợp cỡ cực kỳ lớn
USB	Universal Serial Bus	Bus nối tiếp đa năng
VLSI	Very Large-Scale Integration	Mạch tích hợp cỡ rất lớn
WAR	Write After Read	Ghi sau đọc
WAW	Write After Write	Ghi sau ghi
WE	Write Enable	Cho phép ghi
WO	Write Operand	Ghi toán hạng

CHƯƠNG 1. TỔNG QUAN VỀ KIẾN TRÚC MÁY TÍNH

1.1. KHÁI NIỆM VỀ TỔ CHỨC VÀ KIẾN TRÚC MÁY TÍNH

Trong mô tả máy tính, có hai khái niệm cần được phân biệt là kiến trúc máy tính và tổ chức máy tính. Mặc dù khá khó để có thể đưa ra định nghĩa tóm lược của hai khái niệm này, tuy nhiên chúng ta có thể hiểu như sau:

Kiến trúc máy tính bao gồm các thuộc tính của một hệ thống mà người lập trình có thể nhìn thấy được, hay nói cách khác, đây là các thuộc tính có tác động lên việc thực thi một chương trình máy tính. **Tổ chức máy tính** có thể hiểu là các khối chức năng trong máy tính và sự kết nối giữa chúng để thực hiện các đặc tính của kiến trúc. Ví dụ, các thuộc tính của kiến trúc bao gồm tập lệnh, số bit để biểu diễn các kiểu dữ liệu khác nhau (như ký tự, số,...), cơ chế vào/ra (I/O) và cách kỹ thuật định địa chỉ bộ nhớ. Các thuộc tính của tổ chức bao gồm các đặc điểm phần cứng như: các tín hiệu điều khiển, giao diện giữa máy tính và các thiết bị ngoại vi và các công nghệ bộ nhớ được sử dụng.

Ví dụ, về mặt thiết kế kiến trúc máy tính: ta muốn thực hiện lệnh Nhân. Vậy, về mặt tổ chức sẽ như sau: lệnh này sẽ được thực hiện bởi một đơn vị phần cứng đặc biệt hoặc một cơ chế cho phép thực hiện lặp đi lặp lại phép cộng. Việc lựa chọn một trong hai phương án trên phụ thuộc vào tần suất dự kiến lệnh nhân được sử dụng, tương quan tốc độ tính toán của hai phương pháp, chi phí và kích thước vật lý mỗi một phương án tổ chức.

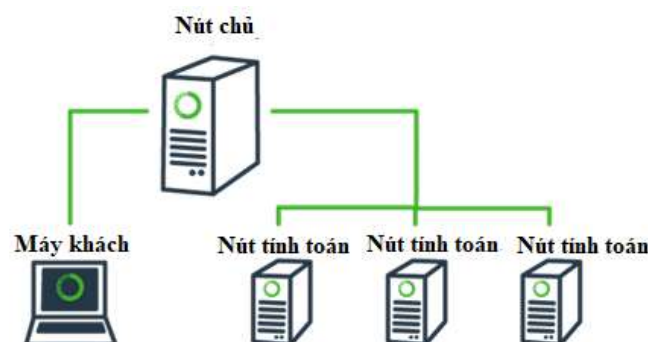
Hiện nay, sự khác nhau giữa kiến trúc và tổ chức vẫn còn khá rõ nét. Nhiều nhà sản xuất máy tính đưa ra một họ các model máy tính, tất cả chúng đều có kiến trúc tương tự nhau nhưng lại khác nhau về tổ chức. Các model có giá thành và hiệu suất khác nhau. Một kiến trúc đặc biệt có thể tồn tại trong nhiều năm và với nhiều model máy tính khác nhau nhưng tổ chức của nó thay đổi theo sự thay đổi của công nghệ. Ví dụ kiến trúc IBM System/370 được đưa ra lần đầu tiên vào năm 1970 với một số model khác nhau. Khách hàng có nhu cầu thấp, đơn giản có thể mua một model rẻ hơn, chậm hơn và nếu có nhu cầu nâng cấp thì sau đó có thể nâng cấp lên một model đắt tiền hơn và nhanh hơn mà không cần phải bỏ đi phần mềm đã được phát triển trên đó. Trong những năm qua, IBM đã giới thiệu nhiều model mới với công nghệ cải tiến để thay thế các model cũ, cung cấp cho khách hàng các hệ máy tính tốc độ cao hơn, chi phí thấp hơn hoặc cả hai. Những model mới hơn vẫn có cùng kiến trúc với model cũ để phần mềm của khách hàng được bảo vệ. Kiến trúc system/370 vẫn tồn tại đến ngày nay và là kiến trúc của dòng sản phẩm máy tính mainframe của hãng IBM.

Với một lớp máy tính khác gọi là máy vi tính, mối quan hệ giữa kiến trúc và tổ chức rất chặt chẽ. Những thay đổi trong công nghệ không chỉ ảnh hưởng đến tổ chức mà còn dẫn đến việc các kiến trúc máy tính mạnh hơn và phức tạp hơn. Tuy nhiên, yêu cầu tương thích giữa các thế hệ máy tính này khá thấp vì vậy việc thiết kế tổ chức và kiến trúc máy tính có nhiều tương tác hơn.

1.2. PHÂN LOẠI MÁY TÍNH

Có nhiều cách thức khác nhau để phân loại máy tính. Người ta có thể phân loại dựa vào ứng dụng, công nghệ, thiết kế, cấu hình,... Phần này sẽ trình bày một số loại máy tính hiện đại được phân loại theo ứng dụng như sau:

- Các **máy tính cá nhân** (PC): Đây là hình thức máy tính được biết đến nhiều nhất. Máy tính cá nhân tập trung vào việc cung cấp hiệu quả sử dụng cho người dùng đơn lẻ với chi phí thấp và thường thực thi phần mềm của bên thứ ba. Lớp máy tính này đã thúc đẩy sự phát triển của nhiều công nghệ máy tính.
- Các **máy chủ** (Server): Hiện nay, bất cứ một doanh nghiệp nào cũng cần một hệ thống công nghệ thông tin (CNTT) chứ không chỉ đơn giản là dùng máy tính PC nữa. Để vận hành và quản lý hệ thống CNTT đó thì doanh nghiệp (hoặc cá nhân) cần phải có máy chủ. Máy chủ là một máy tính được thiết kế với nhiều tính năng vượt trội hơn, năng lực lưu trữ và xử lý dữ liệu cũng lớn hơn máy tính thông thường rất nhiều. Nó được kết nối với mạng máy tính hoặc Internet, có IP tĩnh, có năng lực xử lý cao. Trên đó người ta cài đặt các phần mềm để phục vụ cho các máy tính khác truy cập để yêu cầu cung cấp các dịch vụ và tài nguyên. Một số máy chủ phổ biến có thể kể đến như: máy chủ cơ sở dữ liệu (database server), máy chủ tập tin (file server), máy chủ mail (mail server), máy chủ in (print server), máy chủ web (web server), máy chủ game (game server), máy chủ ứng dụng (application server), hoặc một số loại khác của máy chủ.
- Các **máy tính nhúng**: Đây là loại máy tính lớn nhất, có phạm vi ứng dụng và hiệu suất rộng nhất. Máy tính nhúng bao gồm bộ vi xử lý được tìm thấy trong ô tô, Tivi, mạng các bộ xử lý điều khiển một chiếc máy bay hoặc tàu chở hàng hiện đại,... Hệ thống máy tính nhúng được thiết kế để chạy một ứng dụng hoặc một tập hợp các ứng dụng liên quan thường được tích hợp với phần cứng và được phân phối như một hệ thống duy nhất. Các hệ thống máy tính nhúng có nhiều yêu cầu và ràng buộc khác nhau về giá thành, tối ưu hoá, tái sử dụng, tuổi thọ, chất lượng, điều kiện môi trường, ứng dụng,...
- Máy tính cụm (cluster) (hình 1): là một kiến trúc nhằm nâng cao khả năng sẵn sàng cho các hệ thống mạng máy tính. Kiến trúc này là một nhóm hai hay nhiều máy chủ chuyên dụng chạy một hay nhiều ứng dụng cùng lúc. Các máy chủ này được kết nối lại với nhau để cung cấp khả năng chịu lỗi và cân bằng tải.



Hình 1.1. Ví dụ minh họa về máy tính cụm

- Thiết bị di động cá nhân (PMD): là một thiết bị máy tính đủ nhỏ để cầm và vận hành trong tay. Thông thường, bất kỳ thiết bị máy tính cầm tay nào cũng sẽ có giao diện màn hình phẳng LCD hoặc OLED, cung cấp giao diện màn hình cảm ứng với các nút kỹ thuật số và bàn phím hoặc nút vật lý cùng với bàn phím vật lý. Thiết bị này có thể kết nối Internet và kết nối với các thiết bị khác như hệ thống giải trí trên xe hơi hoặc tai nghe thông qua Wi-Fi, Bluetooth, mạng di động hoặc giao tiếp trường gần (NFC). Máy ảnh tích hợp, máy nghe nhạc kỹ thuật số, khả năng thực hiện và nhận các cuộc gọi điện thoại, trò chơi video và khả năng Hệ thống định vị toàn cầu (GPS) là phổ biến. Năng lượng thường được cung cấp bởi một pin lithium. Thiết bị di động có thể chạy các hệ điều hành di động cho phép các ứng dụng của bên thứ ba chuyên biệt cho các khả năng nói trên được cài đặt và chạy.

1.3. SỰ PHÁT TRIỂN CỦA CÔNG NGHỆ MÁY TÍNH

Chúng ta bắt đầu nghiên cứu về máy tính thông qua việc tìm hiểu lịch sử ra đời và phát triển của công nghệ máy tính. Điều này sẽ giúp cho chúng ta có cái nhìn tổng thể về cấu trúc và chức năng của máy tính.

1.3.1. Thế hệ đầu tiên: đèn ống chân không

1.3.1.1. ENIAC

ENIAC (Electronic Numerical Integrator And Computer) được thiết kế và chế tạo tại trường Đại học Pennsylvania, là máy tính điện tử số đa năng đầu tiên trên thế giới. Dự án này đáp ứng nhu cầu của Hoa Kỳ trong chiến tranh thế giới II. John Mauchly, giáo sư ngành kỹ thuật điện của Đại học Pennsylvania, cùng với một sinh viên của ông là John Eckert đã đề xuất xây dựng một máy tính đa năng sử dụng đèn ống chân không cho bài toán của Tổ chức nghiên cứu đạn đạo quân đội (BRL). Năm 1943, Quân đội chấp nhận đề xuất này, và công việc bắt đầu được triển khai trên ENIAC. Máy ENIAC rất to, nặng tới 30 tấn, chiếm hơn 450m² mặt sàn và chứa hơn 18000 đèn chân không. Khi hoạt động, nó tiêu thụ 140 kilowatt điện. Nó cũng nhanh hơn hẳn so với máy tính cơ điện, có khả năng thực hiện 5000 phép cộng trên giấy.

ENIAC là máy thập phân chứ không phải một máy nhị phân. Tức là, số được biểu diễn dưới dạng thập phân, và phép tính số học thực hiện trong hệ thập phân. Bộ nhớ của nó bao gồm 20 bộ tích lũy (accumulator), mỗi bộ tích lũy có thể trữ một số thập phân 10 chữ số. Mỗi chữ số lại được biểu diễn bởi một vòng gồm 10 đèn ống chân không. Tại một thời điểm, chỉ có một đèn ống chân không ở trạng thái ON, thể hiện một trong 10 chữ số. Hạn chế lớn nhất của ENIAC là nó phải được lập trình thủ công bằng cách thiết lập các công tắc chuyển mạch và cắm rút dây cáp.

ENIAC hoàn thành vào năm 1946, quá muộn để được sử dụng phục vụ chiến tranh. Thay vào đó, nhiệm vụ đầu tiên của nó là thực hiện một loạt các phép tính phức tạp để xác định tính khả thi của bom hydro. Việc sử dụng ENIAC cho một mục đích khác với mục đích xây dựng ban đầu đã chứng minh được tính đa năng của máy này. ENIAC tiếp tục hoạt động dưới sự quản lý của BRL cho đến năm 1955 thì bị tháo rời.

1.3.1.2. Máy tính Von Neumann

Để tránh sự đơn điệu của ENIAC, một giả thiết về một chương trình được thể hiện dưới dạng phù hợp để lưu trữ trong bộ nhớ cùng với dữ liệu. Khi đó, máy tính có thể đọc được lệnh từ

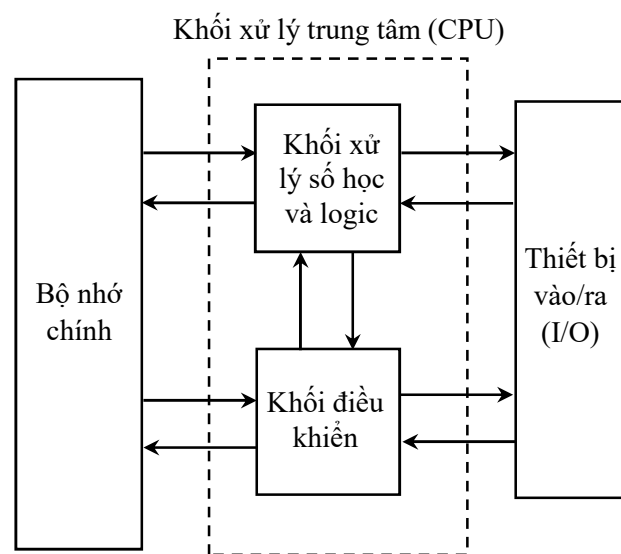
bộ nhớ, và một chương trình có thể được thiết lập hoặc thay đổi bằng cách thiết lập giá trị của một phần trong bộ nhớ.

Ý tưởng này được gọi là **chương trình lưu trữ được**, thường được biết đến là ý tưởng của các nhà thiết kế ENIAC, đặc biệt là nhà toán học John Von Neumann, chuyên gia tư vấn cho dự án ENIAC. Cùng thời điểm đó Alan Turing cũng phát triển ý tưởng này. Công bố đầu tiên của ý tưởng này là do Von Neumann đưa ra năm 1945 trong một máy tính mới, EDVAC (Electronic Discrete Variable Computer).

Năm 1946, Von Neumann và các cộng sự của ông bắt đầu thiết kế một máy tính chương trình lưu trữ mới, gọi là máy tính IAS, tại Viện Nghiên cứu cao cấp Princeton. Mặc dù cho đến năm 1952 vẫn không hoàn thành, máy tính IAS được ghi nhận là nguyên mẫu của tất cả các máy tính đa năng sau này.

Hình 1.2 cho thấy cấu trúc chung của máy tính IAS. Nó bao gồm:

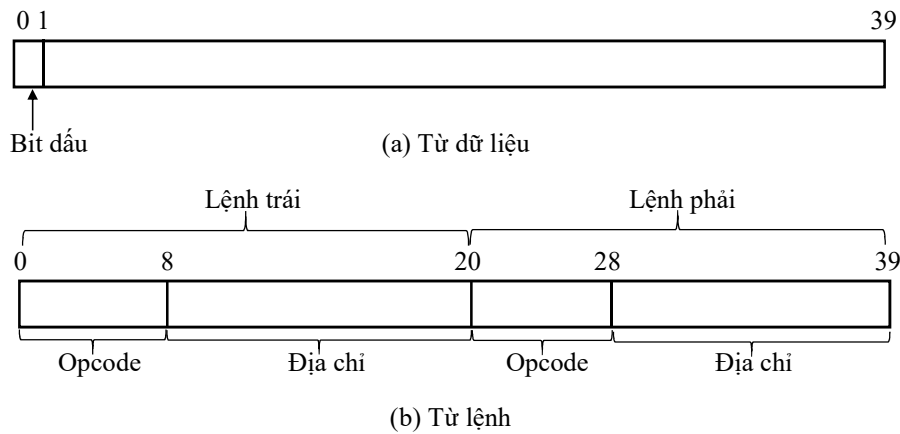
- **Một bộ nhớ chính**, lưu trữ cả dữ liệu và lệnh
- **Một khối số học và logic (ALU)** có khả năng xử lý dữ liệu nhị phân
- **Một khối điều khiển (CU)**, biên dịch các lệnh trong bộ nhớ và cho phép lệnh được thực hiện
- **Thiết bị vào/ra (I/O)** vận hành bởi khối điều khiển



Hình 1.2. Cấu trúc máy tính IAS

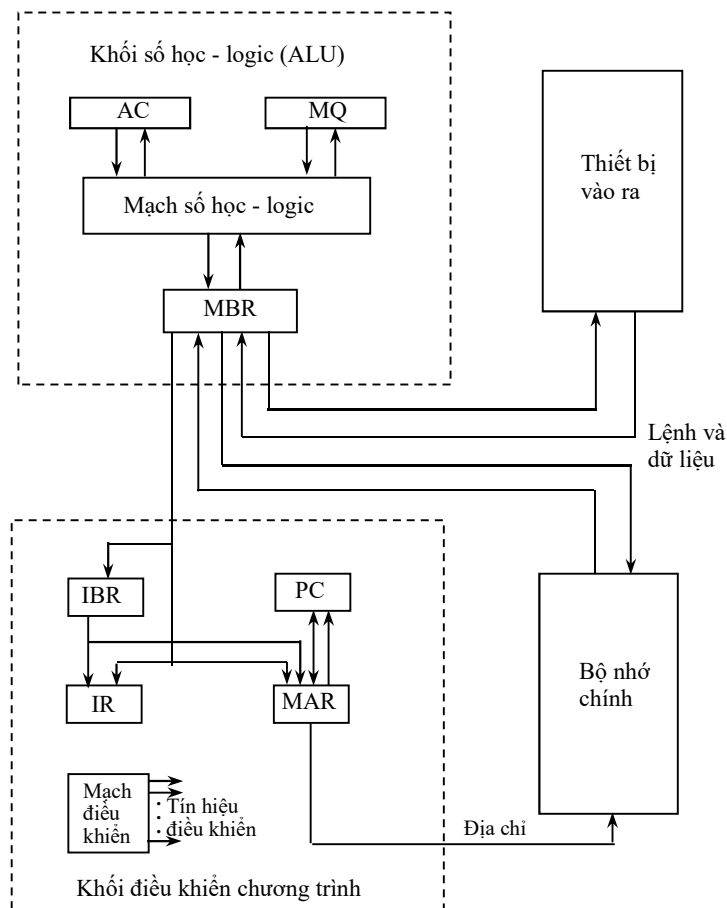
Hầu như tất cả các máy tính ngày nay đều có cấu trúc và chức năng chung giống như trên và do đó được gọi là máy von Neumann. Vì vậy, ta sẽ mô tả một cách ngắn gọn hoạt động của máy tính IAS.

Bộ nhớ của IAS bao gồm 1000 vị trí lưu trữ, được gọi là **từ (word)**, mỗi vị trí có 40 chữ số (bit) nhị phân. Cả dữ liệu và lệnh được lưu trữ ở đó. Số được biểu diễn dưới dạng nhị phân, và mỗi lệnh là một mã nhị phân. Hình 1.3 mô tả các định dạng này. Mỗi số được biểu diễn bởi 1 bit dấu và 39 bit giá trị. Một từ cũng có thể chứa hai lệnh, mỗi lệnh gồm 20 bit. Trong 20 bit lệnh, 8 bit dành cho **mã lệnh (opcode)** chỉ định hành động nào được thực hiện và 12 bit dành cho địa chỉ để xác định một từ cụ thể trong bộ nhớ (đánh số từ 0 đến 999).



Hình 1.3. Định dạng bộ nhớ IAS

Khối điều khiển vận hành máy IAS bằng cách truy xuất các lệnh từ bộ nhớ và thực hiện lần lượt từng lệnh. Hình này cho thấy cả bộ điều khiển và ALU đều chứa các vị trí lưu trữ, được gọi là *thanh ghi*.

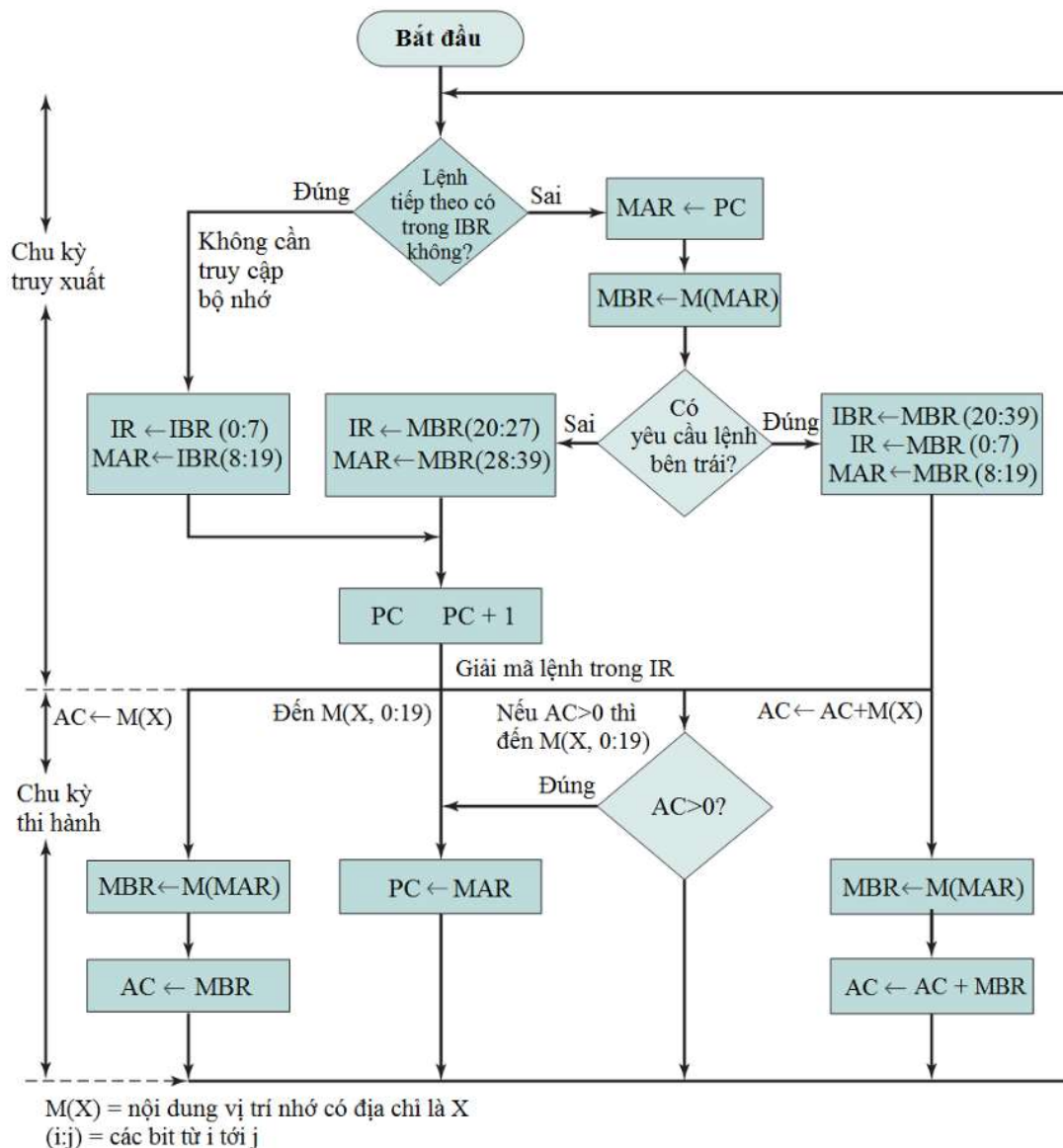


Hình 1.4. Cấu trúc mở rộng của máy IAS

Để giải thích rõ hơn, ta xét một sơ đồ cấu trúc chi tiết ở hình 1.4. Hình vẽ này cho thấy cả khối điều khiển và khối ALU đều có các vị trí lưu trữ, được gọi là *thanh ghi* và được định nghĩa như sau:

- **Thanh ghi đệm bộ nhớ (MBR):** Chứa một từ sắp được lưu vào bộ nhớ hoặc sắp được gửi ra khối I/O, hoặc được dùng để nhận một từ đến từ bộ nhớ hoặc từ các khối I/O.

- **Thanh ghi địa chỉ bộ nhớ (MAR):** Chứa địa chỉ bộ nhớ của từ sắp được đọc hoặc ghi vào MBR.
- **Thanh ghi tập lệnh (IR):** Chứa lệnh với mã lệnh 8 bit sắp được thực hiện.
- **Thanh ghi đệm chứa tập lệnh (IBR):** Được sử dụng để tạm thời lưu trữ lệnh nằm bên phải của 1 từ trong bộ nhớ.
- **Bộ đếm chương trình (PC):** Chứa địa chỉ của cặp lệnh tiếp theo cần truy xuất từ bộ nhớ.
- **Bộ cộng tích lũy (AC) và bộ nhân chia (MQ):** Được sử dụng để tạm thời lưu trữ các toán hạng và kết quả của các phép tính trong ALU. Ví dụ, kết quả của phép nhân hai số 40 bit là một số 80 bit; 40 bit quan trọng hơn được lưu trữ trong AC và 40 bit ít quan trọng hơn lưu trong MQ.



Hình 1.5. Một phần lưu đồ hoạt động của IAS

IAS hoạt động bằng cách thực hiện lặp đi lặp lại từng **chu kỳ lệnh**, như thể hiện trong hình 1.5. Mỗi chu kỳ lệnh bao gồm hai chu kỳ con. Trong **chu kỳ truy xuất**, opcode của lệnh tiếp theo

được nạp vào IR và phần địa chỉ được nạp vào MAR. Lệnh này có thể được lấy từ IBR, hoặc có thể được lấy từ bộ nhớ bằng cách nạp một từ vào MBR, và rồi đưa xuống IBR, IR, và MAR.

Một khi opcode đã được nạp vào IR, **chu kỳ thực thi** được diễn ra. Mạch điều khiển giải mã opcode và thực hiện lệnh bằng cách gửi các tín hiệu điều khiển phù hợp để di chuyển dữ liệu hoặc thực hiện phép tính.

Bảng 1.1. Tập lệnh trong IAS

Kiểu lệnh	Opcode	Biểu diễn ký tự	Mô tả
Truyền dữ liệu	00001010	LOAD MQ	Truyền nội dung thanh ghi MQ tới thanh ghi AC
	00001001	LOAD MQ,M(X)	Truyền nội dung vị trí nhớ X tới MQ
	00100001	STOR M(X)	Truyền nội dung trong AC tới vị trí nhớ X
	00000001	LOAD M(X)	Truyền M(X) tới thanh ghi AC
	00000010	LOAD -M(X)	Truyền -M(X) tới thanh ghi AC
	00000011	LOAD M(X)	Truyền giá trị tuyệt đối của M(X) tới AC
	00000100	LOAD - M(X)	Truyền - M(X) tới thanh ghi AC
Nhánh không điều kiện	00001101	JUMP M(X,0:19)	Lấy lệnh tiếp theo từ nửa trái của M(X)
	00001110	JUMP M(X,20:39)	Lấy lệnh tiếp theo từ nửa phải của M(X)
Nhánh có điều kiện	00001111	JUMP +M(X,0:19)	Nếu số trong AC không âm, lấy lệnh tiếp theo từ nửa trái của M(X)
	00010000	JUMP +M(X,20:39)	Nếu số trong AC không âm, lấy lệnh tiếp theo từ nửa phải của M(X)
Số học	00000101	ADD M(X)	Cộng M(X) với AC; kết quả đặt vào AC
	00000111	ADD M(X)	Cộng M(X) với AC; kết quả đặt vào AC
	00000110	SUB M(X)	Lấy AC trừ M(X); kết quả đặt vào AC
	00001011	MUL M(X)	Nhân M(X) với MQ; các bit quan trọng nhất của kết quả đặt vào AC, các bit ít quan trọng nhất của kết quả đặt vào MQ
	00001100	DIV M(X)	Chia AC cho M(X); thương số đặt vào MQ, phần dư đặt vào AC
	00010100	LSH	Nhân AC với 2; tức là dịch trái một vị trí bit
	00010101	RSH	Chia AC cho 2; tức là dịch phải một vị trí
Sửa đổi địa chỉ	00010010	STOR M(X,8:19)	Thay trường địa chỉ bên trái của M(X) bằng 12 bit ngoài cùng bên phải của AC
	00010011	STOR M(X,28:39)	Thay trường địa chỉ bên phải của M(X) bằng 12 bit ngoài cùng bên phải của AC

Máy tính IAS có tổng cộng 21 lệnh, được liệt kê trong Bảng 1.1. Các lệnh này có thể được phân thành các nhóm như sau:

- **Truyền dữ liệu:** Di chuyển dữ liệu giữa bộ nhớ và các thanh ghi ALU hoặc giữa hai thanh ghi ALU.
- **Rẽ nhánh không điều kiện:** Thông thường, khối điều khiển lần lượt thực hiện các lệnh trong bộ nhớ. Trình tự thực hiện này có thể được thay đổi bởi một lệnh rẽ nhánh, nhằm hỗ trợ cho các hành động lặp đi lặp lại.
- **Rẽ nhánh có điều kiện:** Nhánh có thể được tạo ra tùy thuộc vào điều kiện, do đó sẽ có các điểm quyết định.
- **Số học:** Các phép tính được thực hiện bởi ALU.
- **Sửa đổi địa chỉ:** Cho phép tính toán địa chỉ trong ALU rồi sau đó chèn địa chỉ vào các lệnh đã lưu trữ trong bộ nhớ. Điều này cho phép một chương trình định địa chỉ một cách linh hoạt hơn.

Bảng 1.1 trình bày các lệnh dưới dạng ký hiệu ký tự dễ đọc. Trên thực tế, mỗi lệnh phải tuân theo định dạng ở hình 1.3b. Phần opcode (8 bit đầu tiên) xác định lệnh nào trong số 21 lệnh sẽ được thực hiện. Phần địa chỉ (12 bit còn lại) xác định vị trí nào trong 1000 vị trí bộ nhớ liên quan đến việc thực hiện lệnh.

1.3.1.3. Máy tính thương mại

Những năm 1950 đã chứng kiến sự ra đời của ngành công nghiệp máy tính với hai công ty lớn là Sperry và IBM chiếm lĩnh thị trường.

Năm 1947, Eckert và Mauchly thành lập Công ty Máy tính Eckert-Mauchly để sản xuất máy tính với mục đích thương mại. Máy tính thành công đầu tiên của họ là UNIVAC I (Universal Automatic Computer). Công ty Máy tính Eckert-Mauchly trở thành một nhánh UNIVAC của tập đoàn Sperry-Rand, và tiếp tục sản xuất một loạt máy tính sau này.

UNIVAC I là máy tính thương mại thành công đầu tiên. Nó được dùng cho cả các ứng dụng khoa học và thương mại. Bài báo đầu tiên mô tả hệ thống này đã liệt kê ra các mẫu nhiệm vụ có thể thực hiện là tính toán đại số ma trận, vấn đề về thống kê, hóa đơn bảo hiểm cho công ty bảo hiểm nhân thọ, và các vấn đề hậu cần khác.

UNIVAC II, có dung lượng lưu trữ lớn hơn và hiệu năng cao hơn UNIVAC I, đã được ra mắt vào những năm cuối của thập niên 1950 và cho thấy một số xu hướng của ngành công nghiệp máy tính. Thứ nhất, những tiến bộ trong công nghệ cho phép các công ty tiếp tục chế tạo các máy tính lớn hơn, mạnh hơn. Thứ hai, mỗi công ty cố gắng làm cho các máy mới tương thích ngược với các máy cũ. Điều này có nghĩa là các chương trình viết cho các máy cũ vẫn có thể thực hiện được trên máy mới. Chiến lược này được công nhận với nỗ lực duy trì nền tảng khách hàng; tức là, khi một khách hàng quyết định mua một máy tính mới hơn, anh ta có xu hướng mua sản phẩm mới hơn của cùng một công ty để tránh mất thêm tiền đầu tư vào các chương trình.

Chi nhánh UNIVAC cũng bắt đầu phát triển dòng máy tính 1100, đây là nguồn thu nhập chính của hãng. Mẫu sản phẩm đầu tiên, UNIVAC 1103 chủ yếu dành cho các ứng dụng khoa học, liên quan đến tính toán phức tạp. Các mẫu khác tập trung vào ứng dụng kinh doanh, bao gồm việc xử lý lượng lớn dữ liệu văn bản. Hiện nay, sự phân biệt chức năng này hầu như không còn tồn tại nữa.

IBM, khi ấy là một công ty sản xuất thiết bị xử lý thẻ đục lỗ, cho ra đời máy tính điện tử chương trình lưu trữ đầu tiên – 701 vào năm 1953. Máy 701 phục vụ chính cho ứng dụng tính toán

khoa học. Năm 1955, IBM giới thiệu sản phẩm 702 với một số tính năng phần cứng phù hợp với các ứng dụng kinh doanh. Đó là những mẫu sản phẩm đầu tiên thuộc dòng máy tính 700/7000 giúp IBM vươn lên trở thành nhà sản xuất máy tính hàng đầu.

1.3.2. Thế hệ thứ hai: transistor

Bước tiến lớn đầu tiên trong nền công nghiệp sản xuất máy tính điện tử gắn liền với việc sử dụng bóng bán dẫn transistor thay thế cho đèn ống chân không. Transistor nhỏ hơn, rẻ hơn, và tỏa nhiệt ít hơn đèn ống chân không nhưng lại có thể được sử dụng giống như đèn ống chân không để chế tạo ra máy tính. Nếu như đèn ống chân không kích thước lớn, đòi hỏi dây dẫn, tấm kim loại, bóng thủy tinh và chân không, thì transistor là một thiết bị bán dẫn khá nhỏ, được làm từ silic.

Transistor được phát minh tại Bell Labs vào năm 1947 và tạo ra một cuộc cách mạng điện tử những năm 1950. Tuy nhiên, cho đến những năm cuối của thập niên 1950, các máy tính bán dẫn hoàn toàn mới được bán trên thị trường. IBM không phải là công ty đầu tiên đưa ra công nghệ mới. NCR và RCA là những công ty dẫn đầu với một số máy tính transistor nhỏ. IBM theo ngay sau với dòng 7000.

Việc sử dụng transistor định nghĩa *thế hệ thứ hai* của máy tính. Có thể nói máy tính được phân loại thành các thế hệ dựa trên công nghệ phần cứng cơ bản được sử dụng (Bảng 1.2). Nếu so sánh với thế hệ máy trước, mỗi thế hệ máy mới lại có hiệu suất xử lý tốt hơn, dung lượng bộ nhớ lớn hơn, và kích thước nhỏ hơn.

Bảng 1.2. Các thế hệ máy tính

Thế hệ	Thời gian	Công nghệ	Tốc độ điển hình (hành động trên giây)
1	1946 – 1957	Ống chân không	40,000
2	1958 – 1964	Transistor	200,000
3	1965 – 1971	Mạch tích hợp cỡ nhỏ và vừa	1,000,000
4	1972 – 1977	Mạch tích hợp cỡ lớn	10,000,000
5	1978 – 1991	Mạch tích hợp cỡ rất lớn	100,000,000
6	1991 –	Mạch tích hợp cỡ cực kỳ lớn	1,000,000,000

Bên cạnh đó, cũng có những sự thay đổi khác nữa. Thế hệ thứ hai đã chứng kiến sự ra đời của khối số học và logic phức tạp hơn, sử dụng các ngôn ngữ lập trình bậc cao và sự xuất hiện của phần mềm hệ thống. Phần mềm hệ thống cung cấp khả năng tải chương trình, di chuyển dữ liệu tới các thiết bị ngoại vi, và các thư viện để thực hiện các phép tính phổ biến, giống với các hệ điều hành hiện đại như Windows và Linux.

Thế hệ thứ hai cũng đánh dấu sự xuất hiện của Tập đoàn Thiết bị Số (DEC). DEC được thành lập vào năm 1957 và cho ra mắt máy tính đầu tiên, PDP-1. Máy tính PDP-1 của DEC tạo nên hiện tượng máy tính mini và sau này trở nên cực kỳ nổi bật trong thế hệ thứ ba.

1.3.3. Thế hệ thứ ba: mạch tích hợp

Một transistor được đóng gói riêng được gọi là một linh kiện rời. Trong suốt những năm 1950 và đầu những năm 1960, linh kiện điện tử chủ yếu được sản xuất thành các linh kiện rời – transistor, điện trở, tụ điện, v.v. Các linh kiện rời được sản xuất riêng rẽ, đóng gói trong vỏ riêng, và được hàn hoặc nối dây với nhau trong các bảng mạch điện. Các bảng mạch này sau đó được đặt

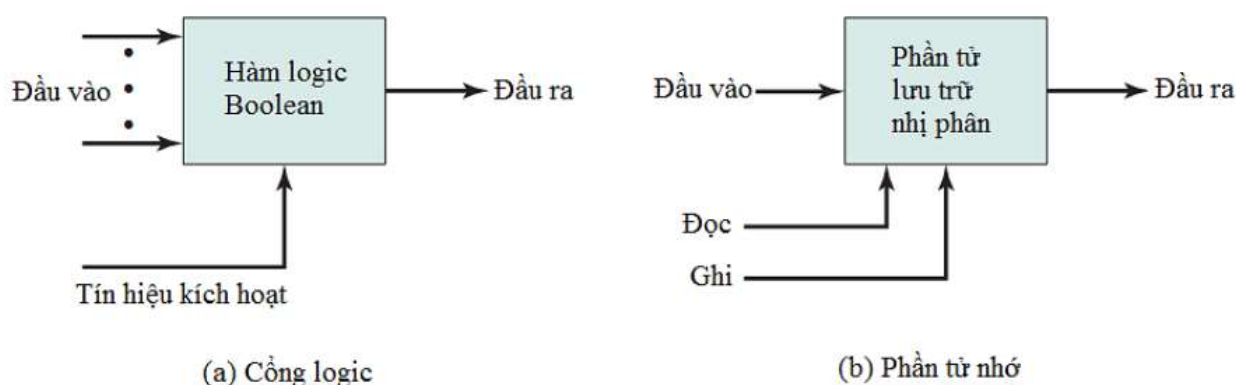
vào trong máy tính, máy hiện sóng (oscilloscope) và các thiết bị điện tử khác. Toàn bộ quá trình sản xuất, từ transistor tới bảng mạch điện, là rất đắt tiền và phức tạp.

Các máy tính thế hệ thứ hai ban đầu có khoảng 10.000 transistor. Con số này dần tăng lên hàng trăm ngàn, khiến cho việc sản xuất các máy tính mới hơn, mạnh hơn ngày càng trở nên khó khăn.

Năm 1958 đánh dấu thành tựu cách mạng điện tử và điểm bắt đầu của kỷ nguyên vi điện tử: phát minh ra **mạch tích hợp**. Mạch tích hợp là linh kiện xác định thế hệ máy tính thứ ba. Trong phần này, ta tìm hiểu sơ lược về công nghệ mạch tích hợp. Sau đó, ta xét hai thành viên quan trọng nhất của thế hệ thứ ba: IBM System/360 và DEC PDP-8.

1.3.3.1. Vi điện tử

Vi điện tử là chỉ "thiết bị điện tử nhỏ". Ngành điện tử số và công nghiệp máy tính luôn có một xu hướng liên tục và nhất quán, đó là kích thước mạch điện tử số ngày càng giảm. Trước khi xem xét tác động và lợi ích của xu hướng này, chúng ta cần tìm hiểu về bản chất của điện tử số.



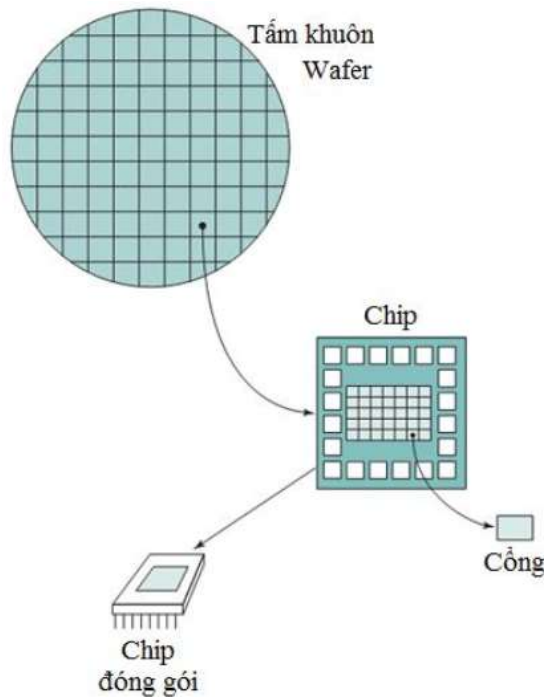
Hình 1.6. Các linh kiện máy tính cơ bản

Như đã đề cập, các chức năng cơ bản của máy tính số là lưu trữ, di chuyển, xử lý và điều khiển. Để thực hiện được các chức năng đó, chỉ cần hai loại linh kiện cơ bản (Hình 1.6) là: cổng logic và phần tử nhớ. Cổng logic là linh kiện thực hiện một hàm Boolean đơn giản, chẳng hạn như IF A AND B TRUE THEN C IS TRUE (cổng AND). Các linh kiện như vậy được gọi là cổng bởi vì chúng kiểm soát lưu lượng dữ liệu giống như cách mà cổng kênh kiểm soát lưu lượng nước. Phần tử nhớ là linh kiện có thể nhớ một bit dữ liệu; có nghĩa là bất kỳ lúc nào linh kiện đó cũng có thể ở một trong hai trạng thái ổn định. Bằng cách kết nối một số lượng lớn các linh kiện cơ bản này, chúng ta có thể xây dựng nên một chiếc máy tính. Chúng ta có thể liên hệ điều này với bốn chức năng cơ bản của máy tính như sau:

- **Lưu trữ dữ liệu:** Thực hiện bởi phần tử nhớ.
- **Xử lý dữ liệu:** Thực hiện bởi cổng logic.
- **Di chuyển dữ liệu:** Đường nối giữa các bộ phận của máy tính được sử dụng để di chuyển dữ liệu đi vào/ra bộ nhớ và từ bộ nhớ qua cổng logic tới bộ nhớ.
- **Điều khiển:** Đường nối giữa các bộ phận của máy tính có thể truyền tín hiệu điều khiển. Ví dụ, một cổng logic sẽ có một hoặc hai đầu vào dữ liệu và một đầu vào tín hiệu điều khiển kích hoạt cổng. Khi tín hiệu điều khiển là ON, cổng thực hiện chức năng của nó trên đầu vào dữ liệu và tạo ra một đầu ra dữ liệu. Tương tự, phần tử nhớ sẽ nhớ bit thông tin ở đầu vào của nó khi tín hiệu điều khiển GHI là ON và sẽ đặt bit nó đang nhớ lên đầu ra của nó khi tín hiệu điều khiển ĐỌC là ON.

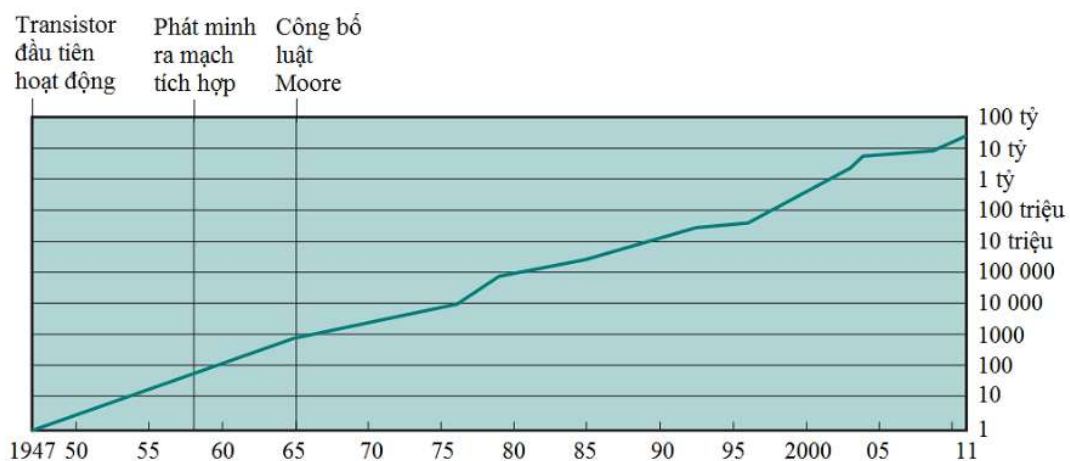
Do đó, một máy tính bao gồm các cổng logic, phần tử nhớ và đường kết nối giữa các bộ phận. Cổng và phần tử nhớ lại được chế tạo từ những linh kiện điện tử số đơn giản.

Các linh kiện như điện trở, transistor và dây dẫn có thể cùng được chế tạo từ chất bán dẫn như silic. Do đó, hoàn toàn có thể chế tạo ra cả một mạch điện hoàn chỉnh trong một miếng silic nhỏ xíu thay vì lắp ráp các linh kiện rời được chế tạo từ các miếng silic riêng biệt vào cùng một mạch điện. Nhiều transistor có thể được sản xuất cùng lúc trên một tấm khuôn silic. Hơn nữa, các transistor này có thể được kết nối với nhau để hình thành các mạch điện.



Hình 1.7. Mối quan hệ giữa tấm khuôn Wafer, Chip và Cổng logic

Hình 1.7 mô tả các khái niệm chính trong mạch tích hợp. Một tấm khuôn silic mỏng (còn gọi là **wafer**) được chia thành một ma trận các ô vuông nhỏ (vài milimet vuông). Các mẫu mạch giống hệt nhau được chế tạo trong mỗi ô, sau đó tấm khuôn wafer được chia thành các **chip**. Mỗi chip bao gồm nhiều cổng logic và/hoặc phần tử nhớ cùng với một số điểm đầu vào và đầu ra. Chip này sau đó được đóng gói kín và được gắn thêm chân để có thể lắp vào các thiết bị ngoài chip. Một vài con chip đã đóng gói này có thể được kết nối với nhau trên một bảng mạch in để tạo ra mạch điện lớn hơn và phức tạp hơn.



Hình 1.8. Mức tăng số lượng transistor trên mạch tích hợp

Ban đầu, chỉ có thể sản xuất và đóng gói một vài cổng logic hoặc phần tử nhớ cùng nhau. Các mạch tích hợp ban đầu này được gọi là tích hợp cỡ nhỏ (SSI – small-scale integration). Sau đó, ngày càng nhiều linh kiện có thể được đóng gói trên cùng một chip. Sự tăng trưởng mật độ tích hợp được minh họa trong hình 1.8; đây là một trong những xu hướng công nghệ đáng chú ý nhất từng được ghi lại. Hình vẽ này phản ánh định luật Moore nổi tiếng, do Gordon Moore, người đồng sáng lập của Intel, đưa ra vào năm 1965. Ông Moore quan sát thấy số lượng transistor có thể đặt trên một con chip đã tăng gấp đôi sau mỗi năm và dự đoán chính xác tốc độ này sẽ duy trì trong tương lai gần. Đáng ngạc nhiên, tốc độ tăng trưởng này tiếp tục năm này qua năm khác, thập kỷ này qua thập kỷ khác. Tốc độ sau đó chậm lại còn tăng gấp đôi sau mỗi 18 tháng vào những năm 1970 và duy trì cho đến ngày nay. Hệ quả của quy luật Moore được đưa ra như sau:

- Giá của một con chip hầu như không thay đổi trong giai đoạn mật độ tăng trưởng nhanh chóng này. Điều này có nghĩa là giá của logic máy tính và mạch nhớ đã giảm rất mạnh.
- Do các bộ phận logic và nhớ được đặt gần nhau hơn trên con chip mật độ cao hơn, chiều dài đường dẫn điện được rút ngắn, tăng tốc độ hoạt động.
- Máy tính trở nên nhỏ gọn hơn và phù hợp để sử dụng ở nhiều môi trường khác nhau.
- Giảm yêu cầu về điện năng tiêu thụ và bộ làm mát.
- Các kết nối trên mạch tích hợp đáng tin cậy hơn nhiều so với các mối hàn. Mạch điện trên mỗi chip càng nhiều, kết nối giữa các con chip càng ít.

1.3.3.2. IBM System/360

Đến năm 1964, IBM đã chiếm lĩnh thị trường máy tính với dòng máy 7000. Trong năm đó, IBM đã cho ra mắt một dòng sản phẩm máy tính mới là System/360. Sự ra mắt sản phẩm mới này dường như là tin không vui đối với khách hàng hiện tại của IBM: bởi dòng sản phẩm 360 không tương thích với các máy IBM trước đó. Do đó, việc chuyển sang sử dụng máy 360 sẽ đem lại khó khăn cho những khách hàng hiện tại. Đây là bước đi táo bạo của IBM, tuy nhiên IBM muốn vượt qua một số khó khăn của kiến trúc 7000 và tạo ra một hệ thống có khả năng phát triển với công nghệ mạch tích hợp mới. Chiến lược táo bạo này đem lại thành quả cả về tài chính và kỹ thuật. 360 là sản phẩm thành công của thập kỷ và đã củng cố vị thế nhà cung cấp máy tính hàng đầu cho IBM, với hơn 70% thị phần. Với một số sửa đổi và mở rộng, kiến trúc của 360 vẫn còn được duy trì cho đến ngày nay trong kiến trúc máy mainframe của IBM.

Bảng 1.3. Các đặc điểm chính của họ System/360

Đặc điểm	Mẫu 30	Mẫu 40	Mẫu 50	Mẫu 65	Mẫu 75
Kích thước bộ nhớ tối đa (byte)	64K	256K	246K	512K	512K
Tốc độ dữ liệu từ bộ nhớ (Mbyte/s)	0.5	0.8	2.0	8.0	16.0
Chu kỳ xử lý (μ s)	1.0	0.625	0.5	0.25	0.2
Tốc độ tương đối	1	3.5	10	21	50
Số kênh dữ liệu tối đa	3	3	4	6	6
Tốc độ dữ liệu tối đa trên một kênh (Kbyte/s)	250	400	800	1250	1250

System/360 là họ máy tính được lập kế hoạch đầu tiên của ngành công nghiệp máy tính. Họ máy này bao trùm một dải rộng hiệu năng và chi phí. Bảng 1.3 liệt kê một số đặc điểm chính của các mẫu máy vào năm 1965 (mỗi thành viên trong họ máy tính được phân biệt bằng một con số). Các mẫu máy này tương thích với nhau, tức là một chương trình viết cho mẫu này có thể chạy được trên một mẫu khác trong cùng dòng máy, chỉ khác biệt về thời gian thực hiện.

System/360 không chỉ quyết định tương lai của IBM mà còn có tác động sâu sắc đến toàn bộ ngành công nghiệp máy tính. Nhiều tính năng của nó đã trở thành tiêu chuẩn cho các sản phẩm máy tính lớn khác.

1.3.3.3. DEC PDP-8

Trong cùng năm mà IBM ra mắt System/360, Công ty thiết bị số (DEC – Digital Equipment Corporation) cũng giới thiệu máy tính PDP-8. Vào thời điểm mà mỗi chiếc máy tính đòi hỏi một phòng máy lạnh, PDP-8 (được gọi là máy tính cỡ nhỏ - minicomputer) đủ nhỏ để đặt vừa trên một chiếc bàn trong phòng thí nghiệm hoặc lắp được vào thiết bị khác. Dù PDP-8 không thể thực hiện được tất cả mọi chức năng như một chiếc máy mainframe, nhưng với mức giá 16.000 USD, nó đủ rẻ để mỗi kỹ thuật viên phòng thí nghiệm có thể sở hữu một chiếc. Trái lại, dòng máy mainframe System/360 được giới thiệu chỉ một vài tháng trước có giá lên tới hàng trăm nghìn đô la.

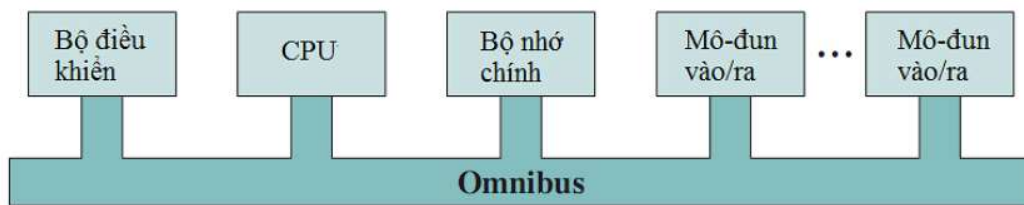
Giá rẻ và kích thước nhỏ của PDP-8 cho phép một nhà sản xuất khác mua PDP-8 và tích hợp nó vào một hệ thống tổng thể để bán lại. Những nhà sản xuất kiểu này được gọi là nhà sản xuất thiết bị gốc (OEM), và thị trường OEM đã trở thành một phân khúc chính của thị trường máy tính.

Bảng 1.4. Sự phát triển của PDP-8

Mẫu	Ra mắt	Giá của bộ xử lý + Bộ nhớ 4K từ 12 bit (\$1000)	Tốc độ dữ liệu từ Bộ nhớ (từ/ μ s)	Thể tích (feet ³)	Cải tiến
PDP-8	1965	16.2	1.26	8.0	Sản xuất dây chuyền tự động
PDP-8/5	1966	8.79	0.08	3.2	Thực hiện lệnh nối tiếp
PDP-8/1	1968	11.6	1.34	8.0	Mạch tích hợp cỡ vừa
PDP-8/L	1968	7.0	1.26	2.0	Tủ chứa nhỏ hơn
PDP-8/E	1971	4.99	1.52	2.2	Omnibus
PDP-8/M	1972	3.69	1.52	1.8	Tủ chứa nhỏ bằng nửa 8/E
PDP-8/A	1975	2.6	1.34	1.2	Bộ nhớ bán dẫn; bộ xử lý dấu chấm động

PDP-8 ngay lập tức trở thành hit và đem lại lợi nhuận lớn cho DEC. Chiếc máy này và các thành viên sau này thuộc họ máy PDP-8 (xem Bảng 1.4) đã được sản xuất với số lượng lớn không kém các máy tính IBM, với khoảng 50.000 máy được bán ra trong vòng vài năm tiếp theo. Theo DEC, PDP-8 đã "thiết lập khái niệm máy tính mini, dẫn tới ngành công nghiệp trị giá nhiều tỷ đô la". Nó cũng đã đưa DEC trở thành nhà cung cấp máy tính mini số một, và cho tới khi PDP-8 hết thời, DEC là nhà sản xuất máy tính số hai, chỉ đứng sau IBM.

Trái ngược với kiến trúc chuyển mạch trung tâm mà IBM sử dụng trên các hệ thống 700/7000 và 360 của mình, các mẫu PDP-8 sau này đã sử dụng một kiến trúc phổ biến đối với các máy vi tính: cấu trúc bus. Điều này được minh họa trong Hình 1.9. Bus PDP-8, được gọi là Omnibus, bao gồm 96 đường dẫn tín hiệu riêng biệt, được sử dụng để truyền tín hiệu điều khiển, địa chỉ và dữ liệu. Bởi vì tất cả các thành phần hệ thống chia sẻ một nhóm đường dẫn tín hiệu chung, việc sử dụng đường dẫn chung có thể được kiểm soát bởi CPU. Kiến trúc này rất linh hoạt, cho phép các mô đun được cắm vào bus để tạo ra các cấu hình khác nhau.



Hình 1.9. Cấu trúc bus PDP-8

1.3.4. Các thế hệ tiếp theo

Sau thế hệ thứ ba, có ít sự đồng thuận chung hơn về việc định nghĩa các thế hệ máy tính. Bảng 1.2 cho thấy đã có một số thế hệ tiếp theo, dựa trên những tiến bộ trong công nghệ mạch tích hợp. Với sự ra đời của mạch tích hợp cỡ lớn (LSI), hơn 1000 linh kiện có thể được đặt trên một con chip tích hợp. Mạch tích hợp cỡ rất lớn (VLSI) có hơn 10.000 linh kiện trên mỗi con chip, trong khi chip tích hợp cỡ cực kỳ lớn (ULSI) hiện tại có thể chứa hơn một tỷ linh kiện.

Với nhịp độ phát triển công nghệ nhanh, tốc độ giới thiệu sản phẩm mới nhanh, tầm quan trọng của phần mềm và truyền thông cũng như phần cứng, việc phân loại máy tính theo thế hệ trở nên không rõ ràng và ít có ý nghĩa. Có thể nói rằng ứng dụng thương mại của các sản phẩm mới đã đem đến sự thay đổi chính vào đầu những năm 1970. Trong phần này, chúng ta đề cập đến hai trong số những kết quả quan trọng nhất.

1.3.4.1. Bộ nhớ bán dẫn

Ứng dụng đầu tiên của công nghệ mạch tích hợp vào máy tính là xây dựng bộ xử lý (khối điều khiển và khối xử lý số học và logic) từ chip tích hợp. Tuy nhiên, công nghệ này cũng có thể được sử dụng để xây dựng bộ nhớ.

Trong những năm 1950 và 1960, hầu hết bộ nhớ máy tính được chế tạo từ các vòng kim loại nhỏ, mỗi vòng có đường kính khoảng 2 mm. Những vòng kim loại này được xâu vào lưới dây đan treo trên tấm màn nhỏ bên trong máy tính. Từ hoá theo một chiều, một vòng kim loại (được gọi là **lõi – core**) biểu diễn bit 1; từ hoá theo chiều khác, nó biểu diễn bit 0. Bộ nhớ lõi từ khá nhanh; chỉ mất một phần triệu giây để đọc một bit lưu trong bộ nhớ. Nhưng nó đắt tiền, cồng kềnh, và mỗi lần đọc dữ liệu trong lõi sẽ xóa luôn dữ liệu được lưu trữ. Vì vậy, cần phải có thêm mạch điện để khôi phục lại dữ liệu ngay khi nó được truy xuất.

Sau đó, vào năm 1970, Fairchild đã sản xuất bộ nhớ bán dẫn dung lượng tương đối lớn đầu tiên. Chip này, với kích thước gần bằng một lõi đơn, có thể chứa 256 bit bộ nhớ. Nó không làm mất dữ liệu và nhanh hơn lõi. Chỉ mất 1/70 tỷ giây để đọc một bit. Tuy nhiên, chi phí cho mỗi bit cao hơn so với lõi.

Đến năm 1974, chi phí cho mỗi bit bộ nhớ bán dẫn giảm xuống thấp hơn chi phí cho mỗi bit của bộ nhớ lõi từ. Sau đó, giá bộ nhớ liên tục giảm mạnh khi mật độ bộ nhớ vật lý tăng nhanh. Điều này dẫn đến việc các máy nhỏ hơn, nhanh hơn với bộ nhớ lớn hơn và đắt hơn. Sự phát triển của công nghệ bộ nhớ, cùng với sự phát triển của công nghệ bộ xử lý đã thay đổi bản chất của máy tính trong vòng chưa đầy một thập kỷ.

Kể từ năm 1970, bộ nhớ bán dẫn đã trải qua 13 thế hệ: 1K, 4K, 16K, 64K, 256K, 1M, 4M, 16M, 64M, 256M, 1G, 4G và 16GB trên một con chip ($1K = 2^{10}$, $1M = 2^{20}$, $1G = 2^{30}$). Mỗi thế hệ lại cung cấp mật độ lưu trữ gấp bốn lần thế hệ trước, cùng với việc giảm chi phí cho mỗi bit và giảm thời gian truy cập.

1.3.4.2. Vi xử lý

Giống như việc mật độ linh kiện trên chip nhớ ngày càng tăng, mật độ linh kiện trên chip xử lý cũng tăng liên tục. Dần dần, ngày càng có nhiều linh kiện được đặt trên một con chip, do đó để xây dựng một bộ xử lý máy tính thì cần ít chip hơn.

Bảng 1.5. Quá trình phát triển của vi xử lý Intel

(a) Bộ xử lý những năm 1970

	4004	8008	8080	8086	8088
Ra mắt	1971	1972	1974	1978	1979
Tốc độ đồng hồ	108 kHz	108 kHz	2 MHz	5 MHz, 8 MHz, 10 MHz	5 MHz, 8 MHz
Độ rộng bus	4 bit	8 bit	8 bit	16 bit	8 bit
Số transistor	2300	3500	6000	29,000	29,000
Kích thước (μm)	10		6	3	6
Bộ nhớ	640 byte	16 kB	64 kB	1 MB	1 MB

(b) Bộ xử lý những năm 1980

	80286	386TM DX	386TM SX	486TM DX CPU
Ra mắt	1982	1985	1988	1989
Tốc độ đồng hồ	6 – 12.5 MHz	16 – 33 MHz	16 – 33 MHz	25 – 50 MHz
Độ rộng bus	16 bit	32 bit	16 bit	32 bit
Số transistor	134,000	275,000	275,000	1.2 tỷ
Kích thước (μm)	1.5	1	1	0.8 – 1
Bộ nhớ	16 MB	4 GB	16 MB	4 GB
Bộ nhớ ảo	1 GB	64 TB	64 TB	64 TB
Cache	-	-	-	8 kB

(c) Bộ xử lý những năm 1990

	486TM SX	Pentium	Pentium Pro	Pentium II
Ra mắt	1991	1993	1995	1997
Tốc độ đồng hồ	16 – 33 MHz	60 – 166 MHz	150 – 200 MHz	200 – 200 MHz
Độ rộng bus	32 bit	32 bit	64 bit	64 bit
Số transistor	1.185 tỷ	3.1 tỷ	5.5 tỷ	7.5 tỷ
Kích thước (μm)	1	0.8	0.6	0.35
Bộ nhớ	4 GB	4 GB	64 GB	64 GB
Bộ nhớ ảo	64 TB	64 TB	64 TB	64 TB
Cache	8 kB	8 kB	512 kB L1 và 1 MB L2	kB L2

(d) Bộ xử lý hiện tại

	Pentium III	Pentium 4	Core 2 Duo	Core i7 EE 990
Ra mắt	1990	2000	2006	2011
Tốc độ đồng hồ	450 – 600 MHz	1.3 – 1.8 GHz	1.06 – 1.2 GHz	3.5 GHz
Độ rộng bus	64 bit	64 bit	64 bit	64 bit
Số transistor	9.5 tỷ	42 tỷ	167 tỷ	1170 tỷ
Kích thước (µm)	250	180	65	32
Bộ nhớ	64 GB	64 GB	64 GB	64 GB
Bộ nhớ ảo	64 TB	64 TB	64 TB	64TB
Cache	512 kB L2	256 kB L2	2 MB L2	1.5 MB L2/ 12MB L3

Năm 1971 đã chứng kiến một bước tiến đột phá, khi Intel phát triển sản phẩm 4004. 4004 là chip đầu tiên chứa tất cả các thành phần của CPU trên một con chip đơn: Vi xử lý ra đời.

Vi xử lý 4004 có thể cộng hai số 4 bit và có thể thực hiện phép nhân chỉ bằng việc lặp đi lặp lại phép cộng. Theo tiêu chuẩn ngày nay, 4004 quả là thô sơ, nhưng nó đánh dấu điểm khởi đầu trong quá trình phát triển liên tục về khả năng và hiệu suất của vi xử lý.

Sự phát triển này có thể nhận thấy dễ dàng nhất qua số lượng bit mà bộ xử lý xử lý được cùng một lúc. Tham số đánh giá tốt nhất là chiều rộng của bus dữ liệu: số bit dữ liệu có thể đưa vào hoặc gửi ra khỏi bộ xử lý tại một thời điểm. Một tham số khác là số bit trong thanh ghi tích lũy (accumulator) hoặc trong tập hợp thanh ghi đa năng. Thông thường, các tham số này trùng nhau, nhưng cũng có trường hợp khác biệt. Ví dụ, một số vi xử lý được phát triển để xử lý các số 16 bit trong thanh ghi nhưng mỗi lần chỉ có thể đọc và ghi 8 bit.

Bước tiến lớn tiếp theo trong quá trình phát triển của vi xử lý là sự ra đời của Intel 8008 vào năm 1972. Đây là vi xử lý 8 bit đầu tiên và độ phức tạp gần gấp đôi so với 4004.

Tuy nhiên, sự kiện lớn tiếp theo có sức ảnh hưởng lớn hơn cả: sự ra đời của Intel 8080 vào năm 1974. Đây là vi xử lý đa năng đầu tiên. Trong khi 4004 và 8008 được thiết kế cho các ứng dụng cụ thể, 8080 được thiết kế để trở thành CPU của máy vi tính đa năng. Giống như 8008, 8080 là vi xử lý 8 bit. Tuy nhiên, 8080 nhanh hơn, có tập lệnh phong phú hơn, và có khả năng định địa chỉ lớn.

Cũng trong thời gian đó, vi xử lý 16 bit đã bắt đầu được phát triển. Tuy nhiên, cho đến những năm cuối thập kỷ 1970, vi xử lý 16 bit đa năng mới xuất hiện. Một trong số đó là 8086. Bước tiến tiếp theo diễn ra vào năm 1981, khi cả Bell Labs và Hewlett-Packard cùng phát triển bộ vi xử lý 32 bit. Intel đã giới thiệu vi xử lý 32 bit của mình, 80386, vào năm 1985 (Bảng 1.5).

1.4. HIỆU SUẤT CỦA MÁY TÍNH

Khi cần có sự đánh giá và lựa chọn giữa các máy tính khác nhau, hiệu suất của máy tính là một thuộc tính quan trọng. Việc đo lường và so sánh chính xác các máy tính khác nhau là rất quan trọng đối với người dùng máy tính và các nhà thiết kế. Do đó, hiểu cách tốt nhất để đo lường

hiệu suất và những hạn chế của phép đo hiệu suất là điều quan trọng. Vì vậy, phần này sẽ mô tả các cách khác nhau mà hiệu suất có thể được định lượng; sau đó là mô tả các tham số để đo lường hiệu suất từ quan điểm của cả người dùng máy tính và nhà thiết kế. Ngoài ra, chúng ta cũng xem xét mối quan hệ giữa các tham số này và đưa ra công thức định lượng hiệu suất bộ xử lý.

Nếu một chương trình đang được chạy trên hai máy tính để bàn khác nhau, thì có thể nói rằng máy tính để bàn nào đó nhanh hơn nếu nó hoàn thành công việc đầu tiên. Nếu trong một trung tâm dữ liệu có một số máy chủ đang thực thi công việc do nhiều người dùng gửi đến, thì có thể nói rằng máy tính nhanh hơn là máy tính hoàn thành nhiều công việc nhất trong ngày. Là một người dùng máy tính cá nhân, chúng ta quan tâm đến việc giảm **thời gian đáp ứng** (response time) – còn được gọi là **thời gian thực thi** (execution time) – là khoảng thời gian từ khi bắt đầu đến khi hoàn thành một tác vụ, bao gồm thời gian truy xuất ổ đĩa, thời gian truy xuất bộ nhớ, thời gian cho các hoạt động vào ra, thời gian cho hệ điều hành, thời gian thực thi CPU,... Với người quản lý trung tâm dữ liệu, họ lại thường quan tâm đến việc tăng thông lượng hoặc băng thông – là lượng công việc được hoàn thành trong một đơn vị thời gian. Do đó, trong hầu hết các trường hợp, chúng ta sẽ cần các tham số hiệu suất khác nhau cũng như các bộ ứng dụng khác nhau để đánh giá, chẳng hạn như các thiết bị di động cá nhân, sẽ tập trung nhiều hơn vào thời gian đáp ứng, so với máy chủ, vốn tập trung nhiều hơn vào thông lượng.

Vậy việc thực hiện những thay đổi sau đây đối với hệ thống máy tính có làm tăng thông lượng hoặc giảm thời gian phản hồi hoặc cả hai hay không?

1. Thay thế bộ xử lý trong máy tính bằng bộ xử lý có phiên bản nhanh hơn
2. Bổ sung thêm bộ xử lý vào hệ thống sử dụng nhiều bộ xử lý cho các tác vụ riêng biệt, ví dụ như: tìm kiếm trên web.

Rõ ràng, việc giảm thời gian đáp ứng hầu như luôn cải thiện thông lượng. Do đó, trong trường hợp 1, cả thời gian đáp ứng và thông lượng đều được cải thiện. Trong trường hợp 2, không có tác vụ nào được hoàn thành nhanh hơn, vì vậy chỉ có thông lượng tăng lên.

Tuy nhiên, nếu nhu cầu xử lý trong trường hợp thứ hai gần bằng thông lượng, hệ thống có thể buộc các yêu cầu phải xếp hàng. Trong trường hợp này, việc tăng thông lượng cũng có thể cải thiện thời gian đáp ứng, vì nó sẽ giảm thời gian chờ trong hàng đợi. Do đó, trong nhiều hệ thống máy tính, việc thay đổi thời gian thực thi hoặc thông lượng của cái này sẽ dẫn đến cái khác.

Để tối đa hóa hiệu suất (P), chúng ta cần giảm thiểu thời gian đáp ứng hoặc thời gian thực thi (ET) cho một số tác vụ. Do đó, chúng ta có thể biểu diễn mối liên hệ giữa hiệu suất và thời gian thực thi cho một máy tính X như sau:

$$P_X = \frac{1}{ET_X} \quad (1.1)$$

Điều này có nghĩa là đối với hai máy tính X và Y, nếu hiệu suất của X lớn hơn hiệu suất của Y thì ta có:

$$P_X > P_Y \text{ hay } \frac{1}{ET_X} > \frac{1}{ET_Y}, ET_Y > ET_X$$

Tức là thời gian thực thi trên Y lớn hơn trên X, và ta nói X nhanh hơn Y.

Tuy nhiên, khi thảo luận về một thiết kế máy tính, để thể hiện một cách định lượng về hiệu suất của hai máy tính khác nhau, chúng ta sẽ sử dụng cụm từ “X nhanh gấp n lần Y” – nghĩa là:

$$\frac{P_X}{P_Y} = n$$

Nếu X nhanh gấp n lần Y thì thời gian thực thi trên Y gấp n lần trên X:

$$\frac{P_X}{P_Y} = \frac{ET_X}{ET_Y} = n$$

Như vậy, thời gian thực thi là thước đo hiệu suất của máy tính: máy tính nào đó thực hiện cùng một lượng công việc trong thời gian nhỏ nhất là máy tính nhanh nhất. Thời gian thực thi chương trình được tính bằng giây cho mỗi chương trình. Tuy nhiên, thời gian này có thể được xác định theo những cách khác nhau, tùy thuộc vào những gì chúng ta tính.

Thời gian thực thi CPU hay đơn giản là **thời gian CPU**, là thời gian CPU dành để tính toán cho tác vụ, không bao gồm thời gian chờ I/O hoặc chạy các chương trình khác. (Tuy nhiên, hãy nhớ rằng thời gian đáp ứng mà người dùng trải qua sẽ là thời gian trôi qua của chương trình, không phải thời gian CPU.)

Với người dùng máy tính, họ thường quan tâm đến thời gian, nhưng khi kiểm tra các chi tiết của máy tính, sẽ rất thuận tiện khi xem xét về hiệu suất qua các tham số. Đặc biệt, các nhà thiết kế máy tính có thể xem xét về máy tính bằng cách sử dụng những tham số liên quan đến **tốc độ** phần cứng thực hiện các chức năng cơ bản. Hầu hết tất cả các máy tính đều được xây dựng bằng cách sử dụng xung nhịp để xác định thời điểm các sự kiện diễn ra trong phần cứng. Những khoảng thời gian rời rạc này được gọi là **chu kỳ xung nhịp** (clock cycles, clock periods, clocks, cycles). Các nhà thiết kế đề cập đến độ dài của một chu kỳ xung nhịp vừa là thời gian cho một chu kỳ xung nhịp hoàn chỉnh (ví dụ: 250 pico giây hoặc 250 ps) vừa là **tốc độ xung nhịp** (R_{CC}) (ví dụ: 4 gigahertz hoặc 4 GHz), nghịch đảo của thời gian xung nhịp.

Hiệu suất CPU và các yếu tố của nó

Người dùng và nhà thiết kế thường kiểm tra hiệu suất bằng cách sử dụng các tham số khác nhau. Nếu giữa chúng có mối liên hệ với nhau thì chúng ta có thể xác định ảnh hưởng của thay đổi thiết kế đối với hiệu suất mà người dùng đã trải qua. Vì chúng ta đang cần xác định hiệu suất của CPU, nên thước đo hiệu suất mấu chốt là **thời gian thực thi CPU** (ET_{CPU}). Một công thức đơn giản liên quan đến các tham số cơ bản (số chu kỳ xung nhịp - N_{CC} và thời gian chu kỳ xung nhịp - T_{CC}) với thời gian thực thi CPU:

$$ET_{CPU} = N_{CC} \times T_{CC} \quad (1.2)$$

hay:

$$ET_{CPU} = N_{CC} / R_{CC} \quad (1.3)$$

Công thức (1.3) cho thấy nhà thiết kế phần cứng có thể cải thiện hiệu suất bằng cách giảm số chu kỳ xung nhịp cần thiết cho một chương trình hoặc độ dài của chu kỳ xung nhịp. Sự cân đối giữa số lượng chu kỳ xung nhịp cần thiết cho một chương trình và độ dài của mỗi chu kỳ luôn đặt ra thách thức. Vì nhiều kỹ thuật giảm số chu kỳ xung nhịp cũng có thể làm tăng thời gian chu kỳ xung nhịp.

Các biểu thức tính hiệu suất ở trên không bao gồm bất kỳ tham chiếu nào đến số lượng lệnh cần thiết cho chương trình. Tuy nhiên, vì trình biên dịch tạo ra các lệnh để thực thi và máy tính phải thực hiện các lệnh để chạy chương trình, nên thời gian thực thi phải phụ thuộc vào số lượng lệnh trong một chương trình. Một biểu thức khác để tính thời gian thực thi của một chương

trình là nó bằng số lượng lệnh (IC) của chương trình được thực thi nhân với số xung nhịp trung bình cho mỗi lệnh (CPI). Do đó, số chu kỳ xung nhịp cần thiết cho một chương trình có thể được viết là:

$$N_{CC} = IC \times CPI \quad (1.4)$$

Thuật ngữ **số chu kỳ xung nhịp trên mỗi lệnh**, là số chu kỳ xung nhịp trung bình mà mỗi lệnh cần để thực thi. Vì các lệnh khác nhau có thể mất khoảng thời gian thực thi khác nhau tùy thuộc vào những gì chúng làm, nên CPI là giá trị trung bình của tất cả các lệnh được thực hiện trong chương trình. CPI cung cấp cách thức để so sánh hai cách triển khai khác nhau của cùng một kiến trúc tập lệnh khi số lượng lệnh thực thi cho một chương trình giống nhau.

Khi đó chúng ta có thể viết công thức tính hiệu suất cơ bản này theo số lượng lệnh (số lệnh được thực thi bởi chương trình), CPI và thời gian chu kỳ xung nhịp:

$$ET_{CPU} = IC \times CPI \times T_{CC} \quad (1.5)$$

hay:

$$ET_{CPU} = IC \times CPI / R_{CC} \quad (1.6)$$

Công thức (1.5), (1.6) đặc biệt hữu ích vì chúng tách biệt ba yếu tố chính ảnh hưởng đến hiệu suất. Chúng ta có thể sử dụng những công thức này để so sánh hai cách triển khai khác nhau hoặc để đánh giá một phương án thiết kế nếu chúng ta biết tác động của nó đối với ba thông số này.

1.5. HỆ ĐẾM VÀ TỔ CHỨC LƯU TRỮ DỮ LIỆU TRONG HỆ THỐNG MÁY TÍNH

1.5.1. Hệ thập phân

Trong cuộc sống hàng ngày, chúng ta sử dụng hệ thống dựa trên các chữ số thập phân (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) để biểu diễn các số, và hệ thống này được gọi là hệ thập phân. Hãy xét ý nghĩa của số 83. Nó có nghĩa là tám chục cộng với ba:

$$83 = (8 \times 10) + 3$$

Số 4728 có nghĩa là bốn nghìn, bảy trăm, hai chục, cộng với tám:

$$4728 = (4 \times 1000) + (7 \times 100) + (2 \times 10) + 8$$

Hệ thập phân có cơ số là 10. Điều này có nghĩa là mỗi chữ số trong một số được nhân với lũy thừa bậc i của 10, với i là vị trí tương ứng của chữ số đó:

$$83 = (8 \times 10^1) + (3 \times 10^0)$$

$$4728 = (4 \times 10^3) + (7 \times 10^2) + (2 \times 10^1) + (8 \times 10^0)$$

Phân phân số thập phân cũng tuân theo nguyên tắc tương tự, nhưng số mũ i là âm.

Như vậy, phân số thập phân 0.256 là 2 phần mười cộng với 5 phần trăm cộng với 6 phần nghìn hay là:

$$0.256 = (2 \times 10^{-1}) + (5 \times 10^{-2}) + (6 \times 10^{-3})$$

Một số có cả phần nguyên và phần phân số thì các chữ số được nhân với 10 mũ i , với i có cả giá trị dương và âm:

$$442.256 = (4 \times 10^2) + (4 \times 10^1) + (2 \times 10^0) + (2 \times 10^{-1}) + (5 \times 10^{-2}) + (6 \times 10^{-3})$$

Trong một số bất kỳ, chữ số ngoài cùng bên trái được gọi là chữ số quan trọng nhất (chữ số có trọng số lớn nhất), bởi vì nó mang giá trị lớn nhất. Các chữ số ngoài cùng bên phải được gọi là chữ số ít quan trọng nhất (chữ số có trọng số nhỏ nhất). Trong số thập phân ở ví dụ trên, số 4 phía bên trái là chữ số có trọng số lớn nhất và số 6 bên phải là chữ số có trọng số nhỏ nhất.

Bảng 1.6 cho thấy mối quan hệ giữa vị trí của chữ số và giá trị được gán cho vị trí đó. Mỗi vị trí có trọng số lớn gấp 10 lần giá trị trọng số của vị trí bên phải nó và bằng một phần mười giá trị trọng số của vị trí bên trái nó. Nếu chúng ta đánh thứ tự vị trí chữ số như ví dụ trong Bảng 1.6, thì vị trí i có trọng số là 10^i .

Bảng 1.6. Vị trí của một số thập phân

4	7	2	2	5	6
100	10	Li	1/10	1/100	1/1000
10^2	10^1	10^0	10^{-1}	10^{-2}	10^{-3}
Vị trí 2	Vị trí 1	Vị trí 0	Vị trí -1	Vị trí -2	Vị trí -3

Tổng quát, nếu số X có biểu diễn dạng thập phân là $X = \{\dots d_2 d_1 d_0 . d_{-1} d_{-2} d_{-3} \dots\}$, giá trị của X là:

$$X = \sum_i (d_i \times 10^i) \quad (1.7)$$

Lưu ý, xét số 509 và hỏi có bao nhiêu giá trị hàng chục trong số đó. Bởi vì có một chữ số 0 ở vị trí hàng chục, có người sẽ nói rằng không có hàng chục. Nhưng trên thực tế ở đây có 50 chục. Chữ số 0 ở vị trí hàng chục có nghĩa là không có phần hàng chục lẻ ra mà không thể gộp vào hàng trăm, hoặc hàng ngàn, vv... Do đó, bởi vì mỗi vị trí chỉ chứa giá trị số lẻ ra mà không thể gộp vào các vị trí cao hơn, mỗi vị trí chữ số phải có giá trị không lớn hơn 9. Vậy 9 là giá trị lớn nhất mà một vị trí có thể chứa.

1.5.2. Hệ đếm có vị trí

Trong hệ đếm có vị trí, mỗi số được biểu diễn bởi một dãy các chữ số trong đó mỗi vị trí chữ số i có trọng số tương ứng r^i , trong đó r là cơ số của hệ đếm đó. Dạng biểu diễn tổng quát của một số thuộc hệ đếm có cơ số r là:

$$(\dots a_2 a_1 a_0 . a_{-1} a_{-2} a_{-3} \dots)_r \quad (1.8)$$

trong đó giá trị của chữ số a_i bất kỳ là một số nguyên trong khoảng $0 \leq a_i < r$. Dấu chấm giữa a_0 và a_{-1} được gọi là dấu thập phân (dấu phân số). Số này có giá trị là:

$$\dots + a_2 r^2 + a_1 r^1 + a_{-1} r^{-1} + a_{-2} r^{-2} + a_{-3} r^{-3} + \dots = \sum_i (a_i \times r^i) \quad (1.9)$$

Bảng 1.7. Vị trí của một số trong hệ cơ số 7

Vị trí	4	3	2	2	0	-1
Trọng số	7^2	7^3	7^2	7^1	7^0	7^{-1}
Giá trị thập phân tương ứng	2401	343	49	7	1	1/7

Hệ thập phân là một trường hợp đặc biệt của một hệ số đếm có vị trí với cơ số 10 và các chữ số trong khoảng từ 0 đến 9.

Ví dụ về một hệ đếm có vị trí khác, hãy xét hệ đếm với cơ số 7. Bảng 1.7 cho thấy giá trị trọng số của các vị trí từ -1 đến 4. Tại mỗi vị trí, chữ số có thể nhận giá trị trong khoảng 0 đến 6.

1.5.3. Hệ nhị phân

Trong hệ thập phân, 10 chữ số khác nhau được sử dụng để biểu diễn các số theo cơ số 10. Trong hệ nhị phân, ta chỉ dùng hai chữ số là 1 và 0. Do vậy, số trong hệ nhị phân được biểu diễn theo cơ số 2.

Để tránh nhầm lẫn, đôi khi chúng ta đặt một số nhỏ bên cạnh số cần biểu diễn để chỉ rõ cơ số của nó. Ví dụ: 83_{10} và 4728_{10} là các số được biểu diễn bằng ký hiệu thập phân hoặc gọi ngắn gọn là số thập phân. Các chữ số 1 và 0 trong ký hiệu nhị phân có cùng ý nghĩa như trong ký hiệu thập phân:

$$0_2 = 0_{10}$$

$$1_2 = 1_{10}$$

Để biểu diễn các số lớn hơn, như với ký hiệu thập phân, mỗi chữ số trong một số nhị phân có một giá trị phụ thuộc vào vị trí của nó. Ví dụ:

$$10_2 = (1 \times 2^1) + (0 \times 2^0) = 2_{10}$$

$$11_2 = (1 \times 2^1) + (1 \times 2^0) = 3_{10}$$

$$100_2 = (1 \times 2^2) + (0 \times 2^1) + (0 \times 2^0) = 4_{10}$$

Ngoài ra, các giá trị phân số được biểu diễn với lũy thừa bậc số mũ âm của cơ số:
 $1001.101 = 2^3 + 2^0 + 2^{-1} + 2^{-3} = 9.625_{10}$

Tổng quát, nếu biểu diễn nhị phân của $Y = \{\dots b_2 b_1 b_0 . b_{-1} b_{-2} b_{-3} \dots\}$, giá trị của Y là:

$$Y = \sum_i b_i \times 2^i \quad (1.10)$$

1.5.4. Chuyển đổi giữa nhị phân và thập phân

Việc chuyển đổi một số từ ký hiệu nhị phân sang ký hiệu thập phân được thực hiện rất dễ dàng. Thực tế, chúng ta đã gặp một số ví dụ trong mục trước. Để chuyển đổi một số từ nhị phân sang thập phân, ta chỉ cần nhân mỗi chữ số nhị phân với lũy thừa của 2 và cộng vào kết quả. Để chuyển từ thập phân sang nhị phân, ta cần thực hiện riêng việc chuyển đổi phần nguyên và phần phân số.

1.5.4.1. Phần nguyên

Đối với phần nguyên, nhớ lại rằng trong ký hiệu nhị phân, số nguyên biểu diễn bởi

$$b_{m-1} b_{m-2} \dots b_2 b_1 b_0 \quad b_i = 0 \text{ hoặc } 1$$

có giá trị là:

$$(b_{m-1} \times 2^{m-1}) + (b_{m-2} \times 2^{m-2}) + \dots + (b_1 \times 2^1) + b_0$$

Giả sử ta cần chuyển đổi một số nguyên thập phân N thành dạng nhị phân. Nếu chia N cho 2, trong hệ thập phân, nhận được thương số N_1 và phần dư R_0 , ta có thể viết lại:

$$N = 2 \times N_1 + R_0 \quad R_0 = 0 \text{ hoặc } 1$$

Tiếp theo, ta chia giá trị N_1 cho 2. Giả sử rằng thương số mới là N_2 và phần dư mới là R_1 . Khi đó:

$$N_1 = 2 \times N_2 + R_1 \quad R_1 = 0 \text{ hoặc } 1$$

vậy là:

$$N = 2(2N_2 + R_1) + R_0 = (N_2 \times 2^2) + (R_1 \times 2^1) + R_0$$

Nếu tiếp tục:

$$N_2 = 2 \times N_3 + R_2$$

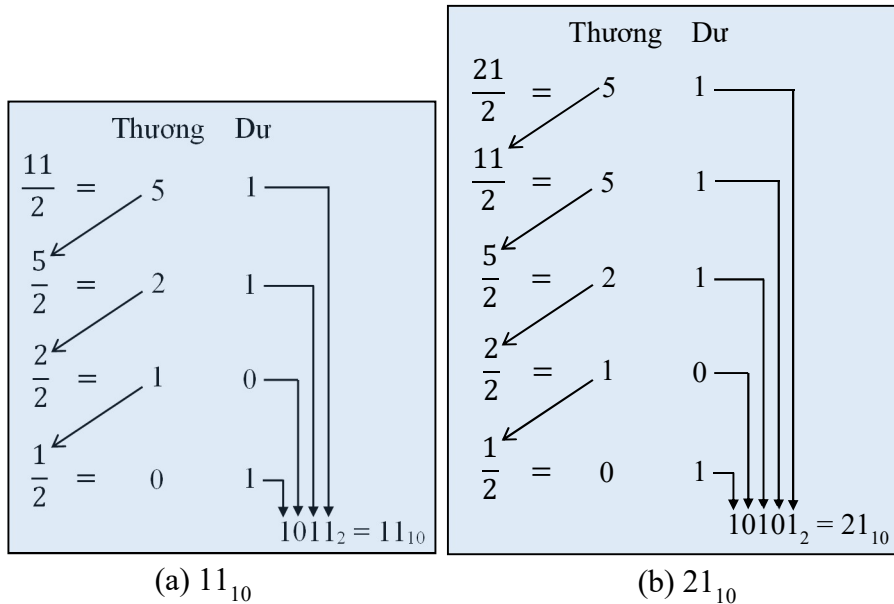
ta có:

$$N = (N_3 \times 2^3) + (R_2 \times 2^2) + (R_1 \times 2^1) + R_0$$

Vì $N > N_1 > N_2 \dots$, tiếp tục thực hiện tương tự cuối cùng sẽ tạo ra một thương số $N_{m-1} = 1$ và phần dư R_{m-2} bằng 0 hoặc 1. Khi đó:

$$N = (1 \times 2^{m-1}) + (R_{m-2} \times 2^{m-1}) + \dots + (R_2 \times 2^2) + (R_1 \times 2^1) + R_0 \quad (1.11)$$

là dạng nhị phân của N . Do đó, ta chuyển đổi từ cơ số 10 sang cơ số 2 bằng cách lặp lại việc chia một số cho 2. Thương số cuối cùng (bằng 1) và các số dư theo thứ tự ý nghĩa tăng dần chính là các chữ số nhị phân của N . Xét hai ví dụ trong Hình 1.10.



Hình 1.10. Ví dụ chuyển đổi từ thập phân sang nhị phân đối với phần nguyên

1.5.4.2. Phần phân số

Đối với phần phân số, nhớ lại rằng trong ký hiệu nhị phân, một số có giá trị trong khoảng từ 0 đến 1 được biểu diễn bởi:

$$0.b_{-1}b_{-2}b_{-3} \dots \quad b_i = 0 \text{ hoặc } 1$$

và có giá trị là:

$$(b_{-1}2^{-1}) + (b_{-2} \times 2^{-2}) + (b_{-3} \times 2^{-3}) \dots$$

Có thể biến đổi thành:

$$2^{-1} \times (b_{-1} + 2^{-1} \times (b_{-2}2^{-1} \times (b_{-3} + \dots) \dots))$$

Cách biểu diễn này gợi ý cho ta về kỹ thuật chuyển đổi. Giả sử ta muốn chuyển đổi số F ($0 < F < 1$) từ ký hiệu thập phân sang nhị phân. Chúng ta biết rằng F có thể được biểu diễn dưới dạng:

$$F = 2^{-1} \times (b_{-1} 2^{-1} \times (b_{-2} 2^{-1} \times (b_{-3} + \dots) \dots))$$

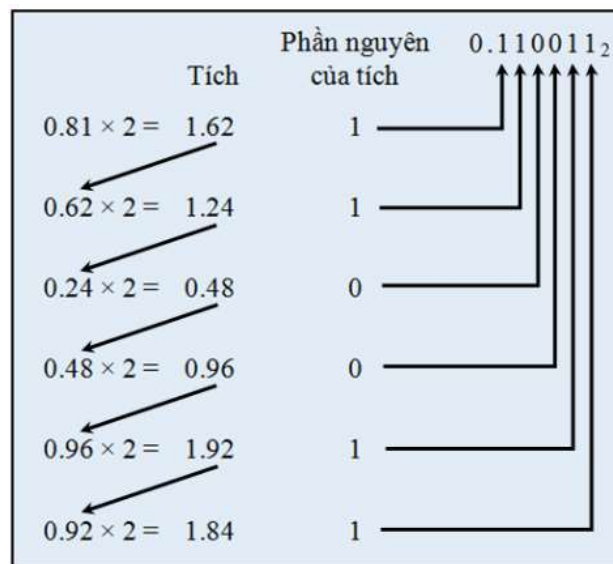
Nếu ta nhân F với 2, ta thu được:

$$2 \times F = b_{-1} 2^{-1} \times (b_{-2} 2^{-1} \times (b_{-3} + \dots) \dots)$$

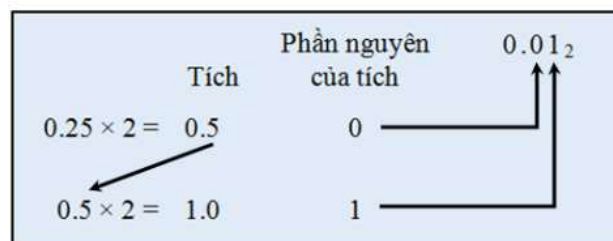
Từ biểu thức này, chúng ta thấy rằng phần nguyên của $(2 \times F)$ phải là 0 hoặc 1 bởi vì $0 < F < 1$, và chính bằng b_{-1} . Vì vậy, chúng ta có thể viết $(2 \times F) = b_{-1} + F_1$, trong đó $0 < F_1 < 1$ và:

$$F_1 = 2^{-1} \times (b_{-2} 2^{-1} \times (b_{-3} 2^{-1} \times (b_{-4} + \dots) \dots))$$

Để tìm b_{-2} , chúng ta lặp lại quá trình trên. Do đó, thuật toán chuyển đổi phần phân số liên quan đến việc lặp lại phép nhân với 2. Ở mỗi bước, phần phân số của tích số trong phép nhân ở bước trước được nhân với 2. Chữ số bên trái dấu thập phân trong tích nhận được là 0 hoặc 1 và sẽ tham gia vào biểu diễn nhị phân của số cần chuyển đổi, bắt đầu với chữ số quan trọng nhất. Phần phân số của tích được sử dụng làm số bị nhân trong bước tiếp theo. Xét hai ví dụ trong Hình 1.11.



(a) $0.81_{10} = 0.110011_2$ (xấp xỉ)



(b) $0.25_{10} = 0.01_2$ (chính xác)

Hình 1.11. Ví dụ chuyển đổi từ thập phân sang nhị phân đối với phần phân số

Kết quả chuyển đổi không nhất thiết phải chính xác; một phân số thập phân với giá trị hữu hạn có thể phải biểu diễn tương ứng bằng một phân số nhị phân vô hạn. Trong trường hợp như

vậy, thuật toán chuyển đổi thường phải tạm dừng sau một số bước đã định, tùy thuộc vào độ chính xác mong muốn.

1.5.5. Ký hiệu thập lục phân

Do các thành phần máy vi tính kỹ thuật số đều thể hiện tính chất nhị phân, tất cả các dạng dữ liệu bên trong máy tính đều được biểu diễn bằng mã nhị phân. Tuy nhiên, cho dù hệ thống nhị phân của máy tính có thuận tiện như thế nào đối với máy tính, thì đối với con người, hệ thống số này lại quá cồng kềnh. Do đó, đa phần các chuyên gia máy tính, những người phải dành nhiều thời gian làm việc với các dữ liệu thô thực tế trong máy tính, đều muốn làm việc với dạng ký hiệu ngắn gọn hơn.

Vậy dạng ký hiệu nào được sử dụng? Ký hiệu thập phân là một khả năng. Nó chắc chắn là ngắn gọn hơn ký hiệu nhị phân, nhưng việc chuyển đổi giữa cơ số 2 và cơ số 10 lại khá nhàm chán.

Thay vào đó, ký hiệu thập lục phân đã được sử dụng. Các chữ số nhị phân được nhóm thành các nhóm bốn bit, được gọi là **nibble**. Mỗi tổ hợp có thể có của bốn chữ số nhị phân được gán cho một ký tự tương ứng như sau:

0000 = 0	0100 = 4	1000 = 8	1100 = C
0001 = 1	0101 = 5	1001 = 9	1101 = D
0010 = 2	0110 = 6	1010 = A	1110 = E
0011 = 3	0111 = 7	1011 = B	1111 = F

Bởi vì có 16 ký tự được sử dụng, nó được gọi là hệ thập lục phân, và 16 ký tự này chính là các chữ số thập lục phân.

Bảng 1.8. Thập phân, nhị phân và thập lục phân

Thập phân (cơ số 10)	Nhị phân (cơ số 2)	Thập lục phân (cơ số 16)
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F
16	0001 0000	10
17	0001 0001	11
18	0001 0010	12
31	0001 1111	1F

100	0110 0100	64
255	1111 1111	FF
256	0001 0000 0000	100

Một dãy chữ số thập lục phân biểu diễn một số nguyên cơ số 16 (Bảng 1.8). Do vậy:

$$2 = (2_{16} \times 16^1) + (C_{16} \times 16^0)$$

$$C_{16} = (2_{10} \times 16^1) + (12_{10} \times 16^0) = 44$$

Do đó, một số thập lục phân là số trong hệ đếm có vị trí với cơ số 16 có thể biểu diễn tổng quát như sau:

$$Z = \sum_i (h_i \times 16^i) \quad (1.12)$$

trong đó 16 là cơ số và mỗi chữ số thập lục phân h_i có giá trị trong khoảng thập phân $0 \leq h_i < 16$, tương đương với khoảng thập lục phân $0 \leq h_i < F$.

Ký hiệu thập lục phân không chỉ được sử dụng để biểu diễn số nguyên mà còn được sử dụng để biểu diễn ngắn gọn một dãy số nhị phân bất kỳ. Lý do cho việc sử dụng ký hiệu thập lục phân là:

- Nó ngắn gọn hơn ký hiệu nhị phân.
- Trong hầu hết các máy tính, dữ liệu nhị phân thường ở dạng bội số của 4 bit, và do đó tương đương với một vài chữ số thập lục phân.
- Việc chuyển đổi giữa ký hiệu nhị phân và thập lục phân rất dễ thực hiện.

Để làm sáng tỏ lý do cuối cùng, ta xét chuỗi nhị phân 110111100001. Chuỗi này tương đương với:

$$11011 \ 1110 \ 0001 = DE11_{16}$$

1.5.6. Biểu diễn số nguyên

Trong hệ thống số nhị phân, các số được biểu diễn dưới dạng các chữ số 0 hoặc 1 cùng với dấu âm (dành cho số âm) và dấu chấm (dành cho số thực có phần phân):

$$-1101.0101 = -13.3125_{10}$$

Với mục đích lưu trữ và xử lý trong máy tính, chúng ta không có các ký hiệu đặc biệt sử dụng để biểu diễn dấu âm và dấu chấm. Nếu chỉ giới hạn trong các số nguyên không âm thì việc biểu diễn hết sức đơn giản:

Một từ 8-bit có thể biểu diễn các số từ 0 đến 255 như sau:

$$00000000 = 0$$

$$00000001 = 1$$

Nghĩa là, nếu một từ ta đổi chuỗi n-bit gồm các chữ số nhị phân $a_{n-1}a_{n-2} \dots a_1a_0$ sang số nguyên không dấu A ở hệ thập phân như sau:

$$A = \sum_{i=0}^{n-1} 2^i a_i \quad (1.13)$$

1.5.6.1. Biểu diễn dấu – độ lớn

Có một số phương pháp biểu diễn số nguyên âm và dương trong máy tính, tất cả các phương pháp này đều có đặc điểm chung là sử dụng bit có trọng số lớn nhất (bit đầu tiên bên trái) làm bit dấu. Nếu bit dấu bằng 0, số đó là số dương, nếu bit dấu bằng 1, số đó là số âm.

Dạng biểu diễn đơn giản nhất trong trường hợp này đó là biểu diễn dấu – độ lớn. Với một từ n -bit, $n - 1$ bit bên phải sẽ là giá trị độ lớn của số:

$$+18 = 00010010$$

$$-18 = 10010010 \text{ (dấu – độ lớn)}$$

Như vậy, ta có công thức quy đổi như sau:

$$A = \begin{cases} \sum_{i=0}^{n-2} 2^i a_i & \text{nếu } a_{n-1} = 0 \\ -\sum_{i=0}^{n-2} 2^i a_i & \text{nếu } a_{n-1} = 1 \end{cases} \quad (1.14)$$

Với phương pháp biểu diễn này, ta thấy có một số nhược điểm như sau. Thứ nhất, khi thực hiện các phép toán cộng hoặc trừ, ta cần phải xét dấu của hai số và sự tương quan về độ lớn để xác định phép toán cần thực hiện. Một nhược điểm nữa là phép biểu diễn này có hai dạng biểu diễn số 0:

$$+0_{10} = 00000000$$

$$-0_{10} = 10000000 \text{ (dấu – độ lớn)}$$

Điều này cũng gây khó khăn cho Đơn vị tính toán số học và logic (ALU) khi kiểm tra trường hợp 0 (phép toán logic này được thực hiện khá thường xuyên trong máy tính). Vì những nhược điểm này, biểu diễn dấu – độ lớn hiếm khi được dùng để biểu diễn số nguyên trong ALU. Thay vào đó, dạng biểu diễn bù hai được sử dụng nhiều hơn.

1.5.6.2. Biểu diễn bù hai

Giống như dấu – độ lớn, biểu diễn bù hai sử dụng bit quan trọng nhất làm bit dấu, nhờ đó, ta dễ dàng kiểm tra một số là số dương hay âm. Tuy nhiên, phần độ lớn sẽ được biểu diễn như sau:

- Với số nguyên dương, độ lớn của số được chuyển từ thập phân sang nhị phân theo cách thông thường, điều này giống với cách biểu diễn số dương của dạng dấu – độ lớn.
- Với số âm, ta lấy bù một của từng bit trong số nguyên dương tương ứng, sau đó cộng kết quả thu được với 1 như cộng giữa hai số nguyên không dấu (bù 2).

Bảng 1.9. Các đặc điểm chính của dạng biểu diễn bù hai

Miền giá trị	Từ -2^{n-1} đến $2^{n-1} - 1$
Biểu diễn số 0	Một cách
Số âm	Với số âm, ta lấy bù một của từng bit trong số nguyên dương tương ứng, sau đó cộng kết quả thu được với 1 như cộng giữa hai số nguyên không dấu (bù 2)
Mở rộng chiều dài bit	Các bit thêm được điền vào bên trái với giá trị của bit dấu
Luật tràn	Khi cộng hai số có cùng dấu (cùng âm hoặc cùng dương), nếu kết quả có dấu ngược lại thì phép toán bị tràn
Luật trừ	Đề trừ A cho B, ta lấy bù hai của B rồi cộng với A

Ví dụ về biểu diễn số 18 và -18 dạng bù hai 8-bit như sau:

$$(+18)_{10} = (00010010)_{\text{bù 2}}$$

Với -18, đầu tiên ta lấy bù 1 (bit 1 đổi thành bit 0 và ngược lại) của 00010010 ta được 11101101, sau đó cộng với 1, ta có:

$$\begin{array}{r} 11101101 \\ + \quad 1 \\ \hline 11101110 \end{array}$$

$$\text{Như vậy: } (-18)_{10} = (11101110)_{\text{bù 2}}$$

Bảng 1.9 liệt kê các đặc điểm chính của dạng biểu diễn bù hai. Diễn giải kỹ hơn các đặc điểm này sẽ được đề cập trong phần này và phần tiếp theo.

Miền giá trị

Một số nguyên n – bit biểu diễn bù hai. Nếu A là số dương, bit dấu $a_{n-1} = 0$. Các bit còn lại biểu diễn độ lớn của số giống như dạng biểu diễn dấu – độ lớn. Như vậy:

$$A = \sum_{i=0}^{n-2} 2^i a_i \quad \text{với } A \geq 0 \quad (1.15)$$

Số 0 được biểu diễn như số dương với bit dấu bằng 0 và tất cả các bit còn lại đều là 0. Từ công thức trên ta có thể thấy miền giá trị của số nguyên dương trong dạng biểu diễn này là từ 0 (tất cả các bit phần độ lớn bằng 0) đến $2^{n-1} - 1$ (tất cả các bit phần độ lớn bằng 1). Với các số lớn hơn, ta cần phải biểu diễn bằng nhiều bit hơn.

Bảng 1.10. Các dạng biểu diễn khác nhau đối với số nguyên 4 – bit

Hệ thập phân	Biểu diễn dấu – độ lớn	Biểu diễn bù hai	Biểu diễn biased
+8	—	—	1111
+7	0111	0111	1110
+6	0110	0110	1101
+5	0101	0101	1100
+4	0100	0100	1011
+3	0011	0011	1010
+2	0010	0010	1001
+1	0001	0001	1000
+0	0000	0000	0111
-0	1000	—	—
-1	1001	1111	0110
-2	1010	1110	0101
-3	1011	1101	0100
-4	1100	1100	0011
-5	1101	1011	0010
-6	1110	1010	0001
-7	1111	1001	0000
-8	—	1000	—

Với một số âm A ($A < 0$), bit dấu $a_{n-1} = 1$. Trọng số của bit dấu này được quy ước là -2^{n-1} . Với quy ước này, ta hoàn toàn có thể sử dụng quy tắc ở trên để tính giá trị của một số từ dạng biểu

diễn bù 2: tổng của giá trị bit nhân với trọng số của mỗi bit. Vậy, giá trị của số A được tính như sau:

$$\text{Bù hai} \quad A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i \quad (1.16)$$

Biểu thức (1.8) định nghĩa biểu diễn bù hai cho cả hai số âm và số dương. Với $a_{n-1} = 0$, $-2^{n-1}a_{n-1} = 0$, phương trình xác định giá trị số nguyên không âm. Với $a_{n-1} = 1$, giá trị -2^{n-1} bị trừ khỏi tổng, ta sẽ có số nguyên âm tương ứng.

Bảng 1.10 so sánh hai dạng biểu diễn dấu – độ lớn và bù hai với các số nguyên 4 – bit. Mặc dù, biểu diễn bù hai tạo cảm giác khó hiểu và khó sử dụng hơn nhưng lại rất phù hợp cho các phép tính số học quan trọng nhất là các phép cộng hoặc trừ. Chính vì vậy, dạng biểu diễn này được sử dụng rộng rãi để biểu diễn số nguyên trong máy tính.

Để xác định giá trị mà một số bù hai biểu diễn ta có thể sử dụng hộp giá trị, trong đó giá trị đầu tiên từ bên phải của hộp 1 (tương ứng với 2^0) và giá trị các vị trí kế tiếp sang phía trái lần lượt tăng gấp đôi giá trị, cho đến vị trí cuối cùng bên trái mang giá trị âm (như trong Hình 1.12a). Số lượng cột trong hộp bằng chính số bit của biểu diễn bù hai, nếu có nhiều bit hơn thì ta thêm vào các cột có giá trị dương, cùng quy luật như vậy.

Hình 1.12b và c minh họa việc sử dụng hộp giá trị để chuyển đổi từ số dạng bù hai sang dạng thập phân và ngược lại.

-128	64	32	16	8	4	2	1

a. Hộp giá trị của biểu diễn bù hai 8-bit

-128	64	32	16	8	4	2	1
1	0	0	0	0	0	1	1

$$-128 \quad \quad \quad +2 \quad +1 = -125$$

b. Chuyển đổi từ 10000011 sang số thập phân

-128	64	32	16	8	4	2	1
1	0	0	0	1	0	0	0

$$-120 = -128 \quad \quad \quad +8$$

c. Chuyển đổi từ số -120 hệ thập phân sang nhị phân

Hình 1.12. Sử dụng hộp giá trị để chuyển đổi thập phân – nhị phân

1.5.6.3. Mở rộng phạm vi

Đôi khi, trong một số trường hợp ta cần phải biểu diễn một số nguyên n-bit dưới dạng m-bit ($m > n$). Việc tăng chiều dài bit được gọi là mở rộng phạm vi, vì việc tăng số bit để biểu diễn số cho phép phạm vi biểu diễn của dãy bit tăng lên.

+18 =	00010010	(Dấu – độ lớn, 8 bit)
+18 =	00000000000010010	(Dấu – độ lớn, 16 bit)
-18 =	10010010	(Dấu – độ lớn, 8 bit)
-18 =	10000000000010010	(Dấu – độ lớn, 16 bit)

Với dạng biểu diễn dấu – độ lớn, để mở rộng phạm vi, ta chỉ cần đặt bit dấu thành bit đầu tiên bên trái và điền thêm các bit 0. Với dạng bù hai, quy tắc trên không còn đúng, ta có ví dụ sau:

+18 =	00010010	(Bù hai, 8 bit)
+18 =	0000000000010010	(Bù hai, 16 bit)
-18 =	11101110	(Bù hai, 8 bit)
-32658 =	1000000011101110	(Bù hai, 16 bit)

Vậy với dạng bù hai, để mở rộng phạm vi, ta thêm vào bên trái của số đó các bit giống với bit dấu. Quy tắc này ta có thể gọi là quy tắc mở rộng dấu.

-18 =	11101110	(Bù hai, 8 bit)
-18 =	1111111111101110	(Bù hai, 16 bit)

Để hiểu được quy tắc này, ta xét một chuỗi n-bit: $a_{n-1}a_{n-2} \dots a_1a_0$ dạng mã bù hai, vậy ta có giá trị A của số đó như sau:

$$A = -2^{n-1}a_{n-1} \sum_{i=0}^{n-2} 2^i a_i \quad (1.17)$$

Khi mở rộng phạm vi chuỗi bit thành m-bit, ta có:

$$A = -2^{m-1}a_{m-1} + \sum_{i=0}^{m-2} 2^i a_i$$

Nếu A là số dương, các bit thêm vào là các bit 0 nên không làm thay đổi gì tổng trên. Nếu A là số âm, ta có hai tổng sau phải bằng nhau:

$$-2^{m-1} + \sum_{i=0}^{m-2} 2^i a_i = -2^{n-1} \sum_{i=0}^{n-2} 2^i a_i$$

$$-2^{m-1} + \sum_{i=n-1}^{m-2} 2^i a_i = -2^{n-1}$$

$$2^{n-1} + \sum_{i=n-1}^{m-2} 2^i a_i = 2^{m-1}$$

$$1 + \sum_{i=0}^{n-2} 2^i + \sum_{i=n-1}^{m-2} 2^i a_i = 1 + \sum_{i=0}^{m-2} 2^i$$

$$\sum_{i=n-1}^{m-2} 2^i a_i = \sum_{i=n-1}^{m-2} 2^i$$

Như vậy, rõ ràng $a_{m-2} = \dots = a_{n-2} = a_{n-1} = 1$.

1.5.7. Biểu diễn dấu chấm động

1.5.7.1. Nguyên tắc

Với biểu diễn số dạng dấu chấm tĩnh (ví dụ: dạng bù hai) ta chỉ có thể biểu diễn một dải các số nguyên dương hoặc âm xung quanh 0. Ngoài ra, để biểu diễn số thực, ta có thể giả định một số bit nhất định cho phần phân số. Tuy nhiên, giải pháp này có một số hạn chế: không thể biểu diễn được các số quá lớn hoặc quá bé.

Với số thập phân, để biểu diễn các số lớn hoặc quá bé, thông thường chúng ta sử dụng cách như sau:

$$976,000,000,000,000 = 9.76 \times 10^{14}$$

Hay

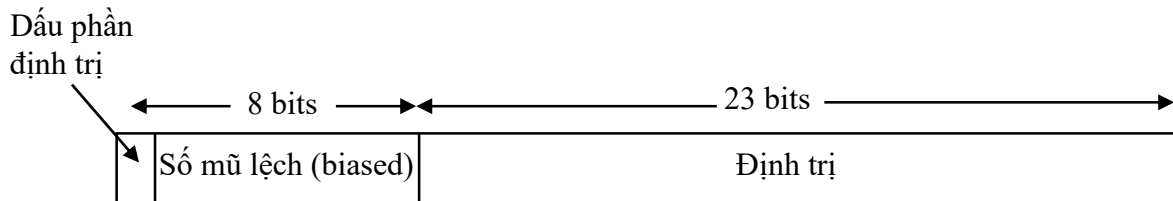
$$0.000000000000976 = 9.76 \times 10^{-14}$$

Với cách này, ta có thể biểu diễn các số quá lớn hoặc quá bé bằng cách ngắn gọn hơn rất nhiều.

Tương tự ta có thể biểu diễn một số nhị phân như sau: $\pm S \times B^{\pm E}$

Trong đó:

- Dấu: cộng hoặc trừ
- Phần định trị S
- Phần số mũ E
- Cơ số B (với số nhị phân, cơ số bằng 2)



a. Định dạng

1.1010001	$\times 2^{10100}$	=	0 10010011 101000100000000000000000	1.6328125	$\times 2^{20}$
-1.1010001	$\times 2^{10100}$	=	1 10010011 101000100000000000000000	-1.6328125	$\times 2^{20}$
1.1010001	$\times 2^{-10100}$	=	0 01101011 101000100000000000000000	1.6328125	$\times 2^{-20}$
-1.1010001	$\times 2^{-10100}$	=	1 01101011 101000100000000000000000	-1.6328125	$\times 2^{-20}$

b. Các ví dụ

Hình 1.13. Định dạng biểu diễn dấu chấm động 32-bit.

Như vậy, khi lưu trữ một số nhị phân, ta chỉ cần lưu trữ dấu, phần định trị S và số mũ E. Còn cơ số B mặc định bằng 2, ta không cần lưu trữ. Định dạng biểu diễn dấu chấm động 32-bit điển hình được minh họa trong Hình 1.13. Bit ngoài cùng bên trái là bit dấu, 8 bit tiếp theo được sử dụng để lưu trữ số mũ. Tuy nhiên, giá trị số mũ không được lưu trữ trực tiếp tại 8-bit này mà được điều chỉnh thành **số mũ lệch (biased)**. Số mũ thực tế sẽ bằng số mũ lệch trừ đi một giá trị cố định, gọi là **độ lệch**. Giá trị độ lệch bằng $(2^{k-1} - 1)$, với k là số bit biểu diễn phần mũ. Trong trường hợp này, có 8 bit phần mũ nên độ lệch là 127 ($2^7 - 1$). Như vậy, với 8-bit trường mũ lệch ta biểu diễn các số trong khoảng từ 0 đến 255, trừ đi độ lệch, giá trị số mũ thực tế sẽ từ -127 đến +128. 23 bit còn lại biểu diễn phần định trị.

Vậy, để biểu diễn một số thực nhị phân bất kỳ dưới định dạng 32-bit như trên, ta phải đưa số đó về dạng tiêu chuẩn như sau: số ngoài cùng bên trái phải bằng 1, sau đó là dấu chấm và phần phân số: $\pm 1.bbb \dots b \times 2^{\pm E}$

Với b là các bit nhị phân (0 hoặc 1). Vì bit ngoài cùng bên trái luôn là 1 nên ta không cần thiết phải lưu trữ bit này. Như vậy, với trường 23-bit ta có thể lưu trữ được 24-bit phần định trị. Vậy nếu số chưa ở dạng tiêu chuẩn, ta phải dịch dấu chấm về vị trí sau bit 1 đầu tiên và điều chỉnh số mũ tương ứng.

Các số nhị phân sau đây có giá trị tương đương nhau:

$$0.11 \times 2^5$$

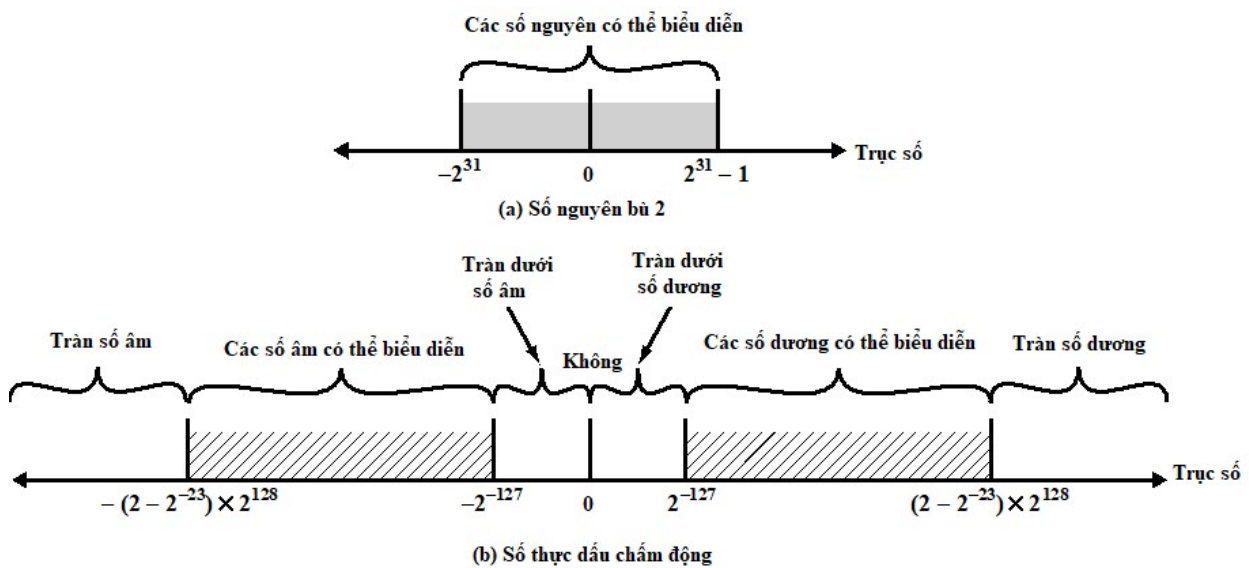
$$1102^6$$

$$0.01102^6$$

$$1.12^4 \text{ (Dạng tiêu chuẩn)}$$

Các ví dụ trong Hình 1.13b được lưu trữ theo định dạng trên. Trong đó, ở bên trái là số thực nhị phân, ở giữa là dạng biểu diễn số thực dấu chấm động 32-bit theo định dạng ở trên, bên phải là giá trị của số ở hệ thập phân. Chú ý một số điểm sau:

- Dấu được lưu trữ ở bit đầu tiên.
- Bit đầu tiên của phần định trị luôn là 1, do vậy không cần lưu trữ.
- Giá trị trong trường số mũ là giá trị số mũ thực tế cộng với 127.
- Cơ số là 2



Hình 1.14. Các số có thể biểu diễn với định dạng 32-bit

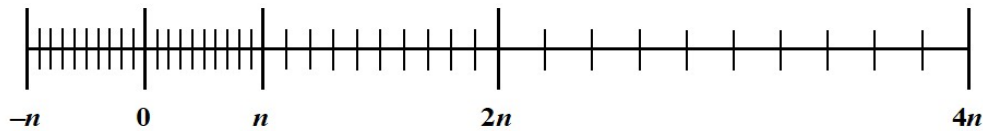
Trong hình 1.14 là miền giá trị mà một từ 32-bit có thể biểu diễn được. Với dạng biểu diễn bù hai ta có thể biểu diễn được các giá trị từ -2^{31} đến $2^{31} - 1$, như vậy có tất cả 2^{32} số khác nhau. Với định dạng dấu chấm động, miền giá trị như sau:

- Các số âm từ $-(2 - 2^{-23}) \times 2^{128}$ đến -2^{-127}
- Các số dương từ 2^{-127} đến $(2 - 2^{-23}) \times 2^{128}$
- Năm miền giá trị không thể biểu diễn:
- Các số âm nhỏ hơn $-(2 - 2^{-23}) \times 2^{128}$, gọi là Tràn trên âm
- Các số âm lớn hơn -2^{-127} , gọi là Tràn dưới âm
- Số không
- Các số dương nhỏ hơn 2^{-127} , gọi là Tràn dưới dương
- Các số dương lớn hơn $(2 - 2^{-23}) \times 2^{128}$, gọi là Tràn trên dương

Như trên ta thấy, định dạng dấu chấm động không biểu diễn được số 0. Tuy nhiên, phần sau ta sẽ sử dụng một biểu diễn quy ước cho số 0. Định dạng dấu chấm động không cho phép biểu diễn nhiều số hơn so với các dạng biểu diễn khác cùng số chiều dài bit. Số lượng tối đa các giá trị khác nhau một từ nhớ 32 bit biểu diễn được vẫn là 2^{32} . Điều mà dạng biểu diễn này thực hiện được

đó là trải các giá trị biểu diễn ra thành miền dài, một miền âm và một miền dương. Trong thực tế, hầu như các số biểu diễn dưới dạng dấu chấm động chỉ là biểu diễn xấp xỉ.

Ngoài ra, vị trí các số được biểu diễn trên trục số không cách đều nhau (Hình 1.15). Các số ở gần gốc tọa độ có khoảng cách gần hơn và ngược lại. Đây là một trong những nhược điểm của các phép toán thực hiện trên biểu diễn dấu chấm động: Các phép toán có kết quả không bằng chính xác giá trị có thể biểu diễn được và phải được làm tròn về giá trị gần nhất có thể biểu diễn được.

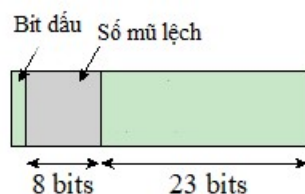


Hình 1.15. Mật độ các số dấu chấm động

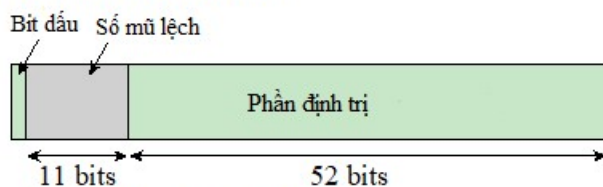
Với định dạng 32-bit ở trên, độ chính xác và miền giá trị có một mối quan hệ với nhau. Nếu chúng ta tăng số bit phần mũ, phạm vi biểu diễn của định dạng đó sẽ tăng lên. Tuy nhiên, với 32-bit ta chỉ biểu diễn được 2^{32} giá trị khác nhau nên khi phạm vi biểu diễn tăng thì độ chính xác sẽ giảm đi. Như vậy, với một định dạng nhất định, phần mũ càng lớn thì miền giá trị càng lớn tuy nhiên độ chính xác càng nhỏ. Cách duy nhất để tăng cả phạm vi và độ chính xác là sử dụng nhiều bit hơn. Do đó, hầu hết các máy tính cung cấp các dạng biểu diễn có độ chính xác đơn và độ chính xác kép. Ví dụ, bộ xử lý có thể hỗ trợ định dạng chính xác đơn là 64 bit và một định dạng chính xác kép là 128 bit.

1.5.7.2. Chuẩn IEEE cho biểu diễn số nhị phân dấu chấm động

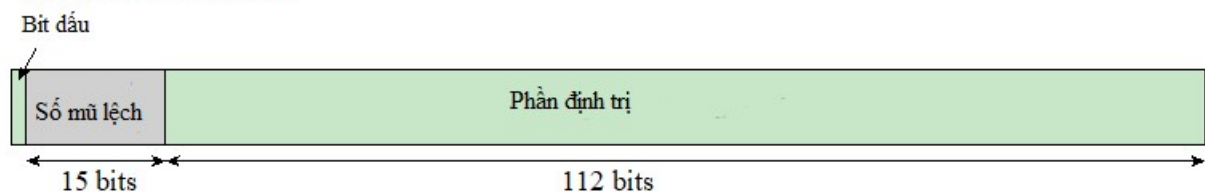
Chuẩn IEEE 754 được đưa ra năm 1985 và sửa đổi năm 2008 quy định về biểu diễn nhị phân dấu chấm động trong máy tính. Chuẩn này được phát triển với mục đích tạo thuận lợi cho các chương trình máy tính có thể tương thích với nhiều bộ xử lý khác nhau cũng như khuyến khích phát triển các chương trình định hướng số học tinh vi hơn. Hiện nay, chuẩn IEEE 754 đã được áp dụng rộng rãi trên hầu hết các bộ xử lý và các bộ tính toán số học hiện tại.



(a) Định dạng nhị phân 32



(b) Định dạng nhị phân 64



(c) Định dạng nhị phân 128

Hình 1.16. Các định dạng IEEE 754

IEEE 754-2008 bao gồm các quy định về cả dạng biểu diễn nhị phân và thập phân dấu chấm động. Trong phần này, chúng ta chỉ quan tâm tới các dạng biểu diễn nhị phân. IEEE 754-2008 định nghĩa các định dạng dấu chấm động khác nhau như sau:

- **Định dạng số học:** Các phép toán được quy định trong chuẩn cần được hỗ trợ các định dạng số học. Định dạng này được sử dụng để biểu diễn toán hạng hoặc kết quả của các phép toán dưới dạng dấu chấm động.
- **Định dạng cơ bản:** Định dạng này bao gồm năm dạng biểu diễn dấu chấm động: ba dạng nhị phân và hai dạng thập phân cùng với các chuẩn mã hóa của chúng và quy định định dạng nào được sử dụng trong số học.
- **Định dạng chuyển đổi:** Một định dạng mã hóa nhị phân có chiều dài cố định, cho phép chuyển đổi dữ liệu giữa các nền tảng khác nhau và có thể được sử dụng để lưu trữ.

Bảng 1.11. Quy định trong chuẩn IEEE 754

Thông số	Định dạng		
	Nhị phân 32	Nhị phân 64	Nhị phân 128
Kích thước (bit)	32	64	128
Số bit trường mũ	8	11	15
Độ lệch	127	1023	16383
Số mũ tối đa	127	1023	16383
Số mũ tối thiểu	-126	-1022	-16382
Miền giá trị xấp xỉ (hệ 10)	$10^{-38}, 10^{+38}$	$10^{-308}, 10^{+308}$	$10^{-4932}, 10^{+4932}$
Số bit trường định trị theo sau	23	52	112
Số lượng số mũ	254	2046	32766
Số lượng các số phân số	2^{23}	2^{52}	2^{112}
Số lượng các giá trị	1.98×3^{31}	1.99×2^{63}	1.99×2^{127}
Số dương nhỏ nhất	2^{-126}	2^{-1022}	2^{-16382}
Số dương lớn nhất	$2^{128} - 2^{104}$	$2^{1024} - 2^{971}$	$2^{16384} - 2^{16271}$

Ba **định dạng nhị phân** cơ bản lần lượt có kích thước bit là **32**, **64** và **128** bit, với số bit trường mũ là 8, 11 và 15 bit (hình 1.16). Bảng 1.10 tóm tắt các đặc tính của ba định dạng này. Hai **định dạng thập phân** cơ bản có độ dài bit là **64** và **128** bit. Tất cả các định dạng cơ bản trên cũng đồng thời là các định dạng số học (có thể được sử dụng cho các phép toán (số học) và định dạng chuyển đổi (độc lập với các nền tảng khác nhau)).

Ngoài ra, chuẩn 754 cũng quy định một số định dạng khác như: **định dạng nhị phân 16**, đây là định dạng chuyển đổi và được dùng để lưu trữ các giá trị không cần độ chính xác cao. Định dạng chuyển đổi còn quy định hai **định dạng nhị phân {k}** và **thập phân {k}** có chiều dài k bit với kích thước phần định trị và phần mũ cụ thể. Số bit của các định dạng này phải là một bội số của 32 bit; do đó $k = 160, 192, \dots$. Các định dạng này cũng là định dạng số học.

Một định dạng khác cũng được định nghĩa trong chuẩn là các định dạng **độ chính xác mở rộng** (gọi tắt là **định dạng mở rộng**) quy định việc mở rộng miền giá trị bằng cách thêm các bit

phần mũ và tăng độ chính xác bằng cách tăng số bit phần định trị. Kích thước của phần định trị và phần mũ phụ thuộc vào việc thực hiện cụ thể, tuy nhiên chuẩn quy định một số ràng buộc về chúng. Định dạng mở rộng nằm trong các loại định dạng số học nên nó thường được sử dụng trong các phép toán trung gian. Do có độ chính xác cao hơn, định dạng mở rộng giảm sai số của phép toán do việc làm tròn; và với phạm vi biểu diễn lớn hơn, định dạng này cũng giảm khả năng một hoạt động bị lỗi do tràn kết quả của phép toán trung gian. Ngoài ra, định dạng mở rộng vẫn mang các ưu điểm của định dạng cơ bản có kích thước lớn mà không bị ảnh hưởng về mặt thời gian xử lý do có độ chính xác cao hơn.

Cuối cùng, IEEE 754-2008 định nghĩa là **định dạng chính xác có thể mở rộng**: cho phép người dùng thiết lập độ chính xác và phạm vi biểu diễn. Các định dạng theo kiểu này có thể được sử dụng cho các phép tính trung gian nhưng chuẩn không đưa ra các ràng buộc về khuôn dạng hoặc kích thước của chúng. Bảng 1.11 là quan hệ giữa các định dạng ở trên và các loại định dạng.

Bảng 1.12. Các định dạng IEEE

Định dạng	Loại định dạng		
	Định dạng số học	Định dạng cơ bản	Định dạng chuyển đổi
Nhị phân 16			X
Nhị phân 32	X	X	X
Nhị phân 64	X	X	X
Nhị phân 128	X	X	X
Nhị phân $\{k\}$ ($k = n \times 32$ với $n > 4$)	X	X	X
Thập phân 64	X	X	X
Thập phân 128	X	X	X
Thập phân $\{k\}$ ($k = n \times 32$ với $n > 4$)	X		X
Độ chính xác mở rộng	X		
Độ chính xác có thể mở rộng	X		

Không chỉ quy định về các định dạng, chuẩn IEEE còn quy định một số mẫu bit biểu diễn một số giá trị đặc biệt như trong Bảng 1.12. Cụ thể chuẩn IEEE quy định về giá trị biểu diễn của các định dạng như sau:

- Giá trị trường mũ của định dạng 32-bit là từ 1 đến 254, tương ứng với số mũ thực tế là từ -126 đến +127. Tương tự như vậy với định dạng 64-bit: từ 1 đến 2046 ứng với số mũ thực tế từ -1022 đến +1023 và 128-bit... Với các số mũ trong khoảng này, các định dạng trên sẽ biểu diễn các số theo dạng dấu chấm động tiêu chuẩn (phần trên đã đề cập đến) với bit 1 ở ngoài cùng bên trái trước dấu chấm. Như vậy, với số 1 là ngầm định, các định dạng trên cho phép ta biểu diễn được 24-bit, 53-bit, hoặc 113-bit phần định trị. Và do đó, người ta gọi trường này là **trường định trị theo sau**.
- Như phần trên đã đề cập, số nhị phân dấu chấm động không biểu diễn được giá trị 0, do đó, chuẩn này quy ước biểu diễn số không là số có phần mũ và phần phân số bằng 0. Tùy thuộc vào bit dấu, ta có số không âm hoặc dương.
- Với tất cả các bit phần mũ bằng 1 và phần phân số bằng 0 biểu diễn âm vô cùng hoặc dương vô cùng tùy thuộc vào bit dấu. Quy ước này cho phép người sử dụng quyết định khi nào xử lý tràn như một điều kiện lỗi hoặc gán giá trị vô cùng và tiếp tục thực thi chương trình.

- Trường mũ bằng không, trường phân số khác không biểu diễn số dạng bán tiêu chuẩn: bit trước dấu chấm bằng 0 (thay vì ngầm định là 1 trong phần trên) và số mũ thực tế là -126 (định dạng 32b) hoặc -1022 (định dạng 64b). Số được biểu diễn là số dương hoặc âm tùy thuộc vào bit dấu.
- Trường mũ gồm tất cả các bit 1, phần phân số là một số bất kỳ khác không được quy ước là giá trị NaN (Not a Number – Không phải là số) và được sử dụng để báo hiệu các điều kiện ngoại lệ khác.

Bảng 1.13. Các quy ước biểu diễn số nhị phân dấu chấm động trong IEEE 754

a. Định dạng nhị phân 32 bit

	Dấu	Số mũ lệch	Phần phân số	Giá trị
Không dương	0	0	0	0
Không âm	1	0	0	-0
Dương vô cùng	0	Tất cả các bit là 1	0	$+\infty$
Âm vô cùng	1	Tất cả các bit là 1	0	$-\infty$
Quiet NaN	0 hoặc 1	Tất cả các bit là 1	$\neq 0$; bit đầu tiên = 1	qNaN
Signaling NaN	0 hoặc 1	Tất cả các bit là 1	$\neq 0$; bit đầu tiên = 0	sNaN
Số khác không dương	0	$0 < e < 255$	f	$1.f \times 2^{e-127}$
Số khác không âm	1	$0 < e < 255$	f	$1.f \times 2^{e-127}$
Số bán tiêu chuẩn dương	0	0	$f \neq 0$	$0.f \times 2^{e-126}$
Số bán tiêu chuẩn âm	1	0	$f \neq 0$	$-0.f \times 2^{e-126}$

b. Định dạng nhị phân 64 bit

	Dấu	Số mũ lệch	Phần phân số	Giá trị
Không dương	0	0	0	0
Không âm	1	0	0	-0
Dương vô cùng	0	Tất cả các bit là 1	0	$+\infty$
Âm vô cùng	1	Tất cả các bit là 1	0	$-\infty$
Quiet NaN	0 hoặc 1	Tất cả các bit là 1	$\neq 0$; bit đầu tiên = 1	qNaN
Signaling NaN	0 hoặc 1	Tất cả các bit là 1	$\neq 0$; bit đầu tiên = 0	sNaN
Số khác không dương	0	$0 < e < 2047$	f	$1.f \times 2^{e-1023}$
Số khác không âm	1	$0 < e < 2047$	f	$1.f \times 2^{e-1023}$
Số bán tiêu chuẩn dương	0	0	$f \neq 0$	$0.f \times 2^{e-1022}$
Số bán tiêu chuẩn âm	1	0	$f \neq 0$	$-0.f \times 2^{e-1022}$

c. Định dạng nhị phân 128 bit

	Dấu	Số mũ lệch	Phần phân số	Giá trị
Không dương	0	0	0	0
Không âm	1	0	0	-0
Dương vô cùng	0	Tất cả các bit là 1	0	$+\infty$
Âm vô cùng	1	Tất cả các bit là 1	0	$-\infty$
Quiet NaN	0 hoặc 1	Tất cả các bit là 1	$\neq 0$; bit đầu tiên = 1	qNaN
Signaling NaN	0 hoặc 1	Tất cả các bit là 1	$\neq 0$; bit đầu tiên = 0	sNaN
Số khác không dương	0	Tất cả các bit là 1	f	$1.f \times 2^{e-16383}$
Số khác không âm	1	Tất cả các bit là 1	f	$1.f \times 2^{e-16383}$
Số bán tiêu chuẩn dương	0	0	$f \neq 0$	$0.f \times 2^{e-16383}$
Số bán tiêu chuẩn âm	1	0	$f \neq 0$	$-0.f \times 2^{e-16383}$

1.6. CÁC THÀNH PHẦN CỦA HỆ THỐNG MÁY TÍNH

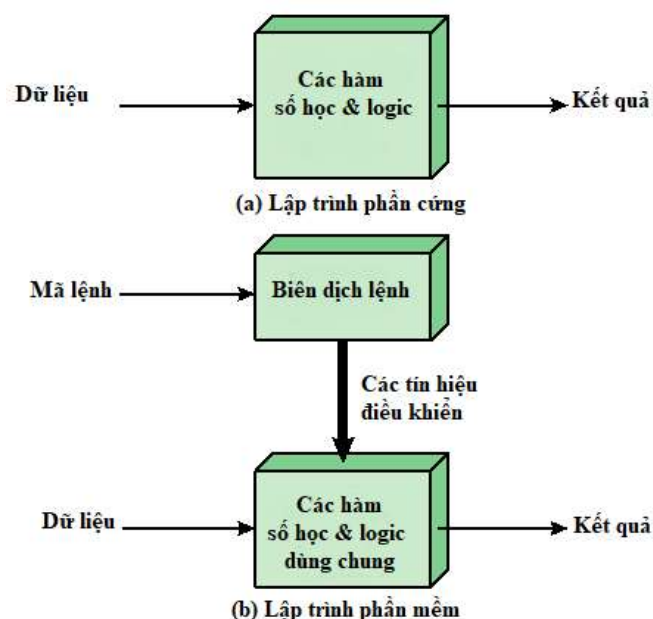
Về mặt tổng quát, một máy tính gồm có bộ xử lý trung tâm (CPU), bộ nhớ và các thành phần vào/ra (I/O). Các thành phần trên được kết nối với nhau theo một quy tắc nhất định để thực thi các chức năng cơ bản của máy tính: thực thi các chương trình máy tính. Vì vậy, chúng ta có thể mô tả một hệ thống máy tính bằng cách mô tả: (1) *hành vi bên ngoài của mỗi thành phần*: dữ liệu và các tín hiệu điều khiển mà nó trao đổi với các thành phần khác; (2) *cấu trúc kết nối và điều khiển* cần thiết để quản lý việc sử dụng cấu trúc kết nối.

1.6.1. Các thành phần máy tính

Trong phần 1.3 chúng ta đã biết thiết kế máy tính được dựa trên kiến trúc cơ bản của John Von Neumann: *kiến trúc Von Neumann* có ba đặc điểm chính như sau:

- Dữ liệu và lệnh được lưu trữ trên một bộ nhớ đọc-ghi
- Nội dung trong bộ nhớ này được đánh địa chỉ theo vị trí (không phân biệt kiểu dữ liệu được lưu trữ trong đó)
- Thực thi lệnh được thực hiện một cách tuần tự (lần lượt từng lệnh một)

Chúng ta có một tập hợp các thành phần logic cơ bản được kết nối theo các cách khác nhau để lưu trữ dữ liệu nhị phân và thực hiện các phép toán số học và logic trên dữ liệu này. Nếu có một phép toán cần được tính toán, ta sẽ phải xây dựng một cấu hình các thành phần logic để thực hiện phép toán này. Có thể hiểu việc nối các thành phần khác nhau để tạo ra cấu hình mong muốn chính là việc lập trình. Kết quả là ta thu được một “*chương trình*” *phần cứng*.



Hình 1.17. Lập trình phần cứng và phần mềm

Giả sử ta xây dựng một cấu hình dùng chung cho tất cả các hàm toán học và logic. Bộ phần cứng này có thể thực hiện được các hàm khác nhau với dữ liệu tùy thuộc vào tín hiệu điều khiển đưa vào phần cứng. Trong trường hợp trên, đầu vào và đầu ra của mạch là dữ liệu và kết quả (Hình 1.17a). Ở trường hợp này, hệ thống nhận dữ liệu, tín hiệu điều khiển và đưa ra kết quả. Như vậy, thay vì phải nối lại dây cho mỗi một chương trình, người lập trình chỉ cần đưa ra một tập các tín hiệu điều khiển hoạt động của phần cứng.

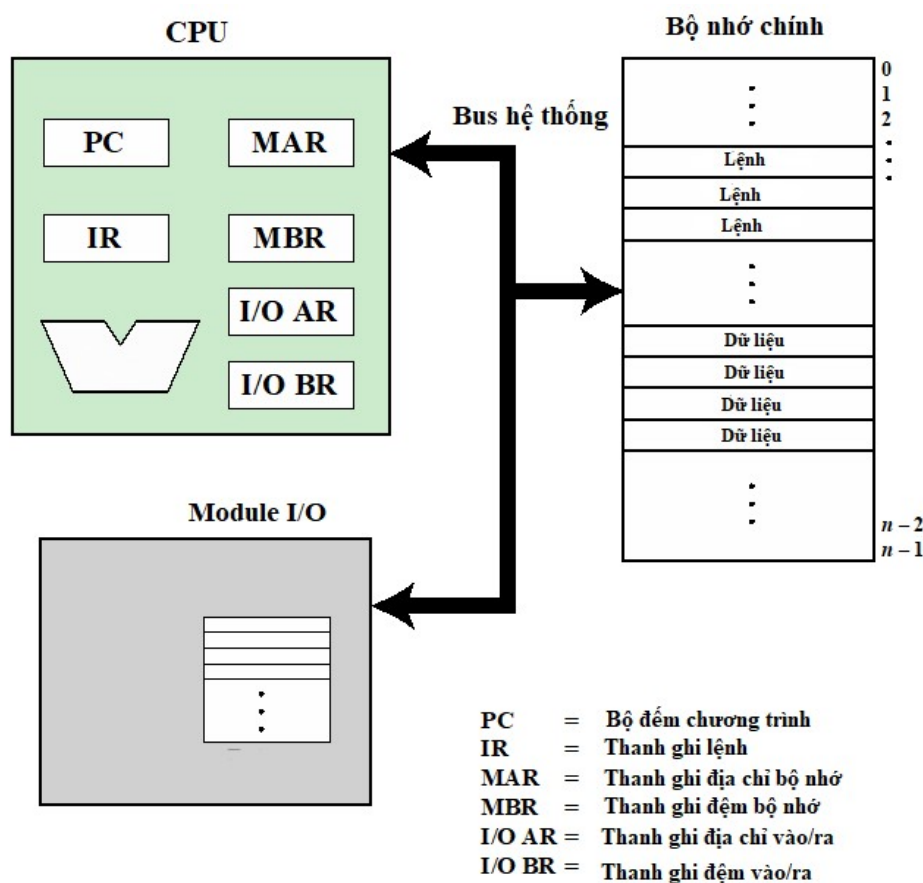
Vậy các tín hiệu điều khiển được đưa vào như thế nào? Thực tế, mỗi chương trình là một chuỗi các bước. Tại mỗi bước, một vài phép toán số học và logic được thực hiện, do vậy cần phải

có một tập các tín hiệu điều khiển tương ứng. Vì vậy, ta sẽ dùng một mã (code) duy nhất cho mỗi tập tín hiệu điều khiển và thiết lập thêm một khối phần cứng có nhiệm vụ nhận mã này và đưa ra các tín hiệu điều khiển tương ứng (Hình 1.17b).

Đến đây, công việc lập trình trở nên đơn giản hơn rất nhiều. Thay vì phải nối lại các bộ phận phần cứng cho mỗi chương trình mới, chúng ta chỉ cần đưa ra một chuỗi các từ mã mới. Mỗi một mã là một câu lệnh được đưa qua phần cứng để biên dịch thành các tín hiệu điều khiển. Chuỗi các mã hoặc lệnh này được gọi là *phần mềm*.

Trong hình 1.17b ta có thể thấy có hai thành phần chính của hệ thống: bộ biên dịch lệnh và một module thực thi các hàm tính toán số học và logic chung. Chúng cấu thành lên CPU. Ngoài ra, cần có một vài thành phần khác nữa mới có thể tạo nên một bộ máy tính. Dữ liệu và lệnh cần được đưa vào hệ thống. Để làm được việc đó, chúng ta cần module vào. Module này chứa các thành phần cơ bản để nhận dữ liệu và lệnh ở một vài dạng ban đầu, biến đổi chúng thành dạng tín hiệu bên trong có thể sử dụng được bởi hệ thống. Để đưa dữ liệu ra, ta cần một module ra. Vậy, chúng ta gọi chung là *các thành phần vào/ra (I/O module)*.

Thêm vào đó, thiết bị vào sẽ đưa lệnh và dữ liệu vào một cách tuần tự, nhưng chương trình đôi khi không thực hiện một cách tuần tự, nó có thể nhảy đến vị trí khác (vd: *lệnh nhảy (JUMP)* (trong IAS). Tương tự, có thể cần phải tham chiếu đến nhiều toán hạng và dữ liệu tại một thời điểm. Do vậy, ta cần phải có một nơi để lưu trữ tạm thời dữ liệu và lệnh. Module đó được gọi là *bộ nhớ* hay *bộ nhớ chính* để phân biệt với bộ nhớ ngoài hay các thiết bị ngoại vi. Von Neumann đã chỉ ra rằng dữ liệu và lệnh có thể cùng lưu trữ trên một bộ nhớ.



Hình 1.18. Các thành phần máy tính

Hình 1.18 minh họa các thành phần chính của máy tính và giao tiếp giữa chúng. CPU trao đổi dữ liệu với bộ nhớ thông qua hai thanh ghi: MAR (thanh ghi địa chỉ bộ nhớ) và MBR (thanh

ghi đệm bộ nhớ). Thanh ghi MAR chứa địa chỉ bộ nhớ cho thao tác đọc hoặc ghi tiếp theo, thanh ghi MBR chứa dữ liệu được đọc từ bộ nhớ hoặc chuẩn bị được ghi vào bộ nhớ. Tương tự, thanh ghi địa chỉ vào/ra (I/OAR) chỉ ra một thiết bị vào/ra cụ thể. Thanh ghi đệm vào/ra (I/OBR) được sử dụng để chuyển tiếp dữ liệu giữa một module vào/ra và CPU.

Module bộ nhớ gồm một tập các vị trí được đánh địa chỉ dạng số liên tục. Mỗi vị trí chứa một số nhị phân có thể là dữ liệu hoặc lệnh. Một module vào/ra truyền dữ liệu từ thiết bị ngoài vào CPU và bộ nhớ và ngược lại. Các bộ đệm bên trong module vào/ra lưu trữ dữ liệu tạm thời cho đến khi nó được gửi đi.

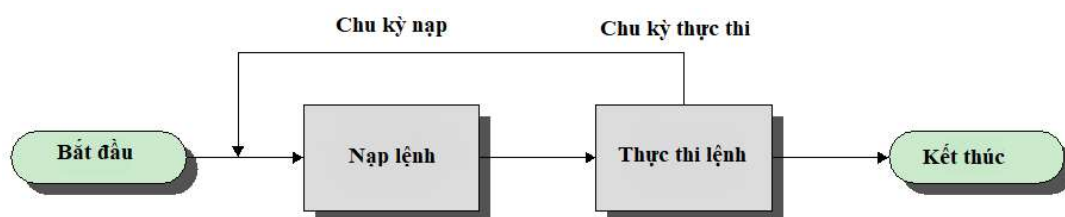
1.6.2. Chức năng của máy tính

Chức năng chính của máy tính là thực thi chương trình gồm nhiều lệnh lưu trữ trong bộ nhớ máy tính. Bộ xử lý sẽ lần lượt thực hiện các lệnh theo một chu trình nhất định. Ở dạng đơn giản nhất, một câu lệnh được thực thi qua hai bước: Bộ xử lý *đọc (truy xuất) lệnh* từ bộ nhớ và *thực thi lệnh* đó. Việc đó được thực hiện lặp đi lặp lại với tất cả các lệnh trong chương trình.

Quá trình thực hiện một lệnh được gọi là **chu kỳ lệnh**. Một chu kỳ lệnh được minh họa trong Hình 1.19 gồm có hai bước như đã nói ở trên đó là: **chu kỳ truy xuất** và **chu kỳ thực thi**.

1.6.2.1. Truy xuất và thực thi lệnh

Bắt đầu một chu kỳ lệnh, bộ xử lý truy xuất lệnh từ bộ nhớ. Trong các máy tính thông thường, thanh ghi PC (program counter: bộ đếm chương trình) chứa địa chỉ của lệnh sẽ được truy xuất tiếp theo. Nếu không có gì đặc biệt, bộ xử lý tự động tăng PC sau mỗi lần truy xuất lệnh để nó sẽ truy xuất đến câu lệnh tiếp theo của chương trình (các câu lệnh được lưu trữ lần lượt tại các địa chỉ bộ nhớ liên tục, câu lệnh sau có địa chỉ tiếp theo lớn hơn câu lệnh trước).



Hình 1.19. Chu kỳ lệnh cơ bản

Ví dụ, ta có một máy tính trong đó mỗi từ lệnh có 16 bit lưu trữ trong bộ nhớ. Bộ xử lý truy xuất đến câu lệnh tại địa chỉ 300. Theo trình tự thông thường, các câu lệnh tiếp theo truy xuất đến sẽ lần lượt là 301, 302, 303,...

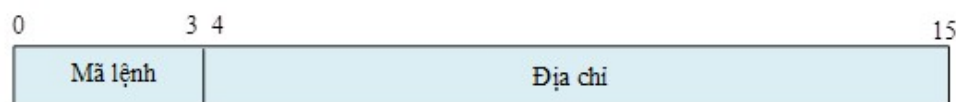
Lệnh sau khi được truy xuất sẽ được ghi vào thanh ghi IR (instruction register – thanh ghi lệnh) trong bộ xử lý. Lệnh chứa các bit chỉ ra hoạt động bộ xử lý cần thực hiện. Bộ xử lý giải mã các bit này và thực hiện hoạt động được yêu cầu. Thông thường, các hoạt động được chia thành bốn nhóm:

- **Bộ xử lý – bộ nhớ:** Dữ liệu được truyền từ bộ xử lý đến bộ nhớ hoặc từ bộ nhớ đến bộ xử lý.
- **Bộ xử lý – Vào/ra:** Dữ liệu được truyền đến và đi từ thiết bị ngoại vi bằng cách truyền dữ liệu giữa bộ xử lý và module vào/ra.
- **Xử lý dữ liệu:** Bộ xử lý thực hiện các phép toán số học và logic lên dữ liệu
- **Điều khiển:** Một lệnh có thể chứa chỉ thị làm thay đổi tuần tự thực hiện các câu lệnh. Ví dụ: bộ xử lý truy xuất câu lệnh ở địa chỉ 149, trong đó chứa chỉ thị yêu cầu truy xuất câu

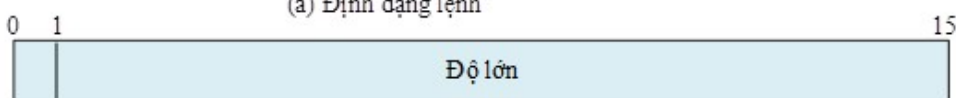
lệnh tiếp theo ở địa chỉ 182. Bộ xử lý sẽ phải thiết lập thanh ghi PC về giá trị 182 thay vì 150 theo trình tự ban đầu.

Việc thực thi một lệnh có thể kết hợp tất cả các hoạt động trên.

Xét một ví dụ đơn giản trong hình 1.20 như sau: ta có một máy giả thiết có các đặc điểm như trong hình. Bộ xử lý chứa một thanh ghi dữ liệu được gọi là thanh ghi AC (accumulator). Cả lệnh và dữ liệu có 16 bit chiều dài do vậy sẽ rất thuận tiện nếu ta tổ chức bộ nhớ thành các từ nhớ 16b. Về cấu trúc lệnh: 4 bit dùng cho phần mã lệnh, như vậy ta có thể có đến $2^4 = 16$ mã lệnh khác nhau và tối đa $2^{12} = 4096$ (4K) từ nhớ trong bộ nhớ có thể được truy cập trực tiếp.



(a) Định dạng lệnh



(b) Định dạng số nguyên

Thanh ghi PC = Địa chỉ của lệnh
 Thanh ghi IR = Lệnh sẽ được thực hiện
 Thanh ghi AC = Thanh ghi lưu trữ dữ liệu làm thời

(c) Các thanh ghi trong CPU

0001 = Đọc dữ liệu từ bộ nhớ vào thanh ghi AC
 0010 = Lưu trữ dữ liệu từ thanh ghi AC vào bộ nhớ
 0101 = Đọc dữ liệu từ bộ nhớ và cộng vào thanh ghi AC

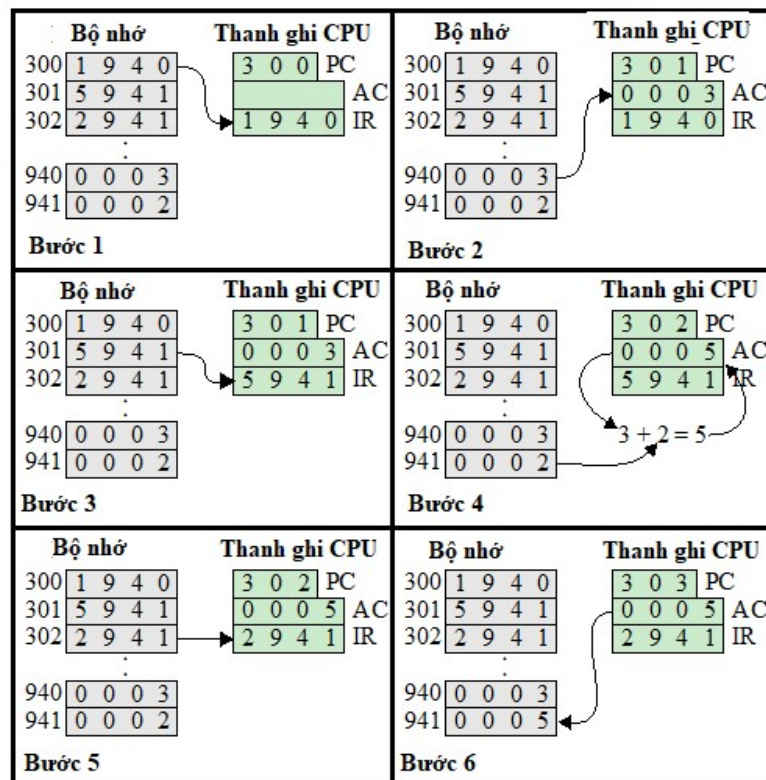
(d) Các mã lệnh được sử dụng

Hình 1.20. Đặc tính của máy giả thiết

Hình 1.21 minh họa hoạt động của bộ xử lý thực hiện một chương trình và trạng thái của các thanh ghi trong quá trình đó. Chương trình này cộng giá trị tại địa chỉ 940 và 941 và ghi kết quả vào ô nhớ 941. Với các lệnh này, ta sẽ có ba chu kỳ truy xuất và ba truy thực thi như sau:

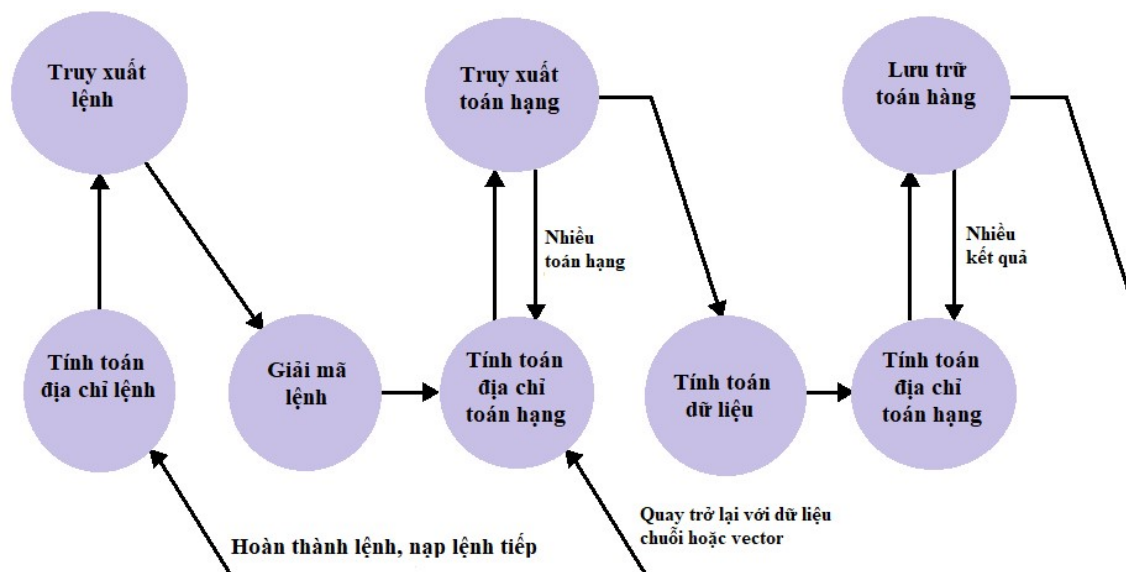
1. Thanh ghi PC có giá trị 300 (địa chỉ của câu lệnh đầu tiên của chương trình). Do vậy, lệnh (có giá trị 1940 theo hệ thập lục phân) được tải vào thanh ghi IR, sau đó thanh ghi PC tăng thêm một đơn vị. Chú ý rằng, quá trình truy xuất này có sự tham gia của hai thanh ghi là MAR và MBR. Tuy nhiên để đơn giản, chúng ta tạm thời bỏ qua các thanh ghi này.
2. 4 bit đầu tiên (tương ứng với chữ số hệ thập lục phân đầu tiên) trong thanh ghi IR chỉ thị rằng dữ liệu sẽ được tải vào thanh ghi AC. 12 bit còn lại là địa chỉ (940) của dữ liệu được tải vào.
3. Lệnh tiếp theo (5941) được đọc vào từ địa chỉ 301, sau đó thanh ghi PC tăng lên một đơn vị.
4. Cộng dữ liệu từ địa chỉ 941 với dữ liệu cũ trong thanh ghi AC, kết quả được lưu trữ lại vào thanh ghi AC.
5. Lệnh tiếp theo (2941) được truy xuất từ địa chỉ 302, thanh ghi PC tăng lên một đơn vị.
6. Nội dung của thanh ghi AC được lưu trữ vào địa chỉ 941.

Trong ví dụ này, mỗi lệnh bao gồm một chu kỳ truy xuất và chu kỳ thực thi. Trong các thế hệ máy tính sau này, một lệnh có thể có nhiều hơn một địa chỉ, đó đó chu kỳ lệnh sẽ có nhiều hơn một lần truy cập bộ nhớ. Ngoài ra, không chỉ tham chiếu đến bộ nhớ, các lệnh còn có thể thực hiện các hoạt động vào/ra thiết bị ngoại vi.



Hình 1.21. Ví dụ về thực thi chương trình (dữ liệu bộ nhớ và thanh ghi biểu diễn dạng HEX)

Hình 1.22 minh họa chi tiết hơn về chu kỳ lệnh dưới dạng đồ thị trạng thái. Với mỗi lệnh cụ thể, một số trạng thái có thể không có, một số trạng thái có thể xuất hiện nhiều hơn một lần.



Hình 1.22. Sơ đồ trạng thái chu kỳ lệnh

Các trạng thái bao gồm:

- **Tính toán địa chỉ lệnh (iac):** Xác định địa chỉ lệnh tiếp theo được thực hiện. Thông thường, việc này được thực hiện bằng cách cộng một số cố định vào địa chỉ của lệnh trước đó. Ví dụ, nếu lệnh dài 16 bit và bộ nhớ được tổ chức dưới dạng các từ nhớ 16b thì ta sẽ cộng 1 vào địa chỉ lệnh trước đó. Nếu bộ nhớ được tổ chức dưới dạng các từ nhớ 8b thì sẽ phải cộng 2 vào địa chỉ trước lệnh trước đó (do một lệnh sẽ được lưu trữ trên 2 từ nhớ).

- **Truy xuất lệnh (if):** Đọc lệnh từ địa chỉ của nó vào bộ xử lý.
- **Giải mã lệnh (iod):** Phân tích lệnh để xác định loại hoạt động cần thực hiện và các toán hạng được sử dụng.
- **Tính toán địa chỉ toán hạng (oac):** Nếu hoạt động cần phải tham chiếu đến một toán hạng trong bộ nhớ hay qua thiết bị ngoại vi thì xác định địa chỉ của toán hạng.
- **Truy xuất toán hạng (of):** Truy xuất toán hạng từ bộ nhớ hoặc đọc từ thiết bị ngoại vi.
- **Tính toán dữ liệu (do):** Thực hiện phép toán được chỉ ra trong lệnh.
- **Lưu trữ toán hạng (os):** Ghi kết quả vào bộ nhớ hoặc đưa ra thiết bị ngoại vi.

Các trạng thái trong phần trên của hình 1.22 là các trạng thái có sự truyền dữ liệu giữa bộ xử lý và bộ nhớ hoặc module vào/ra. Các trạng thái ở dưới của Hình chỉ được thực hiện bên trong bộ xử lý. Mỗi lệnh có thể có nhiều toán hạng hoặc nhiều kết quả tùy thuộc vào các hệ máy tính khác nhau.

1.6.2.2. Ngắt

Hầu như các máy tính đều có cơ chế cho phép các module khác (bộ nhớ, vào/ra) có thể ngắt tiến trình thông thường của bộ xử lý. Bảng 1.14 liệt kê một số loại ngắt phổ biến nhất.

Bảng 1.14. Các loại ngắt

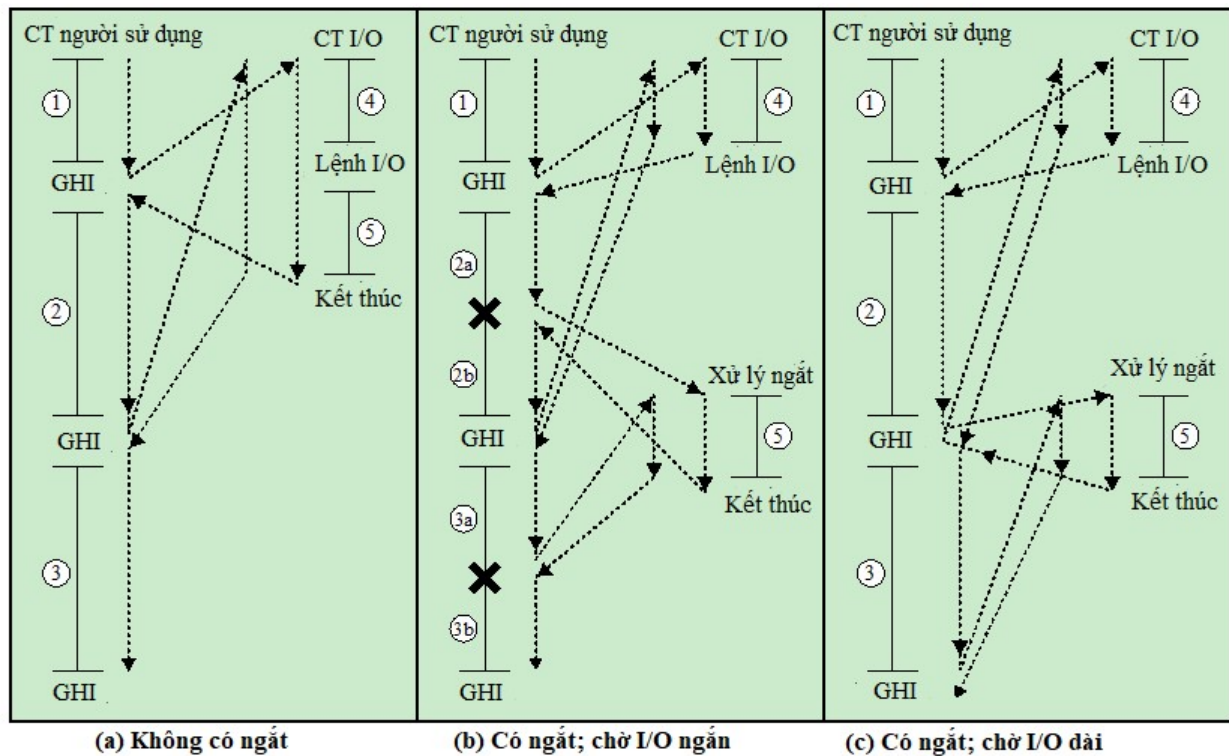
Chương trình	Được tạo ra trong quá trình thực hiện lệnh, chẳng hạn như: tràn số học, chia cho không, cố gắng thực thi một lệnh không được phép hoặc tham chiếu đến bên ngoài không gian bộ nhớ của người dùng.
Định thời	Được tạo ra bởi bộ định thời trong bộ xử lý. Chức năng này cho phép hệ điều hành thực hiện định kỳ một hoạt động nào đó.
Vào/ra	Được tạo ra bởi bộ điều khiển vào/ra để báo hiệu khi một hoạt động vào/ra đã hoàn thành, yêu cầu sự phục vụ của bộ xử lý hoặc để báo lỗi
Lỗi phần cứng	Được tạo ra bởi một lỗi như lỗi nguồn điện hoặc lỗi chẵn lẻ bộ nhớ

Ngắt được đưa ra với mục đích cải thiện hiệu suất của bộ xử lý. Ví dụ, hầu như các thiết bị ngoại đều chậm hơn bộ xử lý. Giả sử bộ xử lý truyền dữ liệu đến một máy in sử dụng chu kỳ lệnh như trong hình 1.19. Sau mỗi lệnh ghi, bộ xử lý phải dừng lại và chờ đợi máy in làm việc. Khoảng thời gian này có thể gấp hàng trăm hoặc hàng nghìn lần một chu kỳ lệnh. Rõ ràng, điều này sẽ làm lãng phí hiệu suất của bộ xử lý.

Hình 1.23a minh họa quá trình này. Một chương trình người sử dụng (user program) thực hiện một loạt lời gọi WRITE. Các đoạn mã 1, 2, 3 là các chuỗi câu lệnh không có giao tiếp với thiết bị ngoại vi. Lời gọi WRITE này gọi một chương trình vào/ra để thực hiện một hoạt động vào/ra. Hoạt động vào/ra này gồm ba phần như sau:

- Một chuỗi lệnh có nhãn 4 trong hình, chuẩn bị cho hoạt động vào/ra. Nó có thể bao gồm việc sao chép dữ liệu vào một bộ đệm và chuẩn bị các tham số cho các lệnh của thiết bị.
- Lệnh vào/ra. Nếu không có ngắt, khi lệnh này được thực hiện, chương trình phải đợi thiết bị ngoại vi thực hiện chức năng được yêu cầu hoặc thực hiện lặp lại hoạt động kiểm tra để xác định xem liệu hoạt động vào/ra đã hoàn thành chưa.
- Một chuỗi lệnh nhãn 5 hoàn thành nốt hoạt động. Chuỗi lệnh này có thể bao gồm việc thiết lập một cờ để báo cho bộ xử lý biết là hoạt động đã hoàn thành hoặc lỗi.

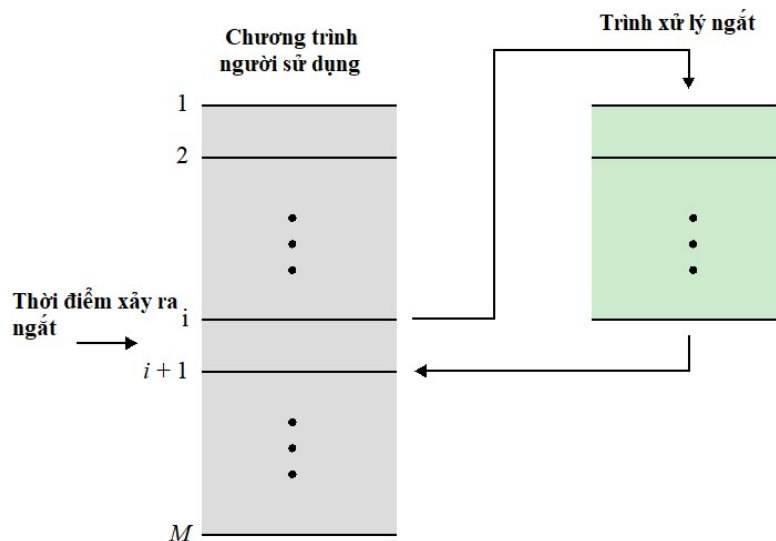
Vì hoạt động vào/ra mất một khoảng thời gian khá lớn để hoàn thành, chương trình vào/ra phải treo để đợi hoạt động hoàn tất, do đó chương trình người sử dụng phải dừng lại ở lời gọi WRITE một khoảng thời gian nhất định.



Hình 1.23. Tiến trình của chương trình không có ngắt và có ngắt

a. Ngắt và chu kỳ lệnh

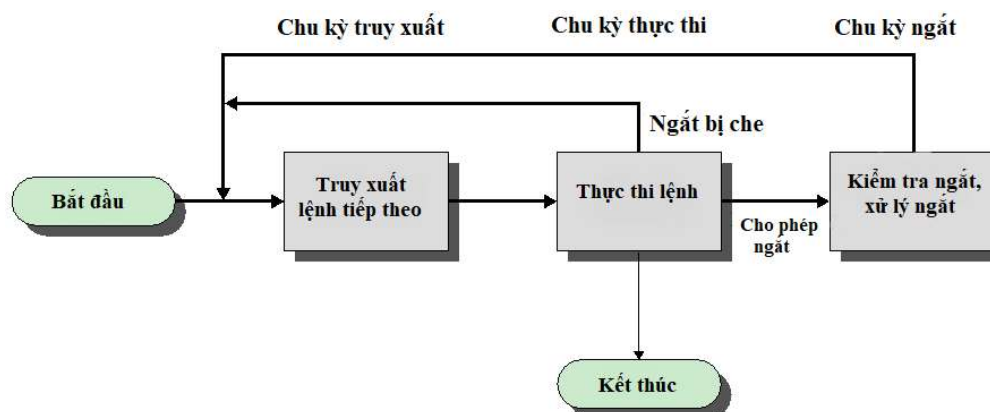
Với ngắt, bộ xử lý có thể thực hiện các lệnh khác trong khi một tiến trình vào/ra đang thực hiện. Xét luồng điều khiển trong Hình 1.23b. Khác với phần a, chương trình người sử dụng thực hiện lời gọi WRITE, lời gọi này gọi đến chương trình vào/ra thực hiện các mã chuẩn bị và lệnh vào/ra thực tế (nhãn 4). Sau khi nhãn này thực hiện xong, luồng điều khiển trở về chương trình người sử dụng. Thiết bị ngoại vi nhận dữ liệu từ bộ nhớ máy tính và thực hiện công việc in, trong khi đó, bộ xử lý tiếp tục thực hiện nhãn 2 của chương trình người sử dụng.



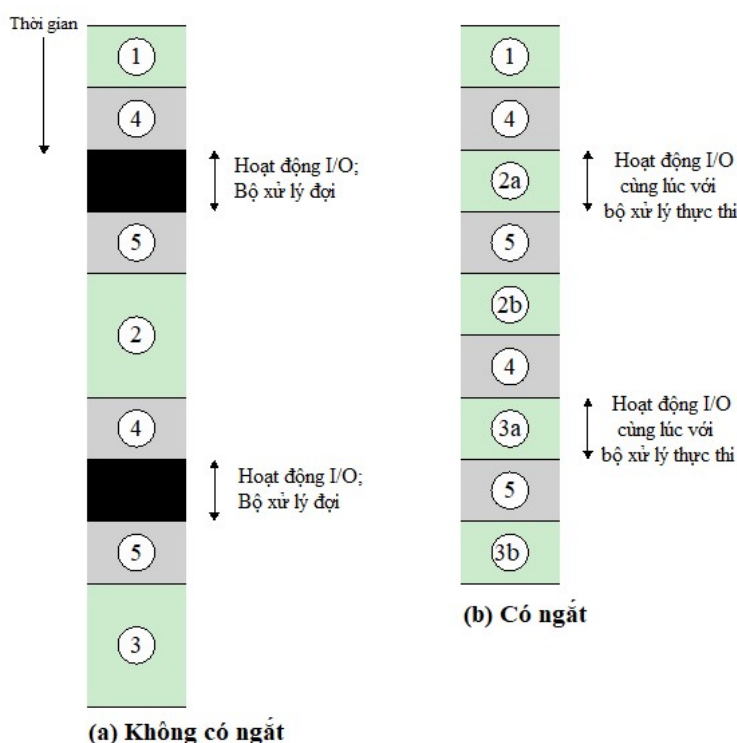
Hình 1.24. Luồng điều khiển trong trường hợp có ngắt

Sau khi máy in in xong và giải phóng bộ nhớ đệm, nghĩa là nó sẵn sàng nhận thêm dữ liệu từ bộ xử lý, module vào/ra sẽ gửi một tín hiệu ngắt đến bộ xử lý. Bộ xử lý đáp ứng bằng cách ngưng hoạt động của chương trình hiện tại tại (nhãn 2), rẽ nhánh sang một chương trình phục vụ thiết bị vào/ra tương ứng (được gọi là **trình xử lý ngắt**) và tiếp tục điều khiển quá trình thực hiện Chương trình vào/ra. Các thời điểm bộ xử lý bị ngắt được ký hiệu bằng dấu sao trong Hình 1.23b.

Đối với chương trình người dùng, khi có yêu cầu ngắt được gửi đến trong lúc chương trình đang được thực thi, chương trình sẽ bị dừng lại (ngắt). Khi việc xử lý ngắt xong, chương trình sẽ được khôi phục lại và tiếp tục được thực thi (Hình 1.24). Do vậy, chương trình người sử dụng không chứa bất cứ đoạn mã đặc biệt nào chứa ngắt, bộ xử lý và hệ điều hành sẽ treo chương trình người sử dụng và khôi phục lại nó tại cùng thời điểm.



Hình 1.25. Chu kỳ lệnh có ngắt



Hình 1.26. Tiến trình thời gian của chương trình: thời gian đợi hoạt động Vào/ra ngắn

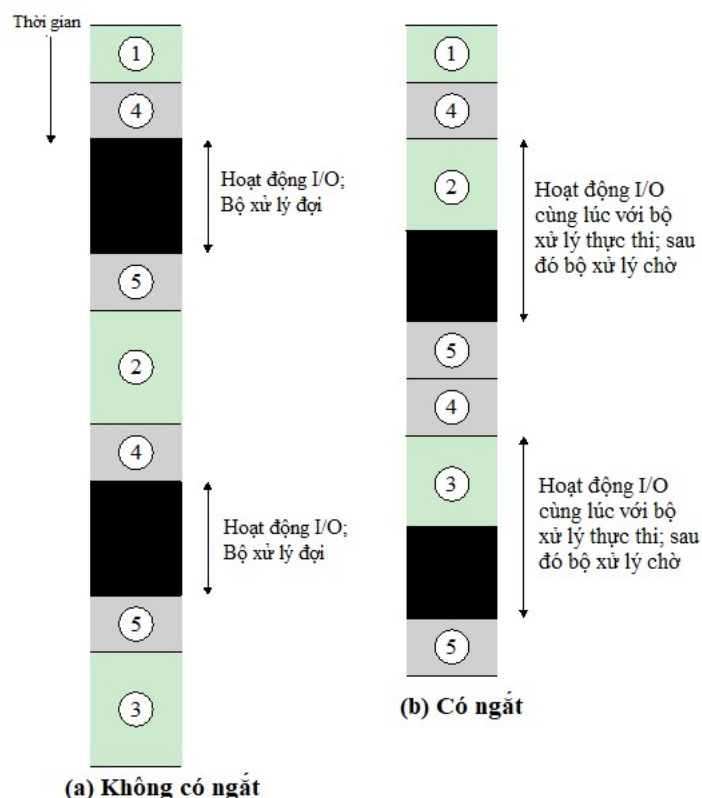
Để thực hiện ngắt, chu kỳ ngắt được thêm vào chu kỳ lệnh như trong hình 1.25. Trong chu kỳ ngắt, bộ xử lý kiểm tra xem liệu có yêu cầu ngắt nào được gửi tới hay không (bằng cách kiểm tra tín hiệu báo ngắt). Nếu không có ngắt nào đang chờ, bộ xử lý thực thi chu kỳ truy xuất và truy

xuất đến câu lệnh tiếp theo của chương trình hiện tại. Nếu có ngắt đang đợi, bộ xử lý thực hiện các công việc sau:

- Treo chương trình hiện tại đang thực thi và lưu nội dung của nó bằng cách lưu địa chỉ của lệnh tiếp theo sẽ được thực thi (nội dung thanh ghi PC) và tất cả các dữ liệu liên quan khác.
- Thiết lập thanh ghi PC về địa chỉ đầu tiên của *trình xử lý ngắt*.

Bộ xử lý thực hiện chu kỳ truy xuất và truy xuất đến lệnh đầu tiên của chương trình xử lý ngắt để phục vụ ngắt. Chương trình xử lý ngắt thường là một chương trình có trong hệ điều hành. Chương trình này sẽ xác định loại ngắt và thực hiện tất cả các hành động cần thiết. Như trong ví dụ trên, trình xử lý ngắt xác định module vào/ra nào đã phát ra ngắt và rẽ nhánh đến chương trình ghi dữ liệu ra module vào/ra. Khi trình xử lý ngắt hoàn thành, bộ xử lý sẽ khôi phục lại chương trình người sử dụng tại thời điểm bị ngắt.

Để hiểu rõ hơn về hiệu quả của việc xử lý ngắt, xét sơ đồ thời gian trong hình 1.26 dựa trên luồng điều khiển trong hình 1.23a và 1.23b. Các đoạn mã của chương trình người sử dụng được tô màu xanh, các đoạn mã của chương trình vào/ra được tô màu ghi. Hình 1.26a là trường hợp khi không sử dụng ngắt. Bộ xử lý sẽ phải đợi đến khi hoạt động vào/ra thực hiện xong.

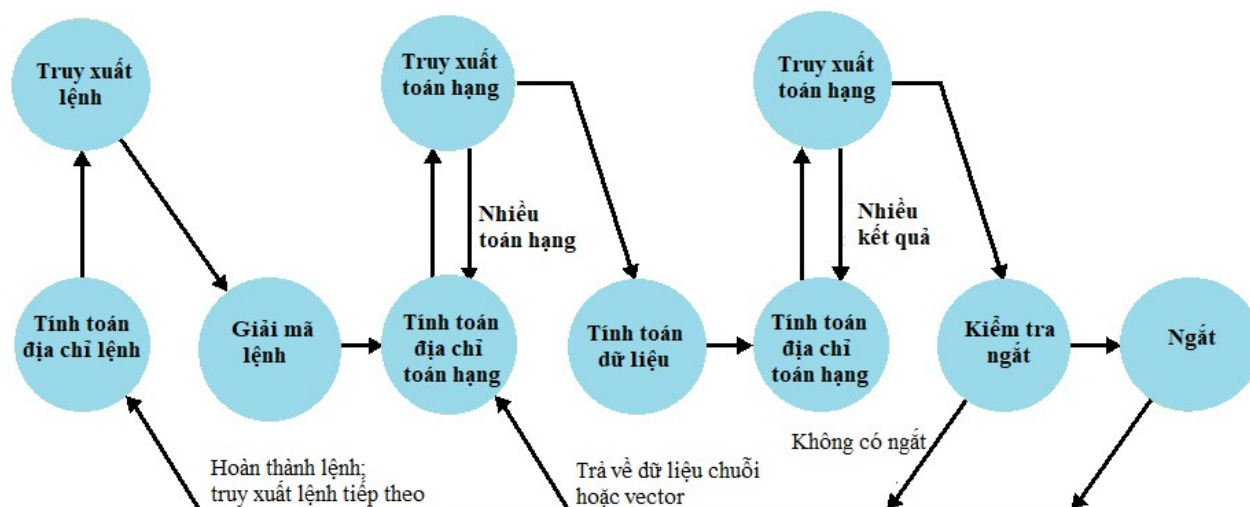


Hình 1.27. Tiến trình thời gian của chương trình: trường hợp thời gian đợi hoạt động vào/ra dài

Hình 1.23b và 1.26b minh họa trường hợp giả thiết rằng khoảng thời gian thực hiện hoạt động vào/ra khá ngắn, ngắn hơn thời gian bộ xử lý thực hiện nhân 2. Do đó, nhân 2 sẽ bị ngắt để phục vụ thiết bị ngoại vi và khôi phục lại sau khi xử lý ngắt xong.

Một trường hợp thường gặp khác (Hình 1.23c) là khi thiết bị ngoại vi chậm hơn nhiều so với bộ xử lý, ví dụ như máy in (printer), hoạt động vào/ra mất nhiều thời gian hơn so với việc bộ xử lý thực hiện các lệnh nhân 2, chương trình người sử dụng đi đến lời gọi GHI (WRITE) trước khi hoạt động vào/ra đầu tiên thực hiện xong. Chương trình người sử dụng tạm thời dừng lại và

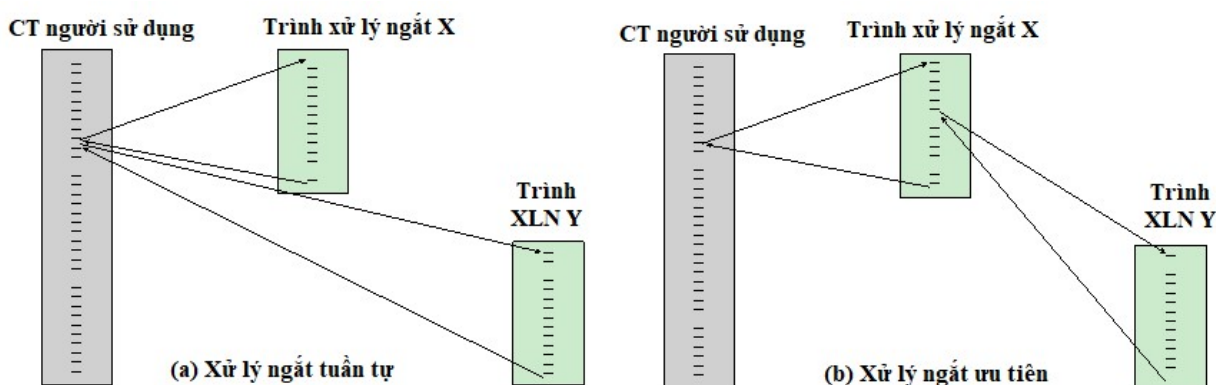
đợi đến khi chương trình vào/ra hoàn thành thì lời gọi hàm GHI mới tiếp tục được xử lý. Hình 1.27 là sơ đồ thời gian của trường hợp này và mặc dù không cần sử dụng ngắt nhưng cơ chế xử lý này vẫn cải thiện hiệu suất của hệ thống. Hình 1.28 là sơ đồ trạng thái chu kỳ lệnh bao gồm cả việc xử lý ngắt (gồm có chu kỳ ngắt).



Hình 1.28. Sơ đồ trạng thái chu kỳ lệnh có ngắt

b. Xử lý nhiều ngắt (đa ngắt)

Các trường hợp ở phần trên chỉ có một ngắt duy nhất tại một thời điểm. Vậy, giả thiết nếu nhiều ngắt được yêu cầu cùng một lúc. Ví dụ như một máy tính có thể đang nhận dữ liệu từ một đường truyền thông tin và nhận kết quả từ máy in. Máy in sẽ đưa ra một ngắt mỗi lần hoàn tất thao tác in. Bộ điều khiển đường truyền cũng gửi yêu cầu ngắt mỗi lần một đơn vị dữ liệu được gửi đến. Giả sử, yêu cầu ngắt này xuất hiện ngay tại thời điểm bộ xử lý đang xử lý ngắt máy in.



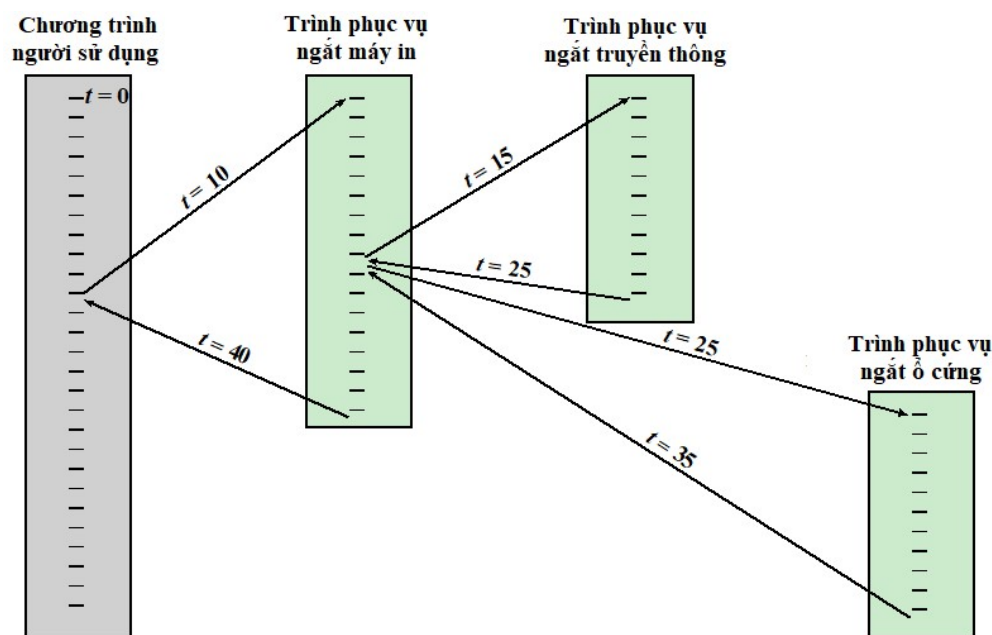
Hình 1.29. Truyền điều khiển trong trường hợp đa ngắt

Có hai giải pháp cho vấn đề này. Thứ nhất, ta vô hiệu hóa tất cả các ngắt được gửi đến trong khi bộ xử lý đang xử lý một ngắt bất kỳ. Như vậy, nếu có yêu cầu ngắt trong khoảng thời gian này, nó sẽ được lưu lại, đến khi nào trình xử lý ngắt hoàn thành, bộ xử lý sẽ kiểm tra xem có ngắt nào đang chờ hay không trước khi quay lại chương trình người sử dụng. (Hình 1.29a).

Nhược điểm của phương pháp trên là nó không giải quyết được vấn đề ưu tiên hoặc yêu cầu thời gian của ngắt. Ví dụ, dữ liệu được gửi đến từ đường truyền thông, nó cần phải được nhận một cách nhanh chóng để giải phóng bộ nhớ đệm cho dữ liệu khác đến. Nếu đợt dữ liệu đầu tiên đi vào chưa được xử lý trước khi đợt thứ hai đến, dữ liệu có thể bị mất.

Phương pháp thứ hai là xác định ưu tiên cho ngắt. Phương pháp này cho phép ngắt có mức ưu tiên cao hơn có thể ngắt quá trình xử lý ngắt có mức ưu tiên thấp hơn (Hình 1.29b). Ví dụ, xét một hệ thống với ba thiết bị vào/ra: máy in, ổ đĩa và một đường truyền thông với các mức ưu tiên lần lượt là 2, 4 và 5.

Hình 1.30 minh họa trường hợp một chương trình người dùng bắt đầu ở thời điểm $t = 0$. Tại $t = 10$, một ngắt máy in xảy ra; thông tin chương trình người sử dụng được đặt trên bộ nhớ ngăn xếp hệ thống và bộ xử lý thực thi **trình phục vụ ngắt (ISR)** máy in. Mặc dù thủ tục này vẫn đang được thực thi, tại $t = 15$, một ngắt đường truyền thông xảy ra. Do đường truyền có mức ưu tiên cao hơn so với máy in, ngắt này được thực hiện. ISR máy in bị ngắt, trạng thái của nó được đẩy lên bộ nhớ ngăn xếp, bộ xử lý thực thi ISR cho đường truyền thông. Trong khi ISR này được thực hiện, một yêu cầu ngắt được gửi đến từ ổ đĩa ($t = 20$). Bởi vì ngắt này có mức ưu tiên thấp hơn, nó bị giữ lại để đợi ISR của đường truyền thông tin kết thúc.



Hình 1.30. Ngắt trong hệ thống hệ thống với ba thiết bị vào/ra

1.6.2.3. Chức năng vào/ra

Chúng ta vừa xem xét hoạt động của máy tính với sự điều khiển của bộ xử lý cũng như sự giao tiếp giữa bộ xử lý và bộ nhớ. Một phần không kém quan trọng khác là sự tương tác giữa bộ xử lý và các thành phần vào/ra. Vấn đề này sẽ được giải thích rõ hơn trong chương 5, tuy nhiên phần này sẽ tóm tắt ngắn gọn một số điểm như sau:

Một module vào/ra (ví dụ: bộ điều khiển ổ đĩa) có thể trao đổi dữ liệu trực tiếp với bộ vi xử lý. Tương tự như cách bộ vi xử lý đọc hoặc ghi dữ liệu với bộ nhớ, nó cũng có thể đọc hoặc ghi dữ liệu với module vào/ra thông qua việc định một địa chỉ cụ thể và trình tự các lệnh thực hiện việc trao đổi dữ liệu này giống như trong hình 1.21 với các lệnh vào/ra thay vì các lệnh tham chiếu bộ nhớ.

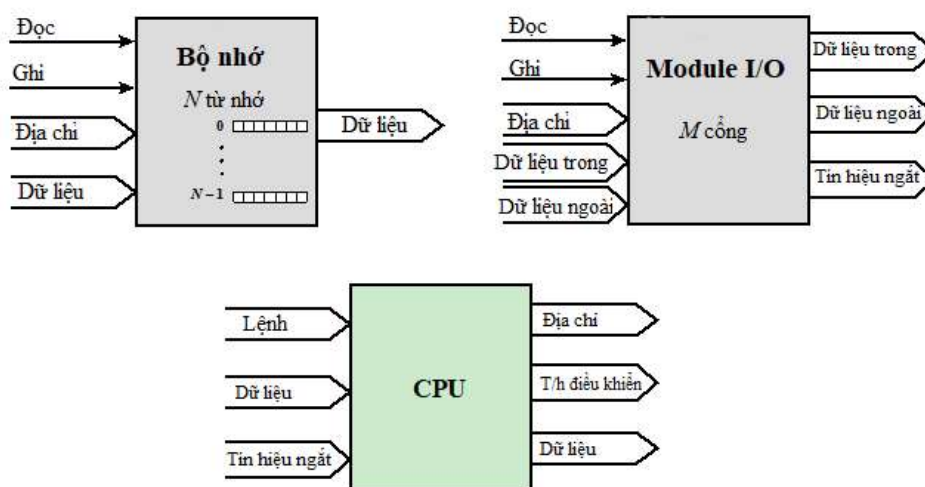
Ngoài ra, các hệ thống máy tính có cơ chế cho phép trao đổi dữ liệu từ module vào/ra trực tiếp với bộ nhớ. Bộ xử lý sẽ cấp cho một module vào/ra quyền đọc hoặc ghi dữ liệu vào bộ nhớ, từ đó việc truyền giữa module vào/ra - memory được thực hiện mà không cần sự tham gia của bộ xử lý - giảm trách nhiệm của bộ xử lý. Cơ chế này được gọi là cơ chế truy cập bộ nhớ trực tiếp (DMA) sẽ được trình bày cụ thể trong chương 5.

1.6.3. Cấu trúc kết nối

Một máy tính bao gồm một tập các thành phần hoặc module theo ba loại cơ bản (bộ xử lý, bộ nhớ, vào/ra) trao đổi thông tin với nhau. Thực chất, một máy tính giống như một mạng lưới các module cơ bản. Do đó, ta cần một bộ phận để kết nối các module với nhau.

Tập hợp các bộ phận kết nối này được gọi là cấu trúc kết nối. Việc thiết kế cấu trúc này phụ thuộc vào sự trao đổi dữ liệu giữa các module. Hình 1.31 minh họa các loại thông tin đầu vào và đầu ra của mỗi loại module:

- **Bộ nhớ:** module bộ nhớ bao gồm N từ nhớ có kích thước bằng nhau. Mỗi từ nhớ được gán cho một địa chỉ dạng số $(0, 1, 2, \dots, N - 1)$. Tín hiệu điều khiển *đọc/ghi* sẽ quyết định *dữ liệu* (từ nhớ) được đọc hoặc ghi vào bộ nhớ. Ví trí của hoạt động (đọc/ghi) được định ra thông qua tín hiệu *địa chỉ*.
- **Module vào/ra:** Nhìn từ bên trong, module vào/ra có chức năng tương tự như bộ nhớ: gồm có hai hoạt động là đọc và ghi. Thêm vào đó, một module vào/ra có thể điều khiển nhiều thiết bị ngoại vi, mỗi thiết bị được giao tiếp với máy tính thông qua một *cổng*, cổng này sẽ được đánh một *địa chỉ duy nhất* (ví dụ: $0, 1, \dots, M - 1$). Các đường dẫn dữ liệu ngoài để giao tiếp dữ liệu với thiết bị ngoại vi này. Ngoài ra, module vào/ra có thể gửi các *tín hiệu ngắt* tới bộ xử lý.
- **Bộ xử lý:** Bộ xử lý đọc *lệnh* và *dữ liệu*, ghi dữ liệu ra sau khi xử lý và sử dụng các *tín hiệu điều khiển* để điều khiển hoạt động chung của hệ thống. Nó cũng nhận *tín hiệu ngắt* từ module vào/ra.
- Từ các loại dữ liệu cần trao đổi giữa các module ở trên, cấu trúc kết nối được thiết kế phải đảm bảo hỗ trợ các loại truyền dữ liệu sau đây:
- **Bộ nhớ đến bộ vi xử lý:** Bộ xử lý đọc lệnh hoặc một đơn vị dữ liệu từ bộ nhớ.
- **Bộ xử lý đến bộ nhớ:** Bộ xử lý ghi một đơn vị dữ liệu vào bộ nhớ
- **Vào/ra đến bộ vi xử lý:** Bộ xử lý đọc dữ liệu từ một thiết bị ngoại vi thông qua một module vào/ra.
- **Bộ xử lý đến Vào/ra:** Bộ vi xử lý gửi dữ liệu đến thiết bị ngoại vi.
- **Vào/ra đến hoặc đi từ bộ nhớ:** Đối với trường hợp này, một module vào/ra được phép trao đổi dữ liệu trực tiếp với bộ nhớ, không qua bộ vi xử lý, sử dụng cơ chế truy cập bộ nhớ trực tiếp – DMA.



Hình 1.31. Các module máy tính

Nhiều năm qua, có một số cấu trúc kết nối được sử dụng. Phổ biến nhất là cấu trúc bus, cấu trúc đa bus và cấu trúc kết nối điểm-điểm kết hợp truyền dữ liệu dạng gói.

1.6.4. Kết nối bus

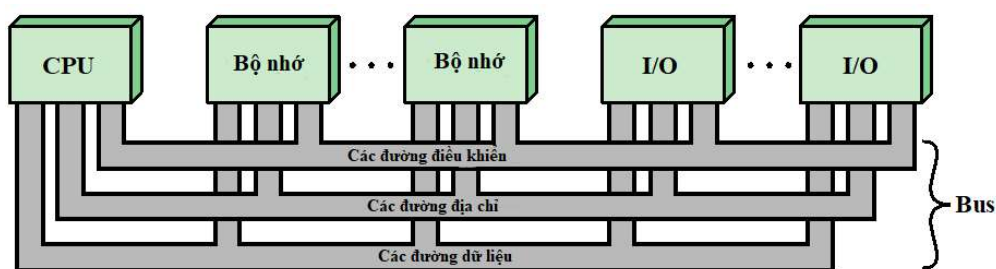
Bus là một đường kết nối giữa hai hoặc nhiều thiết bị. Đặc điểm chính của bus đó là một **phương tiện truyền dẫn chia sẻ**, khi một thiết bị gửi tín hiệu trên bus, tất cả các thiết bị khác gắn vào bus đều nhận được. Nếu hai thiết bị cùng truyền dữ liệu cùng một thời điểm, các tín hiệu sẽ bị chồng chéo lên nhau và bị méo. Do đó, tại một thời điểm chỉ một thiết bị có thể truyền dữ liệu thành công.

Thông thường, bus gồm nhiều đường truyền. Mỗi đường có khả năng truyền các tín hiệu được biểu diễn dưới dạng số nhị phân 1 và 0. Các tín hiệu có thể được truyền nhiều đợt trên một dòng (truyền nối tiếp). Kết hợp nhiều đường trong bus ta có thể truyền nhiều số nhị phân tại cùng một thời điểm (truyền song song). Ví dụ, một khối 8 bit ta có thể truyền đồng thời trong 8 đường bus

Nhìn chung, một hệ thống máy tính sẽ có nhiều bus khác nhau cung cấp các kết nối giữa các thành phần ở cấp độ khác nhau của hệ thống máy tính. Bus kết nối các thành phần chính của máy tính (gồm có: bộ xử lý, bộ nhớ, các module vào/ra) được gọi là bus hệ thống. Các cấu trúc kết nối phổ biến nhất thường sử dụng một hoặc nhiều bus hệ thống.

1.6.4.1. Cấu trúc bus

Thông thường bus hệ thống bao gồm từ năm mươi cho đến hàng trăm đường riêng biệt. Mỗi đường sẽ đảm nhiệm một ý nghĩa hoặc chức năng cụ thể. Mặc dù có rất nhiều thiết kế bus khác nhau, chức năng của các đường bus thường được chia thành ba nhóm như sau: dữ liệu, địa chỉ, điều khiển. Ngoài ra, bus có thể có thêm một số đường cung cấp nguồn cho các module được nối với nó.



Hình 1.32. Sơ đồ kết nối bus

Các **đường dữ liệu** cho phép truyền tải dữ liệu giữa các module của hệ thống. Các đường này tập hợp lại thành **bus dữ liệu**. Bus dữ liệu có thể gồm: 32, 64, 128 đường hoặc nhiều hơn, số đường này được hiểu là độ rộng của bus dữ liệu. Vì mỗi đường chỉ có thể truyền 1 bit tại một thời điểm, số lượng đường sẽ quyết định số lượng bit có thể truyền tại một thời điểm. Vậy độ rộng của bus dữ liệu là một yếu tố quan trọng để đánh giá hiệu suất toàn hệ thống. Ví dụ, nếu bus hệ thống có độ rộng 32 bit, chiều dài của lệnh là 64 bit, vậy bộ xử lý cần truy xuất bộ nhớ hai lần trong mỗi chu kỳ lệnh.

Các **đường địa chỉ** được dùng để xác định nguồn và đích của dữ liệu trong bus dữ liệu. Ví dụ, nếu bộ xử lý cần đọc một từ (8, 16, 32 bit) từ bộ nhớ, nó sẽ đặt địa chỉ của từ đó lên đường địa chỉ. Rõ ràng, độ rộng của **bus địa chỉ** quyết định dung lượng tối đa của bộ nhớ. Thêm vào đó, các đường địa chỉ cũng thường được dùng cho địa chỉ các cổng vào/ra. Thông thường, các bit trọng số cao được dùng để xác định một module, các bit trọng số thấp xác định một vị trí trong bộ nhớ hoặc một cổng vào/ra. Ví dụ, bus địa chỉ 8 bit có các giá trị nhỏ hơn 01111111 sẽ tham chiếu đến 128 vị trí trong module bộ nhớ (module 0), các địa chỉ từ 10000000 trở lên tham chiếu đến các thiết bị được nối với một module vào/ra (module 1).

Các **đường điều khiển** được sử dụng để điều khiển việc truy cập vào và việc sử dụng các đường dữ liệu và địa chỉ. Vì các đường dữ liệu và địa chỉ được chia sẻ với tất cả các thành phần của hệ thống nên cần phải có một phương tiện để kiểm soát việc sử dụng của chúng. Tín hiệu điều khiển truyền cả lệnh và các thông tin định thời giữa các module hệ thống. Tín hiệu định thời chỉ định tính hợp lệ của dữ liệu và địa chỉ. Các tín hiệu lệnh cho biết các thao tác cần thực hiện. Các đường điều khiển bao gồm:

- **Ghi bộ nhớ:** ghi dữ liệu trên bus vào vị trí bộ nhớ có địa chỉ tương ứng trong bus địa chỉ
- **Đọc bộ nhớ:** lấy dữ liệu từ vị trí được định địa chỉ và đặt lên bus
- **Ghi vào/ra:** khiến cho dữ liệu trên bus được đưa ra đầu ra của cổng vào/ra được định địa chỉ
- **Đọc vào/ra:** khiến cho dữ liệu từ cổng vào/ra được định địa chỉ được đặt vào bus
- **Truyền ACK:** chỉ thị rằng dữ liệu đã được nhận vào hoặc được đặt lên bus
- **Yêu cầu bus:** chỉ ra rằng một module đang cần chiếm quyền điều khiển bus
- **Cấp quyền bus:** chỉ ra rằng module yêu cầu đã được cấp quyền điều khiển bus
- **Yêu cầu ngắt:** chỉ ra rằng có một ngắt đang chờ
- **ACK ngắt:** xác nhận yêu cầu ngắt đã được công nhận
- **Đồng hồ:** được sử dụng để đồng bộ hoá các hoạt động
- **Reset:** khởi tạo tất cả các module.

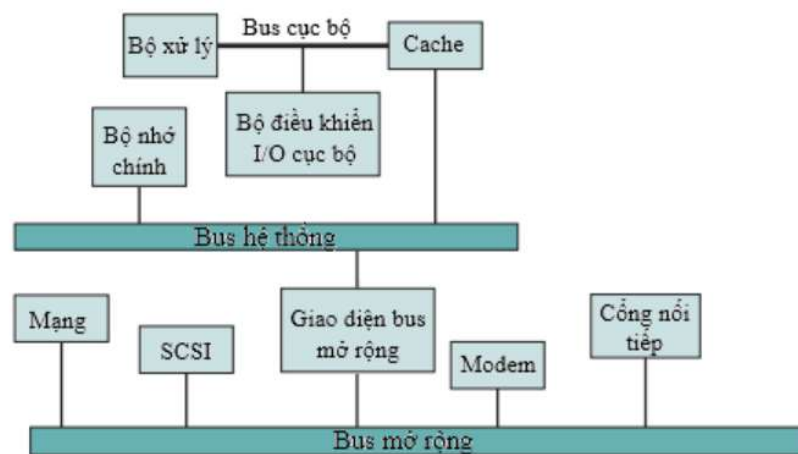
Hoạt động của bus như sau: nếu một module muốn truyền dữ liệu đến một điểm khác, nó cần phải làm hai việc: (1) yêu cầu được sử dụng bus và (2) truyền dữ liệu qua bus. Nếu một module muốn yêu cầu dữ liệu từ một module khác, nó phải (1) yêu cầu được sử dụng bus và (2) truyền yêu cầu đến module khác qua các đường điều khiển và địa chỉ thích hợp. Sau đó nó phải đợi module thứ hai gửi dữ liệu trở lại.

1.6.4.2. Mô hình phân cấp đa bus

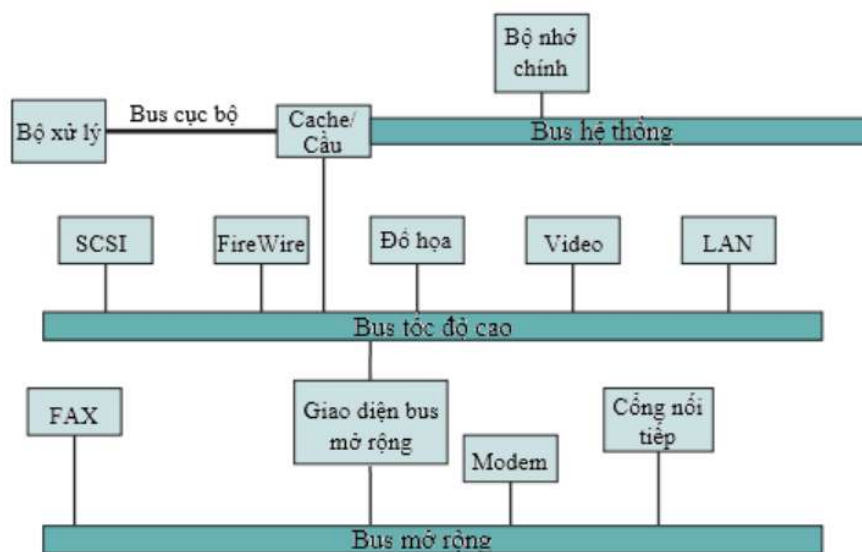
Nếu có một số lượng lớn các thiết bị được nối vào bus, hiệu suất sẽ giảm vì hai nguyên nhân chính sau:

1. Rõ ràng, càng nhiều thiết bị nối vào bus thì chiều dài bus càng tăng gây ra trễ truyền càng lớn. Trễ này quyết định khoảng thời gian cần thiết để các thiết bị phối hợp sử dụng bus. Khi quyền điều khiển bus thường xuyên được chuyển từ một thiết bị này đến thiết bị khác, trễ truyền này sẽ gây ảnh hưởng đáng kể đến hiệu suất.
2. Bus có thể bị hiện tượng nút cổ chai khi nhu cầu truyền dữ liệu vượt quá khả năng của bus. Vấn đề này có thể giải quyết một mức độ nào đó bằng cách tăng tốc độ truyền dữ liệu của nó bằng cách sử dụng bus có độ rộng lớn hơn (ví dụ: tăng bus dữ liệu từ 32 bit lên 64 bit). Tuy nhiên, các thiết bị được kết nối với bus có yêu cầu truyền dữ liệu với tốc độ ngày càng cao (ví dụ: các vi điều khiển video và đồ họa, các giao diện mạng) hệ thống bus ngày càng ít có khả năng đáp ứng kịp.

Theo đó, hầu hết các hệ thống máy tính có cấu trúc kết nối bus thường sử dụng nhiều bus khác nhau (đa bus) dưới dạng hệ thống phân cấp. Hình 1.33a biểu diễn một cấu trúc truyền thống điển hình. Trong đó, một bus cục bộ kết nối bộ vi xử lý với bộ nhớ cache và có thể hỗ trợ một hoặc nhiều thiết bị cục bộ. Bộ điều khiển cache kết nối bộ nhớ cache với cả bus cục bộ và bus hệ thống. Trong các hệ thống hiện đại, bộ nhớ cache thường được đặt trên cùng chip bộ xử lý nên bus bên ngoài không cần thiết. Như sẽ trình bày trong chương 3, việc sử dụng cấu trúc cache sẽ làm giảm số lần bộ vi xử lý truy cập vào bộ nhớ chính. Do đó, bộ nhớ chính không cần phải nối với bus cục bộ mà chỉ cần nối với bus hệ thống. Bằng cách này, một module vào/ra giao tiếp dữ liệu với bộ nhớ chính qua bus hệ thống sẽ không làm ảnh hưởng tới hoạt động của bộ xử lý.



(a) Kiến trúc bus truyền thống



(b) Kiến trúc bus hiệu năng cao

Hình 1.33. Cấu hình bus

Các bộ điều khiển vào/ra hoàn toàn có thể được nối trực tiếp vào bus hệ thống. Ngoài ra, một giải pháp hiệu quả khác là sử dụng một hoặc nhiều bus mở rộng. Giao diện bus mở rộng đệm dữ liệu truyền giữa bus hệ thống và bộ điều khiển vào/ra trên bus mở rộng. Bằng cách đó cho phép hệ thống hỗ trợ nhiều thiết bị vào/ra và đồng thời bảo vệ lưu lượng bộ nhớ-bộ xử lý không bị ảnh hưởng bởi lưu lượng vào/ra.

1.6.5. Kết nối điểm-điểm

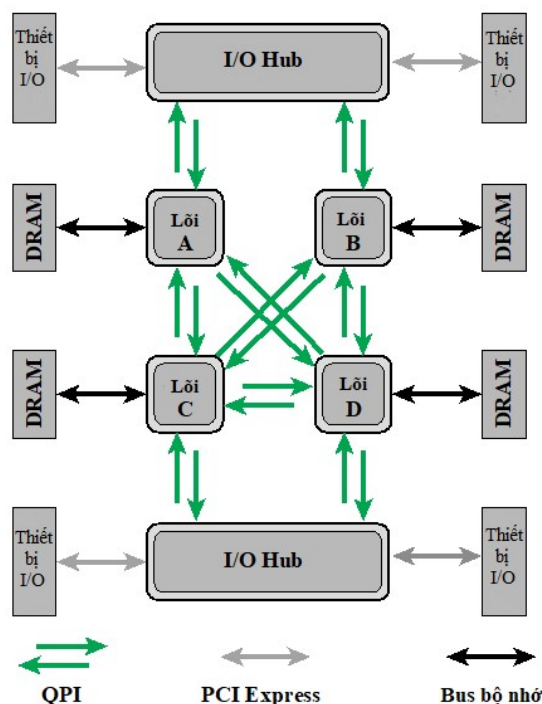
Kiến trúc bus chia sẻ là kiến trúc cơ bản được sử dụng để kết nối giữa bộ vi xử lý và các thành phần khác (bộ nhớ, vào/ra, v.v...) trong nhiều thập kỷ. Nhưng với các hệ thống hiện đại ngày nay, kiến trúc kết nối điểm-điểm có xu hướng được sử dụng ngày càng tăng hơn so với kết nối bus.

Lý do cơ bản của sự thay đổi này là do tần số tín hiệu đồng hồ của bus đồng bộ ngày càng cao thì việc cấp nguồn trở nên khó khăn. Với tốc độ dữ liệu ngày càng cao, việc thực hiện các chức năng đồng bộ và phân xử cũng trở nên khó khăn. Thêm vào đó, sự ra đời của công nghệ chip đa nhân với nhiều bộ vi xử lý và bộ nhớ tích hợp trên một con chip duy nhất, người ta thấy rằng việc sử dụng một cấu trúc bus chia sẻ thông thường làm tăng thêm khó khăn trong việc tăng tốc độ dữ liệu và giảm độ trễ của bus để theo kịp với tốc độ vi xử lý. So với kết nối bus, kết nối điểm-điểm có độ trễ thấp hơn, tốc độ dữ liệu cao hơn, khả năng mở rộng tốt hơn.

Trong phần này, chúng ta sẽ xét một ví dụ quan trọng của hệ thống kết nối điểm-điểm: **QuickPath Interconnect (QPI)** của Intel, được giới thiệu vào năm 2008. Sau đây là một số đặc điểm quan trọng của QPI và các cấu trúc kết nối điểm-điểm khác:

- **Nhiều kết nối trực tiếp:** Các thành phần được kết nối trực tiếp với nhau. Điều này giúp loại bỏ cơ chế phân xử như trong mô hình kết nối bus.
- **Kiến trúc giao thức lớp:** Giống như mô hình phân lớp TCP/IP của mạng máy tính, kết nối QPI sử dụng kiến trúc giao thức lớp chứ không phải là sử dụng các tín hiệu điều khiển như trong mô hình kết nối bus.
- **Truyền dữ liệu dạng gói tin:** Dữ liệu không được gửi dưới dạng chuỗi bit thô mà được gửi dưới dạng các gói tin, mỗi gói tin bao gồm một phần tiêu đề điều khiển và một mã kiểm soát lỗi.

Hình 1.34 là một ví dụ điển hình về kết nối QPI trên một máy tính đa nhân. Các liên kết QPI (chỉ thị bằng các mũi tên màu xanh lá cây) tạo thành một mạng lưới chuyển mạch cho phép dữ liệu di chuyển qua mạng. Kết nối QPI trực tiếp có thể được thiết lập giữa hai nhân. Nếu nhân A trong hình 1.34 cần truy cập đến bộ điều khiển bộ nhớ trong nhân D, nó sẽ phải gửi yêu cầu qua các lõi B hoặc C.



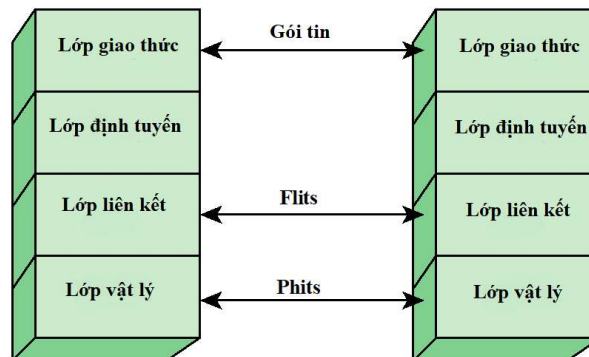
Hình 1.34. Cấu hình đa lõi sử dụng QPI

Ngoài ra, QPI có thể kết nối với module vào/ra thông qua một I/O Hub (IOH). Thông thường trong các hệ thống hiện đại, liên kết giữa IOH với bộ điều khiển thiết bị vào/ra sử dụng một công nghệ kết nối được gọi là PCI Express (PCIe) sẽ được đề cập đến phần sau của chương này. IOH có nhiệm vụ chuyển đổi các giao thức, định dạng QPI sang các giao thức, định dạng PCIe và ngược lại. Mỗi nhân cũng kết nối với một module bộ nhớ (các bộ nhớ này thường sử dụng công nghệ bộ nhớ DRAM (bộ nhớ truy cập ngẫu nhiên) thông qua một bus chuyên dụng.

Kiến trúc giao thức QPI (Hình 1.35) gồm bốn lớp, bao gồm:

- **Lớp vật lý:** bao gồm các đường dây dẫn, các mạch điện và các logic để truyền tín hiệu cũng như hỗ trợ các tính năng phụ cần thiết trong việc truyền và nhận các số 1 và 0. Đơn vị truyền ở lớp này các gói là 20 bit, được gọi là Phit.

- Lớp liên kết: Chịu trách nhiệm truyền tin cậy và điều khiển luồng. Đơn vị dữ liệu của lớp Liên kết là một Flit 80-bit (flow control unit).
- Lớp định tuyến: Cung cấp một framework để truyền các gói dữ liệu
- Lớp giao thức: Bộ quy tắc để trao đổi các gói tin dữ liệu giữa các thiết bị. Một gói bao gồm một số không đổi các Flits.



Hình 1.35. Các lớp QPI

Câu hỏi ôn tập chương 1

1. Kiến trúc máy tính là gì? Kiến trúc máy tính được cấu thành từ những thành phần nào?
2. Phân biệt tổ chức và kiến trúc máy tính.
3. Các chức năng chính của máy tính là gì?
4. Liệt kê và mô tả ngắn gọn các thành phần chính của máy tính.
5. Nêu sơ đồ và các đặc điểm của kiến trúc máy tính von-Neumann. Kiến trúc máy tính von-Neumann hiện đại khác kiến trúc máy tính von-Neumann cổ điển ở những điểm chính nào?
6. Nêu sơ đồ và các đặc điểm của kiến trúc máy tính Harvard. Kiến trúc máy tính Harvard có những ưu điểm gì so với kiến trúc máy tính von-Neumann. Các máy tính hiện đại ngày nay sử dụng kiến trúc nào?
7. Liệt kê và định nghĩa ngắn gọn hai phương pháp xử lý nhiều ngắt.
8. Liệt kê và định nghĩa ngắn gọn các lớp giao thức QPI.
9. Liệt kê và định nghĩa ngắn gọn các lớp giao thức PCIe.
10. Một vi xử lý có độ dài word là 1 byte. Xác định số nguyên nhỏ nhất và lớn nhất có thể biểu diễn được trong các dạng biểu diễn sau:
 - a. Không dấu
 - b. Dấu – độ lớn
 - c. Bù một
 - d. Bù hai
 - e. Packed decimal không dấu
 - f. Packed decimal có dấu
11. Liệt kê và định nghĩa ngắn gọn các trạng thái có thể để thực hiện một lệnh.
12. Liệt kê và định nghĩa ngắn gọn hai phương pháp xử lý nhiều ngắt.
13. Những dạng truyền nào phải có hỗ trợ của cấu trúc kết nối (ví dụ: bus)?
14. Ưu điểm của kiến trúc đa bus so với kiến trúc đơn bus?
15. Liệt kê và định nghĩa ngắn gọn các lớp giao thức QPI.
16. Liệt kê và định nghĩa ngắn gọn các lớp giao thức PCIe.

17. Với mục đích giải quyết một vấn đề ứng dụng, bạn thực hiện một chương trình trên hai hệ thống máy tính. Trên hệ thống A, mã đối tượng đã thực thi 90 triệu thao tác ALU, 45 triệu lệnh tải và 30 triệu lệnh rẽ nhánh. Trên hệ thống B, mã đối tượng đã thực thi 60 triệu thao tác ALU, 55 triệu lệnh tải và 45 triệu lệnh rẽ nhánh. Trong cả hai hệ thống, mỗi thao tác ALU mất 1 chu kỳ xung nhịp, mỗi lệnh tải mất 3 chu kỳ xung nhịp và mỗi lệnh rẽ nhánh mất 5 chu kỳ xung nhịp.

- Tính tần suất xuất hiện của từng loại lệnh được thực hiện trong cả hai hệ thống.
- Tìm CPI cho mỗi hệ thống.
- Giả sử rằng đồng hồ trên hệ thống B nhanh hơn 10% so với đồng hồ trên hệ thống A, hệ thống nào nhanh hơn đối với vấn đề ứng dụng đã cho và bao nhiêu phần trăm?

18. Cho một máy tính có chứa bộ xử lý đặc biệt để thực hiện các phép toán dấu phẩy động. Trong đó 60% các tính toán có thể sử dụng bộ xử lý này. Khi một chương trình sử dụng bộ xử lý dấu phẩy động, tốc độ của bộ xử lý dấu phẩy động nhanh hơn 40% so với khi chương trình không sử dụng nó.

- Tính hệ số cải tiến khi sử dụng bộ xử lý dấu phẩy động.
- Để cải thiện tốc độ, có hai tùy chọn:
 - Cách 1: Chỉnh sửa trình biên dịch để 70% tính toán có thể sử dụng bộ xử lý dấu chấm động. Chi phí của tùy chọn này là \$50K.
 - Tùy chọn 2: Sửa đổi bộ xử lý dấu chấm động. Tốc độ của bộ xử lý dấu phẩy động nhanh hơn 100% so với khi không sử dụng. Giả sử trong trường hợp này 50% các tính toán có thể sử dụng bộ xử lý dấu phẩy động. Chi phí của tùy chọn này là \$60K.

Hỏi tùy chọn nào tốt hơn, tại sao?

19. Cho các bộ xử lý P1, P2 và P3 có cùng một tập lệnh. Bộ P1 có tốc độ xung nhịp 3,5 GHz và CPI là 2,0. P2 có tốc độ xung nhịp 2,4 GHz và CPI là 1,5. P3 có tốc độ xung nhịp 5,0 GHz và có CPI là 2,5.

- Hỏi bộ xử lý nào có hiệu suất cao nhất (số lệnh thực hiện mỗi giây)?
- Tính số xung nhịp và số lệnh khi mỗi bộ xử lý thực hiện một chương trình trong 20 giây.
- Nếu cố gắng giảm 35% thời gian thực hiện nhưng điều này dẫn đến việc tăng 22% CPI. Khi đó tốc độ xung nhịp cần thiết đối với mỗi bộ xử lý là bao nhiêu?

20. Xét hai bộ xử lý có cùng một kiến trúc tập lệnh. Các lệnh được chia thành bốn loại theo CPI của chúng (loại A, B, C và D). Bộ xử lý 1 có tốc độ xung nhịp là 2,5 GHz và CPI tương ứng cho 4 loại lệnh là 1, 2, 3 và 3; Bộ xử lý 2 có tốc độ xung nhịp là 3 GHz và CPI tương ứng cho 4 loại lệnh là 2, 2, 2 và 2. Cho một chương trình gồm có các lệnh với 10% lệnh loại A, 20% lệnh loại B, 50% lệnh loại C và 20% lệnh loại D.

- Hỏi bộ xử lý nào cho hiệu suất tốt hơn? Tính CPI trung bình của mỗi bộ xử lý?
- Tính chu kỳ xung nhịp của 2 bộ xử lý?

CHƯƠNG 2. KIẾN TRÚC TẬP LỆNH VÀ BỘ XỬ LÝ

2.1. GIỚI THIỆU CHUNG

Kiến trúc tập lệnh máy tính hay gọi tắt là tập lệnh máy tính chính là điểm chung mà người thiết kế máy tính và người lập trình máy tính có thể cùng nhìn thấy khi xem xét một máy tính. Từ quan điểm của người thiết kế, tập lệnh máy tính cung cấp các yêu cầu chức năng cho bộ vi xử lý: việc thực hiện hoạt động của bộ xử lý gắn liền với việc thực hiện tập lệnh máy tính. Người sử dụng máy tính muốn lập trình bằng ngôn ngữ máy (hợp ngữ: assembly language) cần quan tâm đến cấu trúc thanh ghi và bộ nhớ, các loại dữ liệu được máy tính hỗ trợ trực tiếp và chức năng của đơn vị số học và logic (ALU).

Hoạt động của bộ xử lý được quyết định bởi các lệnh mà nó thực thi, được gọi là lệnh máy. Tập hợp nhiều lệnh khác nhau mà bộ xử lý có thể thực thi được gọi là tập lệnh của bộ xử lý.

Mỗi lệnh phải chứa những thông tin cần thiết để bộ xử lý có thể thực thi. Hình 1.22, cho thấy các bước trong quá trình thực thi lệnh và đồng thời cũng thể hiện các thành phần của một lệnh máy. Những thành phần đó là:

- **Mã opcode:** Chỉ ra hành động cần thực hiện (ví dụ: ADD, I/O). Hành động cần được thực hiện được xác định bởi một mã nhị phân.
- **Địa chỉ tham chiếu toán hạng nguồn:** Một hành động có thể liên quan đến một hoặc nhiều toán hạng nguồn. Toán hạng nguồn là đầu vào của hành động.
- **Địa chỉ tham chiếu toán hạng đích:** Một hành động có thể tạo ra một kết quả.
- **Địa chỉ tham chiếu lệnh tiếp theo:** Chỉ cho bộ xử lý biết vị trí lệnh tiếp theo cần truy xuất sau khi lệnh này đã hoàn thành.

Địa chỉ của lệnh tiếp theo cần truy xuất có thể là địa chỉ thực hoặc địa chỉ ảo, tùy thuộc vào kiến trúc đang sử dụng. Nói chung, sự phân biệt là minh bạch đối với kiến trúc tập lệnh. Trong hầu hết các trường hợp, lệnh tiếp theo sẽ được truy xuất ngay sau lệnh hiện tại. Trong trường hợp này, không có sự tham chiếu rõ ràng tới lệnh tiếp theo. Nếu cần có sự tham chiếu rõ ràng, khi đó địa chỉ bộ nhớ chính hoặc địa chỉ bộ nhớ ảo phải được cung cấp.

Các toán hạng nguồn và đích có thể nằm trong các vùng sau:

- **Bộ nhớ chính/bộ nhớ ảo:** Phải cung cấp địa chỉ bộ nhớ chính/ảo để phục vụ cho các tham chiếu lệnh tiếp theo.
- **Thanh ghi trong bộ xử lý:** Một bộ xử lý chứa một hoặc nhiều thanh ghi, các thanh ghi này có thể được các lệnh tham chiếu đến. Nếu chỉ có duy nhất một thanh ghi, tham chiếu đến thanh ghi đó có thể là ngầm định. Nếu có nhiều thanh ghi, mỗi thanh ghi phải được gán một tên gọi hoặc chỉ số riêng. Khi đó, lệnh tham chiếu đến thanh ghi nào phải chứa chỉ số của thanh ghi đó.
- **Tức thời:** Giá trị của toán hạng đặt trong một trường nhất định trong lệnh đang thực hiện.
- **Thiết bị vào/ra:** Lệnh phải chỉ ra thiết bị và mô-đun vào/ra liên quan đến hành động. Nếu sử dụng I/O ánh xạ bộ nhớ thì địa chỉ cần xác định chính là một địa chỉ bộ nhớ chính/ảo khác.

2.2. LỆNH MÁY VÀ TOÁN HẠNG

2.2.1. Biểu diễn lệnh

Trong máy tính, mỗi lệnh được biểu diễn bằng một chuỗi các bit. Lệnh được chia ra thành các trường tương ứng với các thành phần cấu thành nên lệnh. Hình 2.1 cho ta một ví dụ về định dạng lệnh. Đa số các tập lệnh đều sử dụng nhiều định dạng lệnh. Trong quá trình thực thi lệnh,

một lệnh sẽ được đọc vào thanh ghi lệnh (IR) trong bộ xử lý. Bộ xử lý phải có khả năng trích xuất dữ liệu từ các trường lệnh khác nhau để thực hiện thao tác yêu cầu. Sẽ rất khó khăn cho lập trình viên và người học nếu phải làm việc với các lệnh máy biểu diễn ở dạng nhị phân. Do đó, việc biểu diễn lệnh máy ở dạng ký tự gọi nhớ đã được sử dụng và phổ biến trong thực tế. Ví dụ như biểu diễn ký tự gọi nhớ được sử dụng cho tập lệnh IAS, trong Bảng 1.1. Opcode được biểu diễn bằng các từ viết tắt, gọi nhớ về hành động tương ứng. Ví dụ,



Hình 2.1. Một định dạng lệnh đơn giản

Các toán tử cũng được biểu diễn bằng ký tự. Ví dụ, lệnh: “ADD R, Y”, có nghĩa là cộng giá trị trong vị trí dữ liệu Y với giá trị nội dung thanh ghi R. Trong ví dụ này, Y là địa chỉ của một vị trí trong bộ nhớ, và R là một thanh ghi cụ thể. Lưu ý rằng hành động này được thực hiện trên nội dung của một vị trí chứ không phải trên địa chỉ của nó.

Do đó, một chương trình ngôn ngữ máy có thể được tạo ra gồm các câu lệnh dạng ký tự. Mỗi opcode ký tự gắn với một biểu diễn nhị phân cố định, và lập trình viên sẽ xác định vị trí của mỗi toán hạng ký tự. Ví dụ, lập trình viên có thể bắt đầu với một loạt các khai báo:

$X = 513$

$Y = 514$

Một chương trình đơn giản sẽ nhận đầu vào dạng ký tự này, chuyển đổi các opcode và địa chỉ tham chiếu toán hạng sang dạng nhị phân và tạo ra các lệnh máy nhị phân.

Rất hiếm, thậm chí không có, lập trình viên lập trình bằng ngôn ngữ máy. Hầu hết các chương trình hiện nay đều được viết bằng ngôn ngữ bậc cao hoặc chí ít là hợp ngữ (assembly). Tuy nhiên, ngôn ngữ máy dạng ký tự vẫn là một công cụ hữu ích để mô tả lệnh máy.

2.2.2. Phân loại lệnh

Giả sử một lệnh ngôn ngữ bậc cao có thể được diễn đạt bằng một ngôn ngữ như BASIC hay FORTRAN. Ví dụ:

$X = X + Y$

Câu lệnh này chỉ thị cho máy tính cộng giá trị lưu trữ trong Y với giá trị được lưu trữ trong X và đặt kết quả vào X. Hành động này có thể được thực hiện với lệnh máy như thế nào? Giả sử rằng các biến số X và Y tương ứng với các vị trí 513 và 514. Giả sử sử dụng một tập lệnh máy đơn giản, hành động này có thể được thực hiện bằng ba câu lệnh:

- a. Tải nội dung của vị trí bộ nhớ 513 vào một thanh ghi.
- b. Cộng nội dung của vị trí bộ nhớ 514 vào thanh ghi đó.

c. Lưu trữ nội dung của thanh ghi vào vị trí bộ nhớ 513.

Có thể thấy, một lệnh đơn giản lại đòi hỏi đến ba câu lệnh máy. Đây là đặc điểm điển hình của mối quan hệ giữa ngôn ngữ bậc cao và ngôn ngữ máy. Ngôn ngữ bậc cao diễn đạt hành động ở dạng đại số ngắn gọn, sử dụng các biến số. Ngôn ngữ máy diễn đạt hành động ở dạng cơ bản gắn với sự di chuyển của dữ liệu đến hoặc từ thanh ghi.

Chúng ta hãy xem xét các loại lệnh có trong một máy tính thực tế. Máy tính cần có một tập hợp các lệnh cho phép người sử dụng tạo ra mọi thao tác xử lý dữ liệu cần thiết.

Hoặc cách khác, ta xem xét khả năng của ngôn ngữ lập trình bậc cao. Bất kỳ chương trình viết bằng ngôn ngữ bậc cao nào cũng phải được dịch sang ngôn ngữ máy để được thực hiện. Do đó, tập lệnh máy phải đủ lớn để diễn đạt được mọi lệnh trong ngôn ngữ bậc cao.

Chúng ta có thể phân loại các loại lệnh như sau:

- Xử lý dữ liệu: Lệnh số học và lệnh logic
- Lưu trữ dữ liệu: Chuyển dữ liệu vào hoặc ra khỏi thanh ghi và/hoặc vị trí bộ nhớ
- Di chuyển dữ liệu: Lệnh vào/ra
- Điều khiển: Lệnh kiểm tra và lệnh rẽ nhánh

Lệnh số học cung cấp khả năng tính toán để xử lý dữ liệu số. Lệnh logic thực hiện trên từng bit của một từ chứ không thực hiện trên từng số, do đó chúng cung cấp khả năng xử lý bất kỳ loại dữ liệu nào mà người dùng muốn. Các hành động này được thực hiện chủ yếu trên dữ liệu trong thanh ghi bộ xử lý. Vì vậy, phải có các lệnh bộ nhớ để di chuyển dữ liệu giữa bộ nhớ và thanh ghi. Lệnh vào/ra được sử dụng để truyền chương trình và dữ liệu vào bộ nhớ, và truyền kết quả tính toán lại cho người dùng. Lệnh kiểm tra được sử dụng để kiểm tra giá trị của một từ dữ liệu hoặc trạng thái của một phép toán. Lệnh rẽ nhánh được sử dụng để rẽ nhánh tới một nhóm câu lệnh khác dựa vào quyết định được đưa ra.

2.2.3. Số lượng địa chỉ trong lệnh

Mô tả số lượng địa chỉ trong mỗi lệnh là một trong những cách truyền thống để mô tả kiến trúc bộ xử lý. Khi thiết kế bộ xử lý ngày càng phức tạp, vai trò của yếu tố này đã giảm dần. Tuy nhiên, việc phân tích số lượng địa chỉ trong lệnh lại rất hữu ích cho người học tại thời điểm này.

Vậy số lượng địa chỉ tối đa cần có trong một lệnh là bao nhiêu? Rõ ràng, lệnh số học và lệnh logic sẽ yêu cầu nhiều toán hạng nhất. Hầu như tất cả các phép tính số học và logic đều có một hoặc hai toán hạng nguồn. Do đó, chúng ta cần tối đa hai địa chỉ để tham chiếu toán hạng nguồn. Kết quả của một hành động phải được lưu trữ, vì vậy cần có một địa chỉ thứ ba, tham chiếu toán hạng đích. Cuối cùng, sau khi hoàn thành một lệnh, lệnh tiếp theo phải được truy xuất, và cần phải có địa chỉ của nó.

Từ đó, ta thấy rằng một lệnh cần phải có tối đa bốn tham chiếu địa chỉ: hai toán hạng nguồn, một toán hạng đích, và địa chỉ của lệnh tiếp theo. Trong hầu hết các kiến trúc, các lệnh thường có một, hai hoặc ba địa chỉ toán hạng, còn địa chỉ của lệnh tiếp theo được ngầm định (lấy từ bộ đếm chương trình). Hầu hết các kiến trúc cũng có một vài lệnh chuyên dụng với nhiều toán hạng hơn. Ví dụ, việc tải và lưu trữ nhiều lệnh của kiến trúc ARM có 17 toán hạng thanh ghi trong một lệnh duy nhất.

Hình 2.2 lần lượt so sánh các lệnh có một, hai và ba địa chỉ được dùng để tính biểu thức $Y = \frac{A-B}{C+(D \times E)}$. Với ba địa chỉ, mỗi lệnh xác định hai vị trí toán hạng nguồn và một vị trí toán hạng đích. Bởi vì chúng ta giả thiết không thay đổi giá trị của bất kỳ vị trí toán hạng nào, một vị trí tạm

thời T được sử dụng để lưu trữ một số kết quả trung gian. Lưu ý rằng có tất cả bốn lệnh và biểu thức ban đầu có năm toán hạng.

Lệnh	Giải thích	
SUB	Y, A, B	$Y \leftarrow A - B$
MPY	T, D, E	$T \leftarrow D \times E$
ADD	T, T, C	$T \leftarrow T + C$
DIV	Y, Y, T	$Y \leftarrow Y \div T$

(a) Lệnh có ba địa chỉ

Lệnh	Giải thích	
MOVE	Y, A	$Y \leftarrow A$
SUB	Y, B	$Y \leftarrow Y - B$
MOVE	T, D	$T \leftarrow D$
MPY	T, E	$T \leftarrow T \times E$
ADD	T, C	$T \leftarrow T + C$
DIV	Y, T	$Y \leftarrow Y \div T$

(b) Lệnh có hai địa chỉ

Lệnh	Giải thích	
LOAD	D	$AC \leftarrow D$
MPY	E	$AC \leftarrow AC \times E$
ADD	C	$AC \leftarrow AC + C$
STOR	Y	$Y \leftarrow AC$
LOAD	A	$AC \leftarrow A$
SUB	B	$AC \leftarrow AC - B$
DIV	Y	$AC \leftarrow AC \div Y$
STOR	Y	$Y \leftarrow AC$

(c) Lệnh có một địa chỉ

Hình 2.2. Chương trình tính biểu thức $Y = \frac{A-B}{C+(D \times E)}$

Định dạng lệnh ba địa chỉ không phổ biến vì để chứa được ba tham chiếu địa chỉ chúng đòi hỏi định dạng lệnh tương đối dài. Với lệnh hai địa chỉ, và đối với các xử lý nhị phân, một địa chỉ sẽ phải đảm nhiệm gấp đôi nhiệm vụ, vừa cho toán hạng nguồn vừa cho kết quả đích. Do đó, lệnh SUB Y, B thực hiện phép tính $Y - B$ và lưu trữ kết quả vào Y.

Định dạng hai địa chỉ làm giảm độ dài lệnh nhưng cũng gây ra một số phiền toái. Để tránh thay đổi giá trị của toán hạng, lệnh MOVE được sử dụng để di chuyển một trong các giá trị đó đến vị trí kết quả hoặc một vị trí tạm thời trước khi thực hiện xử lý. Chương trình ví dụ lúc này mở rộng lên tới sáu lệnh.

Đơn giản hơn nữa là lệnh một địa chỉ. Với định dạng này, địa chỉ thứ hai phải được ngầm định. Điều này rất phổ biến trong các máy trước đây, với địa chỉ ngầm định là một thanh ghi bộ xử lý được gọi là AC (accumulator). Thanh ghi AC chứa một trong hai toán hạng và được sử dụng để lưu trữ kết quả. Trong ví dụ của chúng ta, cần tổng cộng tám lệnh để hoàn thành tính toán Y.

Trên thực tế, một số lệnh có thể thực hiện mà không cần địa chỉ. Lệnh không địa chỉ được áp dụng cho một tổ chức bộ nhớ đặc biệt được gọi là ngăn xếp. Ngăn xếp là một tập hợp các vị trí LIFO (vào sau ra trước). Ngăn xếp ở một vị trí đã biết, và ít nhất hai phần tử trên cùng của ngăn xếp thường nằm trong thanh ghi bộ xử lý. Do đó, lệnh không địa chỉ sẽ tham chiếu đến hai phần tử ở đỉnh ngăn xếp.

Bảng 2.1 tóm tắt cách giải thích các lệnh có không, một, hai hoặc ba địa chỉ. Trong các trường hợp trong bảng, giả sử rằng địa chỉ của lệnh kế tiếp là ngầm định, và một phép toán với hai toán hạng nguồn và một toán hạng kết quả sẽ được thực hiện.

Số lượng địa chỉ trong câu lệnh là một yếu tố thiết kế cơ bản. Càng ít địa chỉ trong lệnh dẫn tới các lệnh đơn giản hơn, đòi hỏi bộ xử lý ít phức tạp hơn. Nó cũng làm cho lệnh ngắn gọn hơn. Mặt khác, chương trình lại có tổng cộng nhiều lệnh hơn, khiến cho thời gian thực hiện lâu hơn, chương trình phức tạp và dài hơn. Ngoài ra, còn có sự khác biệt quan trọng giữa lệnh một địa

chỉ và nhiều địa chỉ. Với lệnh một địa chỉ, lập trình viên thường chỉ có sẵn một thanh ghi đa năng là AC. Với lệnh nhiều địa chỉ, thường có nhiều thanh ghi đa năng hơn. Điều này cho phép một số hành động chỉ cần thực hiện trong thanh ghi. Bởi vì tham chiếu thanh ghi nhanh hơn nhiều so với tham chiếu bộ nhớ, tốc độ thực hiện lệnh sẽ tăng lên. Nhằm có được tính linh hoạt và khả năng sử dụng nhiều thanh ghi, hầu hết các máy tính hiện đại sử dụng đồng thời các lệnh hai và ba địa chỉ.

Bảng 2.1. Cách sử dụng địa chỉ lệnh (Lệnh không rẽ nhánh)

Số lượng địa chỉ	Biểu diễn ký tự	Giải thích
3	OP A, B, C	$A \leftarrow B \text{ OP } C$
2	OP A, B	$A \leftarrow A \text{ OP } B$
1	OP A	$AC \leftarrow AC \text{ OP } A$
0	OP	$T \leftarrow (T - 1) \text{ OP } T$

AC = Thanh ghi tích lũy

T = đỉnh ngăn xếp

(T - 1) = phần tử thứ hai trong ngăn xếp

A, B, C = các vị trí bộ nhớ hoặc thanh ghi

Việc lựa chọn số lượng địa chỉ cho mỗi lệnh trong quá trình thiết kế còn phức tạp bởi các yếu tố khác. Có một câu hỏi đặt ra là khi nào thì tham chiếu đến địa chỉ một vị trí bộ nhớ và khi nào thì tham chiếu địa chỉ một thanh ghi. Bởi số lượng thanh ghi là rất ít (so với số lượng vị trí bộ nhớ), nên địa chỉ tham chiếu thanh ghi cần ít bit biểu diễn hơn. Ngoài ra, như chúng ta sẽ thấy, một máy có thể có nhiều chế độ địa chỉ khác nhau, và việc chỉ định chế độ cụ thể sẽ cần dùng một hoặc nhiều bit. Kết quả là hầu hết các thiết kế bộ xử lý đều có nhiều định dạng lệnh khác nhau.

2.2.4. Các kiểu toán hạng

Lệnh máy thao tác trên dữ liệu. Các kiểu dữ liệu quan trọng nhất là:

- Địa chỉ
- Số
- Ký tự
- Dữ liệu logic

Chúng ta sẽ thấy rằng địa chỉ thực ra cũng là một dạng dữ liệu. Trong nhiều trường hợp, tham chiếu toán hạng trong một lệnh phải được tính toán (bằng một số phép tính) để xác định địa chỉ bộ nhớ chính hoặc ảo. Trong ngữ cảnh này, địa chỉ có thể được coi là số nguyên không dấu.

Các kiểu dữ liệu phổ biến khác là số, ký tự, và dữ liệu logic, và mục này sẽ trình bày ngắn gọn về từng kiểu.

Số

Tất cả các ngôn ngữ máy đều có kiểu dữ liệu dạng số. Ngay cả trong xử lý dữ liệu không phải dạng số, vẫn cần phải có số để thực hiện bộ đếm, độ rộng trường, vv. Một điểm khác biệt quan trọng giữa các con số được sử dụng trong toán học thông thường và số được lưu trữ trong máy tính là số trong máy tính bị giới hạn. Điều này có thể hiểu theo hai nghĩa. Thứ nhất, độ lớn của số biểu diễn được trên máy bị giới hạn và thứ hai, trong trường hợp số dấu chấm động, độ chính xác của số cũng bị hạn chế. Vì vậy, lập trình viên phải hiểu biết rõ về những ảnh hưởng của việc làm tròn, tràn trên (overflow), và tràn dưới (underflow).

Ba kiểu dữ liệu số thông thường trong máy tính là:

- Số nguyên nhị phân hoặc số thực dấu chấm tĩnh nhị phân
- Số thực dấu chấm động nhị phân
- Số thập phân

Ký tự

Dữ liệu cũng thường ở dạng văn bản hoặc chuỗi ký tự. Mặc dù dữ liệu dạng văn bản là tiện lợi nhất cho con người, việc lưu trữ hoặc truyền dữ liệu này qua hệ thống xử lý dữ liệu và truyền thông không thể thực hiện được một cách dễ dàng. Các hệ thống đó chỉ được thiết kế cho dữ liệu nhị phân. Do đó, một số loại mã đã được tạo ra sao cho mỗi ký tự được biểu diễn bởi một chuỗi bit. Mã Morse có lẽ là xuất hiện sớm nhất và là ví dụ rất quen thuộc cho điều này. Ngày nay, mã ký tự IRA (International Reference Alphabet) được sử dụng phổ biến nhất là mã ASCII (American Standard Code for Information Interchange). Mỗi ký tự trong mã này được biểu diễn bởi một mẫu 7 bit duy nhất; do đó, có thể biểu diễn được tổng cộng 128 ký tự khác nhau. Con số này lớn hơn số lượng cần thiết để biểu diễn các ký tự in được, bởi một số mẫu 7 bit được dùng để biểu diễn ký tự điều khiển. Một số ký tự điều khiển liên quan đến điều khiển việc in các ký tự trên một trang; một số khác liên quan đến các thủ tục truyền thông. Các ký tự mã hoá IRA hầu như luôn được lưu trữ và truyền đi theo từng nhóm 8 bit cho mỗi ký tự. Bit thứ tám có thể được đặt thành 0 hoặc được sử dụng như là bit chẵn lẻ (parity) để phát hiện lỗi. Trong trường hợp thứ hai, bit được đặt sao cho tổng số các số 1 nhị phân trong mỗi cụm 8 bit luôn lẻ (parity lẻ) hoặc luôn chẵn (parity chẵn).

Lưu ý, trong bảng mã ASCII, các chữ số thập phân từ 0 đến 9 được biểu diễn bằng mẫu bit 011XXXX với 4 bit ngoài cùng bên phải là các số nhị phân tương ứng từ 0000 đến 1001. Tức là, thập phân 0 tương ứng 0110000, 1 tương ứng 0110001,..., 9 tương ứng 0111001.

Dữ liệu logic

Thông thường, mỗi từ hoặc mỗi đơn vị địa chỉ khác (byte, halfword, v.v...) được coi như một đơn vị dữ liệu. Tuy nhiên, đôi khi ta cần xem xét một đơn vị dữ liệu n bit như là n phần dữ liệu 1 bit riêng, mỗi phần có giá trị 0 hoặc 1. Khi dữ liệu được xem xét theo cách này, chúng được gọi là dữ liệu logic.

Việc xét dữ liệu theo hướng bit có hai ưu điểm. Thứ nhất, đôi khi chúng ta mong muốn lưu trữ một mảng các phần tử dữ liệu nhị phân hoặc logic, trong đó mỗi phần tử chỉ có thể nhận giá trị 1 (true) và 0 (false). Với dữ liệu logic, bộ nhớ có thể được sử dụng hiệu quả nhất cho kiểu lưu trữ này. Thứ hai, đôi khi chúng ta cũng mong muốn thao tác trên các bit của một dữ liệu. Ví dụ, nếu các phép tính dấu chấm động được thực hiện trong phần mềm, ta cần phải có khả năng dịch bit trong một số thao tác.

Lưu ý rằng, trong các ví dụ trước, cùng một dữ liệu đôi khi được xử lý như là logic và đôi khi lại như là số hoặc văn bản. "Kiểu" của một đơn vị dữ liệu được xác định bởi thao tác đang được thực hiện trên nó. Mặc dù điều này hầu như luôn gặp phải trong ngôn ngữ máy, nó không phải là trường hợp thường gặp trong các ngôn ngữ bậc cao.

2.2.5. Các loại hành động

Tổng số lượng opcode trên mỗi máy là rất khác nhau. Tuy nhiên, các loại hành động tổng quát lại giống nhau và có thể tìm thấy trên tất cả các máy. Chúng được phân loại thành các loại điển hình như sau:

- Truyền dữ liệu

- Xử lý số học
- Xử lý logic
- Điều khiển vào/ra
- Chuyển điều khiển (rẽ nhánh)
- Điều khiển hệ thống

Bảng 2.2. Các hành động của tập lệnh

Loại	Lệnh	Mô tả
Truyền dữ liệu	MOVE	Copy dữ liệu từ nguồn đến đích
	LOAD	Nạp dữ liệu từ bộ nhớ đến bộ xử lý
	STORE	Cất dữ liệu từ bộ xử lý đến bộ nhớ
	EXCHANGE	Trao đổi nội dung của nguồn và đích
	CLEAR	Chuyển các bit 0 vào toán hạng đích
	SET	Chuyển các bit 1 vào toán hạng đích
	PUSH	Cất nội dung toán hạng nguồn vào ngăn xếp
	POP	Lấy nội dung đỉnh ngăn xếp đưa đến toán hạng đích
Xử lý số học	ADD	Cộng hai toán hạng
	SUBTRACT	Trừ hai toán hạng
	MULTIPLY	Nhân hai toán hạng
	DIVIDE	Chia hai toán hạng
	ABSOLUTE	Lấy trị tuyệt đối toán hạng
	NEGATE	Đổi dấu toán hạng (lấy bù 2)
	INCREMENT	Tăng toán hạng thêm 1
	DECREMENT	Giảm toán hạng đi 1
Xử lý logic	AND	Thực hiện phép AND hai toán hạng
	OR	Thực hiện phép OR hai toán hạng
	NOT	Đảo bit của toán hạng (lấy bù 1)
	XOR	Thực hiện phép XOR hai toán hạng
	TEST	Kiểm tra điều kiện cụ thể; thiết lập cờ dựa trên kết quả
	COMPARE	So sánh logic hoặc số học của hai hoặc nhiều toán hạng;
	SHIFT	Dịch trái (phải) toán hạng
	ROTATE	Quay vòng trái (phải) toán hạng
Điều khiển vào/ra	INPUT	Truyền dữ liệu từ một cổng hoặc thiết bị I/O xác định đến đích (VD: Bộ nhớ chính hoặc thanh ghi bộ xử lý)
	OUTPUT	Truyền dữ liệu từ nguồn xác định đến cổng hoặc thiết bị I/O
	START I/O	Truyền lệnh đến bộ xử lý I/O để bắt đầu hoạt động I/O
	TEST I/O	Truyền thông tin trạng thái từ hệ thống I/O đến đích xác định
	JUMP (BRANCH)	Truyền không điều kiện; tải địa chỉ xác định vào PC

Truyền điều khiển	JUMP CONDITIONAL	Kiểm tra điều kiện cụ thể; tải địa chỉ xác định vào PC hoặc không làm gì tùy thuộc vào điều kiện
	CALL	Cất nội dung của PC (địa chỉ trở về) vào một vị trí xác định; Nạp địa chỉ lệnh đầu tiên của chương trình con vào PC
	RETURN	Đặt địa chỉ trở về trả lại cho PC để trở về chương trình chính
	SKIP	Tăng PC để bỏ qua lệnh tiếp theo
	SKIP CONDITONAL	Kiểm tra điều kiện cụ thể; bỏ qua lệnh hoặc không làm gì tùy thuộc vào điều kiện
	HALT	Dừng thực thi chương trình
	WAIT (HOLD)	Dừng thực thi chương trình; liên tục kiểm tra điều kiện cụ thể; quay lại thực thi tiếp khi điều kiện được thỏa mãn
	NO OPERATION	Không có hành động nào được thực hiện, nhưng việc thực thi chương trình vẫn được tiếp tục

Bảng 2.2 liệt kê các câu lệnh phổ biến của mỗi loại hành động. Mục này sẽ trình bày ngắn gọn về các loại hành động này, đồng thời thảo luận ngắn gọn về hoạt động của bộ xử lý để thực hiện từng loại hành động cụ thể (tóm tắt trong Bảng 2.3).

Bảng 2.3. Hoạt động của bộ xử lý tương ứng với từng loại hành động

Truyền dữ liệu	Truyền dữ liệu từ vị trí này sang vị trí khác
	Nếu có liên quan đến bộ nhớ: Xác định địa chỉ bộ nhớ Thực hiện chuyển đổi địa chỉ bộ nhớ từ địa chỉ bộ nhớ ảo sang thực Kiểm tra bộ nhớ cache Khởi tạo lệnh đọc/ghi bộ nhớ
	Có thể liên quan đến việc truyền dữ liệu (trước và/hoặc sau truyền dữ liệu)
	Thực hiện hàm trong ALU
	Thiết lập mã điều kiện và cờ
Số học	
Logic	Tương tự như số học
Truyền điều khiển	Cập nhật PC. Đối với lệnh gọi/trả về chương trình con (Call/Return), quản lý việc liên kết và truyền tham số
Điều khiển vào/ra	Ra lệnh lệnh cho mô-đun I/O
	Nếu sử dụng I/O ánh xạ bộ nhớ, xác định địa chỉ ánh xạ bộ nhớ

2.2.5.1. Truyền dữ liệu

Loại lệnh máy cơ bản nhất là lệnh truyền dữ liệu. Lệnh chuyển dữ liệu phải xác định một vài thông tin. Thứ nhất, nó phải xác định vị trí của các toán hạng nguồn và toán hạng đích. Mỗi vị trí này có thể là vị trí bộ nhớ, thanh ghi, hoặc đỉnh ngăn xếp. Thứ hai, nó cũng phải chỉ ra độ dài dữ liệu cần truyền. Thứ ba, giống như với tất cả các lệnh có toán hạng, chế độ định địa chỉ cho mỗi toán hạng phải được xác định.

Trong các loại hành động xử lý, hành động truyền dữ liệu có lẽ là đơn giản nhất. Nếu cả toán hạng nguồn và toán hạng đích đều ở thanh ghi, thì bộ xử lý chỉ đơn giản là chuyển dữ liệu từ thanh ghi này sang thanh ghi khác; đây là hành động diễn ra bên trong bộ xử lý. Nếu một hoặc cả

hai loại toán hạng này nằm trong bộ nhớ, khi đó bộ xử lý phải thực hiện một số hoặc tất cả các hành động sau đây:

1. Tính toán địa chỉ bộ nhớ, dựa trên chế độ địa chỉ.
2. Nếu địa chỉ liên quan đến bộ nhớ ảo, dịch địa chỉ từ địa chỉ bộ nhớ ảo sang thực.
3. Xác định xem toán hạng mong muốn có trong bộ nhớ cache hay không.
4. Nếu không, ra một mệnh lệnh cho mô-đun bộ nhớ.

2.2.5.2. Xử lý số học

Hầu hết máy tính cung cấp các phép tính số học cơ bản như cộng, trừ, nhân và chia. Các phép tính này được cung cấp cho tính toán liên quan đến số nguyên (dấu chấm tĩnh) có dấu, số thực dấu chấm động và số packed decimal.

Bên cạnh đó, còn có các xử lý khác, bao gồm một loạt các lệnh chỉ có một toán hạng; ví dụ:

- Tuyệt đối (Absolute): Lấy giá trị tuyệt đối của toán hạng.
- Đảo (Negate): Đảo dấu toán hạng.
- Tăng (Increment): Tăng toán hạng thêm 1.
- Giảm (Decrement): Giảm toán hạng đi 1.

Việc thực thi một lệnh số học có thể bao gồm vài thao tác truyền dữ liệu để đặt các toán hạng tới đầu vào khối ALU, và để vận chuyển kết quả ở đầu ra ALU. Ngoài ra, khối ALU của bộ xử lý tất nhiên sẽ thực hiện các xử lý mong muốn.

2.2.5.3. Xử lý logic

Hầu hết máy tính cung cấp một loạt các hành động để xử lý các bit riêng lẻ trong một word hoặc một đơn vị địa chỉ khác. Chúng dựa trên các phép toán đại số logic Boolean.

Một số phép toán logic cơ bản có thể được thực hiện trên dữ liệu nhị phân hoặc Boolean được trình bày trong Bảng 2.4. Phép toán NOT đảo ngược giá trị một bit. AND, OR, và XOR là các hàm logic có hai toán hạng phổ biến nhất. EQUAL là một phép thử nhị phân hữu ích.

Bảng 2.4. Các phép toán logic cơ bản

P	Q	NOT P	P AND Q	P OR Q	P XOR Q	P = Q
0	0	1	0	0	0	1
0	1	1	0	1	1	0
1	0	0	0	1	1	0
1	1	0	1	1	0	1

Những phép toán logic này có thể được áp dụng (trên từng bit) cho các đơn vị dữ liệu logic n bit. Vì vậy, nếu hai thanh ghi chứa dữ liệu như sau:

$$(R1) = 10100101$$

$$(R2) = 00001111$$

thì

$$(R1) \text{ AND } (R2) = 00000101$$

trong đó ký hiệu (X) có nghĩa là nội dung của vị trí X. Do đó, phép toán AND có thể được sử dụng làm mặt nạ để chọn một số bit xác định trong một word và lọc đi các bit còn lại.

Xét ví dụ khác, nếu hai thanh ghi là:

$$(R1) = 10100101$$

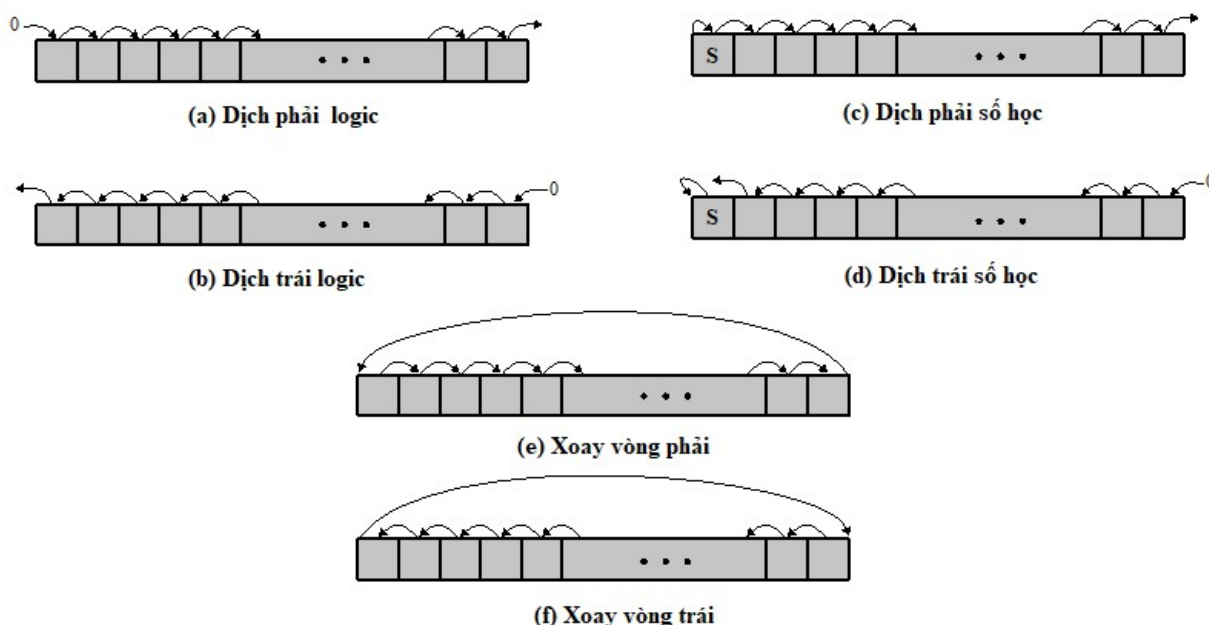
$$(R2) = 11111111$$

thì

$$(R1) \text{ XOR } (R2) = 01011010$$

Với một word được thiết lập để tất cả các bit là 1, phép toán XOR đảo ngược giá trị tất cả các bit trong word còn lại (bù một).

Ngoài phép toán logic trên từng bit, hầu hết máy tính đều cung cấp một loạt các chức năng dịch và xoay vòng. Các phép dịch và xoay vòng cơ bản nhất được minh họa trong hình 2.3. Với phép dịch logic, các bit của một word được dịch chuyển sang trái hoặc phải. Trên một đầu, bit dịch ra ngoài bị mất đi. Ở đầu kia, một bit 0 được dịch vào. Phép dịch logic chủ yếu hữu ích trong việc tách các trường trong một word. Các bit 0 được dịch vào một word thay thế cho những thông tin không mong muốn được dịch ra ngoài ở đầu bên kia.



Hình 2.3. Phép dịch và xoay vòng

Ví dụ, giả sử chúng ta muốn truyền các ký tự của dữ liệu tới một thiết bị I/O, mỗi lần truyền một ký tự. Nếu mỗi word của bộ nhớ dài 16 bit và chứa hai ký tự, chúng ta phải tách rời từng ký tự trước khi gửi đi. Để gửi đi hai ký tự trong một word:

1. Tải word đó vào thanh ghi.
2. Dịch sang phải tám lần. Việc này sẽ làm dịch ký tự còn lại sang nửa bên phải của thanh ghi.
3. Thực hiện hành động I/O. Mô-đun I/O đọc 8 bit trọng số thấp hơn từ bus dữ liệu.

Các bước vừa nêu tương ứng với việc gửi đi ký tự bên trái. Để gửi ký tự bên phải:

1. Tải lại word đó vào thanh ghi.

2. AND với 0000000011111111. Việc này sẽ lọc bỏ ký tự bên trái.
3. Thực hiện hành động I/O.

Các phép dịch số học xử lý dữ liệu như là một số nguyên có dấu và không dịch chuyển bit dấu. Với phép dịch phải số học, bit dấu được sao chép vào vị trí bit ở bên phải nó. Phép dịch trái số học giống như phép dịch trái logic được thực hiện trên tất cả các bit, ngoại trừ bit dấu, tức là bit dấu giữ nguyên. Các phép dịch này có thể làm tăng tốc độ xử lý của các phép toán số học nhất định. Với các số biểu diễn bù hai, phép dịch phải số học tương ứng với một phép chia cho 2 (làm tròn với số lẻ). Cả dịch trái số học và dịch trái logic tương ứng với một phép nhân với 2 khi không có tràn trên (overflow). Nếu xảy ra tràn, phép dịch trái số học và dịch trái logic sẽ tạo ra các kết quả khác nhau, nhưng dịch trái số học vẫn giữ lại dấu của số.

Phép xoay vòng, hoặc dịch vòng, bảo toàn tất cả các bit mà nó xử lý. Phép xoay vòng được sử dụng để lần lượt đưa từng bit về vị trí bit ngoài cùng bên trái, tại đó, nó có thể được nhận diện giá trị bằng cách kiểm tra dấu của dữ liệu (coi như một số có dấu).

Giống như các phép tính số học, phép tính logic cũng liên quan đến hoạt động của khối ALU và có thể bao gồm các thao tác truyền dữ liệu. Bảng 2.5 đưa ra ví dụ minh họa cho các phép dịch và xoay vòng này.

Bảng 2.5. Ví dụ cho phép dịch và xoay vòng

Đầu vào	Hành động	Kết quả
10100110	Dịch phải logic (3 bit)	00010100
10100110	Dịch trái logic (3 bit)	00110000
10100110	Dịch phải số học (3 bit)	11110100
10100110	Dịch trái số học (3 bit)	10110000
10100110	Xoay vòng phải (3 bit)	11010100
10100110	Xoay vòng trái (3 bit)	00110101

2.2.5.4. Điều khiển vào/ra

Các lệnh đầu vào/đầu ra đã được thảo luận chi tiết trong Chương 6. Như chúng ta đã thấy, có nhiều phương pháp được thực hiện, bao gồm I/O bằng lập trình, tách biệt bộ nhớ, I/O bằng lập trình, ánh xạ bộ nhớ, DMA, và việc sử dụng bộ xử lý I/O. Nhiều cách thực hiện chỉ cung cấp một vài lệnh I/O, với các hành động cụ thể được xác định bởi tham số, mã hoặc các word lệnh.

2.2.5.5. Điều khiển hệ thống

Lệnh điều khiển hệ thống là lệnh có thể được thực thi chỉ khi bộ xử lý đang ở trong trạng thái đặc quyền (privileged) nhất định hoặc đang thi hành một chương trình trong vùng đặc quyền (privileged) đặc biệt của bộ nhớ. Thông thường, các lệnh này được dành riêng cho việc sử dụng hệ điều hành.

Ví dụ về hành động điều khiển hệ thống, một lệnh điều khiển hệ thống có thể đọc hoặc thay đổi thanh ghi điều khiển. Một ví dụ khác là lệnh để đọc hoặc thay đổi khóa bảo vệ bộ nhớ.

2.2.5.6. Chuyển điều khiển (rẽ nhánh)

Đối với tất cả các loại hành động đã thảo luận trên đây, lệnh tiếp theo được thực hiện là lệnh liền ngay phía sau lệnh hiện tại trong bộ nhớ. Tuy nhiên, một số lệnh trong chương trình có

chức năng thay đổi trình tự thi hành lệnh. Đối với các lệnh này, bộ xử lý sẽ cập nhật thanh ghi PC bằng địa chỉ của một số lệnh trong bộ nhớ.

Tại sao cần phải có các hành động chuyển điều khiển? Trong số nhiều lý do, một số lý do quan trọng nhất là:

1. Trong thực tế sử dụng máy tính, việc thực hiện một lệnh nhiều hơn một lần và thậm chí hàng nghìn lần là rất cần thiết. Một ứng dụng có thể đòi hỏi thực hiện hàng nghìn đến hàng triệu câu lệnh. Vì vậy, nếu phải viết riêng từng lệnh kể cả lệnh được lặp lại thì sẽ rất khó khăn. Nếu cần xử lý một bảng hoặc danh sách các mục, ta cần một vòng lặp chương trình. Một chuỗi lệnh sẽ được thi hành lặp đi lặp lại để xử lý tất cả dữ liệu.
2. Hầu như mọi chương trình đều liên quan đến việc đưa ra quyết định. Chúng ta muốn máy tính thực hiện một hành động nếu điều kiện này được thỏa mãn, và hành động khác nếu điều kiện khác được thỏa mãn. Ví dụ, một chuỗi lệnh tính toán căn bậc hai của một số. Ở đầu chuỗi, dấu của số đó được kiểm tra. Nếu là số âm, phép tính không được thực hiện, và sẽ báo lỗi.
3. Tạo ra một chương trình máy tính lớn là một nhiệm vụ cực kỳ khó khăn. Sẽ rất hiệu quả nếu có cơ chế để phân chia công việc thành các phần nhỏ hơn và thực hiện lần lượt từng việc.

Tiếp theo, chúng ta sẽ thảo luận về các lệnh chuyển điều khiển thông dụng nhất trong các tập lệnh: lệnh rẽ nhánh, skip, và gọi thủ tục.

Lệnh rẽ nhánh (Branch)

Lệnh rẽ nhánh, còn được gọi là lệnh nhảy (jump), có một trong các toán hạng của nó là địa chỉ của lệnh tiếp theo được thực thi. Thông thường, lệnh này là lệnh rẽ nhánh có điều kiện. Tức là, chỉ rẽ nhánh (cập nhật địa chỉ đã chỉ định trong toán hạng vào thanh ghi PC) khi một điều kiện nhất định được thỏa mãn. Nếu không, lệnh tiếp theo trong chuỗi lệnh được thực thi (tăng giá trị PC như bình thường). Một lệnh rẽ nhánh mà việc rẽ nhánh luôn được thực hiện được gọi là rẽ nhánh không điều kiện.

Có hai cách phổ biến để đặt ra điều kiện trong lệnh rẽ nhánh có điều kiện. Thứ nhất, hầu hết các máy đều có mã điều kiện 1 bit hoặc nhiều bit được thiết lập như là kết quả của một số phép tính. Mã này có thể được coi là một thanh ghi ngắn, hiển thị với người dùng.

Ví dụ, một phép tính số học (ADD, SUBTRACT, v.v.) có thể thiết lập mã điều kiện 2 bit nhận một trong bốn giá trị sau: 0, dương, âm, tràn (overflow). Trên máy này, có thể có bốn lệnh rẽ nhánh có điều kiện khác nhau như sau:

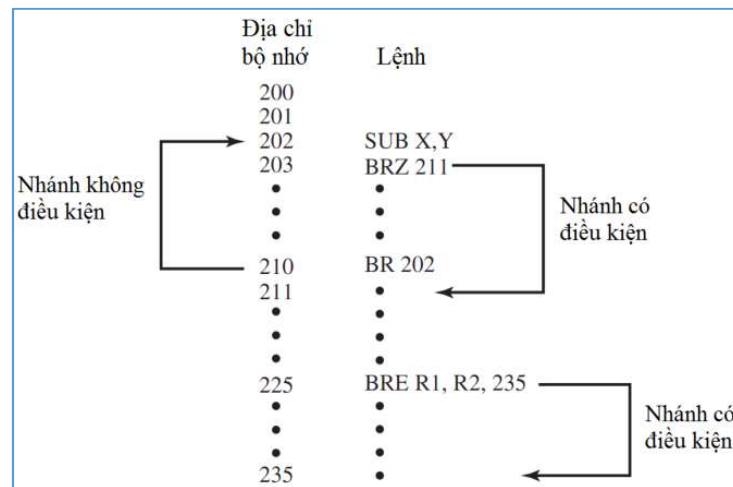
- BRP X Rẽ nhánh đến vị trí X nếu kết quả dương.
- BRN X Rẽ nhánh đến vị trí X nếu kết quả âm
- BRZ X Rẽ nhánh đến vị trí X nếu kết quả là 0.
- BRO X Rẽ nhánh đến vị trí X nếu xảy ra tràn.

Trong tất cả các trường hợp này, kết quả được đề cập đến là kết quả của hành động thiết lập mã điều kiện gần đây nhất. Với định dạng lệnh ba địa chỉ, một phương pháp khác có thể được sử dụng là thực hiện so sánh và chỉ định rẽ nhánh trong cùng một lệnh. Ví dụ:

BRE R1, R2, X Rẽ nhánh đến X nếu nội dung của R1 = nội dung của R2.

Hình 2.4 cho thấy ví dụ về các hành động này. Lưu ý rằng nhánh có thể tiến (một lệnh với địa chỉ cao hơn) hoặc lùi (địa chỉ thấp hơn). Ví dụ này cho ta thấy nhánh vô điều kiện và nhánh có

điều kiện có thể được sử dụng để tạo ra một vòng lặp các lệnh như thế nào. Các lệnh nằm tại các vị trí từ 202 đến 210 sẽ được thi hành lặp đi lặp lại cho đến khi kết quả của phép tính X trừ Y bằng 0.



Hình 2.4. Lệnh rẽ nhánh

Lệnh skip

Một dạng lệnh chuyển điều khiển khác là lệnh skip. Lệnh skip gồm có một địa chỉ ngầm định. Thông thường, skip được ngầm hiểu rằng một lệnh được bỏ qua; do đó, địa chỉ ngầm định bằng với địa chỉ của lệnh tiếp theo cộng với chiều dài một lệnh.

Bởi vì lệnh skip không đòi hỏi trường địa chỉ đích, nên trường này có thể được dành cho những thứ khác. Một ví dụ điển hình là lệnh ISZ (increment-and-skip-if-zero hay là tăng lên và bỏ qua nếu bằng không). Xét đoạn chương trình dưới đây. Trong đoạn chương trình này, hai lệnh chuyển điều khiển được sử dụng để thực hiện một vòng lặp. R1 được thiết lập bằng một số âm, bằng số lần lặp sẽ được thực hiện. Ở cuối vòng lặp, R1 được tăng lên. Nếu nó khác 0, chương trình trở lại phần đầu của vòng lặp. Nếu nó bằng 0, việc rẽ nhánh bị bỏ qua, vòng lặp kết thúc và chương trình tiếp tục thực thi lệnh tiếp theo.

```

301
:
:
309 ISZ R1
310 BR 301

```

Lệnh gọi thủ tục

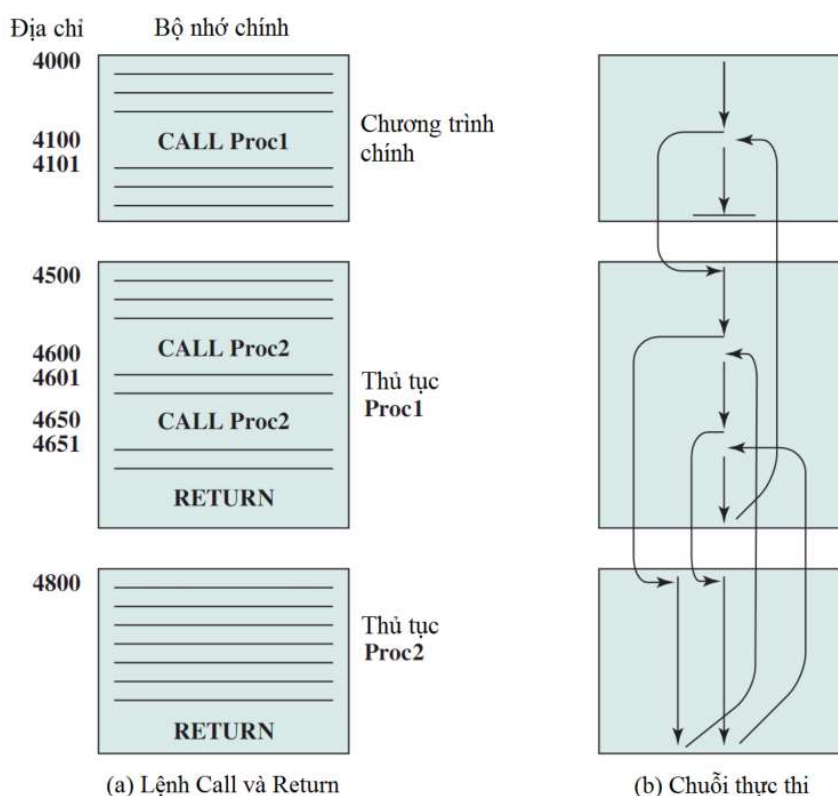
Có lẽ, thủ tục là điểm đổi mới quan trọng nhất trong sự phát triển của ngôn ngữ lập trình. Thủ tục là một chương trình máy tính được kết hợp vào một chương trình lớn hơn.

Thủ tục có thể được gọi tại bất kỳ vị trí nào trong chương trình. Bộ xử lý được chỉ dẫn để đi tới và thực hiện toàn bộ thủ tục rồi sau đó quay trở về điểm phát sinh lệnh gọi thủ tục.

Sử dụng thủ tục có hai ưu điểm là tính kinh tế và tính mô đun. Một thủ tục cho phép sử dụng nhiều lần cùng một đoạn mã. Điều này giúp sử dụng hiệu quả nhất không gian lưu trữ trong hệ thống (chương trình phải được lưu trữ). Thủ tục cũng cho phép công việc lập trình quy mô lớn được chia nhỏ thành các khối công việc nhỏ hơn. Việc này làm giảm đáng kể khối lượng công việc lập trình.

Cơ chế của thủ tục liên quan đến hai lệnh cơ bản: lệnh gọi thủ tục (call) rẽ nhánh từ vị trí hiện tại đến thủ tục, và lệnh trả về (return) để từ thủ tục trở về nơi mà nó được gọi. Cả hai lệnh này đều là các hình thức của lệnh rẽ nhánh.

Hình 2.5a minh họa cho việc sử dụng thủ tục để xây dựng một chương trình. Trong ví dụ này, chương trình chính bắt đầu từ vị trí 4000. Chương trình này gồm có một lệnh gọi thủ tục PROC1, với PROC1 bắt đầu tại vị trí 4500. Khi gặp phải lệnh gọi này, bộ xử lý ngừng thực hiện chương trình chính và bắt đầu thực hiện PROC1 bằng cách truy xuất lệnh tiếp theo tại vị trí 4500. Trong PROC1, có hai lệnh gọi PROC2 biết rằng PROC2 ở vị trí 4800. Trong mỗi trường hợp này, việc thực thi PROC1 bị tạm ngưng và PROC2 được thực thi. Câu lệnh RETURN khiến cho bộ xử lý quay trở lại chương trình gọi và tiếp tục thi hành câu lệnh sau lệnh CALL tương ứng. Hành vi này được minh họa trong hình 2.5b.



Hình 2.5. Thủ tục lồng nhau

Cần lưu ý ba điểm sau:

1. Một thủ tục có thể được gọi từ nhiều vị trí.
2. Một thủ tục có thể gọi một thủ tục khác. Điều này cho phép các thủ tục lồng nhau.
3. Mỗi lệnh gọi thủ tục phải gắn với một lệnh trả về (return) trong chương trình bị gọi.

Do chúng ta muốn gọi một thủ tục từ nhiều điểm trong chương trình, bộ xử lý cần phải lưu lại địa chỉ trả về để có thể trở lại vị trí thích hợp. Có ba nơi để lưu trữ địa chỉ trả về là:

- Thanh ghi
- Điểm bắt đầu thủ tục được gọi
- Đỉnh ngăn xếp

Xét một lệnh ngôn ngữ máy CALL X, là lệnh gọi thủ tục tại vị trí X. Nếu phương án thanh ghi được sử dụng, CALL X tạo ra các hành động sau:

$$RN \leftarrow PC + \Delta$$

$$PC \leftarrow X$$

trong đó RN là thanh ghi luôn được sử dụng cho mục đích này, PC là bộ đếm chương trình, và Δ là chiều dài lệnh. Thủ tục được gọi lúc này có thể lưu lại nội dung thanh ghi RN để sử dụng khi trở lại sau này.

Khả năng thứ hai là lưu trữ địa chỉ trả về tại vị trí bắt đầu thủ tục. Trong trường hợp này, CALL X tương ứng với:

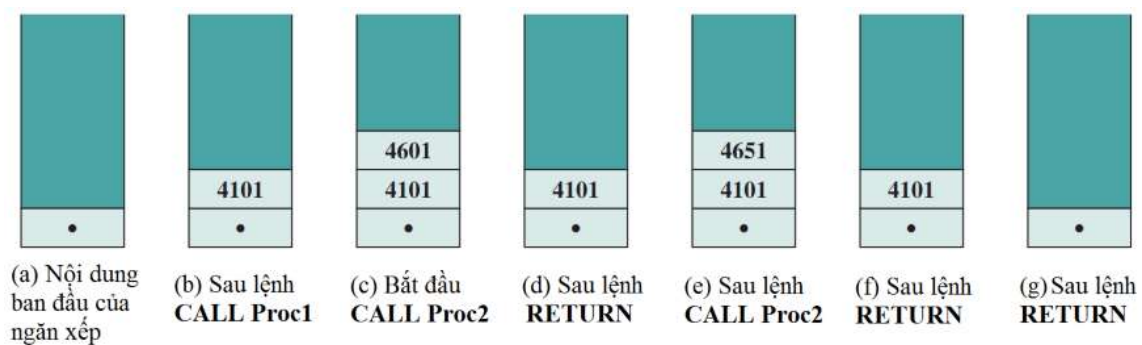
$$X \leftarrow PC + \Delta$$

$$PC \leftarrow X + 1$$

Cách làm này khá tiện lợi. Địa chỉ trả về đã được cất giữ an toàn.

Cả hai phương án này đều hoạt động và đã được sử dụng. Hạn chế duy nhất của các phương án này là chúng làm cho việc sử dụng các thủ tục dùng chung (reentrant) phức tạp hơn. Thủ tục dùng chung là thủ tục mà có thể được mở ra cùng một lúc bởi nhiều lệnh gọi. Thủ tục đệ quy (thủ tục tự gọi chính nó) là một ví dụ về việc sử dụng tính năng này.

Phương pháp tổng quát và mạnh hơn là sử dụng ngăn xếp. Khi bộ xử lý thi hành một lệnh gọi, nó đặt địa chỉ trả về vào ngăn xếp. Khi nó thi hành một lệnh trả về (return), nó sử dụng địa chỉ trên ngăn xếp. Hình 2.6 minh họa việc sử dụng ngăn xếp.



Hình 2.6. Sử dụng ngăn xếp để thực thi các chương trình con lồng nhau ở hình 2.5.

2.3. MÃ HOÁ LỆNH VÀ CÁC CHẾ ĐỘ ĐỊA CHỈ

2.3.1. Mã hoá lệnh

Các lệnh của bộ xử lý được ghi bằng các ký tự dưới dạng gọi nhớ (mnemonic) để người sử dụng dễ nhận biết. Tuy nhiên đối với bộ xử lý thì các lệnh được mã hoá dưới dạng các số 0 và 1 (còn gọi là mã máy), vì đây là dạng biểu diễn thông tin duy nhất mà máy hiểu được. Bởi vậy các lệnh dưới dạng các ký tự gọi nhớ phải được dịch ra mã máy. Việc dịch này được thực hiện bằng các chương trình dịch hợp ngữ.

Vì lệnh cho bộ xử lý được cho dưới dạng mã máy, nên sau khi nhận lệnh bộ xử lý phải thực hiện giải mã lệnh rồi sau đó mới thực hiện lệnh. Một lệnh có thể có độ dài một vài byte tùy theo bộ xử lý. Chẳng hạn, đối với bộ xử lý 8088 một lệnh có độ dài từ 1 - 6 byte. Ví dụ, lệnh MOV dạng mã máy cần ít nhất là 2 byte, trong đó 6 bit của byte đầu dùng để chứa mã lệnh.

Mỗi một lệnh trong tập lệnh đại diện cho một hoạt động cơ bản mà bộ xử lý có khả năng thực hiện. Tất cả các hoạt động mà bộ xử lý có thể thực hiện được biểu diễn dưới dạng tập các lệnh và mỗi lệnh tương ứng với một hoạt động của bộ xử lý. Vì vậy, lệnh là một mã hóa biểu thị

duy nhất tất cả các thông tin cần thiết để thực hiện một chu kỳ lệnh. Dưới đây là các ví dụ về những gì được thể hiện trong một lệnh:

1. Hoạt động phải được thực hiện, ví dụ: cộng hoặc rẽ nhánh.
2. Tên toán hạng nguồn và đích (ví dụ: tên thanh ghi) và các giá trị nếu cần (ví dụ: tức thì trong “add r9, r9, 10”).
3. Cách thức PC được cập nhật. Chẳng hạn: với các lệnh không thuộc điều khiển luồng không chứa bất kỳ thông tin nào về cách cập nhật PC thì PC phải trở thành “PC + 4” (lệnh có độ dài 4 byte); với các lệnh điều khiển luồng sẽ chứa thông tin về độ lệch tới đích trong “beq r9, r0, label”.

Xét máy tính NIOS II với kiến trúc tập lệnh rút gọn (RISC), có một số định dạng lệnh khác nhau như định dạng I, J, R,... Định dạng lệnh xác định thứ tự và loại trường thông tin trong lệnh. Trong đó, định dạng R cung cấp cho ba thanh ghi A, B và C có độ dài 5 bit và trường opcode mở rộng (OPX) gồm 6 bit, cộng với trường khác, X, 5 bit. Các lệnh định dạng R cũng bao gồm trường mã lệnh (OP) của các lệnh định dạng I nhưng sử dụng giá trị duy nhất 0x3A để phân biệt. Vì vậy, khi giải mã, đầu tiên NIOS II xem xét trường OP (bit từ 0 đến 5) và nếu chúng chứa giá trị 0x3A thì nó biết rằng nó phải xét tiếp trường OPX để xác định lệnh là gì. Định dạng lệnh R cụ thể như sau:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
A					B					C					OPX						x					0x3a					

Hình 2.7. Liệt kê tất cả các giá trị OPX và các lệnh tương ứng.

Chẳng hạn lệnh “xor r21, r3, r9” được mã hoá như sau:

C = r21, A = r3 và B = r9, OPX = 0x1E và OP = 0x3A

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
00011					01001					10101					011110						00000					111010					

Hay 0x1a6af03a.

Bảng 2.6. Mã hoá OPX cho các lệnh định dạng R

OPX	Instruction	OPX	Instruction	OPX	Instruction	OPX	Instruction
0x00		0x10	cmplt	0x20	cmpeq	0x30	cmpltu
0x01	eret	0x11		0x21		0x31	add
0x02	roli	0x12	slli	0x22		0x32	
0x03	rol	0x13	sll	0x23		0x33	
0x04	flushp	0x14		0x24	divu	0x34	break
0x05	ret	0x15		0x25	div	0x35	
0x06	nor	0x16	or	0x26	rdctl	0x36	sync
0x07	mulxuu	0x17	mulxsu	0x27	mul	0x37	
0x08	cmpge	0x18	cmpne	0x28	cmpgeu	0x38	
0x09	bret	0x19		0x29	initl	0x39	sub
0x0A		0x1A	srl	0x2A		0x3A	srai
0x0B	ror	0x1B	srl	0x2B		0x3B	sra
0x0C	flushi	0x1C	nextpc	0x2C		0x3C	
0x0D	jmp	0x1D	callr	0x2D	trap	0x3D	
0x0E	and	0x1E	xor	0x2E	wrctl	0x3E	
0x0F		0x1F	mulxss	0x2F		0x3F	

Vậy quá trình giải mã sẽ diễn ra như thế nào khi bộ xử lý nhận được lệnh tiếp theo là 0x1a6af03a hay 0001 1010 0110 1010 1111 0000 0011 1010. Đầu tiên, nó xét 6 bit thấp hơn (trường OP): 111010 sẽ là 0x3A trong hệ hexa, thể hiện định dạng lệnh R, sau đó xét tiếp 6 bit trường OPX: 011110 hay 0x1E là mã lệnh XOR. Thực hiện tương tự đối với việc giải mã các phần còn lại, để cho lệnh “xor r21, r3, r9”.

Như vậy, các yêu cầu đặt ra đối với mã hoá lệnh:

1. Việc mã hoá đảm bảo có thể mã hoá duy nhất (tính một – một) tất cả các kết hợp (hợp lệ) có thể có:
 - a. Hoạt động (cộng và trừ)
 - b. Kiểu dữ liệu (từ, byte)
 - c. Toán hạng nguồn và đích
2. Những tính chất mong muốn:
 - a. Dung lượng bộ nhớ cần thiết để thể hiện một chương trình càng nhỏ càng tốt.
 - b. Giải mã lệnh càng dễ dàng càng tốt.
 - c. Tập lệnh càng đồng nhất càng tốt, nhằm đơn giản hóa phần cứng và tối ưu hóa hoạt động của bộ xử lý.

Thường thì các yêu cầu 2.a và 2.b không giống nhau. Các nhà thiết kế máy tính RISC thường ưu tiên cho 2.b thay vì 2.a: họ đã thiết kế một tập lệnh rất đơn giản để giải mã với chi phí là luôn sử dụng bốn byte cho mỗi lệnh. Các tập lệnh khác như tập lệnh dành cho máy x85 và ARM (bộ xử lý hệ thống nhúng) sử dụng mã hóa lệnh có độ dài thay đổi. Đó là lệnh không sử dụng cùng một số byte để biểu diễn (có ít nhất hai byte nhưng có thể có nhiều hơn).

2.3.2. Các chế độ địa chỉ

Một hay nhiều trường địa chỉ trong định dạng lệnh thông thường có kích thước tương đối nhỏ. Tuy nhiên, chúng ta luôn muốn tham chiếu đến một dải địa chỉ lớn của các vị trí trong bộ nhớ chính hoặc bộ nhớ ảo. Để đạt được mục tiêu này, người ta đã sử dụng một loạt các kỹ thuật địa chỉ. Tuy nhiên, việc sử dụng các kỹ thuật này sẽ dẫn đến một sự đánh đổi giữa: (1) dải địa chỉ và/hoặc tính linh hoạt; (2) số lượng các tham chiếu bộ nhớ trong một lệnh và/hoặc sự phức tạp trong việc tính toán địa chỉ. Trong phần này, chúng ta sẽ tìm hiểu các kỹ thuật hay chế độ địa chỉ phổ biến nhất (Bảng 2.7):

- Chế độ tức thì
- Chế độ trực tiếp
- Chế độ gián tiếp
- Chế độ thanh ghi
- Chế độ gián tiếp thanh ghi
- Chế độ dịch chuyển
- Chế độ ngăn xếp

Trước khi tìm hiểu về các chế độ địa chỉ, có hai vấn đề cần được giải thích. Thứ nhất, hầu như tất cả các kiến trúc máy tính đều hỗ trợ nhiều chế độ địa chỉ. Câu hỏi đặt ra là làm sao để bộ xử lý quyết định chế độ địa chỉ nào đang được sử dụng với một lệnh cụ thể. Có một số kỹ thuật để giải quyết vấn đề này. Thông thường các opcode (mã lệnh) khác nhau sẽ sử dụng các chế độ địa chỉ khác nhau. Ngoài ra, trong định dạng lệnh có thể có một trường mode bao gồm một hoặc nhiều bit. Giá trị của trường này xác định chế độ địa chỉ nào được sử dụng.

Vấn đề thứ hai liên quan đến việc phân giải địa chỉ hiệu dụng (EA). Với một hệ thống không có bộ nhớ ảo, địa chỉ hiệu dụng sẽ là địa chỉ bộ nhớ chính hoặc địa chỉ thanh ghi. Trong hệ

thống có sử dụng bộ nhớ ảo, địa chỉ hiệu dụng là địa chỉ ảo hoặc địa chỉ thanh ghi. Việc ánh xạ từ địa chỉ ảo đến địa chỉ vật lý là chức năng của đơn vị quản lý bộ nhớ (MMU - Memory Management Unit), lập trình viên không thể thấy được.

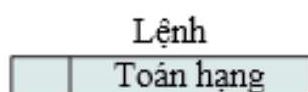
Bảng 2.7. Các chế độ địa chỉ cơ bản

Chế độ	Thuật toán	Ưu điểm	Nhược điểm
Tức thì	Toán hạng = A	Không cần tham chiếu bộ nhớ	Hạn chế về giá trị của toán hạng
Trực tiếp	EA = A	Đơn giản	Không gian địa chỉ hạn chế
Gián tiếp	EA = (A)	Không gian địa chỉ lớn	Tham chiếu bộ nhớ nhiều lần
Thanh ghi	EA = R	Không cần tham chiếu bộ nhớ	Không gian địa chỉ hạn chế
Gián tiếp thanh ghi	EA = A + (R)	Linh hoạt	Phức tạp
Ngăn xếp	EA = đỉnh ngăn xếp	Không cần tham chiếu bộ nhớ	Khả năng ứng dụng ít

2.3.2.1. Địa chỉ tức thì

Dạng đơn giản nhất của chế độ địa chỉ là địa chỉ tức thì, trong đó giá trị toán hạng được đặt luôn trong lệnh:

Toán hạng = A



Hình 2.8. Địa chỉ tức thì

Chế độ này có thể được sử dụng để định nghĩa và sử dụng các hằng số hoặc thiết lập các giá trị ban đầu cho biến. Thông thường, số sẽ được lưu trữ dưới dạng bù hai với một bit dấu. Khi toán hạng được nạp vào thanh ghi dữ liệu, bit dấu được mở rộng sang bên trái để điền đủ vào kích thước thanh ghi. Trong một số trường hợp, giá trị nhị phân tức thì được hiểu là một số nguyên dương không dấu.

Ưu điểm của chế độ này là: trong một chu kỳ lệnh, chỉ cần tham chiếu bộ nhớ để lấy lệnh mà không cần tham chiếu bộ nhớ để lấy toán hạng, do đó tiết kiệm một chu kỳ bộ nhớ hoặc chu kỳ cache. Nhược điểm của chế độ này là kích thước của số bị hạn chế vì chỉ bằng kích thước của trường địa chỉ, mà đối với hầu hết các tập lệnh, kích thước này nhỏ hơn so với độ dài một word (từ nhớ).

Ví dụ:

ADD R1, 5 ; R1 ← R1 + 5

Trong đó, 5 là một toán hạng nguồn được tham chiếu trực tiếp trong câu lệnh: địa chỉ tức thì

2.3.2.2. Địa chỉ trực tiếp

Một dạng khác cũng khá đơn giản là chế độ địa chỉ trực tiếp, trong đó trường địa chỉ chứa địa chỉ hiệu dụng của toán hạng:

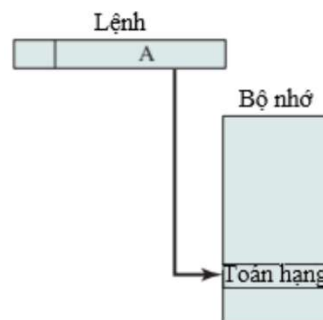
$$EA = A$$

Ví dụ:

ADD R1, A ; R1 ← R1 + (A)

trong đó, A là địa chỉ một vị trí bộ nhớ, (A) là nội dung của vị trí đó. CPU tham chiếu đến toán hạng có địa chỉ A trong bộ nhớ.

Kỹ thuật này khá phổ biến trong các máy tính thế hệ trước đây nhưng ít được sử dụng trong các kiến trúc hiện đại. Chế độ này cần có một tham chiếu bộ nhớ và không có gì đặc biệt. Nhược điểm của chế độ này là không gian địa chỉ hạn chế.



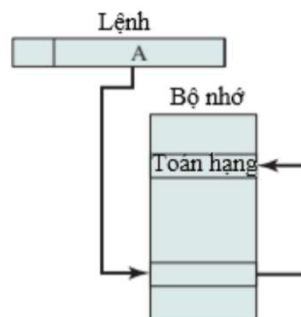
Hình 2.9. Địa chỉ trực tiếp

2.3.2.3. Địa chỉ gián tiếp

Với chế độ địa chỉ trực tiếp, kích thước trường địa chỉ thường ngắn hơn chiều dài từ, do đó hạn chế dài địa chỉ có thể tham chiếu tới. Một giải pháp cho vấn đề này là trường địa chỉ sẽ tham chiếu đến một word trong bộ nhớ mà nội dung của word này chứa địa chỉ toán hạng cần tham chiếu tới. Chế độ này được gọi là chế độ địa chỉ gián tiếp:

$$EA = (A)$$

Như trên đã định nghĩa, dấu ngoặc đơn được hiểu là nội dung của. Ưu điểm của kỹ thuật này là với một từ có độ dài N-bit, không gian địa chỉ là 2^N . Nhược điểm của nó là quá trình thực hiện lệnh đòi hỏi phải tham chiếu bộ nhớ hai lần mới lấy được toán hạng: một lần để lấy địa chỉ toán hạng và một lần lấy giá trị toán hạng.



Hình 2.10. Địa chỉ gián tiếp

Một cải tiến khác của kỹ thuật này là địa chỉ gián tiếp nhiều cấp hoặc địa chỉ gián tiếp nhiều tầng:

$$EA = (\dots (A) \dots)$$

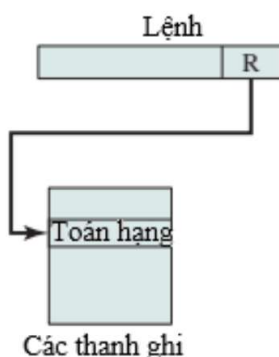
Trong trường hợp này, một bit trong từ nhớ địa chỉ là một cờ gián tiếp (I). Nếu cờ I là 0, thì địa chỉ trong từ đó là địa chỉ hiệu dụng EA. Nếu bit cờ I bằng 1 thì đây vẫn là một địa chỉ gián tiếp. Rõ ràng, việc sử dụng chế độ địa chỉ nhiều tầng này không có ưu điểm nào mà lại có nhược điểm rất lớn đó là số lần truy cập bộ nhớ để lấy ra toán hạng nhiều do đó cải tiến này không được sử dụng nhiều.

2.3.2.4. Địa chỉ thanh ghi

Chế độ địa chỉ thanh ghi cũng tương tự như chế độ địa chỉ trực tiếp. Điểm khác nhau duy nhất là trường địa chỉ dùng để tham chiếu đến thanh ghi chứ không phải tham chiếu đến bộ nhớ chính:

$$EA = R$$

Ví dụ, trường địa chỉ của một lệnh là 5 và lệnh này sử dụng chế độ địa chỉ thanh ghi thì giá trị toán hạng của lệnh nằm trong thanh ghi 5 (R5). Thông thường, một trường địa chỉ tham chiếu đến thanh ghi sẽ có từ 3 đến 5 bit, như vậy có thể tham chiếu đến từ 8 đến 32 thanh ghi đa nhiệm.



Hình 2.11. Địa chỉ thanh ghi

Các ưu điểm của chế độ địa chỉ thanh ghi là (1) trường địa chỉ của lệnh chỉ cần kích thước nhỏ, và (2) không mất một khoảng thời gian tham chiếu bộ nhớ. Thời gian truy cập vào thanh ghi trong bộ xử lý ít hơn nhiều so với thời gian tham chiếu đến một địa chỉ bộ nhớ chính. Nhược điểm của địa chỉ thanh ghi là không gian địa chỉ là rất hạn chế (ít thanh ghi).

Do số lượng thanh ghi hạn chế (ít hơn rất nhiều so với số lượng vị trí bộ nhớ chính) nên việc sử dụng chúng theo cách này chỉ có ý nghĩa nếu chúng được sử dụng hiệu quả. Nếu một toán hạng được nạp vào thanh ghi từ bộ nhớ, sau đó nó chỉ được gọi một lần trong hoạt động và ghi lại vào bộ nhớ thì rõ ràng kỹ thuật này không hiệu quả và còn mất thời gian hơn so với kỹ thuật địa chỉ trực tiếp. Do đó, chế độ này thường được sử dụng khi một toán hạng được gọi đi gọi lại trong nhiều hoạt động, ví dụ một kết quả trung gian trong một phép tính.

Việc quyết định giá trị nào được giữ lại trong thanh ghi và giá trị nào cần được lưu trữ trong bộ nhớ chính sẽ do trình lập trình hoặc trình biên dịch đưa ra. Hầu hết các bộ xử lý hiện đại đều sử dụng nhiều thanh ghi đa năng nên việc sử dụng chúng sao cho hiệu quả là một vấn đề mà các trình hợp ngữ phải đối mặt.

Ví dụ:

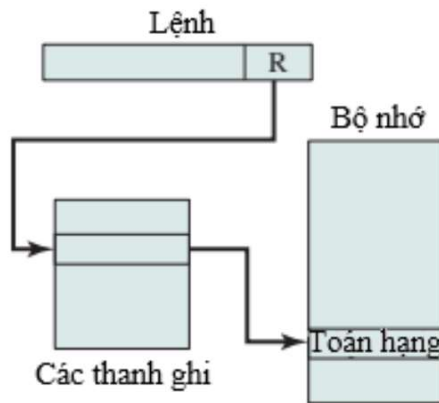
ADD R1, A ; R1 ← R1 + (A)

Trong đó, R1 là một thanh ghi trong bộ xử lý, câu lệnh trên tham chiếu giá trị (A) trong bộ nhớ cộng với giá trị lưu trữ trong thanh ghi R1, kết quả được ghi vào R1.

2.3.2.5. Địa chỉ gián tiếp thanh ghi

Chế độ này cũng tương tự như chế độ địa chỉ gián tiếp, chỉ có một điểm khác biệt là trường địa chỉ của lệnh tham chiếu đến một thanh ghi, thanh ghi này chứa địa chỉ hiệu dụng của toán hạng trong bộ nhớ (khác với chế độ địa chỉ gián tiếp sử dụng một vị trí bộ nhớ để chứa địa chỉ hiệu dụng). Thanh ghi này được gọi là con trỏ. Ta có:

$$EA = (R)$$



Hình 2.12. Địa chỉ gián tiếp thanh ghi

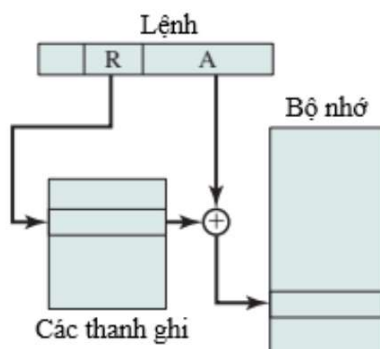
Giống như chế độ địa chỉ gián tiếp, việc sử dụng thanh ghi lưu trữ địa chỉ hiệu dụng tăng không gian địa chỉ của lệnh. Thêm vào đó, kỹ thuật này có tốc độ nhanh hơn do chỉ cần một lần truy xuất bộ nhớ (so với hai lần truy xuất bộ nhớ của chế độ gián tiếp).

2.3.2.6. Địa chỉ dịch chuyển

Đây là một chế độ địa chỉ rất mạnh mẽ kết hợp giữa địa chỉ trực tiếp và địa chỉ gián tiếp thanh ghi. Nó được biết với nhiều tên khác nhau tùy thuộc vào bối cảnh sử dụng nhưng về cơ chế là như nhau. Chúng tôi tạm gọi chế độ này là chế độ địa chỉ dịch chuyển:

$$EA = A + (R)$$

Kỹ thuật này đòi hỏi định dạng lệnh phải có hai trường địa chỉ hoặc phải có một địa chỉ ngầm định. Giá trị của một trường địa chỉ là A, được sử dụng trực tiếp. Trường địa chỉ còn lại hoặc địa chỉ ngầm định (thường quy định theo opcode) tham chiếu đến một thanh ghi. Giá trị lưu trữ trong thanh ghi cộng với A cho ta địa chỉ hiệu dụng của toán hạng.



Hình 2.13. Địa chỉ dịch chuyển

Có ba cách sử dụng phổ biến nhất của chế độ địa chỉ dịch chuyển:

- Địa chỉ tương đối
- Địa chỉ thanh ghi cơ sở

- Địa chỉ indexing

ĐỊA CHỈ TƯƠNG ĐỐI hay còn được gọi là địa chỉ PC-tương đối, thanh ghi tham chiếu được ngầm định là thanh ghi PC. Nghĩa là, địa chỉ lệnh tiếp theo (nội dung PC) được cộng với trường địa chỉ của lệnh để tạo ra EA. Thông thường, trường địa chỉ được coi là một số bù hai trong hoạt động này. Như vậy, địa chỉ hiệu dụng của toán hạng dịch chuyển một đoạn so với địa chỉ của lệnh.

Địa chỉ tương đối khai thác tính cục bộ của các tham chiếu bộ nhớ. Nếu hầu hết các toán hạng được tham chiếu gần với lệnh đang thực hiện, thì việc sử dụng địa chỉ tương đối sẽ giúp giảm bớt số lượng bit địa chỉ cần thiết trong lệnh.

ĐỊA CHỈ THANH GHI CƠ SỞ chế độ này được giải thích là như sau: nội dung của thanh ghi tham chiếu là một địa chỉ bộ nhớ, trường địa chỉ của lệnh chứa một giá trị dịch chuyển (thường là số nguyên không dấu) so với địa chỉ đó. Thanh ghi được tham chiếu có thể được chỉ rõ trong lệnh hoặc ngầm định.

Địa chỉ thanh ghi cơ sở cũng khai thác tính địa phương của các tham chiếu bộ nhớ. Chế độ này cũng giúp thực hiện một kỹ thuật phân đoạn bộ nhớ chính. Ở một số hệ thống, một thanh ghi đoạn ngầm định sẽ chứa địa chỉ đoạn. Một số hệ thống khác, người lập trình có thể lựa chọn một thanh ghi chứa địa chỉ cơ sở đoạn và lệnh phải tham chiếu đến thanh ghi này. Trong trường hợp thứ hai, nếu chiều dài của trường địa chỉ là K và số lượng thanh ghi là N, một lệnh có thể tham chiếu đến một vùng bất kỳ trong N vùng, mỗi vùng có 2K từ.

ĐỊA CHỈ CHỈ MỤC (INDEXING): Trường địa chỉ tham chiếu đến một địa chỉ bộ nhớ chính và đến một thanh ghi chứa một giá trị dịch chuyển dương từ địa chỉ đó. Ta có thể thấy kỹ thuật này ngược lại so với kỹ thuật địa chỉ thanh ghi cơ sở. Do trường địa chỉ của lệnh tham chiếu đến một địa chỉ bộ nhớ nên cần nhiều bit hơn trường địa chỉ tham chiếu đến thanh ghi cơ sở. Tuy nhiên, cả hai kỹ thuật đều có cách tính EA giống nhau và thanh ghi tham chiếu có thể là một thanh ghi ngầm định hoặc cần phải chỉ rõ trong lệnh (tùy thuộc vào từng bộ xử lý khác nhau).

Một ứng dụng quan trọng của chế độ indexing là nó cung cấp một cơ chế hiệu quả để thực hiện các hoạt động lặp đi lặp lại. Ví dụ, một mảng các số được lưu trữ bắt đầu từ vị trí A. Giả sử ta muốn cộng thêm 1 vào mỗi phần tử trong danh sách, ta phải lấy từng giá trị, thực hiện phép cộng với 1 sau đó lưu giá trị lại vào vị trí đó. Như vậy, ta cần thực hiện hoạt động với từng phần tử có địa chỉ hiệu dụng lần lượt là A, A + 1, A + 2, ..., đến vị trí cuối cùng trong mảng. Với chế độ indexing, các hoạt động này có thể được thực hiện dễ dàng.

Giá trị A sẽ được lưu trữ trong trường địa chỉ của lệnh, một thanh ghi được chọn là thanh ghi index và được khởi tạo giá trị bằng 0. Sau mỗi hoạt động, thanh ghi index tăng lên 1 đơn vị.

Vì các thanh ghi chỉ mục (index register) thường được sử dụng cho các tác vụ lặp nên nó thường phải được tăng hoặc giảm giá trị sau mỗi lần được tham chiếu. Vì hoạt động này khá phổ biến nên một số hệ thống sẽ tự động làm điều này như một phần của chu kỳ lệnh, được gọi là tự động lập chỉ mục (autoindexing). Nghĩa là nếu thanh ghi nào đó được dành riêng cho chỉ mục, hoạt động tự động lập chỉ mục được thực hiện ngầm định và tự động. Nếu sử dụng thanh ghi đa dụng làm thanh ghi chỉ mục thì hoạt động tự động lập chỉ mục được thiết lập bằng một bit báo hiệu trong lệnh.

Như vậy, chế độ lập chỉ mục như sau:

$$EA = A + (R)$$

$$(R) + 1 \rightarrow (R)$$

Một số hệ thống máy tính cho phép sử dụng cả chế độ địa chỉ gián tiếp và địa chỉ chỉ mục và có thể sử dụng cả hai chế độ này trong cùng một lệnh. Có hai khả năng: việc định địa chỉ theo chế độ chỉ mục có thể thực hiện trước hoặc sau định địa chỉ gián tiếp.

Nếu được thực hiện sau chế độ địa chỉ gián tiếp, ta gọi là lập chỉ mục sau (postindexing):

$$EA = (A) + (R)$$

Trường địa chỉ của lệnh sẽ trỏ đến một word trong bộ nhớ, nội dung của word này là địa chỉ một từ khác trong bộ nhớ. Địa chỉ này cộng với giá trị thanh ghi chỉ mục sẽ là địa chỉ toán hạng cần truy xuất. Kỹ thuật này rất hữu ích để truy cập các khối dữ liệu có định dạng cố định. Ví dụ, hệ điều hành cần thực hiện các hoạt động giống nhau với mỗi khối dữ liệu, các địa chỉ trong các lệnh tham chiếu đến khối cần trỏ đến một vị trí đầu tiên của khối (địa chỉ = A). Thanh ghi chỉ mục chứa giá trị dịch chuyển đến các vị trí bộ nhớ trong khối đó.

Nếu chế độ chỉ mục được thực hiện trước chế độ gián tiếp, ta gọi là lập chỉ mục trước (preindexing):

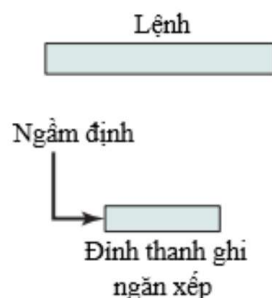
$$EA = (A + (R))$$

Kỹ thuật này thực hiện việc lập chỉ mục đến một địa chỉ giống như chế độ chỉ mục thông thường. Tuy nhiên, giá trị được tham chiếu đến chưa phải là toán hạng, mà là địa chỉ của toán hạng. Ví dụ về việc sử dụng kỹ thuật này là để xây dựng một bảng rẽ nhánh đa đường. Tại một điểm cụ thể trong một chương trình, ta cần phải rẽ nhánh đến một trong các vị trí nhất định tùy thuộc vào điều kiện. Một bảng các địa chỉ được thiết lập bắt đầu từ vị trí A. Bằng cách lập chỉ mục vào bảng này, ta có thể tìm thấy vị trí cần thiết. Thông thường, một tập lệnh sẽ không bao gồm cả lập chỉ mục trước và sau.

2.3.2.7. Địa chỉ ngăn xếp

Chế độ địa chỉ cuối cùng được đề cập tới là địa chỉ ngăn xếp. Ngăn xếp là một mảng các vị trí trong bộ nhớ, còn được gọi là hàng đợi vào sau ra trước – LIFO. Cơ chế làm việc của ngăn xếp như sau: các phần tử sẽ được đưa vào vùng ngăn xếp với cơ chế vào sau ra trước, một con trỏ luôn tham chiếu đến vị trí đỉnh của ngăn xếp (con trỏ này là giá trị địa chỉ được lưu trữ trong một thanh ghi – thanh ghi SP – Stack Pointer). Ngoài ra, trong một số trường hợp, hai phần tử trên cùng của ngăn xếp có thể được lưu trữ trên hai thanh ghi của bộ xử lý, như vậy, phần tử ở đỉnh ngăn xếp là phần tử thứ ba. Chế độ địa chỉ tham chiếu đến các vị trí trong bộ nhớ ngăn xếp về bản chất là chế độ địa chỉ gián tiếp thanh ghi.

Chế độ địa chỉ ngăn xếp là một dạng địa chỉ ngầm định. Các lệnh của máy không cần phải tham chiếu đến địa chỉ bộ nhớ ngăn xếp mà nó sẽ mặc định thao tác với đỉnh ngăn xếp.



Hình 2.14. Địa chỉ ngăn xếp

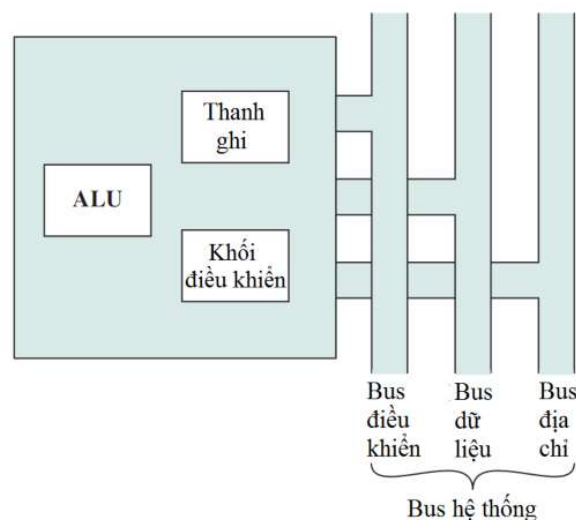
2.4. CẤU TRÚC CPU

2.4.1. Tổ chức của CPU

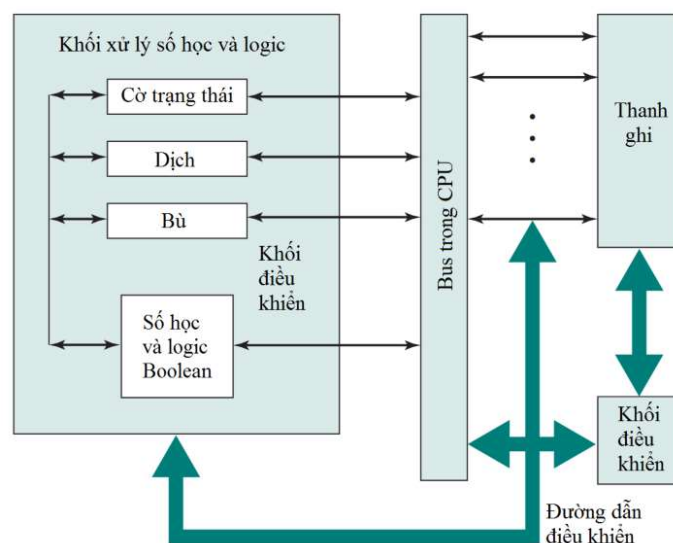
Để hiểu hơn về tổ chức của CPU, ta xét các nhiệm vụ mà nó phải thực hiện:

- Truy xuất lệnh: Bộ xử lý đọc lệnh từ bộ nhớ (thanh ghi, bộ nhớ cache, bộ nhớ chính).
- Giải mã lệnh: Lệnh được giải mã để xác định hành động nào được yêu cầu.
- Truy xuất dữ liệu: Việc thực thi một lệnh có thể yêu cầu đọc dữ liệu từ bộ nhớ hoặc một mô-đun I/O.
- Xử lý dữ liệu: Việc thực thi một lệnh có thể yêu cầu thực hiện một số phép tính số học hoặc logic trên dữ liệu.
- Ghi dữ liệu: Kết thúc việc thực hiện có thể yêu cầu ghi dữ liệu vào bộ nhớ hoặc một mô-đun I/O.

Để thực hiện những việc này, rõ ràng là CPU cần phải lưu tạm thời một số dữ liệu. Nó phải nhớ vị trí của lệnh gần nhất vừa thực thi để có thể biết được nơi truy xuất lệnh tiếp theo. Nó cần lưu trữ tạm thời các lệnh và dữ liệu trong khi một lệnh đang được thực thi. Nói cách khác, CPU cần một bộ nhớ nhỏ bên trong nó.



Hình 2.15. CPU và bus hệ thống



Hình 2.16. Cấu trúc bên trong CPU

Hình 2.15 là sơ đồ khối đơn giản của CPU, cho thấy kết nối của nó với phần còn lại của hệ thống thông qua bus hệ thống. Như đã đề cập ở chương 1, các thành phần chính của bộ xử lý là *khối số học và logic* (ALU) và *khối điều khiển* (CU). ALU thực hiện tính toán hoặc xử lý dữ liệu. Khối điều khiển kiểm soát việc di chuyển dữ liệu và lệnh vào và ra khỏi bộ xử lý và điều khiển hoạt động của ALU. Ngoài ra, hình 2.15 cũng thể hiện bộ nhớ bên trong CPU, bao gồm một tập hợp các vị trí lưu trữ, được gọi là *thanh ghi*.

Hình 2.16 là sơ đồ khối chi tiết hơn của CPU. Các đường truyền dữ liệu và các đường điều khiển logic được thể hiện, bao gồm một thành phần được gọi là bus trong CPU. Thành phần này là cần thiết để truyền dữ liệu giữa các thanh ghi khác nhau với ALU vì trên thực tế, ALU chỉ hoạt động trên dữ liệu nằm trong bộ nhớ nội bộ của bộ xử lý. Hình vẽ này cũng cho thấy các thành phần cơ bản điển hình của ALU.

2.4.2. Tổ chức thanh ghi

Như đã đề cập, hệ thống máy tính sử dụng một hệ thống phân cấp bộ nhớ. Bộ nhớ ở cấp độ cao hơn trong hệ phân cấp thì nhanh hơn, nhỏ hơn và đắt hơn (trên mỗi bit). Tập hợp các thanh ghi trong CPU có chức năng như một cấp độ bộ nhớ phía trên bộ nhớ chính và bộ nhớ đệm trong hệ thống phân cấp. Các thanh ghi trong CPU thực hiện hai vai trò:

- Thanh ghi hiển thị với người dùng: Cho phép lập trình viên hợp ngữ hoặc ngôn ngữ máy giảm thiểu các tham chiếu bộ nhớ chính bằng cách tối ưu hóa việc sử dụng thanh ghi.
- Thanh ghi điều khiển và trạng thái: Được khối điều khiển sử dụng để điều khiển hoạt động của CPU và các chương trình hệ điều hành sử dụng để kiểm soát việc thực thi chương trình.

Tuy nhiên không thể phân loại rõ ràng các thanh ghi vào hai loại này. Ví dụ, trên một số máy tính, bộ đếm chương trình là hiển thị với người dùng (ví dụ, x86), nhưng trong nhiều máy khác thì không hiển thị. Tuy nhiên, ta sẽ sử dụng các hai loại này để tiện cho việc trình bày nội dung tiếp theo.

2.4.2.1. Thanh ghi hiển thị với người dùng

Thanh ghi hiển thị với người dùng là thanh ghi được tham chiếu bằng ngôn ngữ máy mà CPU thực hiện. Ta có thể thể phân loại các thanh ghi này thành các loại sau:

- Đa năng
- Dữ liệu
- Địa chỉ
- Mã điều kiện

Thanh ghi đa năng có thể được lập trình viên gán cho nhiều chức năng khác nhau. Đôi khi việc sử dụng chúng trong tập lệnh là trực giao với hoạt động. Tức là, thanh ghi đa năng bất kỳ có thể chứa toán hạng cho bất kỳ opcode nào. Tuy nhiên, việc sử dụng thanh ghi đa năng thường có những hạn chế. Ví dụ, có thể có thanh ghi chuyên dụng cho các phép toán dấu chấm động và ngăn xếp.

Trong một số trường hợp, thanh ghi đa năng có thể được sử dụng cho chức năng định địa chỉ (ví dụ: thanh ghi gián tiếp, thanh ghi dịch). Trong trường hợp khác, thanh ghi dữ liệu và thanh ghi địa chỉ được phân biệt một phần hoặc phân biệt hoàn toàn. Thanh ghi dữ liệu có thể được sử dụng chỉ để chứa dữ liệu và không được sử dụng trong tính toán địa chỉ toán hạng. Thanh ghi địa chỉ có thể là đa năng, hoặc là chuyên dụng cho một chế độ địa chỉ cụ thể. Ví dụ, một vài thanh ghi địa chỉ gồm:

- Con trỏ segment: Trong máy sử dụng địa chỉ phân đoạn, thanh ghi segment chứa địa chỉ của base của segment. Có thể có nhiều thanh ghi: chẳng hạn, một cho hệ điều hành và một cho tiến trình đang chạy
- Thanh ghi chỉ số: được sử dụng trong địa chỉ chỉ mục và chỉ mục tự động.
- Con trỏ ngăn xếp: Nếu sử dụng chế độ địa chỉ ngăn xếp thì thường có một thanh ghi chuyên dụng trỏ tới đầu ngăn xếp. Điều này cho phép sử dụng địa chỉ ngầm định; tức là, lệnh push, pop và các lệnh ngăn xếp khác không cần phải có toán hạng ngăn xếp rõ ràng.

Loại thanh ghi cuối cùng, hiển thị ít nhất một phần với người dùng, chứa mã điều kiện (còn được gọi là cờ). Mã điều kiện là các bit mà phần cứng của bộ xử lý thiết lập theo kết quả của hành động. Ví dụ, một phép tính số học có thể tạo ra kết quả dương, âm, bằng không, hoặc tràn. Ngoài việc kết quả được lưu trữ trong thanh ghi hoặc bộ nhớ, một mã điều kiện cũng được thiết lập tương ứng. Sau đó, mã này có thể được sử dụng trong lệnh rẽ nhánh có điều kiện.

Các bit mã điều kiện được tập hợp lại trong một hoặc nhiều thanh ghi. Thông thường, chúng tạo thành một phần của thanh ghi điều khiển. Nói chung, các lệnh máy cho phép đọc các bit này bằng tham chiếu ngầm định, nhưng lập trình viên không thể thay đổi chúng.

Trong một số máy, một lệnh gọi chương trình con sẽ dẫn đến việc tắt cả các thanh ghi hiển thị với người dùng được tự động lưu lại, để có thể khôi phục khi trở về. CPU thực hiện việc lưu và khôi phục thanh ghi này như một phần công việc trong quá trình thực thi các lệnh gọi và trả về. Điều này cho phép các chương trình con sử dụng thanh ghi hiển thị với người dùng một cách độc lập. Trên các máy khác, người lập trình có trách nhiệm lưu lại nội dung của các thanh ghi hiển thị với người dùng trước một lệnh gọi chương trình con, bằng cách thêm vào chương trình các lệnh phù hợp.

2.4.2.2. Thanh ghi điều khiển và trạng thái

Nhiều thanh ghi CPU được sử dụng để điều khiển hoạt động của CPU. Phần lớn thanh ghi không hiển thị với người dùng. Một số khác có thể hiển thị với các lệnh máy được thực hiện trong chế độ điều khiển hoặc chế độ hệ điều hành.

Tất nhiên, các máy khác nhau sẽ có tổ chức thanh ghi khác nhau và sử dụng các thuật ngữ khác nhau. Bốn thanh ghi sau đây là cần thiết để thực thi lệnh:

- Bộ đếm chương trình (PC): Chứa địa chỉ của lệnh sắp được truy xuất
- Thanh ghi lệnh (IR): Chứa lệnh vừa được truy xuất gần đây nhất
- Thanh ghi địa chỉ bộ nhớ (MAR): Chứa địa chỉ của một vị trí trong bộ nhớ
- Thanh ghi đệm bộ nhớ (MBR): Chứa một từ dữ liệu sắp được ghi vào bộ nhớ hoặc từ vừa được đọc ra.

Không phải mọi CPU đều có các thanh ghi kiểu MAR và MBR. Tuy nhiên, mọi CPU cần phải có một cơ chế đệm tương tự như MAR và MBR sao cho các bit truyền lên bus hệ thống được phân tầng và các bit đọc từ bus dữ liệu được lưu tạm thời.

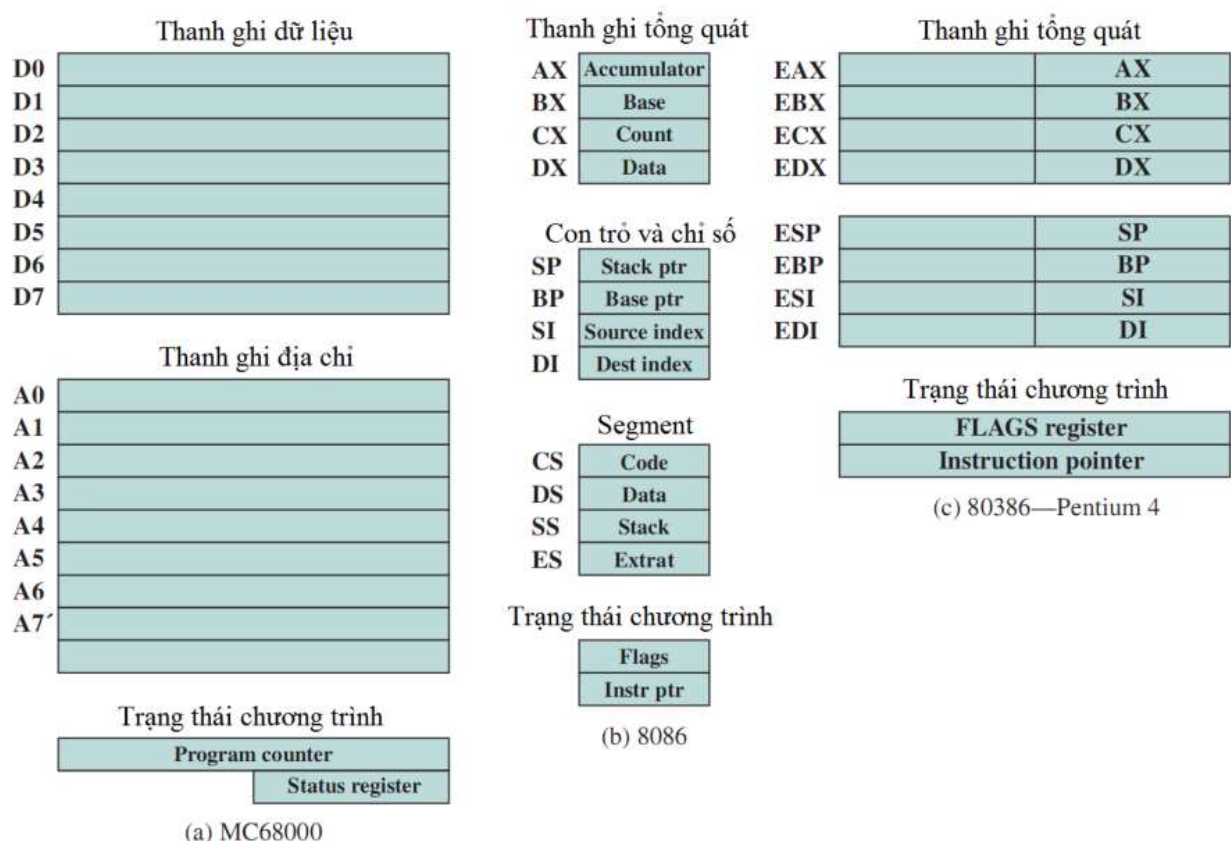
Thông thường, CPU cập nhật PC sau mỗi lần truy xuất lệnh để PC luôn trỏ đến lệnh tiếp theo sẽ được thực thi. Lệnh rẽ nhánh hoặc lệnh skip cũng sẽ thay đổi nội dung của PC. Lệnh đã truy xuất được tải vào thanh ghi IR, nơi phân tích các opcode và nhận diện toán hạng. Dữ liệu được trao đổi với bộ nhớ nhờ thanh ghi MAR và MBR. Trong một hệ thống sử dụng bus, MAR kết nối trực tiếp tới bus địa chỉ, và MBR kết nối trực tiếp với bus dữ liệu. Các thanh ghi hiển thị với người dùng trao đổi dữ liệu với MBR.

Bốn thanh ghi này được sử dụng trong việc di chuyển dữ liệu giữa bộ xử lý và bộ nhớ. Trong CPU, dữ liệu phải được đưa đến ALU để xử lý. ALU có thể có quyền truy cập trực tiếp vào MBR và các thanh ghi hiển thị với người dùng. Ngoài ra, có thể có thêm các thanh ghi đệm đặt gần ALU; các thanh ghi này phục vụ như thanh ghi đầu vào và đầu ra cho ALU và trao đổi dữ liệu với MBR và các thanh ghi hiển thị với người dùng.

Nhiều CPU được thiết kế bao gồm một thanh ghi hoặc một tập hợp thanh ghi gọi là từ trạng thái chương trình (program status word - PSW), có chứa thông tin trạng thái. PSW thường chứa mã điều kiện cùng với các thông tin trạng thái khác. Các trường hoặc cờ phổ biến bao gồm:

- Sign: Chứa bit dấu của kết quả của phép tính số học cuối cùng.
- Zero: Thiết lập khi kết quả bằng 0.
- Carry: Thiết lập nếu một phép tính có nhớ (phép cộng) hoặc vay (phép trừ) vào bit có bậc lớn hơn. Được sử dụng cho các phép tính số học nhiều từ.
- Equal: Thiết lập nếu kết quả so sánh logic là bằng nhau.
- Overflow: Được sử dụng để chỉ định sự tràn số học.
- Interrupt Enable/Disable: Được sử dụng để cho phép hoặc vô hiệu hoá gián đoạn.
- Supervisor: Cho biết bộ xử lý đang thực hiện trong chế độ giám sát hay chế độ người dùng. Một số lệnh privileged chỉ có thể được thực hiện trong chế độ giám sát, và một số vùng bộ nhớ chỉ có thể được truy cập trong chế độ giám sát.

2.4.3. Ví dụ tổ chức thanh ghi CPU



Hình 2.17. Ví dụ tổ chức thanh ghi vi xử lý

Ta sẽ dễ dàng hình dung hơn nếu xem xét và so sánh tổ chức thanh ghi của một số hệ thống. Trong phần này, ta xem xét hai vi xử lý 16 bit được thiết kế trong cùng khoảng thời gian: Motorola MC68000 và Intel 8086. Hình 2.17a và b mô tả tổ chức thanh ghi của từng vi xử lý; các thanh ghi nội bộ thuần túy, chẳng hạn như MAR, không được thể hiện.

MC68000 phân chia các thanh ghi 32 bit của nó thành 8 thanh ghi dữ liệu và 9 thanh ghi địa chỉ. 8 thanh ghi dữ liệu được sử dụng chủ yếu cho thao tác dữ liệu và cũng được sử dụng để định địa chỉ (như thanh ghi index). Độ rộng của thanh ghi cho phép các hoạt động dữ liệu 8, 16 và 32 bit, được xác định bởi opcode. Các thanh ghi địa chỉ chứa địa chỉ 32 bit (không phân đoạn); hai trong số các thanh ghi này cũng được sử dụng làm con trỏ ngăn xếp, một cho người dùng và một cho hệ điều hành, tùy thuộc vào chế độ thực thi hiện tại. Cả hai thanh ghi này được đánh số 7, bởi vì tại một thời điểm chỉ có thể sử dụng một trong hai. MC68000 cũng có thanh ghi PC 32 bit và thanh ghi trạng thái 16 bit.

Intel 8086 có cách tổ chức thanh ghi khác. Mỗi thanh ghi có mục đích riêng, mặc dù một số thanh ghi cũng có thể được sử dụng như là đa năng. 8086 chứa bốn thanh ghi dữ liệu 16 bit, có thể đánh địa chỉ cho từng byte hoặc từng 16 bit, và bốn thanh ghi index và con trỏ 16 bit. Trong một số lệnh, các thanh ghi dữ liệu có thể là đa năng. Trong các lệnh khác, thanh ghi được sử dụng ngầm định. Ví dụ, lệnh nhân luôn sử dụng thanh ghi tích lũy (accumulator). Bốn thanh ghi con trỏ cũng được sử dụng ngầm trong một số hành động, mỗi thanh ghi chứa một offset phân đoạn. Cũng có bốn thanh ghi segment 16 bit. Ba trong bốn thanh ghi segment được sử dụng một cách riêng biệt, ngầm định, lần lượt trỏ đến segment của lệnh hiện tại (hữu ích cho lệnh rẽ nhánh), một segment chứa dữ liệu, và một segment chứa ngăn xếp. Việc sử dụng ngầm định và dành riêng này cho phép mã hoá ngắn gọn nhưng tính linh hoạt giảm. 8086 cũng bao gồm một con trỏ lệnh và một tập hợp các cờ điều khiển và cờ trạng thái 1 bit.

Cần nhấn mạnh quan điểm của sự so sánh này. Không có cách tổ chức thanh ghi CPU nào được cho là tốt nhất. Cũng như việc thiết kế tập lệnh và rất nhiều vấn đề thiết kế CPU khác, lựa chọn thiết kế nào vẫn là vấn đề thuộc về quan điểm và thị hiếu.

Một điểm tham khảo thứ hai liên quan đến thiết kế tổ chức thanh ghi được minh họa trong Hình 2.17c. Hình này cho thấy tổ chức thanh ghi hiển thị với người dùng của Intel 80386, bộ xử lý 32 bit được phát triển mở rộng từ 8086. 80386 sử dụng thanh ghi 32 bit. Tuy nhiên, để cung cấp khả năng tương thích cho các chương trình được viết trên máy tính cũ, 80386 vẫn giữ lại tổ chức thanh ghi cũ và nhúng vào tổ chức mới. Do điểm hạn chế trong thiết kế này, độ linh hoạt trong thiết kế tổ chức thanh ghi trong các kiến trúc bộ xử lý 32 bit giảm đi.

2.5. CHU KỲ LỆNH

Trong mục 1.6, ta đã trình bày về chu kỳ lệnh của CPU. Nhắc lại, một chu kỳ lệnh gồm các giai đoạn sau:

- Truy xuất: Đọc lệnh tiếp theo từ bộ nhớ vào bộ xử lý
- Thực thi: Giải nghĩa opcode và thực hiện hành động được chỉ định
- Ngắt: Nếu ngắt được kích hoạt và đã xảy ra một ngắt, lưu trạng thái xử lý hiện tại và phục vụ ngắt đó.

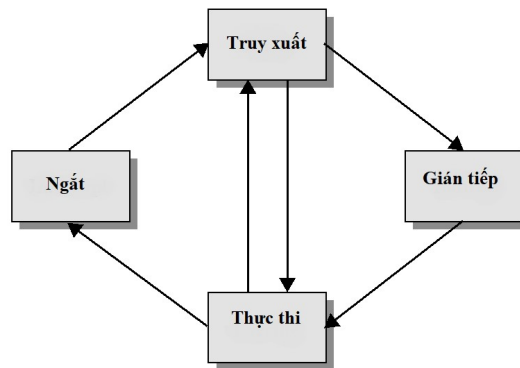
Bây giờ ta sẽ tìm hiểu kỹ hơn về chu kỳ lệnh. Trước tiên, cần phải giới thiệu một giai đoạn bổ sung, được gọi là chu kỳ gián tiếp.

2.5.1. Chu kỳ gián tiếp

Ta đã thấy rằng việc thực thi một lệnh có thể liên quan đến một hoặc nhiều toán hạng trong bộ nhớ, mỗi toán hạng yêu cầu một lần truy cập bộ nhớ. Hơn nữa, nếu sử dụng địa chỉ gián tiếp thì cần có thêm các truy cập bộ nhớ bổ sung.

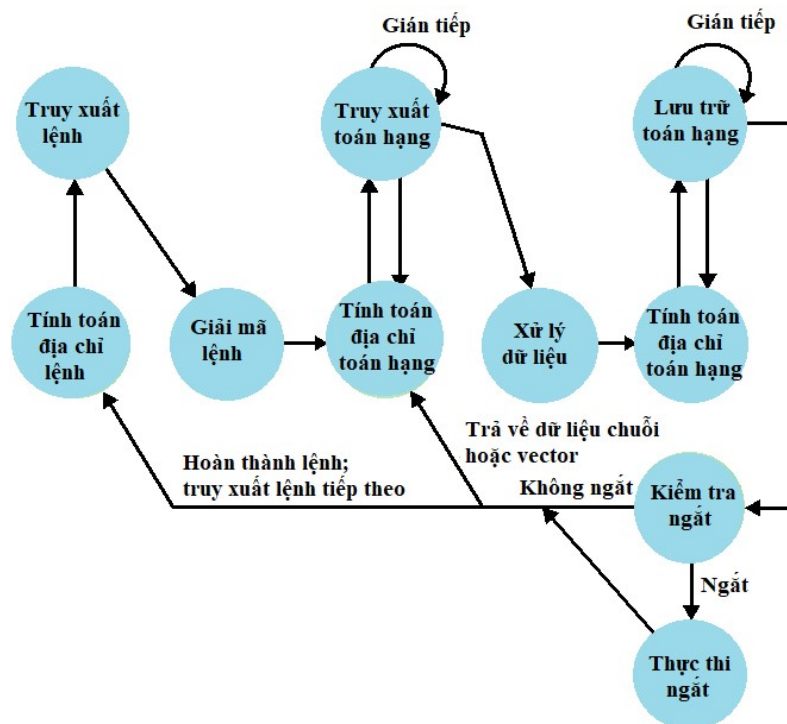
Có thể coi việc truy xuất địa chỉ gián tiếp như một giai đoạn lệnh khác như thể hiện trong hình 2.18. Luồng hoạt động chính bao gồm truy xuất lệnh và thực thi lệnh diễn ra luân phiên. Sau

khi một lệnh được truy xuất, nó được kiểm tra để xác định xem có liên quan đến địa chỉ gián tiếp hay không. Nếu có, các toán hạng yêu cầu được truy xuất bằng cách sử dụng địa chỉ gián tiếp. Sau khi thực thi, có thể xử lý một gián đoạn trước khi truy xuất lệnh tiếp theo.



Hình 2.18. Chu kỳ gián tiếp

Quá trình này cũng có thể được mô tả như hình 2.19. Hình vẽ này mô tả chính xác hơn bản chất của chu kỳ lệnh. Khi một lệnh được truy xuất, các nhận diện toán hạng của lệnh đó phải được xác định. Sau đó, từng toán hạng đầu vào trong bộ nhớ sẽ được truy xuất, và quá trình này có thể đòi hỏi địa chỉ gián tiếp. Các toán hạng lưu ở thanh ghi thì không cần phải truy xuất. Còn khi opcode được thực hiện, có thể cần một quá trình tương tự để lưu kết quả trong bộ nhớ chính.

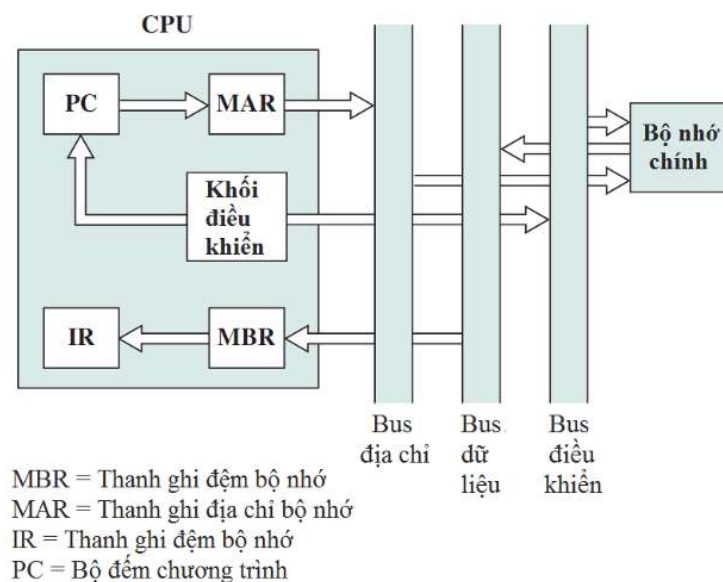


Hình 2.19. Sơ đồ trạng thái chu kỳ lệnh gián tiếp

2.5.2. Luồng dữ liệu

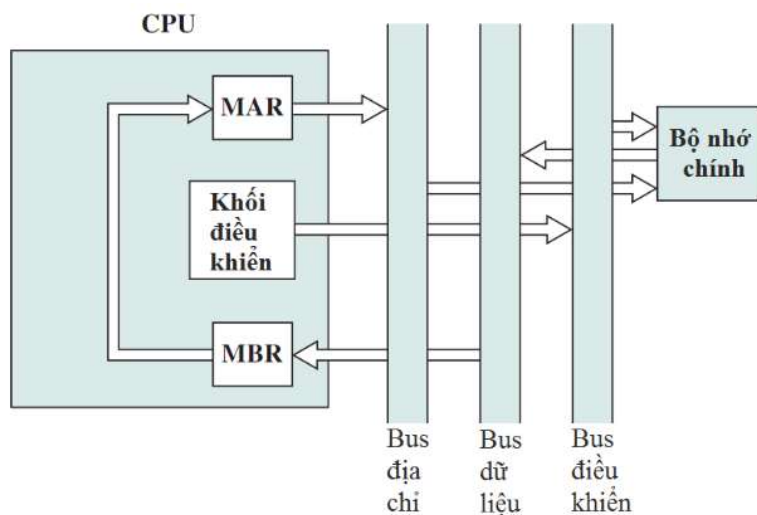
Chuỗi sự kiện chi tiết diễn ra trong chu kỳ lệnh phụ thuộc vào thiết kế của CPU. Tuy nhiên, ta có thể chỉ ra những sự kiện chính phải diễn ra. Giả sử một bộ xử lý sử dụng thanh ghi địa chỉ bộ nhớ (MAR), thanh ghi đệm bộ nhớ (MBR), bộ đếm chương trình (PC) và thanh ghi lệnh (IR).

Trong chu kỳ truy xuất, một lệnh được đọc từ bộ nhớ. Hình 2.20 thể hiện luồng dữ liệu trong chu kỳ này. PC chứa địa chỉ của lệnh tiếp theo được truy xuất. Địa chỉ này được chuyển đến MAR và được đặt lên bus địa chỉ. Khối điều khiển yêu cầu một lần đọc bộ nhớ; kết quả được đặt trên bus dữ liệu và sao chép vào MBR rồi sau đó chuyển tới IR. Trong khi đó, PC được tăng lên 1, chuẩn bị cho lần truy xuất tiếp theo.



Hình 2.20. Luồng dữ liệu, chu kỳ truy xuất

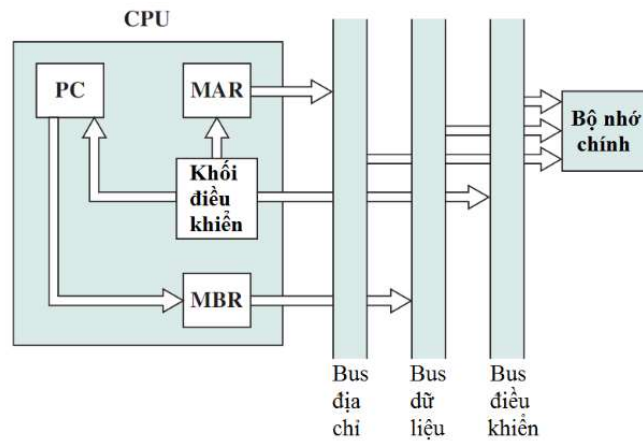
Khi chu kỳ truy xuất kết thúc, khối điều khiển kiểm tra nội dung của IR để xác định xem nó có chứa một nhận diện toán hạng sử dụng địa chỉ gián tiếp hay không. Nếu có, một chu kỳ gián tiếp sẽ được thực hiện. Như thể hiện trong hình 2.21, đây là một chu kỳ đơn giản. N bit ngoài cùng bên phải của MBR, chứa tham chiếu địa chỉ, được chuyển tới MAR. Sau đó khối điều khiển yêu cầu một lần đọc bộ nhớ, để lấy được địa chỉ của toán hạng mong muốn vào MBR.



Hình 2.21. Luồng dữ liệu, chu kỳ gián tiếp

Chu kỳ truy xuất và gián tiếp rất đơn giản và có thể dự đoán được. Chu kỳ thực thi có nhiều hình thức diễn ra; hình thức đó phụ thuộc vào lệnh máy nào đang nằm trong IR. Chu kỳ này có thể liên quan đến việc truyền dữ liệu giữa các thanh ghi, đọc hoặc ghi vào bộ nhớ hoặc I/O, và/hoặc sự tham gia của ALU.

Giống với chu kỳ truy xuất và gián tiếp, chu kỳ ngắt đơn giản và có thể dự đoán được (Hình 2.22). Nội dung hiện tại của PC phải được lưu lại để CPU có thể tiếp tục hoạt động bình thường sau ngắt. Do đó, nội dung của PC được chuyển sang MBR để ghi vào bộ nhớ. Vị trí bộ nhớ đặc biệt dành riêng cho mục đích này (chẳng hạn, con trỏ ngăn xếp) được nạp vào MAR từ bộ điều khiển. Địa chỉ của trình xử lý ngắt được nạp vào PC. Kết quả là chu kỳ lệnh tiếp theo sẽ bắt đầu bằng việc truy xuất lệnh thích hợp (lệnh đầu tiên trong xử lý ngắt).



Hình 2.22. Luồng dữ liệu, chu kỳ ngắt

2.6. KỸ THUẬT ỐNG LỆNH

Khi hệ thống máy tính phát triển, ta có thể đạt được hiệu suất cao hơn bằng cách tận dụng các cải tiến trong công nghệ, ví dụ như sử dụng mạch điện nhanh hơn. Bên cạnh đó, cải tiến tổ chức bộ xử lý cũng giúp cải thiện hiệu suất. Ta đã thấy một số ví dụ về điều này, chẳng hạn như việc sử dụng nhiều thanh ghi thay vì một thanh ghi tích lũy ACC đơn lẻ, và việc sử dụng bộ nhớ cache. Một phương pháp tổ chức khác, khá phổ biến, là ống lệnh.

2.6.1. Chiến lược ống lệnh

Ống lệnh tương tự như việc sử dụng dây chuyền lắp ráp trong nhà máy sản xuất. Dây chuyền lắp ráp dựa trên việc một sản phẩm đi qua nhiều khâu sản xuất khác nhau. Bằng cách đưa quá trình sản xuất vào dây chuyền lắp ráp, các sản phẩm ở các khâu khác nhau có thể được thực hiện đồng thời. Quá trình này được gọi là đường ống (pipelining).

Để áp dụng khái niệm này trong thực thi lệnh, ta phải ghi nhớ rằng một lệnh gồm có nhiều giai đoạn. Ví dụ, hình 2.19 chia chu kỳ lệnh thành 10 hành động theo trình tự. Rõ ràng, việc áp dụng kỹ thuật ống lệnh là hoàn toàn khả thi.

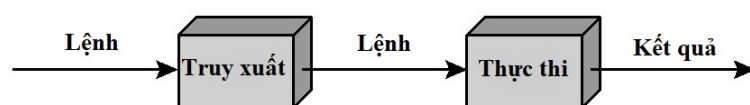
Chẳng hạn, xét một phương án đơn giản, chia quá trình xử lý lệnh thành hai giai đoạn: truy xuất lệnh và thi hành lệnh. Trong quá trình thi hành một lệnh, có nhiều khoảng thời gian trong đó hành động được thực hiện không đòi hỏi việc truy cập vào bộ nhớ chính.

Trong thời gian này, ta có thể truy xuất lệnh tiếp theo song song với việc thi hành lệnh hiện tại. Hình 2.23a mô tả phương pháp này. Ống lệnh có hai giai đoạn độc lập. Giai đoạn thứ nhất truy xuất một lệnh và lưu đệm nó lại. Khi giai đoạn thứ hai đang rảnh, giai đoạn thứ nhất chuyển lệnh vừa đệm cho nó. Trong khi giai đoạn thứ hai đang thi hành lệnh, giai đoạn thứ nhất sẽ tận dụng mọi chu kỳ bộ nhớ không được sử dụng để truy xuất và đệm lệnh tiếp theo. Điều này được gọi là truy xuất lệnh trước (prefetch) hoặc truy xuất nối chồng (fetch overlap). Lưu ý rằng phương pháp này, do liên quan đến việc đệm lệnh, đòi hỏi nhiều thanh ghi hơn. Nói chung, ống lệnh đòi hỏi nhiều thanh ghi để lưu trữ dữ liệu giữa các giai đoạn.

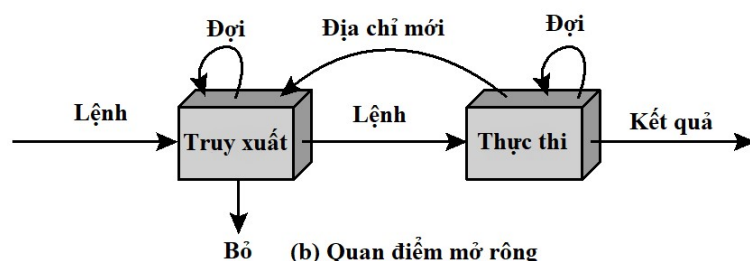
Rõ ràng là quá trình này sẽ tăng tốc việc thực thi lệnh. Nếu các giai đoạn truy xuất và thi hành diễn ra trong khoảng thời gian bằng nhau, thời gian chu kỳ lệnh sẽ giảm một nửa.

Tuy nhiên, nếu ta xét kỹ hơn về pipeline (Hình 2.23b), ta sẽ thấy rằng việc tăng gấp đôi tốc độ thực thi là không thực tế vì hai lý do:

- Thời gian thi hành thường dài hơn thời gian truy xuất. Việc thi hành liên quan đến việc đọc, lưu trữ toán hạng và thực hiện một số phép toán. Do đó, giai đoạn truy xuất có thể phải đợi một lúc trước khi có thể giải phóng bộ đệm.
- Trong một lệnh rẽ nhánh có điều kiện, địa chỉ của lệnh tiếp theo cần truy xuất là không biết trước. Do đó, giai đoạn truy xuất phải đợi cho đến khi nó nhận được địa chỉ lệnh tiếp theo từ giai đoạn thi hành. Giai đoạn thi hành sau đó có thể phải đợi để lệnh tiếp theo được truy xuất.



(a) Quan điểm đơn giản



(b) Quan điểm mở rộng

Hình 2.23. Ống lệnh hai giai đoạn

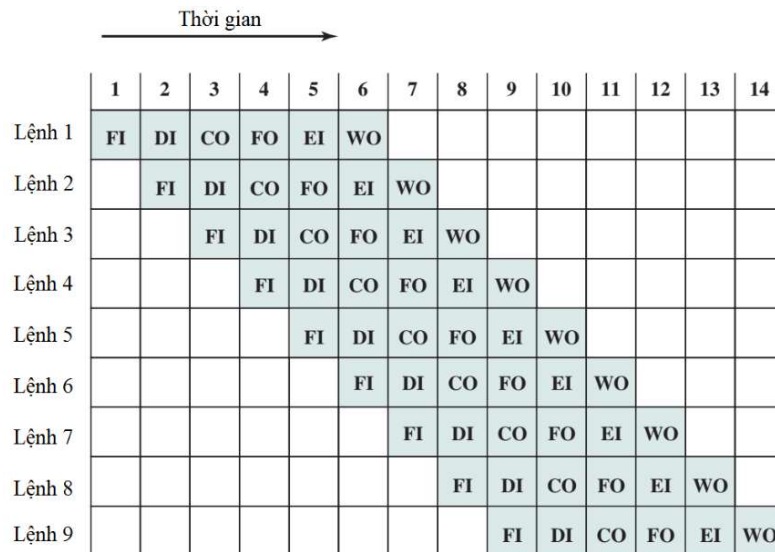
Dự đoán có thể làm giảm thời gian lãng phí ở điều thứ hai nêu trên. Một quy tắc đơn giản như sau: Khi một lệnh rẽ nhánh có điều kiện chuyển từ giai đoạn truy xuất sang thi hành, giai đoạn truy xuất truy xuất lệnh tiếp theo ngay sau lệnh rẽ nhánh trong bộ nhớ. Sau đó, nếu việc rẽ nhánh không được thực hiện thì cũng không mất thời gian. Nếu việc rẽ nhánh được thực hiện, lệnh đã truy xuất phải được loại bỏ và một lệnh khác được truy xuất.

Mặc dù các yếu tố này có làm giảm hiệu quả tiềm năng của ống lệnh hai giai đoạn, tốc độ thực thi lệnh vẫn tăng lên đáng kể. Để tăng tốc nhanh hơn nữa, ống lệnh phải có nhiều giai đoạn hơn. Giả sử việc xử lý lệnh được phân chia thành 6 giai đoạn như sau:

1. Truy xuất lệnh (FI – Fetch instruction): Đọc lệnh mong đợi tiếp theo vào bộ đệm.
2. Giải mã lệnh (DI – Decode instruction): Xác định opcode và nhận diện toán hạng.
3. Tính toán các toán hạng (CO – Calculate operands): Tính toán địa chỉ hiệu dụng của từng toán hạng nguồn. Điều này có thể bao gồm tính toán địa chỉ dịch, gián tiếp thanh ghi, gián tiếp hoặc các hình thức địa chỉ khác.
4. Truy xuất toán hạng (FO – Fetch operands): Truy xuất từng toán hạng từ bộ nhớ. Không cần truy xuất toán hạng từ thanh ghi.
5. Thực thi lệnh (EI – Execute instruction): Thực hiện hành động được chỉ định và lưu trữ kết quả (nếu có) trong vị trí toán hạng đích đã định.
6. Ghi toán hạng (WO – Write operand): Lưu kết quả vào bộ nhớ.

Với sự phân chia này, các giai đoạn khác nhau diễn ra trong khoảng thời gian gần bằng nhau. Để tiện cho việc minh họa, ta giả sử các khoảng thời gian là bằng nhau. Hình 2.24 cho thấy

rằng một ống lệnh 6 giai đoạn có thể làm giảm thời gian thực thi của 9 câu lệnh từ 54 đơn vị thời gian xuống còn 14 đơn vị thời gian.



Hình 2.24. Biểu đồ thời gian của hành động ống lệnh

Sơ đồ này giả thiết rằng mỗi lệnh đi qua tất cả sáu giai đoạn của ống lệnh. Điều này không phải lúc nào cũng đúng. Ví dụ, một lệnh LOAD không cần giai đoạn WO. Tuy nhiên, để đơn giản hóa phần cứng của ống lệnh, ta giả sử rằng mỗi lệnh cần có cả sáu giai đoạn này. Ngoài ra, giả thiết rằng tất cả các giai đoạn có thể được thực hiện song song, và không có xung đột bộ nhớ. Giả thiết, các giai đoạn FI, FO, và WO có liên quan đến việc truy cập bộ nhớ. Trong sơ đồ trên, tất cả các truy cập có thể xảy ra đồng thời. Hầu hết các hệ thống bộ nhớ sẽ không cho phép điều này. Tuy nhiên, giá trị mong muốn có thể nằm trong cache, hoặc giai đoạn FO hoặc WO có thể rỗng. Do đó, phần lớn thời gian, xung đột bộ nhớ sẽ không làm chậm ống lệnh.

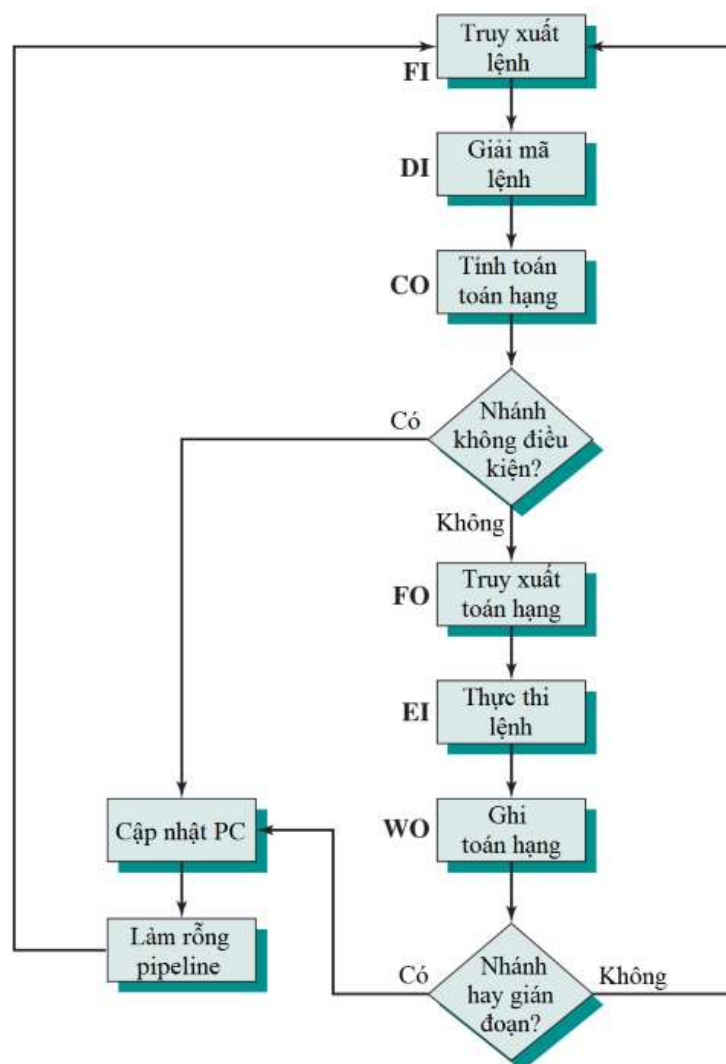


Hình 2.25. Ảnh hưởng của nhánh có điều kiện tới hoạt động của pipeline lệnh

Còn một số yếu tố khác gây hạn chế việc nâng cao hiệu suất. Nếu sáu giai đoạn không diễn ra trong khoảng thời gian bằng nhau, sẽ có khoảng thời gian chờ đợi giữa các giai đoạn ống lệnh, như đã thảo luận ở phần ống lệnh hai giai đoạn. Lệnh rẽ nhánh có điều kiện cũng là một khó khăn khác, vì nó có thể loại bỏ một vài lệnh đã truy xuất. Tương tự, ngắt cũng là một sự kiện không

đoán trước được. Hình 2.25 cho thấy ảnh hưởng của nhánh có điều kiện, sử dụng chương trình giống như hình 2.24.

Giả sử lệnh 3 là nhánh có điều kiện rẽ tới lệnh 15. Nếu lệnh này chưa được thi hành, sẽ không có cách nào biết được lệnh nào tiếp theo cần được thực thi. Trong ví dụ này, ống lệnh chỉ cần tải lệnh tiếp theo theo thứ tự (lệnh 4) và xử lý. Trong hình 2.24, nhánh không được thực hiện, khi đó ta nhận được lợi ích hiệu suất toàn phần. Trong hình 2.25, nhánh được chọn. Điều này chỉ được xác định khi kết thúc đơn vị thời gian 7. Tại thời điểm này, ống lệnh phải xóa sạch các lệnh không hữu ích. Trong đơn vị thời gian 8, lệnh 15 đi vào ống lệnh. Không có lệnh nào được hoàn thành trong các đơn vị thời gian từ 9 đến 12; đây là sự trả giá về hiệu quả hoạt động do ta không thể đoán trước được nhánh. Hình 2.26 cho thấy logic cần thiết để ống lệnh xử lý nhánh và ngắt.

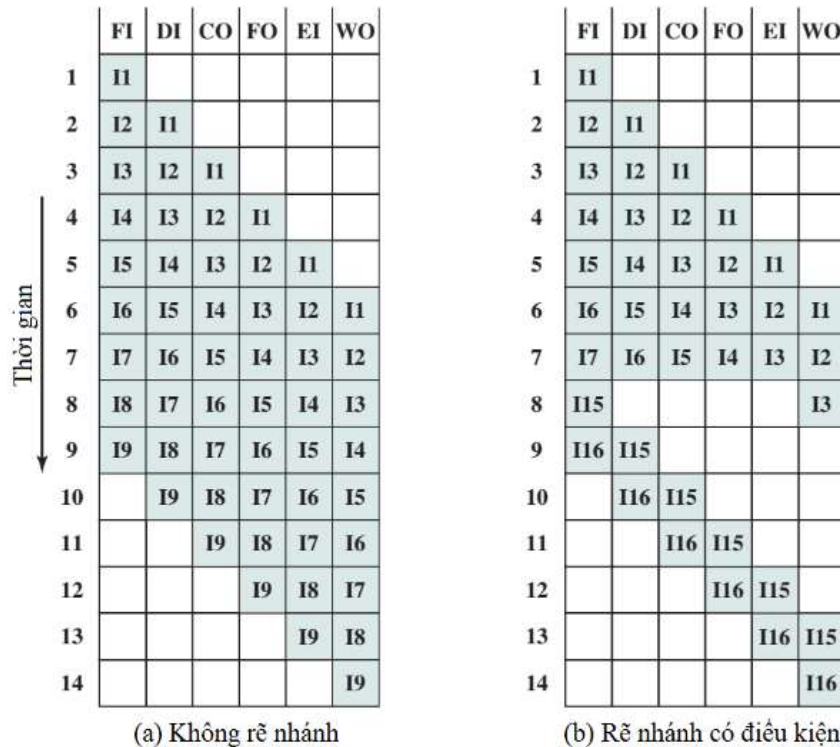


Hình 2.26. Ống lệnh sáu giai đoạn

Cũng có một số vấn đề khác phát sinh trong ống lệnh sáu giai đoạn. Giai đoạn CO có thể phụ thuộc vào nội dung của một thanh ghi mà nội dung này có thể bị thay đổi bởi một lệnh phía trước vẫn còn nằm trong ống lệnh. Các xung đột thanh ghi và xung đột bộ nhớ khác có thể xảy ra. Hệ thống phải có logic để xử lý đối với kiểu xung đột này.

Để làm rõ hoạt động của ống lệnh, ta có thể xem xét nó bằng một cách mô tả khác. Hình 2.24 và 2.25 cho thấy diễn biến theo thời gian trên trục ngang, với mỗi dòng là diễn biến của một lệnh. Hình 2.27 thể hiện cùng một chuỗi sự kiện, nhưng diễn biến thời gian theo chiều dọc từ trên xuống, mỗi dòng thể hiện trạng thái của pipeline ở một thời điểm nhất định. Trong Hình 2.27a

(tương ứng với Hình 2.24), ở thời điểm 6, ống lệnh lấp đầy bởi 6 lệnh khác nhau trong các giai đoạn thi hành khác nhau; và ống lệnh vẫn đầy cho tới thời điểm 9; ta giả sử rằng lệnh I9 là lệnh cuối cùng được thực thi. Trong Hình 2.27b, (tương ứng với Hình 2.25), ống lệnh đầy ở thời điểm 6 và 7. Tại thời điểm 7, lệnh 3 đang trong giai đoạn thực thi và thực hiện rẽ nhánh đến lệnh 15. Tại đây, lệnh I4 đến I7 được giải phóng khỏi ống lệnh, do đó tại thời điểm 8, chỉ có hai lệnh trong pipeline, I3 và I15.



Hình 2.27. Một cách mô tả khác về ống lệnh

2.6.2. Xung đột trong ống lệnh

Trong tiểu mục trước, chúng ta đã đề cập đến một số tình huống có thể làm giảm hiệu suất ống lệnh. Trong tiểu mục này, ta xem xét vấn đề này một cách có hệ thống hơn.

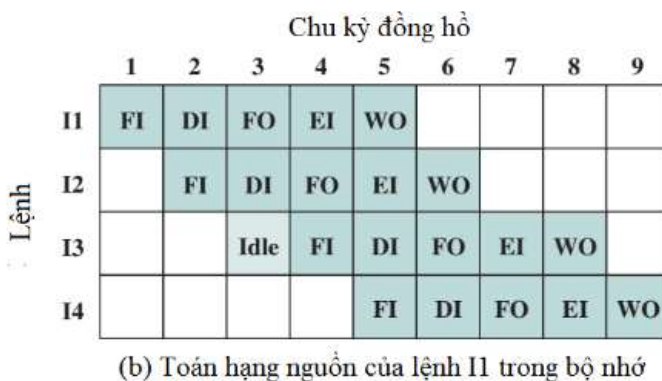
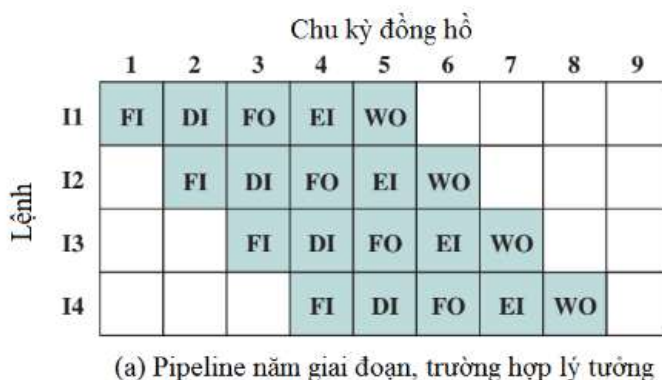
Xung đột trong ống lệnh, hay là xung đột trong pipeline, xảy ra khi một phần hoặc toàn bộ ống lệnh phải ngừng lại vì điều kiện không cho phép tiếp tục thực hiện. Sự ngừng ống lệnh như vậy còn được gọi là pipeline bubble. Có ba loại xung đột trong ống lệnh: tài nguyên, dữ liệu và điều khiển.

2.6.2.1. Xung đột tài nguyên

Xung đột tài nguyên xảy ra khi hai lệnh (hoặc nhiều hơn) đang nằm trong ống lệnh cần sử dụng nguồn tài nguyên giống nhau. Kết quả là các lệnh này không thể thực hiện song song mà phải được thực hiện lần lượt trong một phần của ống lệnh. Xung đột tài nguyên đôi khi được gọi là xung đột cấu trúc.

Xét một ví dụ đơn giản về xung đột tài nguyên. Giả sử một ống lệnh đơn giản gồm năm giai đoạn, trong đó mỗi giai đoạn chiếm một chu kỳ đồng hồ. Hình 2.28a mô tả trường hợp lý tưởng, trong đó cứ mỗi chu kỳ đồng hồ lại có một lệnh mới đi vào ống lệnh. Bây giờ giả sử rằng bộ nhớ chính có một cổng duy nhất và tất cả các hành động truy xuất lệnh, đọc và ghi dữ liệu phải được thực hiện lần lượt từng hành động một. Đồng thời, bỏ qua bộ nhớ cache. Trong trường hợp này, hành động đọc toán hạng hoặc ghi toán hạng từ bộ nhớ không thể được thực hiện song song

với hành động truy xuất lệnh. Điều này được mô tả ở Hình 2.28b, trong đó giả sử rằng toán hạng nguồn của lệnh I1 nằm trong bộ nhớ chứ không nằm ở thanh ghi. Do đó, giai đoạn truy xuất lệnh của ống lệnh phải nhàn rỗi (idle) trong một chu kỳ trước khi bắt đầu truy xuất lệnh đối với lệnh I3. Hình 2.8 giả thiết rằng tất cả các toán hạng khác đều nằm trong thanh ghi.



Hình 2.28. Ví dụ về xung đột tài nguyên

Một ví dụ khác về xung đột tài nguyên là tình huống trong đó nhiều lệnh đã sẵn sàng vào giai đoạn thi hành lệnh EI và chỉ có một ALU duy nhất. Giải pháp cho xung đột tài nguyên này là tăng cường các tài nguyên sẵn có, chẳng hạn như sử dụng nhiều cổng vào bộ nhớ chính và nhiều khối ALU.

2.6.2.2. Xung đột dữ liệu

Xung đột dữ liệu xảy ra khi có xung đột trong việc truy cập vị trí toán hạng. Nói chung, xung đột có thể ở dạng này: Hai lệnh trong một chương trình sẽ được thi hành theo trình tự và cả hai đều truy cập vào một toán hạng bộ nhớ hoặc toán hạng thanh ghi cụ thể. Nếu hai lệnh này được thi hành theo trình tự nghiêm ngặt thì không có vấn đề xảy ra. Tuy nhiên, nếu chúng được thực hiện trong ống lệnh, giá trị toán hạng có thể được cập nhật và sẽ tạo ra kết quả khác với kết quả của việc thi hành tuần tự nghiêm ngặt. Nói cách khác, chương trình tạo ra kết quả không chính xác do sử dụng ống lệnh.

Ví dụ, xét chuỗi lệnh máy x86 sau:

ADD EAX,	EBX /* EAX = EAX + EBX
SUB ECX,	EAX /* ECX = ECX - EAX

Lệnh đầu tiên cộng nội dung của thanh ghi 32 bit EAX với EBX và lưu trữ kết quả trong EAX. Lệnh thứ hai lấy nội dung của ECX trừ EAX từ và lưu trữ kết quả trong ECX. Hình 2.29 mô tả diễn biến trong ống lệnh.

Lệnh ADD chỉ cập nhật thanh ghi EAX khi kết thúc giai đoạn 5, xảy ra ở chu kỳ đồng hồ 5. Tuy nhiên, lệnh SUB cần giá trị đó vào đầu giai đoạn 2, xảy ra ở chu kỳ đồng hồ 4. Để duy trì hoạt động chính xác, ống lệnh phải ngừng lại trong hai chu kỳ đồng hồ. Do đó, khi không có các phần cứng đặc biệt và các thuật toán phòng tránh cụ thể, xung đột dữ liệu kiểu này dẫn đến việc sử dụng ống lệnh không hiệu quả.

Có ba loại xung đột dữ liệu:

- **Đọc sau ghi (RAW):** Một lệnh thay đổi nội dung của một vị trí bộ nhớ hoặc thanh ghi và lệnh tiếp theo đọc dữ liệu trong vị trí bộ nhớ hoặc thanh ghi đó. Xung đột xảy ra nếu việc đọc diễn ra trước khi hành động ghi hoàn tất.
- **Ghi sau đọc (WAR):** Một lệnh đọc nội dung của một vị trí bộ nhớ hoặc thanh ghi và lệnh tiếp theo ghi vào vị trí đó. Xung đột xảy ra nếu hành động ghi hoàn thành trước khi diễn ra hành động đọc.
- **Ghi sau ghi (WAW):** Hai lệnh cùng ghi vào một vị trí. Xung đột xảy ra nếu các hành động ghi diễn ra theo trình tự ngược lại với trình tự đã dự định.

Hình 2.29 đưa ra ví dụ về xung đột RAW.

		Chu kỳ đồng hồ									
		1	2	3	4	5	6	7	8	9	10
ADD EAX, EBX		FI	DI	FO	EI	WO					
SUB ECX, EAX			FI	DI	Idle		FO	EI	WO		
I3				FI			DI	FO	EI	WO	
I4							FI	DI	FO	EI	WO

Hình 2.29. Ví dụ về xung đột dữ liệu

2.6.2.3. Xung đột điều khiển

Xung đột điều khiển, còn được gọi là xung đột rẽ nhánh, xảy ra khi ống lệnh đưa ra quyết định sai về dự báo nhánh và do đó đưa sai các lệnh vào pipeline rồi sau này phải bỏ chúng đi. Chúng ta sẽ thảo luận các giải pháp để tránh xung đột điều khiển trong phần tiếp theo.

2.6.3. Tránh xung đột rẽ nhánh

Đảm bảo một luồng lệnh ổn định đưa đến các giai đoạn đầu tiên của ống lệnh là một trong những vấn đề chính trong thiết kế ống lệnh. Như chúng ta đã thấy, trở ngại chính đối với vấn đề này là lệnh rẽ nhánh có điều kiện. Chừng nào mà lệnh này còn chưa được thi hành, sẽ không thể xác định nhánh đó có được thực hiện hay không.

Có nhiều phương pháp để tránh xung đột với rẽ nhánh có điều kiện như sau:

- Sử dụng nhiều luồng
- Truy xuất trước mục tiêu rẽ nhánh
- Bộ đếm vòng lặp
- Dự báo rẽ nhánh
- Rẽ nhánh chậm

2.6.3.1. Sử dụng nhiều luồng

Lệnh rẽ nhánh có thể gây ảnh hưởng cho một ống lệnh đơn giản vì nó phải chọn một trong hai lệnh cho lần truy xuất tiếp theo và có thể chọn sai. Một phương pháp được đưa ra là sao chép lại phần mở đầu của ống lệnh và cho phép ống lệnh truy xuất cả hai lệnh, sử dụng hai luồng khác nhau. Phương pháp này có hai vấn đề:

- Nhiều ống lệnh kéo theo việc có trễ tranh chấp để truy cập vào thanh ghi và bộ nhớ.
- Có thể có thêm các lệnh rẽ nhánh đi vào ống lệnh (thêm luồng khác) trước khi quyết định rẽ nhánh ban đầu được giải quyết. Mỗi lệnh như vậy lại cần thêm một luồng bổ sung nữa.

Mặc dù có những hạn chế như vậy, phương pháp này thực sự có thể cải thiện hiệu suất. Ví dụ về máy có nhiều luồng pipeline là IBM 370/168 và IBM 3033.

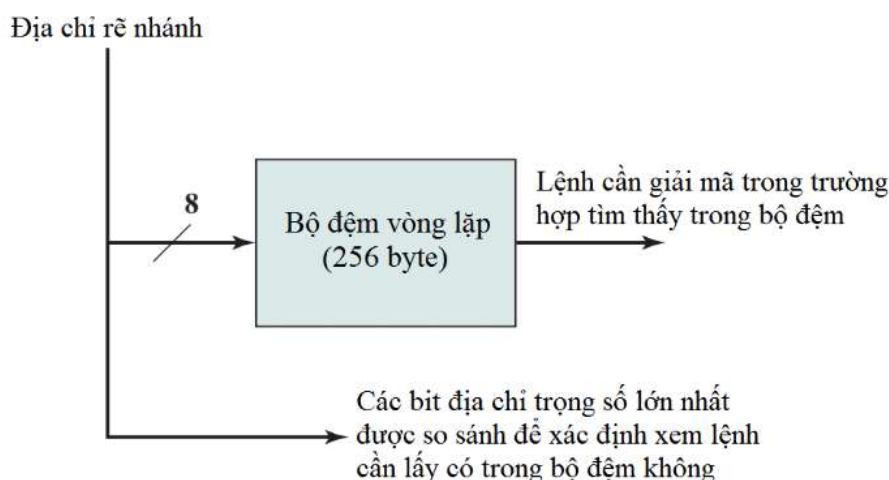
2.6.3.2. Truy xuất trước mục tiêu rẽ nhánh

Khi nhận được một lệnh rẽ nhánh có điều kiện, mục tiêu của lệnh rẽ nhánh đó được truy xuất trước, thêm vào lệnh phía sau lệnh rẽ nhánh. Mục tiêu này sau đó được lưu lại cho đến khi lệnh nhánh được thi hành. Nếu rẽ nhánh được thực hiện thì mục tiêu của nó đã được truy xuất trước. IBM 360/91 sử dụng phương pháp này.

2.6.3.3. Bộ đệm vòng lặp

Bộ đệm vòng lặp là một bộ nhớ nhỏ, tốc độ rất cao và chứa n lệnh được truy xuất gần đây nhất, theo thứ tự. Nó được duy trì bởi giai đoạn truy xuất lệnh của ống lệnh. Nếu một rẽ nhánh được thực hiện, trước tiên, phần cứng phải kiểm tra xem mục tiêu rẽ nhánh có nằm trong bộ đệm hay không. Nếu có, lệnh tiếp theo sẽ được truy xuất ra từ bộ đệm. Bộ đệm vòng lặp có ba lợi ích:

1. Do có sử dụng truy xuất trước, bộ đệm vòng lặp sẽ chứa một vài lệnh tuần tự phía trước địa chỉ truy xuất lệnh hiện tại. Do đó, sẽ có sẵn một số lệnh được truy xuất theo trình tự mà không mất thời gian truy cập bộ nhớ như thông thường.
2. Nếu một rẽ nhánh xảy ra mà mục tiêu của nó chỉ đứng trước địa chỉ của lệnh rẽ nhánh một vài vị trí, mục tiêu sẽ ở sẵn trong bộ đệm. Điều này rất hữu ích cho chuỗi khá thường gặp là IF-THEN và IF-THEN-ELSE.
3. Phương pháp này đặc biệt phù hợp để xử lý các vòng lặp; do đó được gọi tên là bộ đệm vòng lặp. Nếu bộ đệm vòng lặp đủ lớn để chứa tất cả các lệnh trong một vòng lặp, thì những lệnh đó chỉ cần được truy xuất từ bộ nhớ một lần, cho lần lặp đầu tiên. Đối với những lần lặp tiếp theo, tất cả các lệnh cần thiết đã nằm trong bộ đệm.



Hình 2.30. Bộ đệm vòng lặp

Bộ đệm vòng lặp hoạt động theo nguyên lý tương tự với bộ nhớ cache dành riêng cho lệnh. Điểm khác biệt là bộ đệm vòng lặp chỉ giữ lại các lệnh theo trình tự và kích thước nhỏ hơn nhiều và do đó giá rẻ hơn.

Hình 2.30 đưa ra ví dụ về bộ đệm vòng lặp. Nếu bộ đệm chứa 256 byte, và địa chỉ được đánh theo byte, khi đó 8 bit địa chỉ có trọng số nhỏ nhất được sử dụng để đánh số bộ đệm. Các bit địa chỉ có trọng số lớn nhất còn lại được kiểm tra để xác định xem mục tiêu rẽ nhánh có nằm trong vùng bao trùm bởi bộ đệm hay không. Bộ đệm vòng lặp được sử dụng trong các máy CDC (Star-100, 6600, 7600), CRAY-1 và Motorola 68010.

2.6.3.4. Dự đoán nhánh

Nhiều kỹ thuật khác nhau có thể được sử dụng để dự đoán liệu một rẽ nhánh có được thực hiện hay không. Một số kỹ thuật phổ biến hơn cả là:

- Dự đoán không bao giờ được thực hiện
- Dự đoán luôn luôn được thực hiện
- Dự đoán theo opcode
- Công tắc được thực hiện/không được thực hiện
- Bảng lịch sử rẽ nhánh

Ba kỹ thuật đầu tiên là phương pháp tĩnh: chúng không phụ thuộc vào lịch sử thi hành tính đến trước thời điểm lệnh rẽ nhánh có điều kiện. Hai kỹ thuật sau cùng là phương pháp động: chúng phụ thuộc vào lịch sử thi hành. Hai phương pháp đầu tiên là đơn giản nhất.

Chúng hoặc luôn luôn giả định rằng rẽ nhánh sẽ không được thực hiện và tiếp tục truy xuất các lệnh theo trình tự hoặc luôn luôn giả định rằng rẽ nhánh sẽ được thực hiện và luôn truy xuất từ mục tiêu rẽ nhánh. Phương pháp dự đoán không bao giờ được thực hiện là phương pháp phổ biến nhất trong tất cả các phương pháp dự báo nhánh.

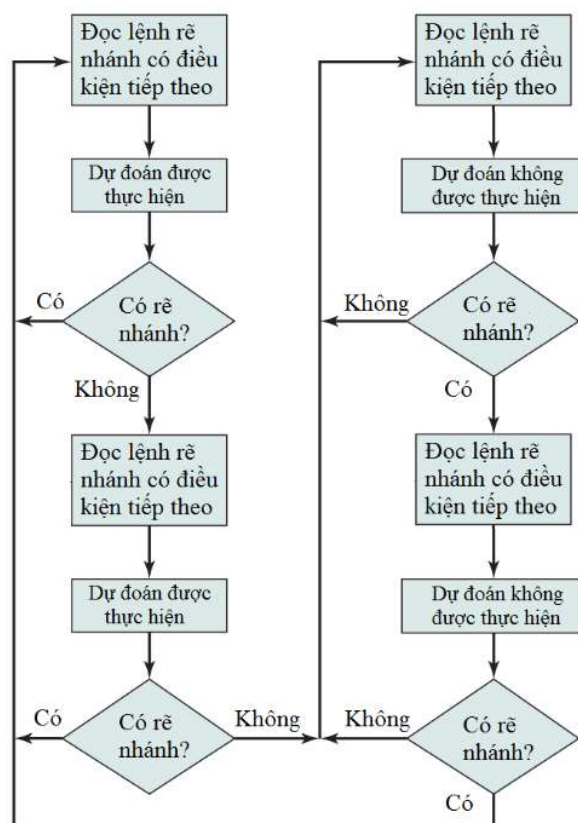
Phương pháp tĩnh cuối cùng đưa ra quyết định dựa trên opcode của lệnh rẽ nhánh. CPU giả định rằng đối với một số loại opcode rẽ nhánh nhất định thì nhánh sẽ luôn được thực hiện, và đối với một số loại khác thì nhánh sẽ không được thực hiện. Chiến lược này đạt được tỷ lệ truy xuất trước thành công lớn hơn 75%.

Các phương pháp rẽ nhánh động cố gắng nâng cao độ chính xác của dự đoán bằng cách ghi lại lịch sử các lệnh rẽ nhánh có điều kiện trong một chương trình. Chẳng hạn, một hoặc nhiều bit được gắn với một lệnh rẽ nhánh có điều kiện và có thể phản ánh lịch sử gần đây của lệnh đó. Các bit này được gọi là công tắc được thực hiện/không được thực hiện, chúng định hướng cho CPU đưa ra một quyết định cụ thể vào lần tiếp theo gặp lại lệnh.

Thông thường, các bit lịch sử này không được gắn với lệnh trong bộ nhớ chính. Thay vào đó, chúng được giữ trong bộ nhớ tạm thời. Một khả năng có thể là gắn các bit này với mọi lệnh rẽ nhánh có điều kiện nằm trong cache. Khi một lệnh này được thay thế khỏi cache, lịch sử của nó mất đi. Một khả năng khác là duy trì một bảng nhỏ chứa các lệnh rẽ nhánh được thi hành gần đây với mỗi lệnh nhập vào có một hoặc nhiều bit lịch sử. CPU có thể truy cập vào bảng một cách kết hợp, giống như bộ nhớ cache, hoặc bằng cách sử dụng các bit trọng số nhỏ trong địa chỉ lệnh của rẽ nhánh.

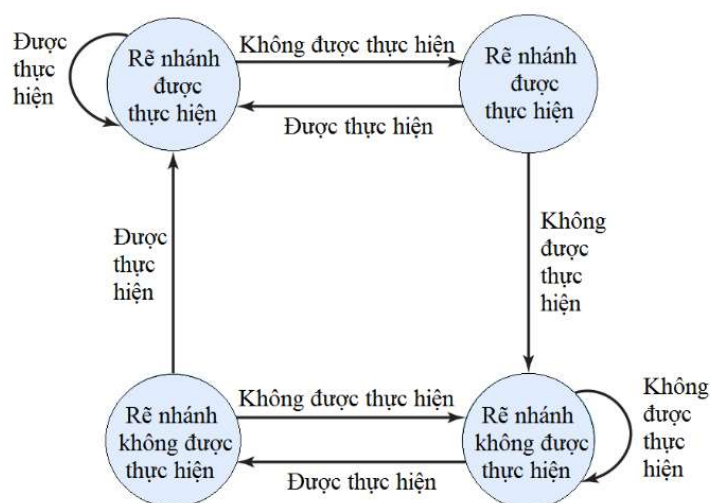
Nếu sử dụng một bit đơn, thông tin duy nhất có thể ghi lại được là liệu lần thi hành gần nhất của lệnh này có rẽ nhánh hay không. Việc sử dụng một bit đơn xuất hiện nhược điểm trong trường hợp một lệnh rẽ nhánh có điều kiện gần như luôn luôn được thực hiện, như một lệnh lặp

chẳng hạn. Nếu chỉ dùng một bit lịch sử, mỗi lần sử dụng vòng lặp sẽ xảy ra hai lần lỗi dự đoán: một lần khi bắt đầu vào vòng lặp, và một khi ra khỏi vòng lặp.



Hình 2.31. Lưu đồ dự đoán nhánh

Nếu sử dụng hai bit, hai bit này có thể được dùng để ghi lại kết quả của hai lần thực thi lệnh rẽ nhánh gần nhất, hoặc để ghi lại một trạng thái nhất định. Hình 2.31 minh họa cho một phương pháp điển hình. Giả sử rằng thuật toán bắt đầu ở góc phía trên bên trái của lưu đồ. Chừng nào mà một lệnh rẽ nhánh có điều kiện tiếp theo gặp phải có thực hiện rẽ nhánh, quá trình ra quyết định sẽ dự đoán rằng lệnh tiếp theo sẽ lại thực hiện rẽ nhánh.



Hình 2.32. Lưu đồ trạng thái dự đoán nhánh

Nếu một lần dự đoán dự đoán sai, thuật toán tiếp tục dự đoán rằng rẽ nhánh tiếp theo được thực hiện. Chỉ khi hai rẽ nhánh liên tiếp không được thực hiện thì thuật toán sẽ chuyển sang phía tay phải của lưu đồ. Sau đó, thuật toán sẽ dự đoán rằng các nhánh không được thực hiện cho đến

khi hai nhánh liên tiếp được thực hiện. Do đó, thuật toán này yêu cầu hai lần dự đoán sai liên tiếp để thay đổi quyết định dự đoán. Quá trình ra quyết định có thể được mô tả ngắn gọn hơn như trong hình 2.32.

Việc sử dụng các bit lịch sử, như vừa mô tả, có một nhược điểm: Nếu quyết định được đưa ra là sẽ thực hiện rẽ nhánh thì sẽ không thể truy xuất được lệnh mục tiêu chừng nào còn chưa giải mã được địa chỉ của lệnh mục tiêu (chính là một toán hạng trong lệnh rẽ nhánh có điều kiện). Rõ ràng là, hiệu quả sẽ cao hơn nếu việc truy xuất lệnh có thể được bắt đầu ngay sau khi có quyết định rẽ nhánh. Để làm được như vậy, cần phải lưu lại nhiều thông tin hơn trong một bộ đệm mục tiêu rẽ nhánh hoặc bảng lịch sử rẽ nhánh.

Bảng lịch sử rẽ nhánh là một bộ nhớ cache nhỏ gắn với giai đoạn truy xuất lệnh trong ống lệnh. Mỗi mục trong bảng gồm có ba phần: địa chỉ của một lệnh rẽ nhánh, một vài bit lịch sử ghi lại trạng thái sử dụng lệnh đó, và thông tin về lệnh mục tiêu. Trường thứ ba thường chứa địa chỉ của lệnh mục tiêu. Trong một số đề xuất, trường thứ ba này có thể chứa luôn lệnh mục tiêu. Quan hệ ràng buộc trong vấn đề này rất rõ ràng: Lưu trữ địa chỉ mục tiêu sẽ làm cho kích thước bảng nhỏ hơn nhưng thời gian truy xuất lệnh lâu hơn so với lưu trữ lệnh mục tiêu.

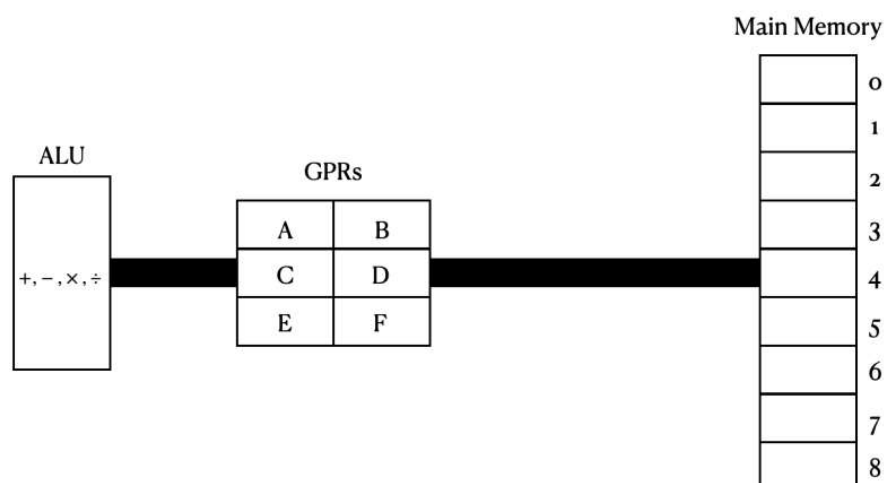
2.6.3.5. Nhánh trễ

Có thể cải thiện hiệu suất pipeline bằng cách tự động sắp xếp lại các lệnh trong một chương trình, sao cho lệnh rẽ nhánh xuất hiện muộn hơn.

2.7. CISC VÀ RISC

CISC là viết tắt của Complex Instruction Set Architecture, và RISC là viết tắt của Reduced Instruction Set Architecture. Đây là hai loại máy tính sử dụng kiến trúc tập lệnh khác nhau. CISC sử dụng kiến trúc bộ nhớ - bộ nhớ và RISC sử dụng kiến trúc thanh ghi - thanh ghi. Dưới đây là một ví dụ để minh họa CISC và RISC.

Hình 2.33 thể hiện cơ chế lưu trữ của một máy tính thông thường. Nó đại diện cho một máy tính có hệ thống bộ nhớ hai cấp không có bộ nhớ đệm. Các thanh ghi đa dụng GPR là bộ nhớ mà các hoạt động có thể truy cập trực tiếp trong hầu hết các bộ xử lý, không giống như bộ nhớ đệm và bộ nhớ chính là những nơi lưu trữ dữ liệu.



Hình 2.33. Cơ chế lưu trữ

- Bộ nhớ chính có 9 từ nhớ được đánh địa chỉ bằng số thập phân từ 0 đến 8.
- ALU chịu trách nhiệm về tất cả các tính toán nhưng chỉ có thể thao tác với dữ liệu đã được tải vào sáu thanh ghi GPR: A, B, C, D, E hoặc F.

- Giả sử chúng ta muốn cho máy tính thực hiện phép tính tích của hai số, một số được lưu ở vị trí 2 và số còn lại được lưu ở vị trí 5 trong bộ nhớ chính, và kết quả lưu lại ở vị trí 2.

CISC

Nếu máy tính là máy CISC, bộ xử lý CISC sẽ chuẩn bị một lệnh cụ thể cho phép nhân, giả sử nó được đặt tên là MUL. Khi được thực thi, lệnh này tải hai số từ bộ nhớ chính vào hai thanh ghi riêng biệt, nhân các toán hạng trong ALU, lưu trữ kết quả trong một thanh ghi thích hợp, rồi lưu trữ lại bộ nhớ chính. Toàn bộ quá trình nhân hai số có thể được hoàn thành chỉ với một lệnh:

MUL 2 2 5

Mục tiêu chính của kiến trúc CISC là hoàn thành một tác vụ với càng ít dòng lệnh càng tốt và xây dựng các lệnh phức tạp trực tiếp vào phần cứng.

Vì vậy, phần cứng bộ xử lý của các máy tính CISC có khả năng hiểu một lệnh phức tạp và thực hiện một loạt các hoạt động đằng sau nó.

Ưu điểm chính của máy CISC là:

- Các trình biên dịch phải thực hiện rất ít công việc để dịch một chương trình ngôn ngữ bậc cao thành chương trình hợp ngữ, vì các lệnh phức tạp đã gần với các ngôn ngữ bậc cao.
- Do độ dài của một chương trình hợp ngữ tương đối ngắn, nên cần rất ít RAM để lưu các lệnh.

RISC

Nếu máy tính là máy RISC, lệnh MUL 2 2 5 ở trên sẽ được chia thành bốn lệnh, chẳng hạn như:

- LOAD A 2 # nạp số tại vị trí 2 vào thanh ghi A
- LOAD B 5 # nạp số tại vị trí 5 vào thanh ghi B
- PROD C A B # tiến hành $A \times B$ và lưu kết quả trở lại C
- STORE 2 C # lưu trữ giá trị tại C trở lại vị trí 2

Ý tưởng của kiến trúc RISC là làm cho phần cứng trở nên đơn giản hơn bằng cách sử dụng “lệnh đơn giản” thực hiện một việc mỗi lần.

Vì vậy, các trình biên dịch cho máy RISC phải làm nhiều việc hơn. Một câu lệnh ngôn ngữ bậc cao có thể cần được dịch thành một số lệnh đơn giản để cho phép các toán hạng được tải từ bộ nhớ chính sang GPR trước khi thực hiện một thao tác và sau đó lưu kết quả hoạt động trở lại bộ nhớ chính. Do đó, cần nhiều RAM hơn để lưu trữ các lệnh.

Ưu điểm chính của máy RISC là:

- Giả sử mỗi lệnh đơn giản có thể được thực hiện trong một chu kỳ đồng hồ, thì việc hoàn thành một lệnh phức tạp MUL 2 2 5 có thể mất cùng một khoảng thời gian để thực hiện bốn lệnh đơn giản. Tuy nhiên, vì tất cả các lệnh đơn giản đều thực thi trong một khoảng thời gian đồng nhất, nên có thể tận dụng ống lệnh để giảm thời gian thực hiện.
- Phần cứng của máy RISC đơn giản hơn nên cần ít không gian phần cứng hơn.
- RISC nói chung là công suất thấp hơn. Hầu hết các hệ thống nhúng sử dụng kiến trúc RISC.

Các CPU hiện tại là CISC trên RISC với vi mã phức tạp hơn để loại bỏ các vấn đề của việc triển khai CISC vật lý thực tế.

Câu hỏi ôn tập chương 2

1. Chức năng của thanh ghi trong bộ xử lý là gì?
2. Thanh ghi hiển thị với người dùng thường hỗ trợ những loại dữ liệu nào?
3. Chế độ địa chỉ của vi xử lý là gì? Mô tả chế độ địa chỉ tức thì. Cho ví dụ minh họa.
4. Chế độ địa chỉ của vi xử lý là gì? Mô tả chế độ địa chỉ trực tiếp. Cho ví dụ minh họa.
5. Chế độ địa chỉ của vi xử lý là gì? Mô tả chế độ địa chỉ chỉ số. Cho ví dụ minh họa.
6. Chế độ địa chỉ của vi xử lý là gì? Mô tả chế độ địa chỉ tương đối. Cho ví dụ minh họa.
7. Nêu sơ đồ khối tổng quát và chu trình xử lý lệnh của CPU.
8. Nêu sơ đồ khối và chức năng của các khối điều khiển (CU) và khối tính toán số học và logic (ALU).
9. Lệnh máy tính là gì? Chu kỳ lệnh là gì? Nêu các pha điển hình trong chu kỳ thực hiện lệnh. Nêu dạng lệnh tổng quát và các thành phần của nó.
10. Cơ chế xử lý xen kẽ dòng lệnh (ống lệnh – pipeline) là gì? Nêu các đặc điểm của cơ chế ống lệnh (có hình vẽ minh họa).
11. Tại sao một pipeline lệnh hai giai đoạn không rút ngắn một nửa thời gian chu kỳ lệnh so với khi không sử dụng pipeline?
12. Liệt kê và giải thích ngắn gọn các giải pháp để pipeline lệnh giải quyết các vấn đề xảy ra với lệnh rẽ nhánh có điều kiện?
13. Dự báo nhánh sử dụng các bit lịch sử như thế nào?
14. Giả sử một máy tính với độ dài word là 8 bit. Nếu phép tính cuối cùng được thực hiện trên máy này một phép cộng, trong đó hai toán hạng là 00000010 và 00000011. Hãy xác định giá trị của các cờ sau?
 - a. Carry
 - b. Zero
 - c. Overflow
 - d. Sign
15. Dùng các chế độ 0, 1, 2, 3 địa chỉ để viết chương trình tính $X=(A+BxC)/(D-ExF)$ sử dụng các lệnh tổng quát?
16. Nêu một hướng giải quyết xung đột dữ liệu trong pipeline khi thực hiện đoạn chương trình sau:
ADD R4, R2, #300 ; $R4 \leftarrow R2+300$
ADD R1, R2, R3 ; $R1 \leftarrow R2+R3$
CMP R1, #100 ; so sánh R1 với 100
SUB R5, #2000 ; $R5 \leftarrow R5 - 2000$

biết rằng mỗi lệnh được chia thành 5 giai đoạn trong pipeline: Đọc lệnh (IF), giải mã & đọc toán hạng (ID), truy nhập bộ nhớ (MEM), thực hiện (EX) và lưu kết quả (WB).

17. Cho đoạn chương trình sau (R1, R2 là các thanh ghi):
 - (1) LOAD R2, #400
 - (2) LOAD R1, #1200
 - (3) STORE (R1), R2
 - (4) SUBTRACT R2, #20
 - (5) ADD 1200, #10
 - (6) ADD R2, (R1)
 - a. Xác định chế độ địa chỉ và ý nghĩa của từng lệnh;

- b. Xác định giá trị của thanh ghi R2 sau khi thực hiện xong lệnh số (6).
18. Cho một dãy số nguyên gồm 100 phần tử lưu trong bộ nhớ bắt đầu từ địa chỉ 1500. Viết đoạn chương trình sử dụng tập lệnh của CPU tính:
- Đếm số các phần tử bằng không trong dãy và lưu kết quả vào ô nhớ có địa chỉ 2000.
 - Tổng của tất cả các số không âm trong dãy và lưu kết quả vào ô nhớ có địa chỉ 2048.
19. Cho một dãy số nguyên gồm 100 phần tử lưu trong bộ nhớ bắt đầu từ địa chỉ 1000. Viết chương trình sử dụng tập lệnh của CPU tính:
- Tổng của các số dương – lưu kết quả vào ô nhớ có địa chỉ 2000.
 - Tổng của tất cả các số trong dãy – lưu kết quả vào ô nhớ có địa chỉ 2010.
20. Cho một dãy số nguyên gồm 10 phần tử lưu trong bộ nhớ bắt đầu từ địa chỉ 2000. Viết đoạn chương trình sử dụng tập lệnh của CPU tính:
- Đếm số các số âm trong dãy và lưu kết quả vào ô nhớ có địa chỉ 3000.
 - Tổng của tất cả các số trong dãy – lưu kết quả vào ô nhớ có địa chỉ 4050.
21. Xét chương trình hợp ngữ sau:

```

I1: Move R3, R7    /R3 ← (R7)/
I2: Load R8, (R3) /R8 ← Memory (R3)/
I3: Add R3, R3, 4   /R3 ← (R3) + 4/
I4: Load R9, (R3) /R9 ← Memory (R3)/
I5: BLE R8, R9, L3  /Branch if (R9) > (R8)/

```

Hỏi chương trình này có các phụ thuộc WAW, RAW và WAR?

22.

- a. Xác định các phụ thuộc RAW, WAR và WAW trong chuỗi lệnh sau:

```

I1: R1 = 100
I2: R1 = R2 + R4
I3: R2 = R4 - 25
I4: R4 = R1 + R3
I5: R1 = R1 + 30

```

- b. Đổi tên các thanh ghi ở câu (a) để tránh các vấn đề phụ thuộc. Xác định các tham chiếu đến các giá trị thanh ghi ban đầu bằng cách sử dụng chỉ số dưới "a" cho tham chiếu thanh ghi.
23. Cho hình vẽ thể hiện việc "phát lệnh theo thứ tự/hoàn thành lệnh theo thứ tự" ở hình vẽ dưới đây.

Decode		Execute		Write		Cycle
I1	I2					1
	I2		I1			2
	I2		I1			3
I3	I4		I2			4
I5	I6		I4	I3		5
I5	I6	I5	I3			6
		I5	I6			7
				I3	I4	8
						9
				I5	I6	

- a. Xác định lý do có khả năng nhất khiến I2 không thể bước vào giai đoạn thực thi cho đến chu kỳ thứ tư. Liệu "phát lệnh theo thứ tự/hoàn thành không theo thứ tự" hoặc "phát lệnh không theo thứ tự/hoàn thành không theo thứ tự" có khắc phục được điều này không? Nếu vậy, cái nào?
- b. Xác định lý do tại sao I6 không thể vào giai đoạn ghi cho đến chu kỳ thứ chín. Liệu "phát lệnh theo thứ tự/hoàn thành không theo thứ tự" hoặc "phát lệnh không theo thứ tự/hoàn thành không theo thứ tự" có khắc phục được điều này không? Nếu vậy, cái nào?

CHƯƠNG 3. HỆ THỐNG BỘ NHỚ

Mặc dù khái niệm có vẻ đơn giản nhưng bộ nhớ máy tính lại là thành phần đa dạng nhất về công nghệ, tổ chức, hiệu suất và giá thành trong hệ thống máy tính. Không có một công nghệ riêng lẻ nào là tối ưu để đáp ứng mọi yêu cầu về bộ nhớ cho một hệ thống máy tính. Kết quả là, hệ thống máy tính thường được trang bị một hệ thống bộ nhớ phân cấp, một số loại bộ nhớ bên trong hệ thống (truy cập trực tiếp bởi bộ xử lý) và một số bên ngoài (bộ xử lý truy cập được thông qua một mô-đun I/O). Chương này tập trung vào các thành phần bộ nhớ trong và bộ nhớ ngoài.

3.1. TỔNG QUAN HỆ THỐNG NHỚ

3.1.1. Các đặc tính của hệ thống bộ nhớ

Để trình bày về bộ nhớ máy tính dễ dàng hơn, ta phân loại các hệ thống bộ nhớ dựa theo các đặc tính chính của chúng. Các đặc tính quan trọng nhất được liệt kê trong Bảng 3.1.

Thuật ngữ **vị trí** trong bảng 3.1 đề cập đến việc bộ nhớ nằm bên trong hay bên ngoài máy tính. Bộ nhớ trong thường được hiểu tương đương với bộ nhớ chính. Nhưng còn có những dạng bộ nhớ trong khác. Bộ xử lý cần có bộ nhớ cục bộ của nó, ở dạng thanh ghi. Hơn nữa, như chúng ta thấy, khối điều khiển của bộ xử lý cũng có thể yêu cầu bộ nhớ trong của riêng nó. Cache là một dạng khác của bộ nhớ trong. Bộ nhớ ngoài bao gồm các thiết bị lưu trữ ngoài, chẳng hạn như đĩa và băng, có thể được bộ xử lý truy cập tới thông qua bộ điều khiển I/O.

Bảng 3.1. Đặc tính quan trọng của hệ thống bộ nhớ máy tính

Vị trí	Hiệu suất
Bên trong (thanh ghi, cache, bộ nhớ chính)	Thời gian truy cập
Bên trong (đĩa quang, đĩa từ, băng)	Chu kỳ nhớ
Dung lượng	Tốc độ truyền tải
Số byte	Loại vật lý
Số từ	Bán dẫn
Đơn vị truyền	Từ
Word	Quang học
Block	Quang từ
Phương pháp truy cập	Tính chất vật lý
Tuần tự	Khả biến/Bất biến
Trực tiếp	Xóa được/Không xóa được
Ngẫu nhiên	Tổ chức
Kết hợp	Môđun bộ nhớ

Một đặc tính rõ ràng của bộ nhớ là **dung lượng**. Đối với bộ nhớ trong, dung lượng thường được biểu diễn theo byte (1 byte = 8 bit) hoặc từ. Độ dài từ phổ biến là 8, 16 và 32 bit. Dung lượng bộ nhớ ngoài thường được biểu diễn theo byte. Một khái niệm liên quan khác là **đơn vị truyền tải**. Đối với bộ nhớ trong, đơn vị truyền tải bằng số đường dây điện đi vào và đi ra khỏi môđun bộ nhớ. Nó có thể bằng độ dài từ, nhưng thường là lớn hơn, chẳng hạn 64, 128 hoặc 256 bit. Để hiểu rõ hơn, hãy xem xét ba khái niệm liên quan của bộ nhớ trong:

- **Từ nhớ (Word):** là đơn vị của tổ chức bộ nhớ. Kích thước của word thường bằng số bit được sử dụng để biểu diễn một số nguyên và bằng độ dài lệnh. Tuy nhiên, có nhiều trường hợp ngoại lệ. Ví dụ: CRAY C90 (mẫu siêu máy tính CRAY cũ) có độ dài từ 64 bit nhưng biểu diễn số nguyên bằng 46 bit. Kiến trúc Intel x86 có nhiều độ dài lệnh khác nhau còn kích thước word bằng 32 bit.
- **Đơn vị địa chỉ:** Trong một số hệ thống, đơn vị địa chỉ chính là word. Tuy nhiên, nhiều hệ thống định địa chỉ ở mức byte. Tổng quát, nếu địa chỉ dài A bit thì số lượng đơn vị địa chỉ $N = 2^A$.
- **Đơn vị truyền tải:** Đối với bộ nhớ chính, đơn vị truyền tải là số bit được đọc hoặc ghi vào bộ nhớ tại một thời điểm. Đơn vị truyền tải không nhất thiết phải bằng từ hoặc đơn vị địa chỉ. Đối với bộ nhớ ngoài, dữ liệu thường được truyền theo từng khối (khối lớn hơn nhiều so với một từ).

Phương pháp truy cập đơn vị dữ liệu cũng là một đặc điểm phân biệt các loại bộ nhớ khác nhau. Có những phương pháp truy cập sau:

- **Truy cập tuần tự:** Bộ nhớ được tổ chức thành các đơn vị dữ liệu, được gọi là bản ghi. Việc truy cập phải thực hiện theo một dãy tuần tự. Thông tin địa chỉ đã lưu được sử dụng để phân biệt các bản ghi và hỗ trợ cho quá trình truy cập. Phương pháp này sử dụng cơ chế đọc-ghi chia sẻ, tức là phải di chuyển từ vị trí hiện tại đến vị trí mong muốn, bỏ qua các bản ghi trung gian. Do đó, thời gian để truy cập tới một bản ghi tùy ý là rất biến động. Băng là một ví dụ sử dụng truy cập tuần tự.
- **Truy cập trực tiếp:** Sử dụng cơ chế đọc-ghi chia sẻ. Tuy nhiên, mỗi khối hoặc bản ghi có một địa chỉ duy nhất dựa trên vị trí thực tế. Truy cập được thực hiện bằng cách truy cập trực tiếp sẽ tìm đến một vùng lân cận, đếm, hoặc chờ cho tới khi đến được vị trí mong muốn. Thời gian truy cập cũng là biến động. Đĩa là một ví dụ sử dụng truy cập trực tiếp.
- **Truy cập ngẫu nhiên:** Mỗi vị trí trong bộ nhớ có cơ chế định địa chỉ riêng. Thời gian truy cập tới một vị trí xác định là không đổi và không phụ thuộc vào chuỗi các truy cập trước đó. Do đó, một vị trí bất kỳ có thể được lựa chọn ngẫu nhiên, được định địa chỉ và truy cập trực tiếp. Bộ nhớ chính và một số hệ thống cache là truy cập ngẫu nhiên.
- **Kết hợp:** Đây là một kiểu bộ nhớ truy cập ngẫu nhiên cho phép so sánh các vị trí bit mong muốn trong một word để tìm ra cặp giống nhau và thực hiện việc này đồng thời trên tất cả các word. Vì vậy, một word được lấy ra dựa trên một phần nội dung của nó chứ không dựa trên địa chỉ của nó. Giống như bộ nhớ truy cập ngẫu nhiên thông thường, mỗi vị trí có cơ chế định địa chỉ riêng; thời gian truy cập không đổi, không phụ thuộc vị trí hoặc các mẫu truy cập trước. Bộ nhớ cache có thể sử dụng kiểu truy cập kết hợp.

Theo quan điểm của người dùng, hai đặc điểm quan trọng nhất của bộ nhớ là dung lượng và hiệu suất. Ba tham số hiệu suất được sử dụng là:

- **Thời gian truy cập (độ trễ):** Đối với bộ nhớ truy cập ngẫu nhiên, đó là thời gian cần thiết để thực hiện một thao tác đọc hoặc ghi, tức là thời gian từ lúc bộ nhớ nhận được một địa chỉ đến lúc dữ liệu được lưu lại hoặc sẵn sàng để sử dụng. Đối với bộ nhớ không phải truy cập ngẫu nhiên, đó là thời gian cần thiết để cơ chế đọc-ghi đến được vị trí mong muốn.
- **Chu kỳ bộ nhớ:** Khái niệm này chủ yếu áp dụng cho bộ nhớ truy cập ngẫu nhiên và bằng thời gian truy cập cộng với thời gian cần thiết trước khi có thể bắt đầu truy cập thứ

hai. Lưu ý rằng chu kỳ bộ nhớ là khái niệm liên quan đến hệ thống bus, không liên quan đến bộ xử lý.

- **Tốc độ truyền tải:** Đây là tốc độ truyền dữ liệu vào hoặc ra khỏi một khối nhớ. Đối với bộ nhớ truy cập ngẫu nhiên, tốc độ truyền tải bằng $1/(\text{chu kỳ})$.

Bộ nhớ còn được phân biệt theo **kiểu vật lý**. Đó là bộ nhớ bán dẫn, bộ nhớ bề mặt từ (đĩa, băng), bộ nhớ quang học và bộ nhớ quang từ.

Một số đặc tính vật lý của lưu trữ dữ liệu là rất quan trọng. Trong bộ nhớ khả biến, thông tin bị phân rã một cách tự nhiên hoặc bị mất đi khi nguồn điện tắt. Trong bộ nhớ bất biến, thông tin một khi đã được ghi thì không bị suy giảm trừ khi có chủ đích thay đổi; không cần năng lượng điện để giữ lại thông tin. Các bộ nhớ bề mặt từ là bất biến. Bộ nhớ bán dẫn (bộ nhớ trên các mạch tích hợp) có thể là khả biến hoặc bất biến. Bộ nhớ không xóa được không thể bị thay đổi, trừ khi phá hủy khối lưu trữ. Bộ nhớ bán dẫn không xóa được được gọi là **bộ nhớ chỉ đọc** (ROM – read-only memory). Tất nhiên, bộ nhớ không xóa được trong thực tế cũng phải là bất biến.

Đối với bộ nhớ truy cập ngẫu nhiên, **tổ chức** là một vấn đề thiết kế chính. Ở đây, tổ chức là sự sắp xếp các bit để tạo thành word.

3.1.2. Phân cấp bộ nhớ

Các ràng buộc trong thiết kế bộ nhớ máy tính có thể được tóm gọn bằng ba câu hỏi: Dung lượng bao nhiêu là đủ? Tốc độ ra sao? Giá thành như thế nào?

Câu hỏi đầu tiên là câu hỏi mở. Dù dung lượng lớn thế nào cũng sẽ có nhiều ứng dụng được phát triển để sử dụng hết dung lượng đó. Câu hỏi về tốc độ, theo nghĩa nào đó, dễ trả lời hơn. Để đạt được hiệu suất cao nhất, bộ nhớ phải có khả năng theo kịp bộ xử lý. Tức là, khi bộ xử lý đang thực thi lệnh, ta không muốn nó phải tạm dừng để chờ các lệnh hoặc toán hạng. Về câu hỏi cuối, đối với một hệ thống thực tế, giá thành bộ nhớ phải hợp lý cân đối với giá thành của các thành phần khác.

Cần có sự cân bằng giữa dung lượng, thời gian truy cập và giá thành bộ nhớ. Giữa ba đặc điểm này tồn tại mối quan hệ như sau:

- Truy cập nhanh hơn, chi phí cho mỗi bit lớn hơn
- Dung lượng lớn hơn, chi phí cho mỗi bit nhỏ hơn
- Dung lượng lớn hơn, truy cập chậm hơn

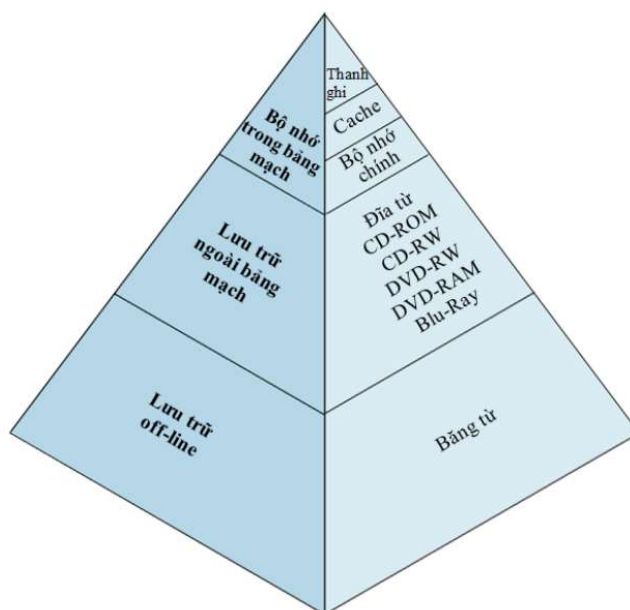
Người thiết kế luôn muốn sử dụng các công nghệ bộ nhớ cung cấp bộ nhớ dung lượng lớn, bởi vì dung lượng là rất cần thiết và đồng thời chi phí cho mỗi bit sẽ thấp. Tuy nhiên, để đáp ứng yêu cầu về hiệu suất, người thiết kế cần phải sử dụng những bộ nhớ đắt tiền, dung lượng tương đối nhỏ với thời gian truy cập nhanh.

Giải pháp cho vấn đề này là không phụ thuộc vào một thành phần hoặc công nghệ bộ nhớ riêng lẻ mà sử dụng một hệ thống phân cấp bộ nhớ. Một hệ thống phân cấp điển hình được mô tả trong hình 3.1. Xét từ trên xuống dưới trong hệ thống phân cấp, xu hướng như sau:

- a. Chi phí cho mỗi bit giảm
- b. Dung lượng tăng
- c. Thời gian truy cập dài hơn
- d. Giảm tần suất truy cập bộ nhớ của bộ xử lý

Do đó, bộ nhớ nhỏ hơn, đắt hơn, nhanh hơn được bổ sung bởi bộ nhớ lớn hơn, rẻ hơn, chậm hơn. Mấu chốt của cách tổ chức này xuất phát từ ý (d): giảm tần suất truy cập bộ nhớ. Việc sử

dụng hai cấp độ bộ nhớ để giảm thời gian truy cập trung bình về nguyên tắc có thể thực hiện được, nhưng chỉ khi áp dụng các điều kiện từ (a) đến (d). Bằng cách sử dụng các công nghệ khác nhau, sẽ có nhiều hệ thống bộ nhớ thỏa mãn các điều kiện từ (a) đến (c). Điều kiện (d) cũng thường có hiệu lực.



Hình 3.1. Phân cấp bộ nhớ

Cơ sở cho tính hợp lệ của điều kiện (d) là một nguyên tắc được gọi là **tính cục bộ của tham chiếu**. Trong quá trình thực thi một chương trình, bộ xử lý tham chiếu các lệnh và dữ liệu trong bộ nhớ. Các tham chiếu bộ nhớ này có xu hướng cụm. Các chương trình thường chứa một số vòng lặp và chương trình con. Mỗi khi vào một vòng lặp hoặc chương trình con, có nhiều tham chiếu lặp đi lặp lại tới một tập hợp nhỏ các lệnh. Tương tự, các thao tác trên bảng và mảng liên quan đến việc truy cập vào một cụm word dữ liệu. Trong một khoảng thời gian dài, các cụm được sử dụng sẽ thay đổi, nhưng trong một khoảng thời gian ngắn, bộ xử lý chủ yếu làm việc với các cụm tham chiếu bộ nhớ nhất định.

Theo đó, có thể tổ chức dữ liệu trên hệ thống phân cấp sao cho tỷ lệ truy cập vào các cấp thấp hơn nhỏ hơn đáng kể so với các cấp bộ nhớ phía trên. Xét ví dụ hệ thống nhớ hai cấp. Giả sử bộ nhớ cấp 2 chứa tất cả các lệnh và dữ liệu của chương trình. Các cụm hiện đang được tham chiếu có thể được đặt tạm thời ở bộ nhớ cấp 1. Đôi khi, một trong các cụm ở cấp 1 phải được đưa về cấp 2 để dành chỗ cho một cụm mới lên cấp 1. Tuy nhiên, đa phần các tham chiếu sẽ là tham chiếu tới lệnh và dữ liệu trong bộ nhớ cấp 1.

Nguyên tắc này có thể được áp dụng trên nhiều hơn hai cấp độ bộ nhớ, như hệ thống phân cấp được thể hiện trong hình 3.1. Loại bộ nhớ nhanh nhất, nhỏ nhất và đắt nhất là các thanh ghi bên trong bộ xử lý. Thông thường, một bộ xử lý chứa vài chục thanh ghi, tuy nhiên, một số khác có chứa hàng trăm thanh ghi. Bộ nhớ chính là hệ thống bộ nhớ chính bên trong máy tính. Mỗi vị trí trong bộ nhớ chính có một địa chỉ duy nhất. Bộ nhớ chính thường được mở rộng bằng bộ nhớ cache nhỏ hơn, tốc độ cao hơn. Cache thường không hiển thị với lập trình viên hoặc, thậm chí, với cả bộ xử lý. Nó là một thiết bị để di chuyển dữ liệu giữa bộ nhớ chính và thanh ghi của bộ xử lý để cải thiện hiệu suất.

Ba dạng bộ nhớ vừa được mô tả thường là khả biến và sử dụng công nghệ bán dẫn. Việc sử dụng ba cấp độ bộ nhớ lợi dụng thực tế rằng bộ nhớ bán dẫn rất đa dạng chủng loại, khác nhau

về tốc độ và chi phí. Dữ liệu được lưu trữ lâu hơn trên thiết bị lưu trữ bên ngoài, trong đó phổ biến nhất là ổ đĩa cứng, đĩa từ, băng và bộ nhớ quang học. Bộ nhớ ngoài bất biến còn được gọi là bộ nhớ thứ cấp hoặc bộ nhớ phụ. Các bộ nhớ này được sử dụng để lưu trữ các chương trình và các tập tin dữ liệu; thường hiển thị với lập trình viên ở mức tập tin và bản ghi. Đĩa cũng được sử dụng để cung cấp một phần mở rộng cho bộ nhớ chính được gọi là bộ nhớ ảo.

3.2. BỘ NHỚ CHÍNH

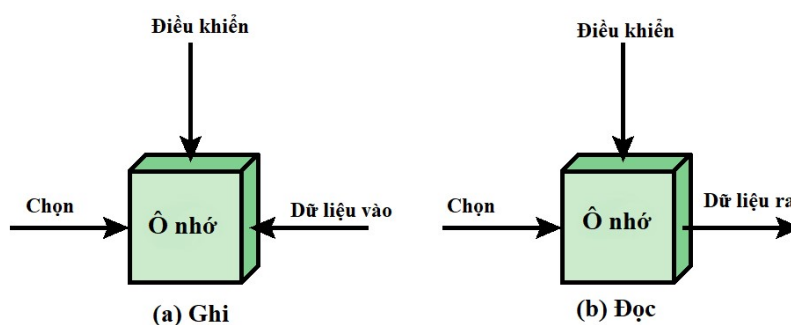
Trong các máy tính trước đây, bộ nhớ máy tính thường dưới dạng các vòng tròn từ tính được gọi là lõi (core). Ngày nay, với sự phát triển của công nghệ vi điện tử, các bộ nhớ chính được sản xuất dưới dạng chip bán dẫn đã thay thế bộ nhớ từ tính. Trong phần này, chúng ta sẽ tìm hiểu công nghệ bộ nhớ bán dẫn.

3.2.1. Tổ chức

Thành phần cơ bản của bộ nhớ bán dẫn là ô nhớ (memory cell). Mặc dù có nhiều công nghệ điện tử được sử dụng trong đó, tất cả các tế bào bộ nhớ bán dẫn đều có các tính chất chung như sau:

- Chúng có hai trạng thái ổn định biểu diễn tương ứng các số nhị phân 1 và 0.
- Chúng có khả năng được ghi vào (ít nhất một lần) trước khi thiết lập trạng thái ổn định.
- Chúng có khả năng đọc để cảm nhận được trạng thái.

Hình 3.2 mô tả hoạt động của một ô nhớ. Thông thường, một ô nhớ có ba dây dẫn nối vào có khả năng truyền tải được tín hiệu điện. Đường **Chọn** được dùng để chọn một ô nhớ để đọc hoặc viết. Đường **điều khiển** chỉ ra hoạt động đọc hoặc ghi đối với ô nhớ đó. Với hoạt động ghi, đường **dữ liệu vào** sẽ đưa vào trạng thái 0 hoặc 1. Với hoạt động đọc, đường **dữ liệu ra** sẽ cảm nhận trạng thái của ô nhớ và chuyển thành tín hiệu tương ứng đi ra. Chi tiết về tổ chức bên trong, chức năng của các ô nhớ phụ thuộc vào công nghệ mạch tích hợp sử dụng và nằm ngoài phạm vi của cuốn sách này. Ở đây, chúng ta sẽ coi nó là một ô nhớ riêng biệt có thể thực hiện các hoạt động đọc/ghi lên đó.



Hình 3.2. Các hoạt động của ô nhớ

3.2.2. DRAM và SRAM

Tất cả các loại bộ nhớ mà chúng ta sẽ tìm hiểu trong chương này là đều có cơ chế truy cập ngẫu nhiên: truy cập trực tiếp vào các vị trí trong bộ nhớ thông qua logic địa chỉ nối dây.

Bảng 3.2 liệt kê các loại bộ nhớ bán dẫn chính. Phổ biến nhất là bộ nhớ RAM – bộ nhớ truy cập ngẫu nhiên. Thực tế, đây chỉ là cách đặt tên, còn tất cả các loại bộ nhớ được liệt kê trong bảng đều có cơ chế truy cập ngẫu nhiên. Đặc điểm để phân biệt các loại bộ nhớ trong đó là bộ nhớ RAM cho phép đọc/ghi dữ liệu vào nó một cách dễ dàng và nhanh chóng. Cả hai hoạt động đọc và ghi đều được thực hiện thông qua việc sử dụng tín hiệu điện.

Một đặc điểm khác của bộ nhớ RAM khác với các bộ nhớ khác là dữ liệu ghi trên nó không ổn định khi không có nguồn điện duy trì (tính điện động). Do đó, RAM phải được cấp nguồn liên tục. Nếu nguồn cấp bị gián đoạn, dữ liệu trên RAM sẽ bị mất nên thông thường RAM chỉ có thể được sử dụng để lưu trữ dữ liệu tạm thời. Hai loại RAM truyền thống được sử dụng trong máy tính là RAM động (DRAM) và RAM tĩnh (SRAM).

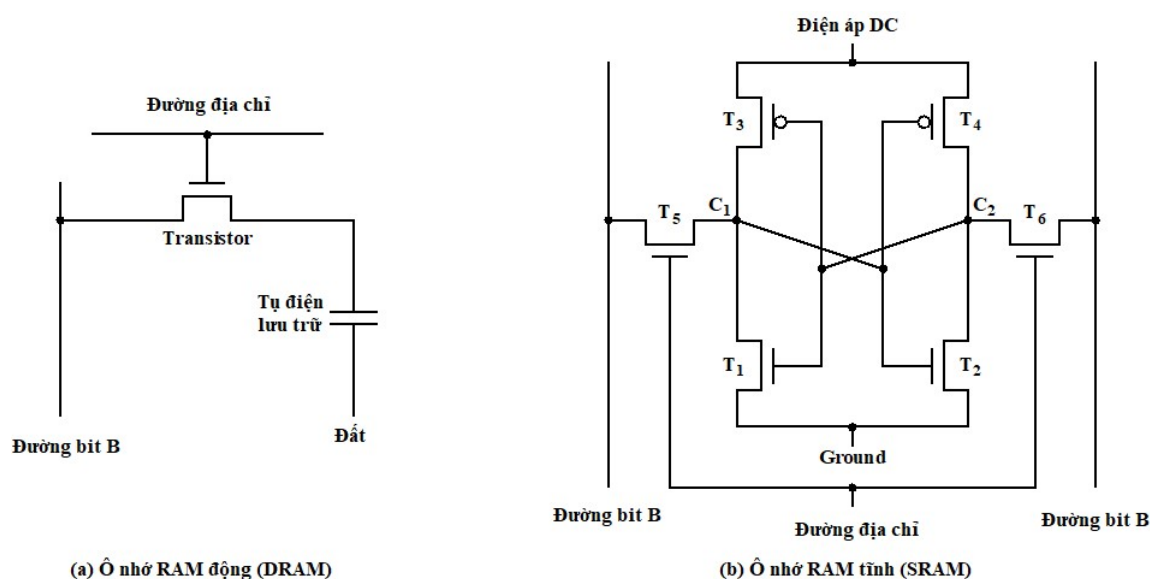
RAM ĐỘNG

Có hai công nghệ RAM chính là: RAM động và RAM tĩnh. RAM động (DRAM) có các ô nhớ lưu trữ dữ liệu dưới dạng các mạch nạp điện vào tụ. Việc có hoặc không có điện tích trên tụ tương ứng với một giá trị nhị phân 1 hoặc 0. Bởi vì điện tích trên tụ điện có xu hướng xả ra tự nhiên, RAM động đòi hỏi phải có dòng làm tươi định kỳ để duy trì lưu trữ dữ liệu.

Bảng 3.2. Các loại bộ nhớ bán dẫn

Loại bộ nhớ	Nhóm	Khả năng xóa	Cơ chế ghi	Tính chất
Bộ nhớ truy cập ngẫu nhiên (RAM)	Bộ nhớ đọc-ghi	Bằng điện, mức byte	Điện	Điện động
Bộ nhớ chỉ đọc (ROM)	Bộ nhớ chỉ đọc	Không được	Mặt nạ	Điện tĩnh
ROM lập trình được (PROM)			Bằng điện	
PROM xóa được	Bộ nhớ hầu như chỉ đọc	Tia UV, mức chip		
PROM xóa được bằng điện		Bằng điện, mức byte		
Bộ nhớ Flash		Bằng điện, mức khối		

Hình 3.3a là sơ đồ một ô nhớ DRAM điển hình. Đường địa chỉ được kích hoạt khi có một hoạt động đọc hoặc ghi đối với ô nhớ này. Transistor hoạt động như một công tắc được đóng lại (cho phép dòng điện đi qua) nếu điện áp được đặt lên đường địa chỉ và mở (không có dòng điện đi qua) nếu không có điện áp trên đường địa chỉ.



Hình 3.3. Cấu trúc ô nhớ bộ nhớ thông thường

Đối với hoạt động ghi, một tín hiệu điện áp được đặt lên đường Bit line; điện áp cao tương ứng với bit 1 còn điện áp thấp đại diện cho bit 0. Tín hiệu địa chỉ đưa vào đường địa chỉ sẽ đóng mạch công tắc cho phép nạp điện vào tụ.

Đối với hoạt động đọc, khi đường địa chỉ có tín hiệu, transistor sẽ đóng, điện tích lưu trữ trên tụ điện sẽ phóng ra đường bit line và đi đến một bộ khuếch đại cảm nhận. Bộ khuếch đại cảm nhận so sánh điện áp tụ điện với giá trị tham chiếu và xác định xem ô nhớ đó chứa giá trị logic 1 hay 0. Tín hiệu đọc ra từ ô nhớ sẽ giải phóng tụ điện do đó cần phải nạp lại điện áp cho tụ để khôi phục lại giá trị lưu trữ cho ô nhớ.

Mặc dù các ô nhớ DRAM lưu trữ một bit (0 hoặc 1), về cơ bản đây là một thiết bị tương tự. Các tụ điện có thể lưu trữ bất kỳ giá trị điện tích nào trong một khoảng nhất định; do đó cần có một giá trị ngưỡng xác định xem điện tích đó có giá trị 1 hoặc 0.

RAM TĨNH

RAM tĩnh (SRAM) là một thiết bị kỹ thuật số sử dụng các thành phần logic tương tự như trong bộ vi xử lý. Trong một SRAM, các giá trị nhị phân được lưu trữ bằng các mạch logic flip-flop truyền thống. Bộ nhớ RAM tĩnh lưu trữ dữ liệu lâu dài miễn là có nguồn điện được cấp cho nó.

Hình 3.3b là một ô nhớ SRAM điển hình. Bốn transistor $T1$, $T2$, $T3$, $T4$ được kết nối với nhau tạo ra trạng thái logic ổn định. Trong trạng thái logic 1, điểm $C1$ ở mức cao và điểm $C2$ ở mức thấp; $T1$ và $T4$ tắt và $T2$ và $T3$ đang bật. Với trạng thái logic 0, điểm $C1$ ở mức thấp và điểm $C2$ ở mức cao; $T1$ và $T4$ bật và $T2$ và $T3$ tắt. Cả hai trạng thái đều ổn định miễn là điện áp dc được cấp liên tục cho ô nhớ.

Tương tự DRAM, đường địa chỉ SRAM được sử dụng để mở hoặc đóng công tắc. Đường địa chỉ điều khiển hai transistor $T5$ và $T6$. Khi tín hiệu địa chỉ được đưa vào đường này, hai bóng bán dẫn được đóng, cho phép một hoạt động đọc hoặc ghi lên ô nhớ. Đối với hoạt động ghi, giá trị cần ghi vào được đưa vào đường B , trong khi giá trị bù của nó được đưa vào đường $\bar{B}$. Các giá trị này đặt các transistor $T1$, $T2$, $T3$, $T4$ ở vào một trạng thái thích hợp. Đối với hoạt động đọc, giá trị bit được đọc từ đường B .

SO SÁNH SRAM VÀ DRAM

Cả RAM tĩnh và RAM động đều là bộ nhớ điện động; nghĩa là nguồn điện phải liên tục được cấp cho bộ nhớ để duy trì các giá trị bit. Một ô nhớ của DRAM đơn giản hơn và nhỏ hơn ô nhớ của SRAM. Do đó, DRAM có mật độ dày hơn (do các ô nhớ nhỏ hơn = nhiều ô nhớ hơn trên một đơn vị diện tích) và ít tốn kém hơn SRAM tương ứng. Mặt khác, DRAM cần có các mạch làm tươi. Tuy nhiên, đối với những bộ nhớ lớn hơn, chi phí mạch làm tươi sẽ được bù lại bởi chi phí của các ô nhớ DRAM nhỏ hơn so với SRAM. Do đó, DRAM thường được sử dụng nhiều hơn để thực hiện các bộ nhớ có dung lượng lớn. Một điểm cuối cùng đó là tốc độ của SRAM nhanh hơn DRAM nên SRAM thường được sử dụng cho bộ nhớ cache (cả cache trên chip hoặc cache ngoài chip) còn DRAM được sử dụng cho bộ nhớ chính.

3.2.3. Các loại ROM – Bộ nhớ chỉ đọc

Bộ nhớ chỉ đọc (ROM) lưu trữ dữ liệu vĩnh viễn không thể thay đổi. ROM là bộ nhớ điện tĩnh do dữ liệu không bị mất đi ngay cả khi không có nguồn điện duy trì. Mặc dù có thể đọc dữ liệu từ ROM, nhưng không thể ghi dữ liệu mới vào đó. Ứng dụng quan trọng của ROM là vi chương trình và một số ứng dụng khác bao gồm:

- Thư viện các hàm thường xuyên được sử dụng
- Các chương trình hệ thống
- Các bảng chức năng

Đối với một số yêu cầu nhất định, ưu điểm của ROM là dữ liệu hoặc chương trình được lưu trữ vĩnh viễn và không bao giờ được nạp từ một thiết bị lưu trữ thứ hai.

ROM được chế tạo giống như các chip mạch tích hợp khác, dữ liệu được ghi vào chip như một phần của quá trình chế tạo. Điều này có phát sinh hai vấn đề:

- Chi phí cho việc ghi dữ liệu vào tương đối lớn dù rằng hàng ngàn bản ROM được sản xuất
- Không có chỗ cho sai sót. Nếu chỉ một bit bị ghi sai thì toàn bộ lô ROM sẽ bị hủy bỏ.

Khi chỉ cần một số lượng ít ROM lưu trữ nội dung nhất định, một loại ROM thay thế rẻ hơn là **PROM** (ROM có thể lập trình được). Giống như ROM, PROM là bộ nhớ điện tĩnh và chỉ có thể được ghi vào một lần. Quá trình ghi PROM được thực hiện bằng điện và có thể được thực hiện bởi nhà cung cấp hoặc khách hàng (không cần ghi trong quá trình sản xuất). Cần có một thiết bị đặc biệt để ghi hoặc "lập trình" lên chip. PROM linh hoạt và tiện lợi còn ROM thì phù hợp với việc sản xuất hàng loạt.

Một biến thể khác của bộ nhớ chỉ đọc là bộ nhớ hầu như chỉ đọc. Bộ nhớ dạng này phù hợp với các ứng dụng trong đó hoạt động đọc nhiều hơn rất nhiều so với hoạt động ghi nhưng cần lưu trữ dạng điện tĩnh. Có ba loại phổ biến: EPROM, EEPROM và bộ nhớ flash.

Bộ nhớ **EPROM** (Bộ nhớ chỉ đọc, Lập trình được, Xóa được) có thể đọc và ghi bằng điện giống như PROM. Tuy nhiên, trước khi thực hiện hoạt động ghi, tất cả các ô nhớ phải được xóa về trạng thái ban đầu bằng cách chiếu một tia cực tím mạnh qua một cửa sổ được thiết kế trên chip. Quá trình xóa này có thể thực hiện liên tục; mỗi lần xóa mất đến khoảng 20 phút. Do đó, nội dung trong EPROM có thể xóa đi, ghi lại nhiều lần và, giống như ROM và PROM, dữ liệu của nó có thể lưu trữ vĩnh viễn. Với dung lượng lưu trữ tương đương, EPROM đắt hơn PROM, nhưng ưu điểm là khả năng cập nhật lại dữ liệu nhiều lần.

Một công nghệ ROM khác là cũng rất phổ biến là bộ nhớ **EEPROM** (Bộ nhớ chỉ đọc, xóa được bằng điện). Với bộ nhớ này, các nội dung mới được ghi vào mà không cần xóa các nội dung trước đó; chỉ cập nhật địa chỉ byte hoặc các byte. Quá trình ghi diễn ra lâu hơn đáng kể so với quá trình đọc khoảng vài trăm micro giây cho mỗi byte. EEPROM kết hợp ưu điểm của tính điện tĩnh với tính linh hoạt trong việc cập nhật dữ liệu, sử dụng các đường bus dữ liệu, địa chỉ, điều khiển thông thường. EEPROM đắt hơn EPROM và mật độ bit cũng thưa hơn (ít bit hơn trong mỗi chip).

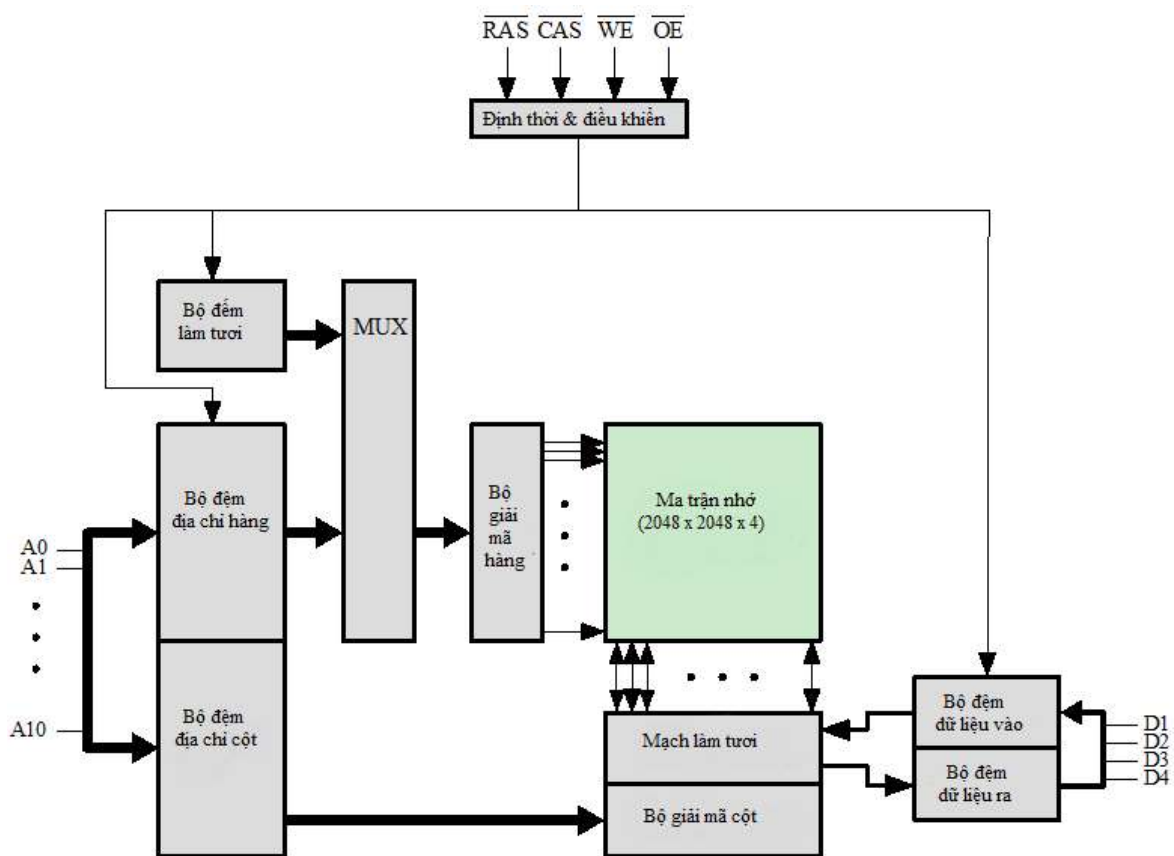
Một dạng bộ nhớ bán dẫn khác là **bộ nhớ flash**. Được giới thiệu lần đầu vào giữa những năm 1980, bộ nhớ flash nằm giữa EPROM và EEPROM về cả chi phí và chức năng. Giống như EEPROM, bộ nhớ flash sử dụng công nghệ xóa bằng điện có thể xóa toàn bộ bộ nhớ trong một hoặc vài giây, nhanh hơn nhiều so với EPROM. Ngoài ra, có thể xóa một khối bộ nhớ chứ không nhất định phải xóa toàn bộ chip. Tuy nhiên, bộ nhớ này không cho phép xóa mức byte. Giống như EPROM, bộ nhớ flash sử dụng chỉ một transistor cho mỗi bit do đó chip flash mật độ cao hơn EEPROM.

3.2.4. Chip Logic

Cũng như các sản phẩm mạch tích hợp, bộ nhớ bán dẫn được đóng gói trong chip (Hình 1.8) dưới dạng một ma trận các ô nhớ.

Trong hệ thống phân cấp bộ nhớ nói chung, chúng ta thấy rằng có sự tương quan giữa tốc độ, dung lượng và chi phí. Sự tương quan này cũng đóng một vai trò quan trọng trong việc thiết kế tổ chức các ô nhớ trên một chip và logic chức năng của chip đó. Đối với bộ nhớ bán dẫn, một trong những vấn đề thiết kế chính là số bit dữ liệu có thể được đọc/ghi cùng một lúc. Về mặt tổ chức, sự bố trí các ô nhớ trong ma trận giống như sự sắp xếp các từ trong bộ nhớ. Ma trận được tổ chức thành W từ, mỗi từ B bit. Ví dụ, một chip nhớ dung lượng 16-Mbit có thể được tổ chức thành 1M từ 16-bit. Một loại tổ chức bộ nhớ khác là 1-bit-mỗi-chip, trong đó dữ liệu được đọc/ghi một 1 bit tại một thời điểm. Phần tiếp theo chúng ta sẽ tìm hiểu tổ chức bộ nhớ DRAM, tổ chức ROM tương tự nhưng đơn giản hơn.

Hình 3.4 minh họa một tổ chức điển hình của bộ nhớ DRAM 16-Mbit với 4-bit được đọc/ghi tại một thời điểm. Về mặt logic, ma trận bộ nhớ được tổ chức thành bốn ma trận vuông 2048×2048 phần tử. Các phần tử của ma trận được nối với nhau theo cả hai chiều ngang (hàng) và chiều dọc (cột). Một đường ngang nối tất cả các đầu Select của tất cả các ô nhớ; một đường thẳng đứng nối các đầu Data-In/Sense của các ô nhớ.



Hình 3.4. Tổ chức bộ nhớ DRAM 16Mb ($4M \times 4$)

Đường địa chỉ chứa thông tin địa chỉ của từ cần được chọn. Số lượng các đường cần thiết sẽ là $\log_2 W$ (với W là số lượng từ nhớ trong bộ nhớ). Như trong ví dụ trên của chúng tôi, cần phải có ít nhất 11 đường địa chỉ để chọn ra được một trong 2048 hàng, 11 đường này được đưa vào bộ giải mã hàng: thực hiện việc chuyển từ 11 đường đầu vào thành 2048 đường đầu ra (nghĩa là từ 11 đường đầu vào, bộ giải mã này sẽ kích hoạt một đường đầu ra tương ứng trong 2048 đường nối hàng) ($2^{11} = 2048$).

11 đường địa chỉ nữa để chọn ra một trong 2048 cột (mỗi cột 4 bit). Bốn đường dữ liệu đưa dữ liệu vào hoặc ra từ ô nhớ thông qua một bộ đếm dữ liệu. Với đầu vào (ghi), bộ điều khiển bit của mỗi đường bit được kích hoạt giá trị 1 hoặc 0 theo giá trị của đường dữ liệu tương ứng. Với

đầu ra (đọc), giá trị của mỗi đường bit được truyền qua bộ khuếch đại cảm nhận và đưa ra đường dữ liệu. Đường hàng chọn ra một dòng các ô nhớ trong ma trận để đọc hoặc ghi.

Bởi vì DRAM này chỉ có 4 bit được đọc/ghi cùng lúc tại một thời điểm nên thường ta phải kết hợp nhiều DRAM bằng cách sử dụng một bộ điều khiển bộ nhớ (memory controller) để có thể thực hiện việc đọc/ghi một từ nhớ từ bus.

Lưu ý rằng, đầu vào chip DRAM chỉ có 11 đường địa chỉ (A0-A10), một nửa số lượng cần thiết cho ma trận nhớ 2048×2048 . Việc này được thực hiện để tiết kiệm cho số chân. 22 đường địa chỉ được đưa qua một bộ chọn bên ngoài chip và được ghép thành 11 đường địa chỉ và đưa vào chip theo hai nhíp. Lần đầu, 11 tín hiệu địa chỉ được truyền cho chip để xác định địa chỉ hàng của ma trận, và sau đó là 11 tín hiệu địa chỉ khác được đưa vào cho địa chỉ cột. Các tín hiệu $\overline{RAS}$ (tín hiệu chọn địa chỉ hàng) và $\overline{CAS}$ (tín hiệu chọn địa chỉ cột) sẽ được đưa vào cùng với hai nhíp trên để chip có thể xác định được địa chỉ hàng hay cột.

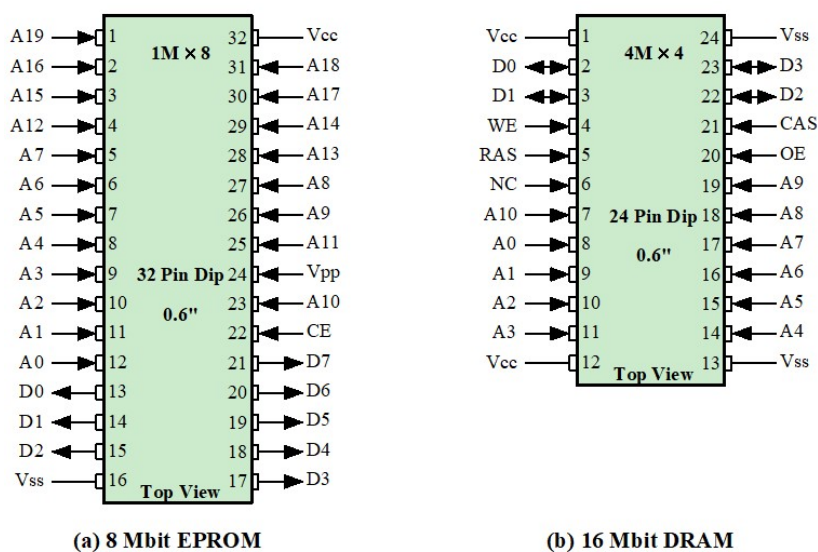
Chân $\overline{WE}$ (cho phép ghi) và $\overline{OE}$ (cho phép đọc) tương ứng với việc chỉ thị thực hiện hoạt động ghi/đọc lên chip. Hai chân khác không được thể hiện trong Hình 3.4 là chân Vss (chân nối đất) và chân Vcc (chân nối nguồn).

Ngoài ra, ta có thể thấy việc ghép kênh địa chỉ cùng với việc sử dụng ma trận nhớ vuông giúp cho việc gấp bốn lần kích thước bộ nhớ với mỗi thế hệ mới của chip bộ nhớ. Thêm một chân vào (pin) nữa dành cho đường địa chỉ làm tăng gấp đôi số hàng và số cột vì vậy kích thước của bộ nhớ tăng lên 4 lần.

Trong hình 3.4 cũng minh họa mạch làm tươi các ô nhớ của DRAM. Ta đã biết, tất cả các ô nhớ DRAM đòi hỏi phải làm tươi sau một thời gian nhất định. Trong quá trình làm tươi, các chip DRAM sẽ bị vô hiệu hóa một khoảng thời gian trong khi tất cả các ô nhớ dữ liệu được làm tươi. Bộ đếm làm tươi sẽ đến tất cả các giá trị hàng. Đối với mỗi hàng, dữ liệu thông qua mạch làm tươi sẽ được đọc ra và ghi vào cùng một vị trí, nhờ đó, dữ liệu trong mỗi ô nhớ được khôi phục lại mức điện áp chính xác.

3.2.5. Đóng gói chip

Như đã đề cập trong Chương 1, các mạch tích hợp được đóng gói trong các chip và có các chân (pin) để giao tiếp với bên ngoài.



Hình 3.5. Sơ đồ chân và tín hiệu của các chip bộ nhớ

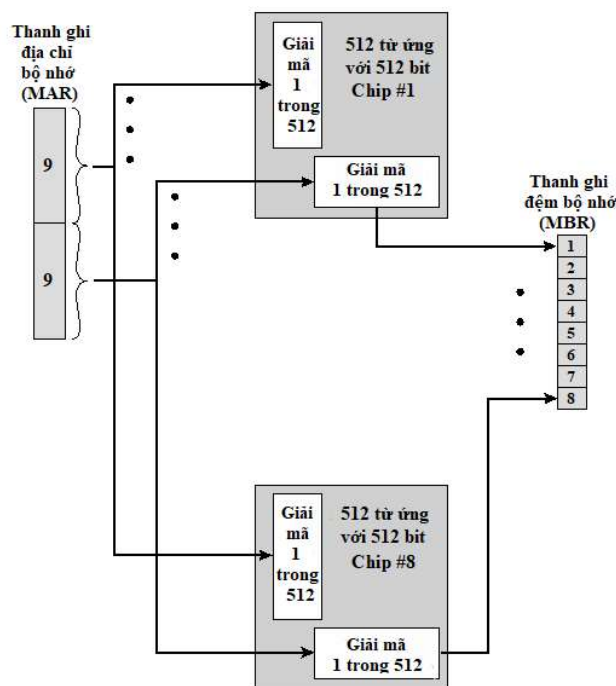
Hình 3.5a minh họa một chip EPROM. Chip có dung lượng 8Mbit được tổ chức thành $1M \times 8$. Số chân của chip là 32 chân với kích thước tiêu chuẩn. Các chân hỗ trợ các đường tín hiệu sau:

- Địa chỉ từ nhớ được truy cập. Với 1M từ nhớ cần tổng cộng 20 chân địa chỉ: A0 – A19
- 8 đường dữ liệu: D0 – D7
- Chân cấp nguồn Vcc
- Chân nối đất: Vss
- Chân CE (cho phép chip): vì có thể có nhiều chip cùng kết nối với cùng bus địa chỉ, chân CE sẽ chỉ ra địa chỉ đó có dành cho chip này hay chip khác. Chân CE được kích hoạt bởi một bộ logic nối với các bit trọng số cao của bus địa chỉ (ví dụ trong trường hợp này các bit địa chỉ trên A19)
- Chân Vpp (Điện áp chương trình) được cung cấp trong quá trình lập trình (các hoạt động ghi).

Hình 3.5b là một cấu hình DRAM 16-Mbit điển hình. Chip này được tổ chức dưới dạng $4M \times 4$. Có một số điểm khác biệt so với một chip ROM ở hình a như sau: Do RAM là bộ nhớ đọc/ghi nên các chân dữ liệu (D0 ÷ D3) có dạng đầu vào/đầu ra (mũi tên hai chiều). Các chân WE (Write Enable – Cho phép ghi) và OE (Output Enable – cho phép đưa dữ liệu ra) sẽ là các tín hiệu quyết định hoạt động ghi hay đọc. Do DRAM được truy cập theo hàng và cột và dữ liệu địa chỉ đưa vào theo phương thức ghép kênh nên chỉ cần có 11 chân địa chỉ vào để xác định một vị trí bộ nhớ ($2^{11} \times 2^{11} = 2^{22} = 4M$). Chức năng các chân RAS và CAS đã được trình bày ở trên. Cuối cùng, chân NC (No Connect – không có kết nối) được thêm vào để số chân của chip là số chẵn.

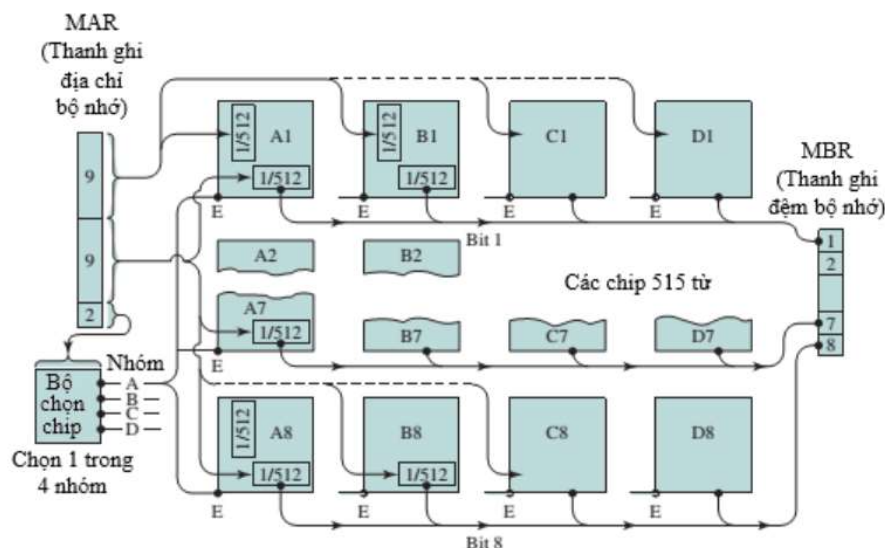
3.2.6. Tổ chức module nhớ

Nếu một chip RAM tổ chức dưới dạng 1 bit/một vị trí nhớ, thì rõ ràng chúng ta sẽ cần ít nhất số lượng chip bằng số bit của một từ nhớ. Ví dụ tổ chức một module bộ nhớ trong hình 3.6 có dung lượng $256K \times 8b$ (256K từ nhớ, mỗi từ 8-bit). Với 256K từ nhớ ta cần phải có 18-bit địa chỉ. Địa chỉ này được đưa vào 8 chip $256K \times 1b$ (mỗi chip cung cấp một đầu dữ liệu vào/ ra 1 bit).



Hình 3.6. Tổ chức bộ nhớ 256Kbyte

Tổ chức này hoạt động miễn là kích thước của bộ nhớ bằng số bit trong mỗi chip. Trong trường hợp cần phải xây dựng một module bộ nhớ có kích lớn hơn, ta phải tổ chức dưới dạng một ma trận chip. Hình 3.7 minh họa một tổ chức bộ nhớ kích thước 1M từ nhớ, mỗi từ 8b. Trong đó ta có bốn cột, mỗi cột có 8 chip kích thước $256K \times 1b$. Như vậy, kích thước một cột là $256K \times 8b$, hay 256K từ nhớ (mỗi từ 8b). Với 1M từ nhớ ta cần 20 đường địa chỉ, trong đó: 18 bit thấp được chuyển đến tất cả 32 chip, 2 bit cao được đưa vào một bộ giải mã ra thành một tín hiệu CS tới một trong bốn cột của module.



Hình 3.7. Tổ chức bộ nhớ 1Mbyte

3.2.7. Tổ chức bộ nhớ đan xen

Bộ nhớ chính gồm một tập các chip nhớ DRAM. Một số chip có thể được nhóm lại với nhau để tạo thành một dải bộ nhớ. Các dải bộ nhớ này có thể được tổ chức theo cách “đan xen”: Mỗi dải nhớ có thể phục vụ yêu cầu đọc hoặc ghi một cách độc lập, như vậy một hệ thống nhớ gồm K dải nhớ có thể phục vụ K yêu cầu cùng một lúc, tăng hiệu năng đọc và ghi của bộ nhớ lên K lần. Nếu các từ liên tiếp của bộ nhớ được lưu giữ tại các dải nhớ khác nhau thì rõ ràng việc truyền một khối dữ liệu sẽ được thực hiện song song trên cả K dải nhớ, như vậy tốc độ truyền nhanh hơn rất nhiều.

3.2.8. Tổ chức dram mở rộng

Như Chương 2 đã đề cập, một trong những yếu tố ảnh hưởng đến hiệu năng của một hệ thống máy tính đó là tình trạng nghẽn do việc giao tiếp giữa bộ xử lý và bộ nhớ chính. Bộ xử lý có tốc độ và hiệu năng ngày càng cao, nếu bộ nhớ không đáp ứng được nó sẽ gây tình trạng trên. Trong nhiều thập kỷ, khối cơ bản của bộ nhớ chính vẫn là chip DRAM và kiến trúc này hầu như có rất ít sự thay đổi kể từ đầu những năm 1970. Do đó, việc cải thiện chip DRAM truyền thống là một trong những yêu cầu quan trọng trong quá trình phát triển của các hệ thống máy tính.

Ở phần trước chúng ta đã có một kiến trúc giúp cải thiện hiệu suất của bộ nhớ chính DRAM đó là việc sử dụng một hoặc nhiều cache SRAM tốc độ cao. Tuy nhiên, do SRAM đắt hơn DRAM nên việc tăng kích thước SRAM quá lớn lại làm giảm lợi nhuận hệ thống.

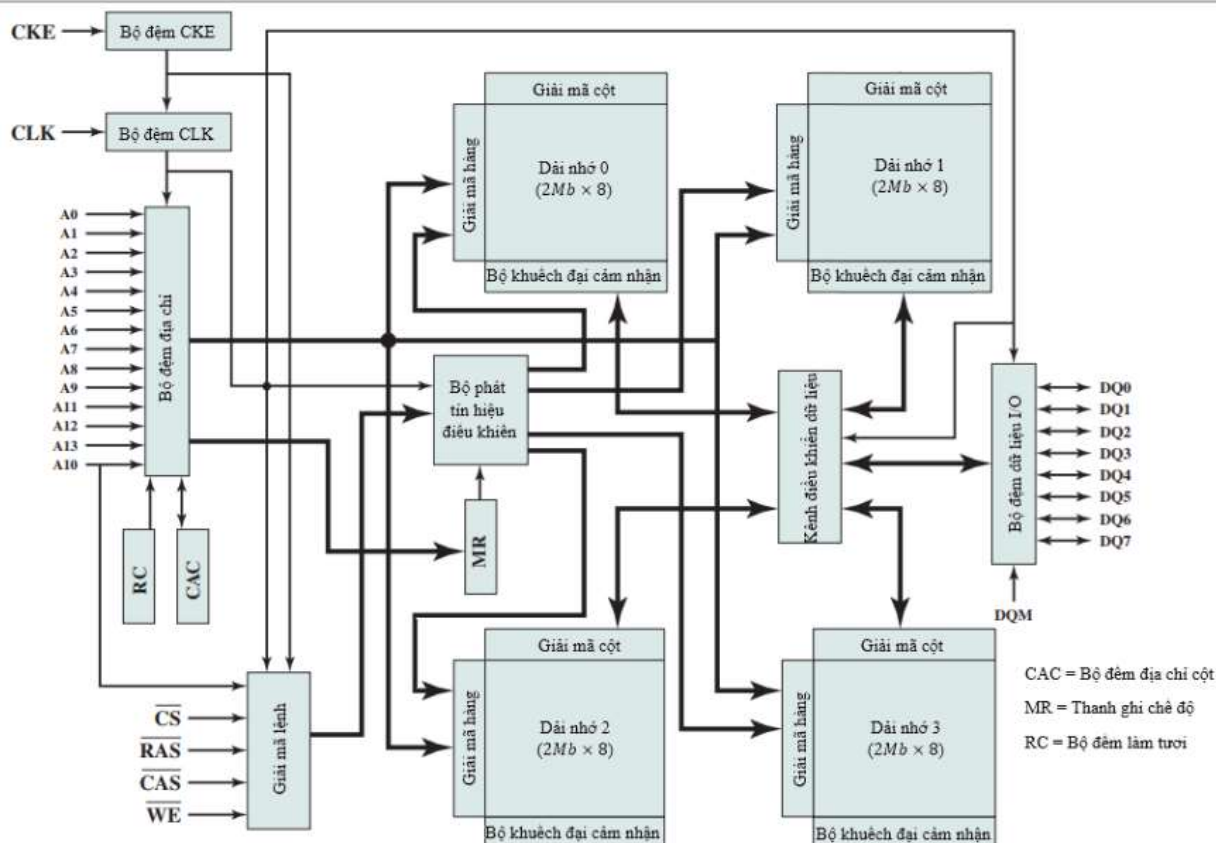
Song song với nó, trong những năm gần đây, người ta cũng đưa ra một số cải tiến trong kiến trúc DRAM, trong đó một số sản phẩm được sử dụng rộng rãi trên thị trường là SDRAM, DDR-DRAM và RDRAM. Bảng 3.3 so sánh hiệu năng của các kiến trúc này.

Bảng 3.3. So sánh hiệu suất của các chip DRAM thay thế

	Tần số đồng hồ	Tốc độ truyền	Thời gian truy cập	Số lượng chân
SDRAM	166	1.3	18	168
DDR	200	3.2	12.5	184
RDRAM	600	4.8	12	162

SDRAM – DRAM đồng bộ

Một trong những loại DRAM được sử dụng rộng rãi nhất là SDRAM (DRAM đồng bộ). Không giống như DRAM truyền thống, việc trao đổi dữ liệu của SDRAM với bộ xử lý được đồng bộ với tín hiệu đồng hồ và chạy với tốc độ cao của bộ xử lý/bus bộ nhớ mà không cần chịu trạng thái chờ đợi.



Hình 3.8. DRAM đồng bộ (SDRAM)

Với DRAM thông thường, bộ xử lý gửi địa chỉ và tín hiệu điều khiển đến bộ nhớ, chỉ ra vị trí cụ thể của dữ liệu trong cần được đọc ra hoặc ghi vào DRAM. Sau một khoảng trễ nhất định (thời gian truy cập) DRAM thực hiện việc ghi hoặc đọc dữ liệu. Khoảng thời gian trễ truy cập này là khoảng thời gian để DRAM thực hiện một số chức năng bên trong, ví dụ như kích hoạt các đường địa chỉ hàng và cột, cảm nhận dữ liệu hoặc đưa dữ liệu ra bộ đệm đầu ra. Bộ vi xử lý phải chờ đợi khoảng thời gian trễ này do đó làm chậm hiệu suất hệ thống.

Với truy cập đồng bộ, SDRAM đưa dữ liệu vào hoặc ra dưới sự điều khiển của đồng hồ hệ thống. Bộ vi xử lý thực thi lệnh và thông tin địa chỉ. SDRAM sẽ trả lời sau một số chu kỳ đồng hồ nhất định. Như vậy, bộ xử lý có thể thực hiện các tác vụ khác trong khi SDRAM xử lý yêu cầu.

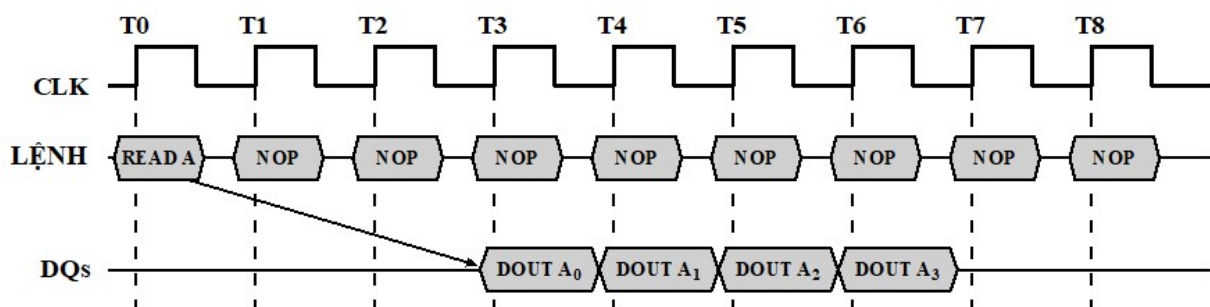
Hình 3.8 là tổ chức bộ nhớ SDRAM IBM 64-MB, Bảng 3.4 liệt kê các chân của chip. SDRAM sử dụng một chế độ truyền nhóm (burst mode) để giảm bớt thời gian thiết lập địa chỉ và thời gian nạp địa chỉ hàng và cột. Với chế độ này, một chuỗi bit có thể được truy cập một cách nhanh chóng sau lần truy cập đầu tiên.

Bảng 3.4. Các chân SDRAM

Từ A0 đến A13	Các đầu vào địa chỉ
CLK	Đầu vào tín hiệu đồng hồ
CKE	Cho phép đồng hồ
CS	Chọn Chip
RAS	Chuyển địa chỉ hàng
CAS	Chuyển địa chỉ cột
WE	Cho phép ghi
DQ0 to DQ7	Vào/ra dữ liệu
DQM	Mặt nạ dữ liệu

Thanh ghi chế độ MR và bộ điều khiển là một điểm khác biệt của SDRAM so với DRAM thông thường. Nó cung cấp một cơ chế tùy chỉnh SDRAM cho phù hợp với các yêu cầu cụ thể của hệ thống. Thanh ghi MR xác định độ dài nhóm (burst): số đơn vị dữ liệu được đặt đồng bộ vào bus. Thanh ghi này cũng cho phép người lập trình có thể điều chỉnh độ trễ giữa việc nhận yêu cầu đọc và việc bắt đầu truyền dữ liệu.

SDRAM hoạt động tốt nhất khi truyền một khối dữ liệu lớn ví dụ như dữ liệu của các ứng dụng xử lý văn bản, bảng tính và các ứng dụng đa phương tiện.



Hình 3.9. Tiến trình thời gian hoạt động Đọc SDRAM (kích thước burst = 4, độ trễ $\overline{CSA} = 2$)

Hình 3.9 là ví dụ về hoạt động của SDRAM với kích thước burst là 4 và độ trễ là 2. Lệnh Đọc nhóm bắt đầu khi tín hiệu ở các chân $\overline{CS}$ và $\overline{CAS}$ ở mức thấp trong khi $\overline{RAS}$ và $\overline{WE}$ ở mức cao tại sườn lên của xung đồng hồ. Tín hiệu địa chỉ xác định địa chỉ cột bắt đầu của burst, thanh ghi MR thiết lập kiểu burst (tuần tự hoặc đan xen) và độ dài burst (1, 2, 4, 8 hoặc toàn bộ page). Độ trễ từ lúc bắt đầu lệnh cho đến khi dữ liệu từ ô nhớ đầu tiên xuất hiện ở đầu ra bằng giá trị độ trễ $\overline{CAS}$ đặt trong thanh ghi MR.

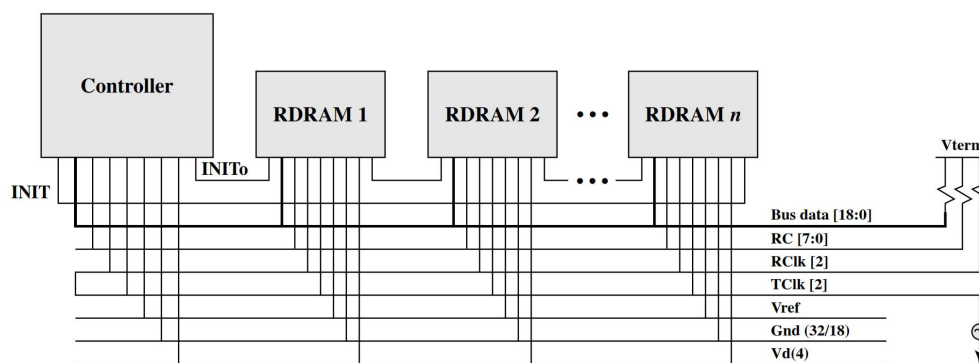
Một phiên bản nâng cấp của SDRAM là DDR-SDRAM (SDRAM tốc độ dữ liệu gấp đôi). Với SDRAM, ta thấy mỗi chu kỳ xung nhịp một dữ liệu được gửi đi, còn với DDR SDRAM dữ liệu được gửi hai lần trong mỗi chu kỳ đồng hồ.

Rambus DRAM

RDRAM là loại RAM được Intel sử dụng cho các bộ vi xử lý Pentium và Itanium và trở thành đối thủ cạnh tranh chính của SDRAM. Chip RDRAM đóng gói theo chiều dọc, với tất cả các chân ở một bên. Chip trao đổi dữ liệu với bộ vi xử lý thông qua 28 dây, chiều dài chip ngắn hơn 12 cm. Bus có thể định địa chỉ được đến 320 chip RDRAM với tốc độ lên đến 1.6 GBps.

Bus RDRAM truyền thông tin địa chỉ và điều khiển bằng cách sử dụng giao thức hướng khối không đồng bộ. Sau khoảng thời gian 480 ns truy cập ban đầu, nó cho tốc độ truyền dữ liệu là 1.6 GBps. Thay vì bị điều khiển bởi các tín hiệu RAS, CAS, R/W, CE như với DRAM thông thường, RDRAM nhận các yêu cầu bộ nhớ qua một bus tốc độ cao. Yêu cầu này chứa thông tin địa chỉ, loại hoạt động (đọc hoặc ghi) và số byte hoạt động yêu cầu.

Hình 3.10 minh họa cấu trúc RDRAM. Cấu trúc bao gồm một bộ điều khiển và một số module RDRAM được kết nối với nhau thông qua một bus chung. Bộ điều khiển được đặt ở một đầu của chip, các đường bus được nối song song. Bus gồm có 18 đường dữ liệu (16 đường dữ liệu thực, hai đường chẵn lẻ) hoạt động với tốc độ gấp đôi tín hiệu đồng hồ; nghĩa là 1 bit được gửi ở sườn lên và 1 bit được gửi ở sườn xuống của xung đồng hồ. Nhờ đó, tốc độ tín hiệu trên mỗi đường là 800 Mbps. Một nhóm gồm 8 đường RC được sử dụng để truyền các tín hiệu địa chỉ và điều khiển. Tín hiệu đồng hồ được truyền từ đầu bên kia đến bộ điều khiển sau đó vòng trở lại. Một module RDRAM gửi dữ liệu đến bộ điều khiển đồng bộ với tín hiệu đồng hồ, bộ điều khiển gửi dữ liệu đến một RDRAM đồng bộ với tín hiệu đồng hồ theo hướng ngược lại. Các đường bus còn lại bao gồm đường điện áp tham chiếu, đường nối đất và đường nối nguồn.



Hình 3.10. Cấu trúc RDRAM

DDR SDRAM

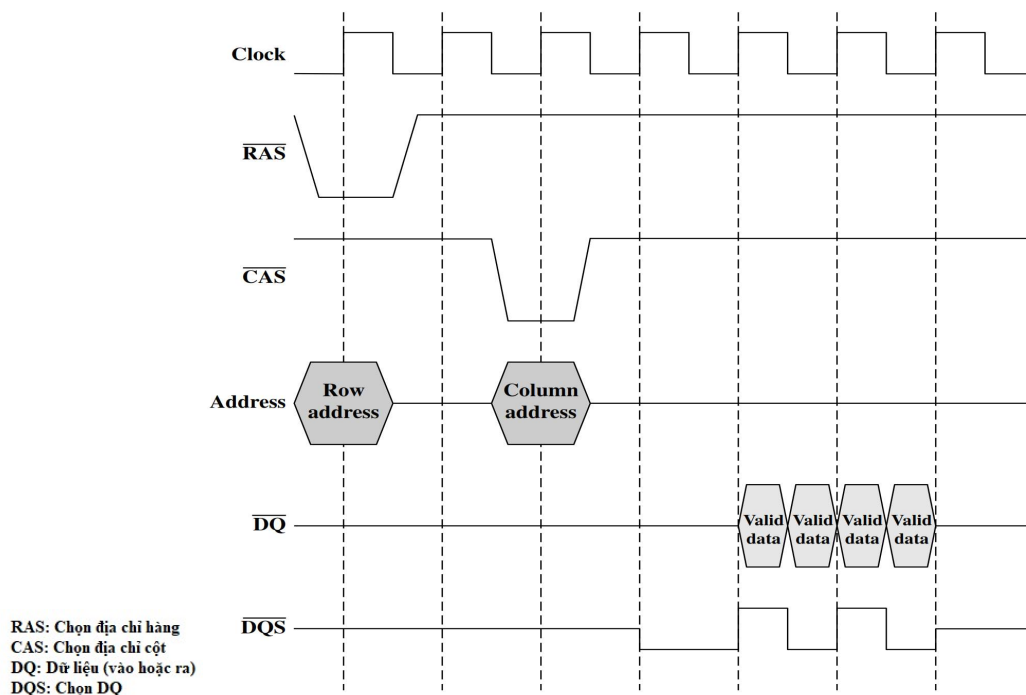
Như phần trên đã đề cập, tốc độ SDRAM bị hạn chế vì nó chỉ có thể gửi dữ liệu đến bộ xử lý một lần mỗi chu kỳ đồng hồ bus. DDR SDRAM cải tiến hơn SDRAM ở chỗ nó cho phép gửi dữ liệu hai lần mỗi chu kỳ đồng hồ, một ở sườn lên và một ở sườn xuống của xung đồng hồ.

DDR DRAM được đưa ra bởi Hiệp hội Công nghệ bán dẫn JEDEC và đã được nhiều công ty sản xuất cũng như được sử dụng rộng rãi trong các máy tính để bàn và máy chủ.

Hình 3.11 là sơ đồ thời gian hoạt động Đọc DDR. Việc truyền dữ liệu được đồng bộ với sườn lên và xuống của xung đồng hồ. Nó cũng được đồng bộ với tín hiệu dữ liệu hai chiều (DQS) do bộ điều khiển bộ nhớ phát ra khi thực hiện hoạt động Đọc và do DRAM phát ra trong quá trình Ghi.

Tiếp đến là các phiên bản cải tiến của công nghệ DDR: DDR2, DDR3,... Trong đó, DDR2 tăng tốc độ truyền dữ liệu bằng cách tăng tần số hoạt động của chip RAM và tăng bộ đệm tiền truy xuất từ 2 bit lên thành 4 bit cho mỗi chip. Bộ đệm tiền truy xuất một bộ nhớ cache nằm trên chip RAM, nó cho phép chip RAM chuẩn bị trước vị trí các bit để đặt chúng vào bus dữ liệu càng nhanh càng tốt. DDR3 được giới thiệu trong năm 2007, trong đó kích thước bộ đệm tăng lên tới 8 bit.

Về mặt lý thuyết, một module DDR có thể truyền dữ liệu với tốc độ khoảng từ 200 đến 600 MHz; một module DDR2 truyền với tốc độ từ 400 đến 1066 MHz; và tốc độ của DDR3 lên từ 800 đến 1600 MHz. Tuy nhiên trong thực tế, tốc độ này có thể nhỏ hơn.



Hình 3.11. Đồ thị thời gian DDR SDRAM

Cache DRAM

CDRAM - Cache DRAM được phát triển bởi hãng Mitsubishi. Bộ nhớ này tích hợp một cache SRAM nhỏ (16 Kb) vào chip DRAM. Bộ nhớ SRAM này được sử dụng theo hai cách: như một bộ nhớ cache thực sự với các line 64-bit hoặc như một bộ đệm hỗ trợ việc truy cập liên tiếp một khối dữ liệu.

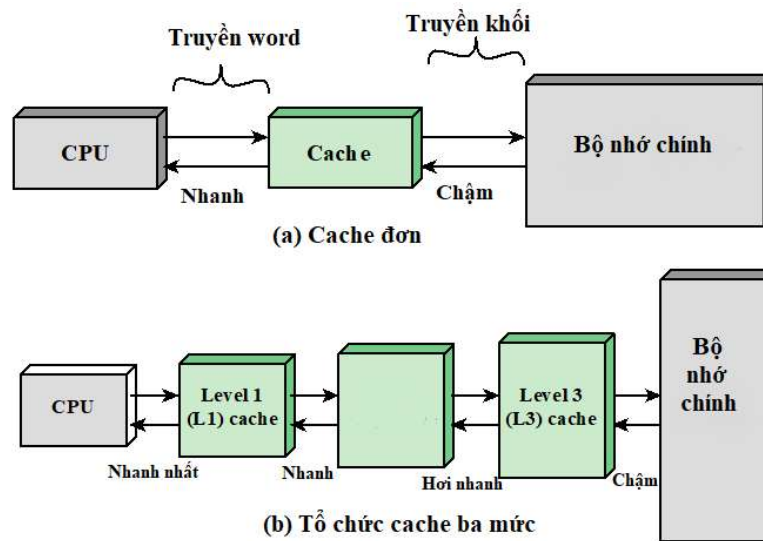
3.3. BỘ NHỚ ĐỆM

3.3.1. Nguyên lý bộ nhớ cache

Bộ nhớ Cache được thiết kế để kết hợp thời gian truy cập của bộ nhớ tốc độ cao, đắt tiền với dung lượng lớn của bộ nhớ tốc độ thấp và rẻ hơn. Điều này được minh họa trong Hình 3.12a. Trong đó, có một bộ nhớ chính tương đối lớn và chậm cùng với một bộ nhớ cache nhỏ hơn, nhanh hơn. Cache chứa bản sao của một phần bộ nhớ chính.

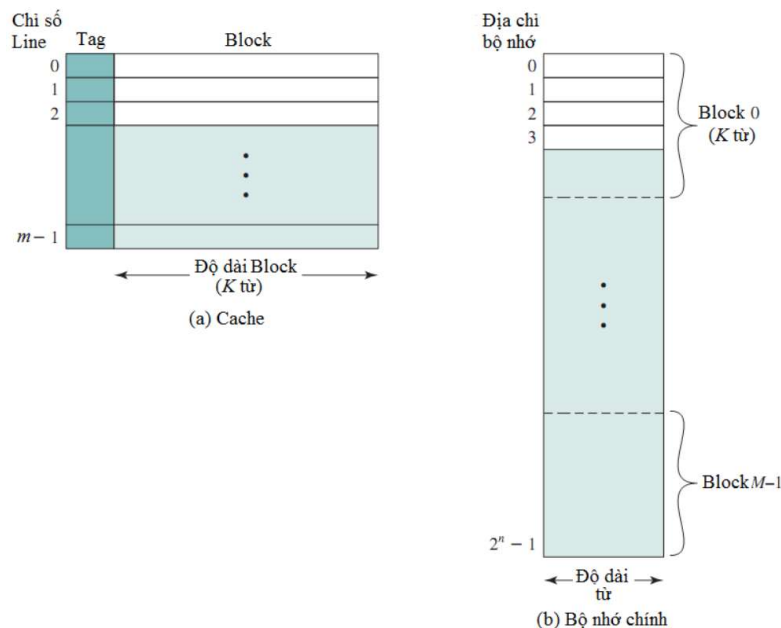
Khi bộ xử lý muốn đọc một word của bộ nhớ chính, trước tiên phải kiểm tra để xác định xem word đó có nằm trong cache hay không. Nếu có, word này được truyền từ cache đến bộ xử lý. Nếu không, một khối trong bộ nhớ chính, gồm một vài word có chứa word mong muốn, được đọc vào cache và rồi word đó được gửi đến bộ xử lý. Do tính cục bộ của tham chiếu, khi một khối dữ liệu được truy xuất vào bộ nhớ cache để đáp ứng một tham chiếu bộ nhớ, có khả năng là sắp tới sẽ có sự tham chiếu tới cùng vị trí bộ nhớ đó hoặc tới các word khác trong khối đó.

Hình 3.12b miêu tả việc sử dụng nhiều cấp bộ nhớ cache. Cache L2 chậm hơn và thường lớn hơn cache L1, cache L3 chậm hơn và thường lớn hơn cache L2.



Hình 3.12. Cache và bộ nhớ chính

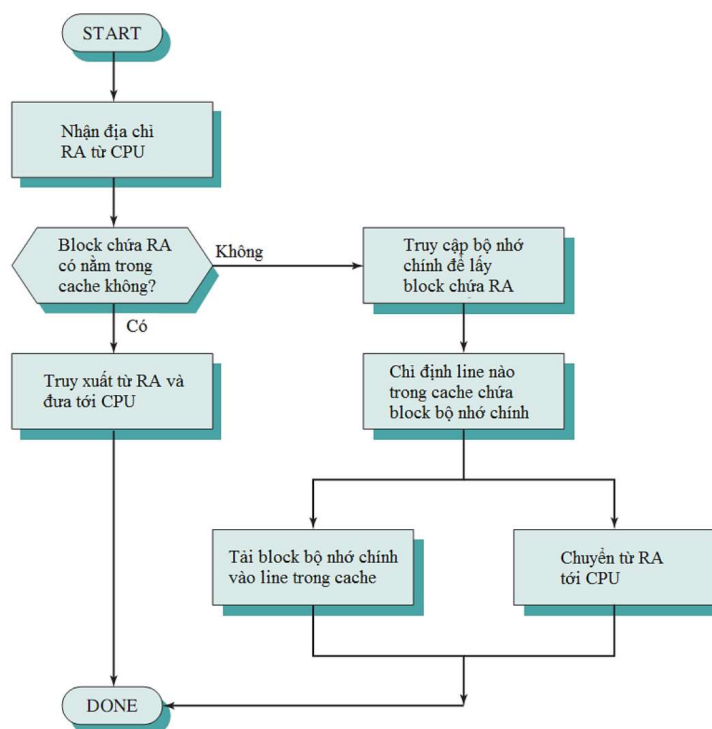
Hình 3.13 mô tả cấu trúc của một hệ thống cache/bộ nhớ chính. Bộ nhớ chính có tới 2^n word, mỗi word có một địa chỉ n bit duy nhất. Vì mục đích ánh xạ, bộ nhớ này được coi là bao gồm K khối có độ dài cố định. Tức là, có $M = 2^n/K$ khối trong bộ nhớ chính. Bộ nhớ cache bao gồm m khối, được gọi là **line**. Mỗi line có K word, cộng với một vài bit của trường tag. Mỗi line còn chứa các bit điều khiển (không được thể hiện), chẳng hạn như một bit cho biết line đó kể từ khi được nạp vào cache đã được thay đổi hay chưa. Độ dài của một line, không bao gồm tag và các bit điều khiển, được gọi là **kích thước line**. Kích thước line nhỏ nhất là 32 bit, với mỗi "word" là một byte; trong trường hợp này kích thước line là 4 byte. Số lượng line ít hơn đáng kể so với số lượng khối bộ nhớ chính ($m \ll M$).



Hình 3.13. Cấu trúc Cache/Bộ nhớ chính

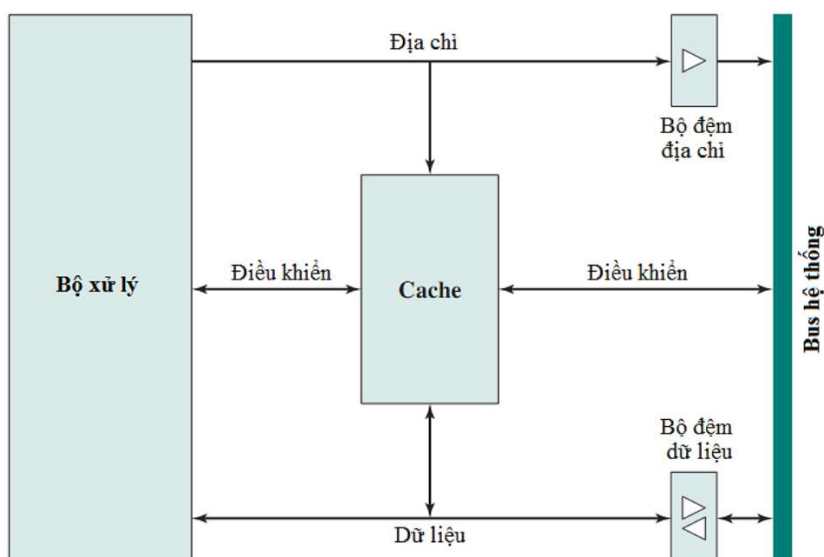
Tại một thời điểm, một vài tập con của các khối bộ nhớ chính nằm trong các line của cache. Nếu một word trong một khối của bộ nhớ chính được đọc, khối đó được truyền đến một trong các

line của cache. Bởi vì có nhiều khối hơn line, một line sẽ không được dành riêng vĩnh viễn cho một khối cụ thể. Do đó, mỗi line bao gồm một **tag** để xác định khối nào đang được lưu trữ. Tag thường là một phần của địa chỉ bộ nhớ chính.



Hình 3.14. Hành động đọc cache

Hình 3.14 mô tả hành động đọc. Bộ xử lý tạo ra địa chỉ đọc (RA - read address) của một word cần đọc. Nếu word đã nằm trong bộ nhớ cache, nó sẽ được gửi đến bộ xử lý. Nếu không, khối chứa word đó được nạp vào bộ nhớ cache, và word được gửi đến bộ xử lý. Hình 3.14 cho thấy hai hành động cuối cùng diễn ra song song và phản ánh tổ chức điển hình của bộ nhớ cache như mô tả trong hình 3.15.



Hình 3.15. Tổ chức cache điển hình

Trong tổ chức này, cache kết nối với bộ xử lý thông qua các đường dữ liệu, điều khiển và địa chỉ. Đường dữ liệu và địa chỉ còn nối tới bộ đếm dữ liệu và địa chỉ; các bộ đếm này lại nối tới bus hệ thống để từ đó nối tới bộ nhớ chính. Khi có một cache hit, bộ đếm dữ liệu và bộ đếm địa

chỉ bị vô hiệu hóa, chỉ có giao tiếp giữa bộ xử lý và cache, không có lưu lượng trên bus hệ thống. Khi có một cache miss, địa chỉ mong muốn được tải lên bus hệ thống và dữ liệu được đưa ra qua bộ đệm dữ liệu tới cả bộ nhớ cache và bộ xử lý. Trong cách tổ chức khác, cache được đặt giữa bộ xử lý và bộ nhớ chính đối với tất cả đường dữ liệu, địa chỉ và điều khiển. Trong trường hợp này, đối với một cache miss, trước tiên word mong muốn được đọc vào cache rồi sau đó chuyển từ cache sang bộ xử lý.

3.3.2. Các yếu tố thiết kế cache

Mặc dù có nhiều cách thực hiện bộ nhớ cache, có một vài yếu tố thiết kế cơ bản được dùng để phân loại và phân biệt các kiến trúc bộ nhớ cache. Bảng 3.5 liệt kê các yếu tố chính đó.

Bảng 3.5. Các yếu tố thiết kế cache

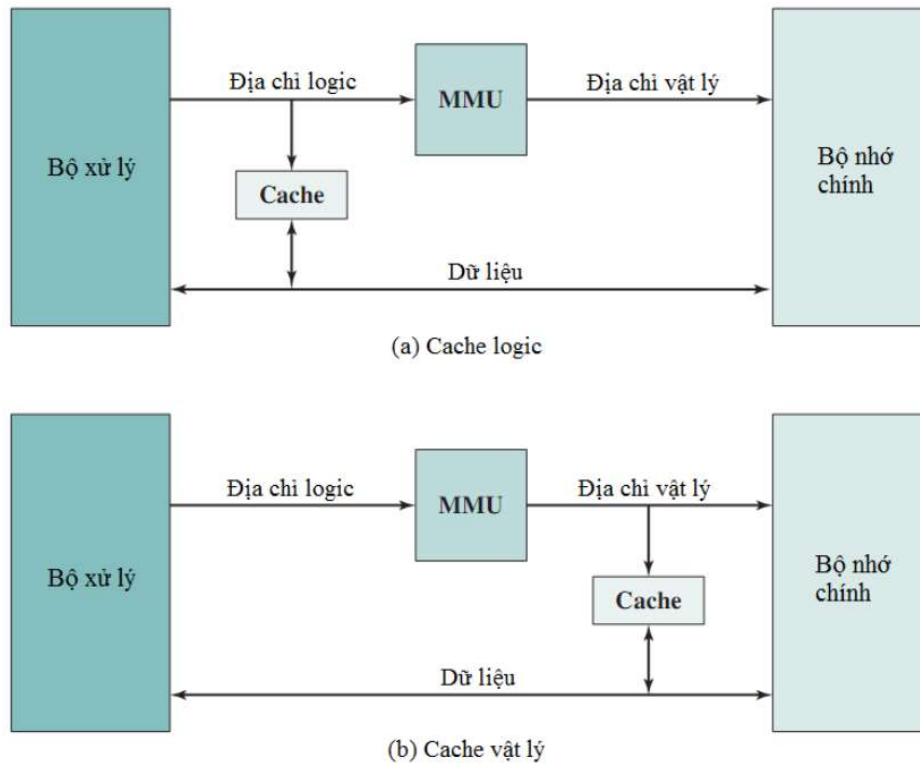
Địa chỉ cache	Chính sách ghi
Logic	Write through
Vật lý	Write back
Kích thước cache	Kích thước line
Hàm ánh xạ	Số lượng cache
Trực tiếp	Một cấp hay hai cấp
Kết hợp	Thống nhất hay phân chia
Kết hợp tập hợp	
Thuật toán thay thế	
Ít được sử dụng gần đây nhất (LRU)	
Vào trước ra trước (FIFO)	
Ít được sử dụng nhất (LFU)	
Ngẫu nhiên	

Địa chỉ cache

Hầu hết các bộ xử lý không nhúng, và nhiều bộ vi xử lý nhúng, có hỗ trợ bộ nhớ ảo. Về cơ bản, bộ nhớ ảo là một tiện ích cho phép các chương trình định địa chỉ bộ nhớ từ quan điểm logic, mà không cần quan tâm dung lượng vật lý sẵn có của bộ nhớ chính. Khi bộ nhớ ảo được sử dụng, các trường địa chỉ của lệnh máy chứa các địa chỉ ảo. Để đọc và ghi ở bộ nhớ chính, một đơn vị quản lý bộ nhớ (MMU) sẽ dịch địa chỉ ảo sang địa chỉ vật lý trong bộ nhớ chính.

Khi địa chỉ ảo được sử dụng, nhà thiết kế hệ thống có thể lựa chọn để đặt cache giữa bộ xử lý và MMU hoặc giữa MMU và bộ nhớ chính (Hình 3.16). Cache logic, còn được gọi là cache ảo, lưu trữ dữ liệu bằng địa chỉ ảo. Bộ xử lý truy cập cache trực tiếp mà không phải đi qua MMU. Cache vật lý lưu trữ dữ liệu nhờ sử dụng địa chỉ vật lý của bộ nhớ chính.

Một ưu điểm rõ ràng của cache logic là tốc độ truy cập bộ nhớ cache nhanh hơn so với tốc độ truy cập cache vật lý, vì cache có thể đáp ứng trước khi MMU thực hiện dịch địa chỉ. Nhược điểm là hầu hết các hệ thống bộ nhớ ảo cung cấp cho các ứng dụng cùng một không gian địa chỉ bộ nhớ ảo giống nhau. Tức là, mỗi ứng dụng đều thấy một bộ nhớ ảo bắt đầu ở địa chỉ 0. Do đó, cùng một địa chỉ ảo trong hai ứng dụng khác nhau tương ứng với hai địa chỉ vật lý khác nhau. Vì vậy, bộ nhớ cache phải được làm sạch hoàn toàn với mỗi chuyển đổi ngữ cảnh của ứng dụng hoặc phải thêm các bit bổ sung vào mỗi line của cache để xác định xem địa chỉ này tương ứng với không gian địa chỉ ảo nào.



Hình 3.16. Cache vật lý và cache logic
Bảng 3.6. Kích thước cache của một số bộ xử lý

Bộ xử lý	Loại	Năm	Cache L1	Cache L2	Cache L3
IBM 360/85	Mainframe	1968	16 to 32 kB	—	—
PDP-11/70	Minicomputer	1975	1 kB	—	—
VAX 11/780	Minicomputer	1978	16 kB	—	—
IBM 3033	Mainframe	1978	64 kB	—	—
IBM 3090	Mainframe	1985	128 to 256 kB	—	—
Intel 80486	PC	1989	8 kB	—	—
Pentium	PC	1993	8 kB/8 kB	256 to 512 KB	—
PowerPC 601	PC	1993	32 kB	—	—
PowerPC 620	PC	1996	32 kB/32 kB	—	—
PowerPC G4	PC/server	1999	32 kB/32 kB	256 KB to 1 MB	2 MB
IBM S/390 G6	Mainframe	1999	256 kB	8 MB	—
Pentium 4	PC/server	2000	8 kB/8 kB	256 KB	—
IBM SP	High-end server/supercomputer	2000	64 kB/32 kB	8 MB	—
CRAY MTA ^b	Supercomputer	2000	8 kB	2 MB	—
Itanium	PC/server	2001	16 kB/16 kB	96 KB	4 MB
Itanium 2	PC/server	2002	32 kB	256 KB	6 MB
IBM POWER5	High-end server	2003	64 kB	1.9 MB	36 MB
CRAY XD-1	Supercomputer	2004	64 kB/64 kB	1MB	—
IBM POWER6	PC/server	2007	64 kB/64 kB	4 MB	32 MB
IBM z10	Mainframe	2008	64 kB/128 kB	3 MB	24-48 MB
Intel Core i7 EE 990	Workstation/server	2011	6 × 32 kB/32 kB	1.5 MB	12 MB
IBM zEnterprise 196	Mainframe/Server	2011	24 × 64 kB/128 kB	24 × 1.5 MB	24 MB L3 192 MB L4

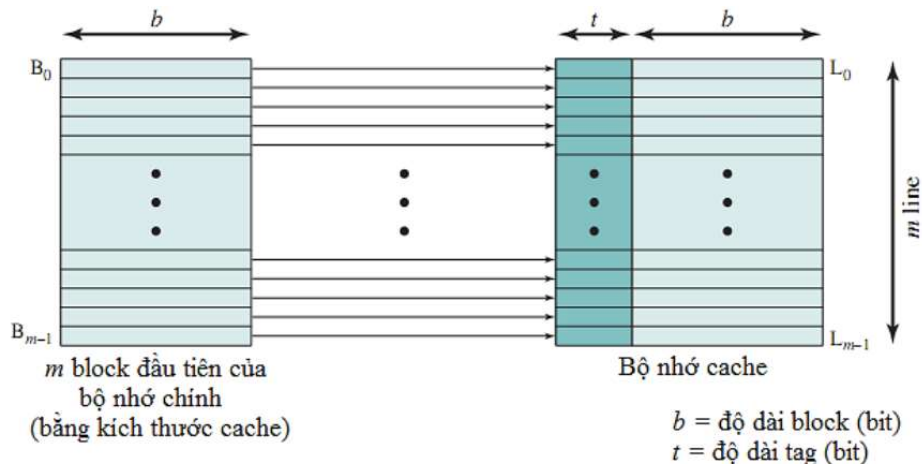
Lưu ý:

^a Hai giá trị được ngăn cách bởi dấu gạch chéo tương ứng với cache lệnh và cache dữ liệu.

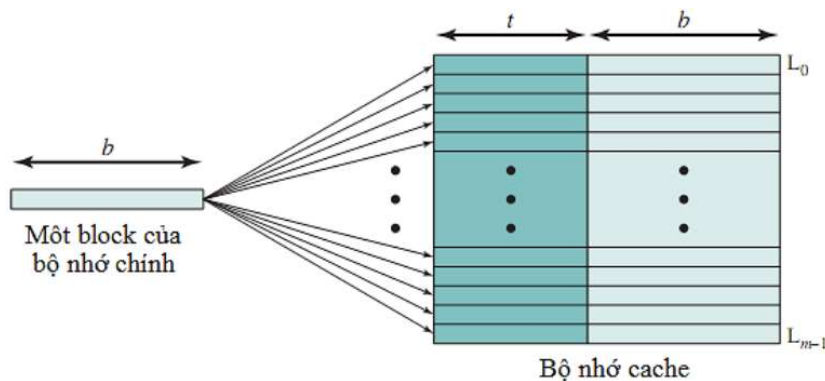
^b Cả hai cache đều là cache lệnh; không có cache dữ liệu.

Kích thước cache

Chúng ta muốn kích thước cache đủ nhỏ để tổng chi phí trung bình cho mỗi bit gần bằng với chi phí đó khi chỉ sử dụng riêng bộ nhớ chính và đủ lớn để tổng thời gian truy cập trung bình gần bằng với thời gian đó khi chỉ sử dụng riêng cache. Có một số động lực khác để giảm thiểu kích thước cache. Cache càng lớn thì có càng nhiều cổng logic tham gia vào việc định địa chỉ cache. Kết quả là các bộ nhớ cache lớn có xu hướng chậm hơn cache nhỏ - kể cả khi được xây dựng bằng cùng công nghệ mạch tích hợp và đặt ở cùng một vị trí trên chip và bảng mạch. Diện tích chip và bảng mạch cũng giới hạn kích thước bộ nhớ cache. Có thể nói, không có một kích thước cache nào là "tối ưu". Bảng 3.6 liệt kê các kích thước bộ nhớ cache của một số bộ xử lý từ trước tới nay.



(a) Ảnh xạ trực tiếp



(b) Ảnh xạ kết hợp

Hình 3.17. Ảnh xạ từ Bộ nhớ chính sang Cache: Trực tiếp và Kết hợp

3.3.3. Hàm ánh xạ

Bởi vì số lượng line trong cache ít hơn số khối bộ nhớ chính, cần có thuật toán ánh xạ các khối bộ nhớ chính vào các line trong cache. Hơn nữa, cần có phương tiện để xác định khối bộ nhớ chính nào đang chiếm một line cache. Hàm ánh xạ được sử dụng sẽ quyết định cách thức tổ chức cache. Ba kỹ thuật ánh xạ có thể được sử dụng là: trực tiếp, kết hợp toàn phần và tập kết hợp (hình 3.17). Ta sẽ lần lượt tìm hiểu từng kỹ thuật thông qua ví dụ cụ thể.

Ví dụ 3.1 Đối với cả ba trường hợp, có một số giả thiết sau:

- Cache có thể chứa 64 Kbyte.
- Dữ liệu được truyền giữa bộ nhớ chính và cache theo từng khối 4 byte. Do đó, cache gồm $16K = 214$ line, mỗi line có 4 byte.

Ánh xạ trực tiếp

Đây là kỹ thuật ánh xạ đơn giản nhất, ánh xạ mỗi khối của bộ nhớ chính vào một line cache xác định duy nhất. Hàm ánh xạ được biểu diễn bởi

$$i = j \bmod m$$

trong đó:

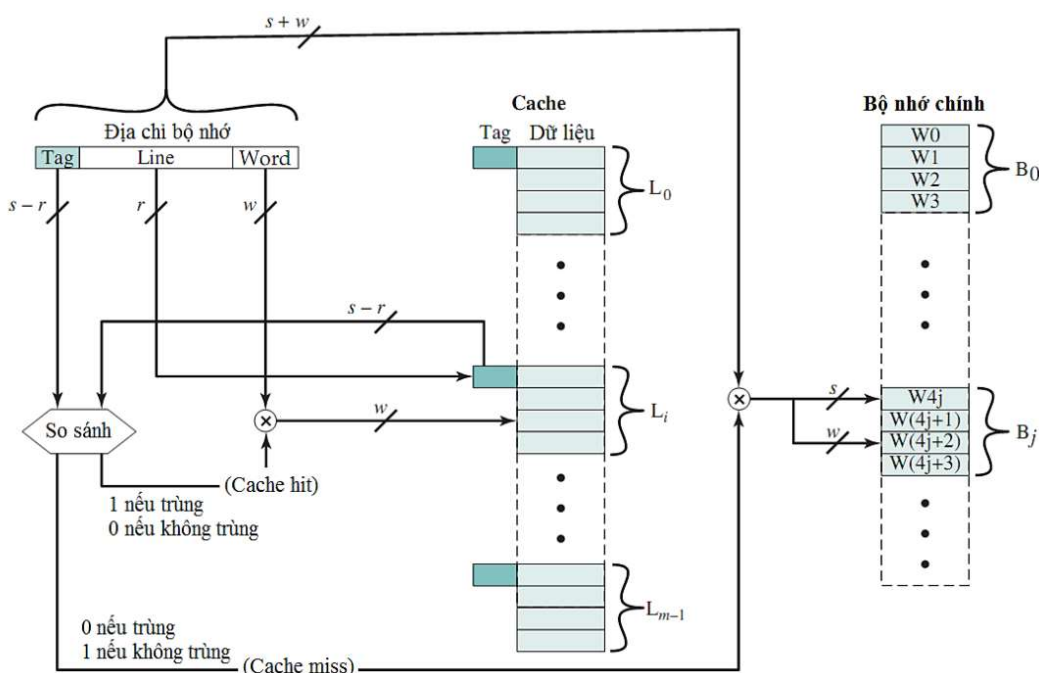
i = chỉ số line cache

j = chỉ số khối bộ nhớ chính

m = tổng số line trong cache

Hình 3.17a mô tả sự ánh xạ m khối đầu tiên của bộ nhớ chính. Mỗi khối trong bộ nhớ chính được ánh xạ vào một line duy nhất trong cache. m khối tiếp theo của bộ nhớ chính cũng được ánh xạ vào cache theo cách tương tự; tức là, khối B_m của bộ nhớ chính ánh xạ vào line L_0 of cache, khối B_{m+1} ánh xạ vào line $L_1, \dots$

Hàm ánh xạ này được thực hiện dễ dàng bằng địa chỉ bộ nhớ chính. Hình 3.18 mô tả cơ chế tổng quát. Đối với mục đích truy cập bộ nhớ cache, mỗi địa chỉ bộ nhớ chính được xem như bao gồm ba trường. w bit có trọng số nhỏ nhất xác định một word hoặc một byte trong một khối của bộ nhớ chính; trong hầu hết các máy tính hiện đại, địa chỉ được đánh ở mức byte. s bit còn lại xác định một khối trong 2^s khối của bộ nhớ chính. Logic cache chia s bit này thành trường tag $s - r$ bit (phần bit có trọng số lớn nhất) và trường line r bit.



Hình 3.18. Tổ chức cache ánh xạ trực tiếp

Trường line xác định một line trong $m = 2^r$ line của cache. Tóm lại:

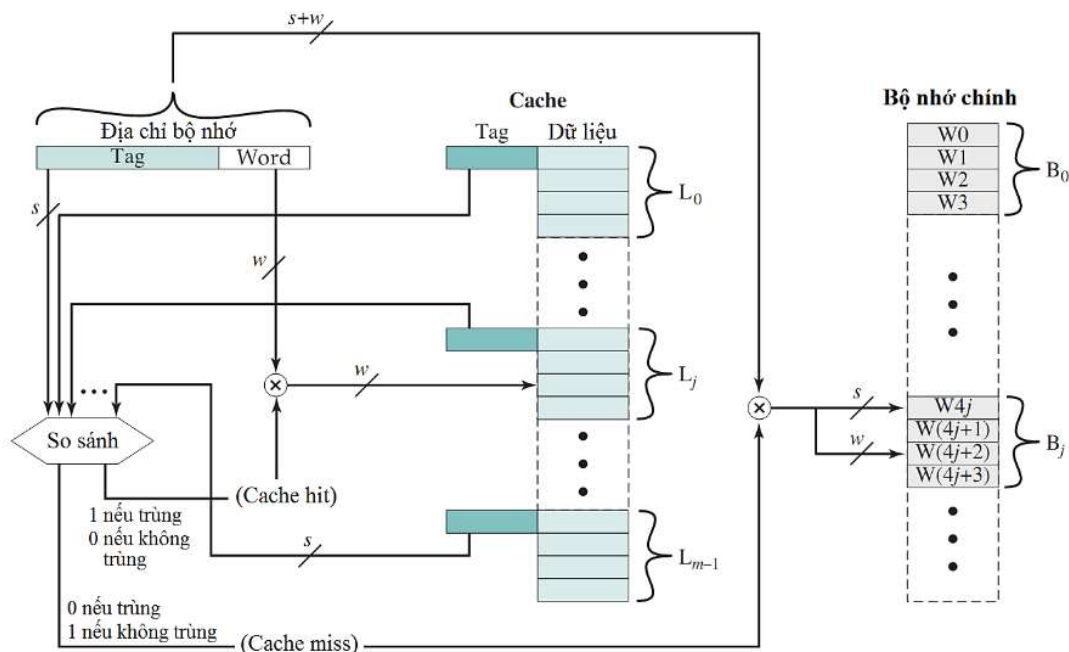
- Độ dài địa chỉ = $(s + w)$ bits
- Số lượng đơn vị địa chỉ = 2^{s+w} word hoặc byte
- Kích thước khối = kích thước line = 2^w word hoặc byte
- Số khối trong bộ nhớ chính = $2^{s+w}/2^w = 2^s$
- Số line trong cache = $m = 2^r$
- Kích thước cache = 2^{r+w} word hoặc byte
- Kích thước tag = $(s - r)$ bit

Kỹ thuật ánh xạ trực tiếp đơn giản và không tốn kém để thực hiện. Nhược điểm chính là chỉ có một vị trí cố định trong cache dành cho một khối bất kỳ. Vì vậy, nếu một chương trình tham chiếu lặp đi lặp lại các word thuộc hai khối khác nhau nhưng được ánh xạ vào cùng một line, khi đó các khối sẽ liên tục bị đổi chỗ trong cache, và tỷ lệ truy cập cache thành công sẽ rất thấp.

Một phương pháp để giảm cache miss là ghi nhớ dữ liệu đã bị loại bỏ khỏi cache để phòng trường hợp sau đó lại cần nó lần nữa. Vì dữ liệu bị loại bỏ đã từng được truy xuất, nên nó có thể được sử dụng lại. Việc tái sử dụng này có thể khả thi nếu sử dụng một cache nạn nhân. Cache nạn nhân là bộ nhớ cache kết hợp toàn phần, có kích thước từ 4 đến 16 line, nằm giữa cache L1 ánh xạ trực tiếp và cấp độ bộ nhớ tiếp theo.

Ánh xạ kết hợp toàn phần

Ánh xạ kết hợp toàn phần khắc phục nhược điểm của ánh xạ trực tiếp bằng cách cho phép mỗi khối bộ nhớ chính được nạp vào bất kỳ line nào trong cache. Trong trường hợp này, địa chỉ bộ nhớ được coi là bao gồm trường Tag và trường Word. Trường Tag xác định một khối bộ nhớ chính duy nhất. Để xác định một khối có nằm trong bộ nhớ cache hay không, logic điều khiển cache phải kiểm tra tag của tất cả các line cùng lúc để tìm ra giá trị tag trùng. Hình 3.19 minh họa cho logic này.



Hình 3.19. Tổ chức cache kết hợp toàn phần

Lưu ý rằng không có trường nào trong địa chỉ tương ứng với chỉ số line, do đó chỉ số line trong cache không được xác định bằng định dạng địa chỉ. Tóm lại:

- Độ dài địa chỉ = $(s + w)$ bit
- Số lượng đơn vị địa chỉ = 2^{s+w} word hoặc byte
- Kích thước khối = Kích thước line = 2^w word hoặc byte
- Số lượng khối trong bộ nhớ chính = $2^{s+w}/2^w = 2^s$
- Số lượng line trong cache = không xác định
- Kích thước tag = s bit

Với ánh xạ kết hợp toàn phần, việc lựa chọn khối nào bị thay thế khi có một khối mới được đọc vào bộ nhớ cache trở nên linh hoạt hơn. Các thuật toán thay thế được thiết kế sao cho tỷ lệ

truy cập cache thành công là cao nhất. Nhược điểm chính của ánh xạ kết hợp toàn phần là yêu cầu mạch điện phức tạp hơn để có thể kiểm tra tag của tất cả các line trong cache đồng thời.

Ánh xạ tập kết hợp

Ánh xạ tập kết hợp thể hiện ưu điểm của cả phương pháp ánh xạ trực tiếp và kết hợp đồng thời lại giảm được nhược điểm của chúng.

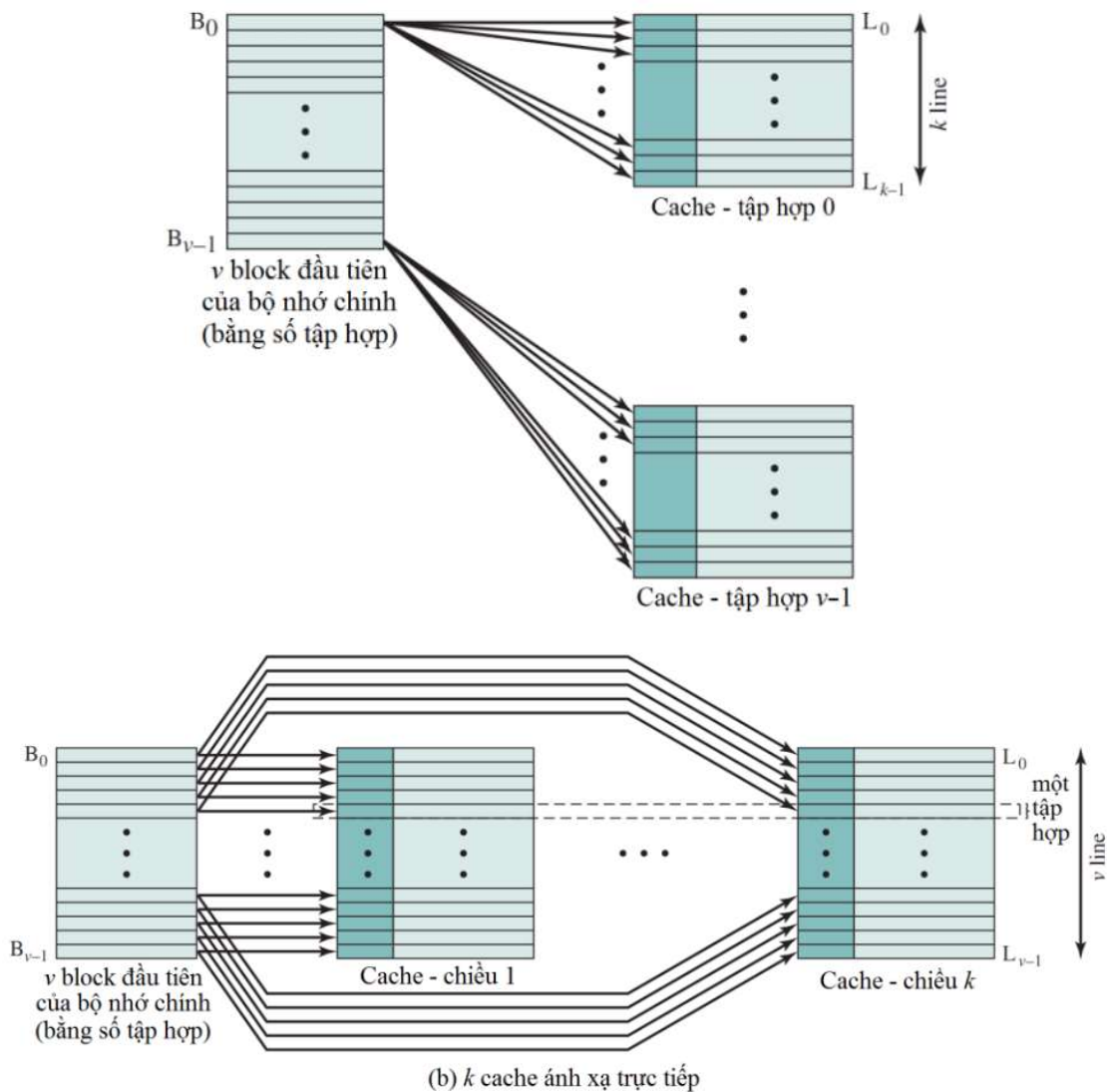
Trong trường hợp này, bộ nhớ cache gồm một số tập hợp, mỗi tập hợp gồm một số line. Hàm ánh xạ được thể hiện như sau:

$$m = v * k$$

$$i = j \bmod v$$

trong đó:

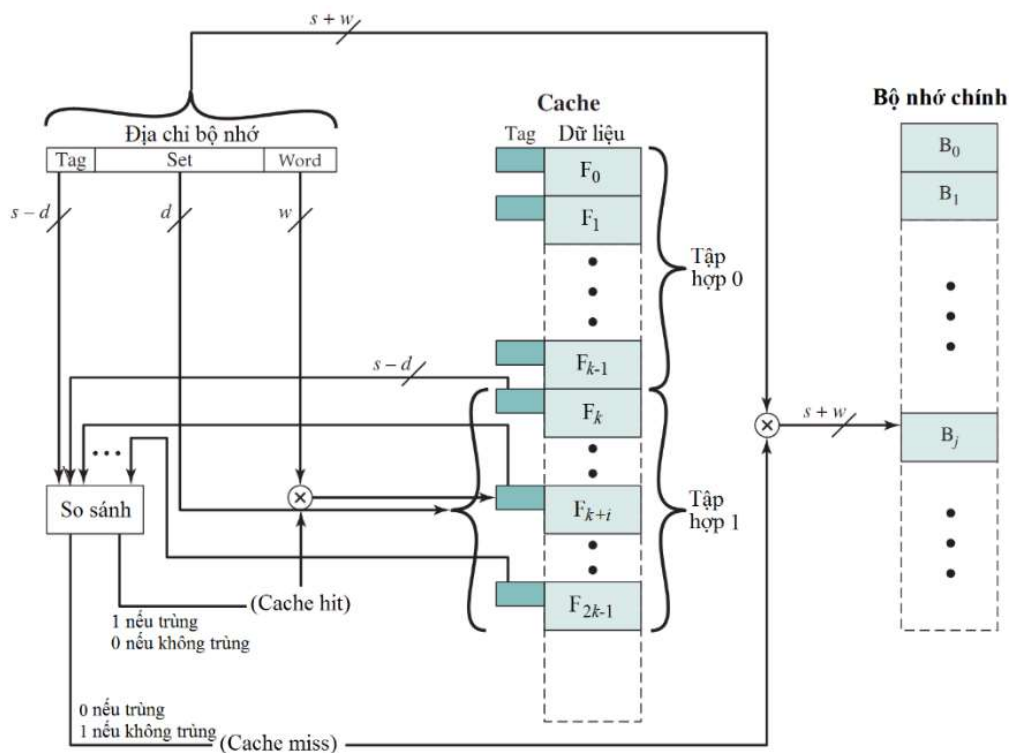
- i = chỉ số của tập hợp trong cache
- j = chỉ số của khối trong bộ nhớ chính
- m = số lượng line trong cache
- v = số lượng tập hợp
- k = số lượng line trong một tập hợp



Hình 3.20. Ánh xạ từ Bộ nhớ chính sang Cache: Tập kết hợp k line

Ta gọi đây là ánh xạ tập kết hợp k line. Với Ánh xạ tập kết hợp, khối B_j có thể được ánh xạ vào bất kỳ line nào của tập i . Hình 3.20a minh họa sự ánh xạ v khối đầu tiên của bộ nhớ chính. Giống với ánh xạ kết hợp, mỗi word có thể ánh xạ vào nhiều line trong cache.

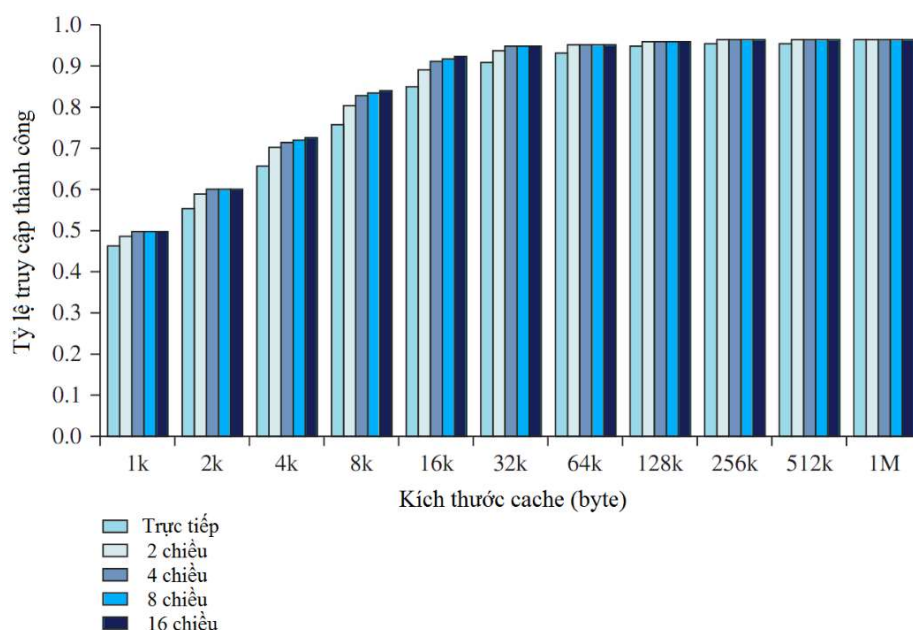
Đối với ánh xạ tập kết hợp, một word có thể ánh xạ được vào tất cả các line trong một tập hợp nhất định, do đó khối B_0 ánh xạ vào tập 0 , khối B_1 ánh xạ vào tập 1 , v.v... Vì vậy, một bộ nhớ cache tập kết hợp có thể được thực hiện như v bộ nhớ cache kết hợp. Cũng có thể thực hiện bộ nhớ cache tập kết hợp như k cache ánh xạ trực tiếp, như thể hiện trong Hình 3.20b. Mỗi cache ánh xạ trực tiếp được gọi là một chiều, gồm v line. Vậy v khối đầu tiên của bộ nhớ chính được ánh xạ trực tiếp vào v line của mỗi chiều; nhóm v khối tiếp theo của bộ nhớ chính được ánh xạ tương tự, v.v... Việc thực hiện ánh xạ trực tiếp thường được sử dụng khi mức độ kết hợp nhỏ (tức là giá trị k nhỏ) trong khi việc thực hiện ánh xạ kết hợp thường được sử dụng khi mức độ kết hợp cao hơn. Đối với ánh xạ tập kết hợp, logic điều khiển cache diễn giải địa chỉ bộ nhớ thành ba trường: Tag, Set và Word. Trong đó, d bit trường Set chỉ định một trong $v = 2^d$ tập hợp. s bit của trường Tag và Set xác định một trong 2^s khối của bộ nhớ chính.



Hình 3.21. Tổ chức cache tập kết hợp k line

Trong trường hợp đặc biệt khi $v = m$, $k = 1$, kỹ thuật tập kết hợp trở thành ánh xạ trực tiếp, và khi $v = 1$, $k = m$, nó sẽ trở thành ánh xạ kết hợp. Việc sử dụng hai line trong một tập hợp ($v = m/2$, $k = 2$) là tổ chức tập kết hợp phổ biến nhất. Nó cải thiện đáng kể tỷ lệ truy cập thành công so với ánh xạ trực tiếp. Tập kết hợp bốn chiều ($v = m/4$, $k = 4$) đem lại sự cải thiện vừa phải. Tuy nhiên, tăng hơn nữa số line trong một tập hợp lại không có nhiều hiệu quả.

Hình 3.22 là kết quả của một nghiên cứu mô phỏng với hiệu suất cache tập kết hợp là hàm biến thiên theo kích thước cache. Sự chênh lệch hiệu suất giữa kết hợp trực tiếp và kết hợp hai chiều là đáng kể khi kích thước cache nhỏ hơn hoặc bằng 64 kB. Cũng lưu ý rằng sự chênh lệch hiệu suất giữa hai chiều và bốn chiều ở 4 kB nhỏ hơn nhiều so với sự chênh lệch ở kích thước cache 8 kB. Độ phức tạp của cache tăng tỷ lệ với mức độ kết hợp, và thể hiện rõ khi kích thước cache nhỏ hơn 16 Kbyte. Điểm lưu ý cuối cùng là, ngoài khoảng 32 kB, việc tăng kích thước cache không làm tăng hiệu suất một cách đáng kể.



Hình 3.22. Sự biến thiên của mức độ kết hợp theo kích thước cache

Kết quả của hình 3.22 dựa trên mô phỏng quá trình thực thi của trình biên dịch GCC. Các ứng dụng khác nhau có thể mang lại kết quả khác nhau, tuy nhiên kết quả so sánh tỷ lệ truy cập cache thành công với kích thước cache vẫn thể hiện xu hướng giống như hình 3.22.

3.3.4. Thuật toán thay thế

Khi bộ nhớ cache đã đầy, nếu một khối mới được đưa đến cache, một trong những khối hiện đang nằm trong cache phải được thay thế. Đối với ánh xạ trực tiếp, chỉ có một line định sẵn cho một khối bất kỳ và không có sự lựa chọn nào khác. Đối với các kỹ thuật kết hợp và tập kết hợp, cần phải có thuật toán thay thế. Để đạt được tốc độ cao, thuật toán đó phải được thực hiện trong phần cứng. Ta sẽ đề cập đến bốn trong số các thuật toán phổ biến nhất. Thuật toán hiệu quả nhất là LRU (ít được sử dụng gần đây nhất): Thay thế khối nằm trong cache lâu nhất mà không có tham chiếu đến nó. Đối với tập kết hợp hai chiều, thuật toán này được thực hiện dễ dàng. Mỗi line chứa một bit USE. Khi một line được tham chiếu, bit USE của nó được đặt lên 1 và bit USE của line còn lại trong tập hợp đó được đặt thành 0. Khi một khối mới được đọc vào tập hợp, line có bit USE bằng 0 được thay thế. Do ta giả thiết rằng các vị trí bộ nhớ được sử dụng gần đây hơn có nhiều khả năng được tham chiếu tiếp, LRU cho tỷ lệ truy cập thành công tốt nhất. LRU cũng tương đối dễ thực hiện trong cache kết hợp toàn phần. Cơ chế bộ nhớ cache duy trì một danh sách riêng chứa chỉ mục của tất cả các line trong cache. Khi một line được tham chiếu, nó được di chuyển lên phía trên của danh sách. Khi cần thay thế, line ở cuối danh sách được sử dụng. Do sự đơn giản trong thực hiện, LRU là thuật toán thay thế phổ biến nhất.

Một thuật toán khác là FIFO (Vào trước ra trước): Thay thế khối trong tập hợp đã nằm trong cache lâu nhất. FIFO được thực hiện dễ dàng bằng kỹ thuật round-robin. Một thuật toán khác nữa là LFU (ít được sử dụng nhất): Thay thế khối trong tập hợp mà số lần tham chiếu đến nó là ít nhất. LFU có thể được thực hiện bằng cách sử dụng một bộ đếm ở mỗi line. Một kỹ thuật khác không dựa trên sự sử dụng khối (tức là không phải LRU, LFU, FIFO) là chọn một line ngẫu nhiên trong số các line. Các nghiên cứu đã chỉ ra rằng việc thay thế ngẫu nhiên đem lại hiệu suất chỉ kém hơn một chút so với thuật toán dựa trên việc sử dụng khối.

3.3.5. Chính sách ghi

Khi một khối đang nằm trong cache được thay thế, có 2 trường hợp cần được xem xét. Nếu khối cũ trong cache chưa từng thay đổi giá trị thì có thể ghi đè khối mới lên mà không cần ghi khối

cũ ra trước. Nếu có ít nhất một thao tác ghi đã được thực hiện trên một word trong line của cache thì bộ nhớ chính phải được cập nhật bằng cách ghi line đó vào một khối của bộ nhớ chính trước khi đưa khối mới vào cache. Có nhiều chính sách ghi khác nhau, với ràng buộc về hiệu suất và tính thương mại. Có hai vấn đề cần phải đối mặt. Một là, nhiều thiết bị có thể truy cập vào bộ nhớ chính. Ví dụ, một mô-đun I/O có thể đọc/ghi trực tiếp vào bộ nhớ. Nếu một word đã thay đổi chỉ được cập nhật trong cache, thì word tương ứng ở bộ nhớ chính không còn hợp lệ. Hơn nữa, nếu thiết bị I/O thay đổi word trong bộ nhớ chính, thì word trong cache lại không hợp lệ. Hai là, vấn đề phức tạp hơn xảy ra khi nhiều bộ xử lý được gắn vào cùng một bus và mỗi bộ xử lý lại có cache cục bộ riêng. Khi đó, nếu một word bị thay đổi trong một cache, nó có thể làm mất ý nghĩa một word trong các cache khác.

Kỹ thuật đơn giản nhất được gọi là write through. Sử dụng kỹ thuật này, tất cả các hành động ghi được thực hiện đồng thời ở cả bộ nhớ chính và cache, đảm bảo rằng nội dung bộ nhớ chính luôn luôn hợp lệ. Một mô-đun bộ xử lý – cache bất kỳ có thể giám sát lưu lượng truy cập vào bộ nhớ chính để duy trì tính nhất quán trong cache của chính nó. Nhược điểm chính của kỹ thuật này là nó tạo ra lưu lượng bộ nhớ đáng kể và có thể tạo ra nút nghẽn cổ chai.

Một kỹ thuật khác, được gọi là write back, giảm thiểu việc ghi vào bộ nhớ chính. Với write back, việc cập nhật giá trị mới chỉ được thực hiện trong cache. Khi có một cập nhật xảy ra, một bit được gọi là dirty bit hoặc use bit trong line được đặt lên 1. Khi một khối cần được thay thế trong cache, nó được ghi trở lại bộ nhớ chính nếu và chỉ nếu dirty bit bằng 1. Vấn đề đối với write back là nhiều phần của bộ nhớ chính là không hợp lệ, và do đó các truy cập bằng mô-đun I/O chỉ được cho phép thực hiện thông qua cache. Điều này làm cho mạch phức tạp hơn và khả năng tạo nút nghẽn cổ chai.

3.3.6. Kích thước line

Một yếu tố thiết kế khác là kích thước line. Khi 1 khối dữ liệu được lấy ra và đặt vào cache, line sẽ chứa không chỉ word mong muốn mà còn một số word liền kề. Khi kích thước khối tăng dần, ban đầu tỷ lệ truy cập thành công cũng tăng do nguyên lý cục bộ của tham chiếu, theo đó dữ liệu trong vùng lân cận của một word được tham chiếu có thể sẽ được tham chiếu trong tương lai gần. Khi kích thước khối tăng thêm, nhiều dữ liệu hữu ích hơn được đưa vào cache. Tuy nhiên, tỷ lệ truy cập thành công bắt đầu giảm khi khối lớn hơn nữa và xác suất sử dụng thông tin mới truy xuất sẽ nhỏ hơn xác suất tái sử dụng thông tin phải được thay thế. Có hai ảnh hưởng rõ rệt là:

- Khối lớn hơn làm giảm số lượng khối trong cache. Bởi vì mỗi lần truy xuất khối mới lại ghi đè lên nội dung cũ hơn trong cache, số lượng khối càng ít thì dữ liệu càng nhanh chóng bị ghi đè ngay sau khi chúng được truy xuất.
- Khi 1 khối lớn hơn, mỗi word thêm vào lại càng xa word mong muốn và do đó ít có khả năng là cần thiết trong tương lai gần.

Mối quan hệ giữa kích thước line và tỷ lệ truy cập thành công là rất phức tạp, tùy thuộc vào đặc tính cục bộ của mỗi chương trình cụ thể, và không có giá trị tối ưu nhất định. Kích thước line từ 8 đến 64 byte được xem là hợp lý, gần như tối ưu.

3.3.7. Số lượng cache

Khi cache mới ra mắt, hệ thống điển hình có một bộ nhớ cache đơn. Gần đây, việc sử dụng nhiều bộ nhớ cache đã trở nên phổ biến. Hai khía cạnh của vấn đề thiết kế này là số lượng cấp độ cache

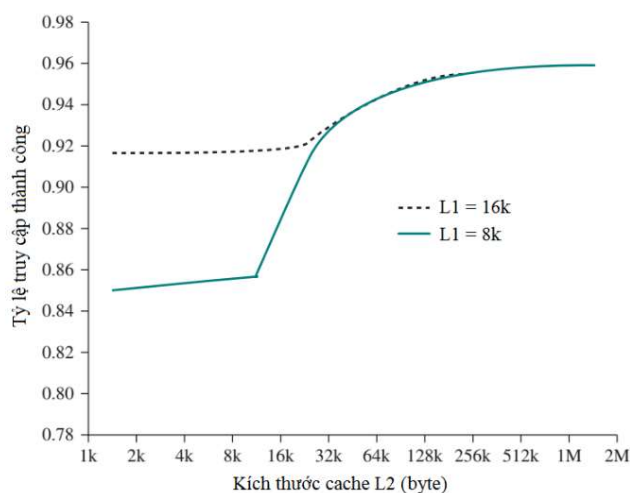
Cache nhiều cấp độ

Khi mật độ logic tăng lên, việc đưa bộ nhớ cache lên trên chip cùng với bộ xử lý là khả thi: ta có cache trên chip. So với cache có thể truy cập thông qua bus bên ngoài, cache trên chip làm giảm bớt việc bộ xử lý phải sử dụng bus bên ngoài và do đó tăng tốc độ thực hiện và tăng hiệu suất tổng thể của hệ thống. Khi đã tìm thấy lệnh hoặc dữ liệu được yêu cầu trong cache trên chip, việc truy cập bus được loại bỏ. Do các đường dữ liệu trong bộ xử lý ngắn hơn so với độ dài của bus, việc truy cập bộ nhớ cache trên chip sẽ nhanh hơn rất nhiều so với chu kỳ bus. Hơn nữa, trong giai đoạn này, bus được tự do để hỗ trợ các nhu cầu truyền khác.

Việc sử dụng một bộ nhớ cache trên chip mở ra câu hỏi liệu có còn cần cache ngoài chip hay không. Câu trả lời là có, và hầu hết các thiết kế hiện đại có cả cache trên chip và cache bên ngoài. Tổ chức đơn giản nhất là bộ nhớ cache hai cấp, với cache trên chip được gán là cấp 1 (L1) và cache bên ngoài được gán là cấp 2 (L2). Lý do cho việc sử dụng cache L2 là: Nếu không có cache L2 và bộ xử lý yêu cầu truy cập vào một vị trí bộ nhớ không có trong cache L1, thì bộ xử lý phải truy cập vào bộ nhớ DRAM hoặc ROM thông qua bus.

Do tốc độ bus thường chậm và thời gian truy cập bộ nhớ chính lâu, điều này dẫn đến hiệu suất kém. Mặt khác, nếu sử dụng bộ nhớ cache L2 SRAM (RAM tĩnh), thì thông tin bị thiếu có thể được truy xuất nhanh chóng. Nếu SRAM đủ nhanh để kịp với tốc độ của bus thì dữ liệu có thể được truy cập bằng một giao dịch trạng thái không đợi, kiểu truyền bus nhanh nhất.

Có hai đặc điểm thiết kế đáng lưu ý đối với các bộ nhớ cache nhiều cấp độ. Một là, đối với cache L2 ngoài chip, nhiều thiết kế không sử dụng bus hệ thống làm đường dẫn giữa cache L2 và bộ xử lý, mà sử dụng một đường dẫn dữ liệu riêng biệt, để giảm bớt gánh nặng cho hệ thống bus. Hai là, khi các thành phần của bộ xử lý liên tục thu nhỏ kích thước, một số bộ xử lý hiện nay kết hợp cả cache L2 trên chip xử lý, cải thiện hiệu suất.



Hình 3.23. Tỷ lệ truy cập thành công tổng cộng (L1 và L2) với L1 8 Kbyte và 16 Kbyte

Hình 3.23 cho thấy kết quả của một nghiên cứu mô phỏng sự thay đổi hiệu suất bộ nhớ cache hai cấp theo kích thước cache. Giả thiết rằng cả hai cache kích thước line nhw nhau. Hình vẽ thể hiện tỷ lệ truy cập cache thành công tổng cộng. Tức là, một lần truy cập thành công được tính nếu dữ liệu mong muốn xuất hiện trong cache L1 hoặc L2. Hình vẽ này cho thấy ảnh hưởng của L2 lên tổng số truy cập thành công khi kích thước L1 cố định. L2 bắt đầu có ảnh hưởng đáng kể đến tổng số lần truy cập cache thành công khi kích thước của nó gấp đôi kích thước cache L1. Lưu ý rằng, trong đường biểu diễn cho cache L1 có kích thước 8 Kbyte, phần dốc nhất tương ứng với cache L2 có kích thước là 16 Kbyte. Còn đối với cache L1 16 Kbyte, phần dốc nhất của đường biểu diễn ứng với kích thước cache L2 là 32 Kbyte. Trước đó, cache L2 có ảnh hưởng rất ít đến hiệu suất tổng cộng của bộ nhớ cache. Nhu cầu sử dụng cache L2 lớn hơn cache L1 để cải thiện

hiệu suất là hoàn toàn hợp lý. Nếu cache L2 có kích thước line và dung lượng bằng với cache L1, nội dung của nó sẽ gần như giống hệt cache L1.

Khi diện tích sẵn có trên chip dành cho bộ nhớ cache ngày càng tăng, hầu hết các bộ xử lý hiện đại đã chuyển cache L2 lên chip xử lý và có thêm cache L3. Ban đầu, bộ nhớ cache L3 có thể truy cập được qua bus bên ngoài. Gần đây, hầu hết các bộ xử lý đã tích hợp cache L3 trên chip. Việc dùng thêm cấp độ cache thứ ba có thể góp phần cải thiện hiệu suất. Hơn nữa, các hệ thống lớn hiện nay, chẳng hạn như hệ thống mainframe IBM Enterprise, kết hợp 3 cấp độ cache trên chip và cache thứ tư dùng chung cho nhiều chip.

Cache hợp nhất và cache phân chia

Khi cache trên chip mới xuất hiện, có nhiều thiết kế cho bộ nhớ cache đơn để lưu trữ các tham chiếu tới cả dữ liệu và lệnh. Gần đây, việc chia tách bộ nhớ cache thành hai phần: một dành cho lệnh và một dành cho dữ liệu dần trở nên phổ biến. Hai bộ nhớ cache này ngang hàng nhau, thường là hai cache L1. Khi bộ xử lý cần truy xuất lệnh từ bộ nhớ chính, trước tiên nó sẽ kiểm tra cache lệnh L1, còn khi bộ xử lý cần truy xuất dữ liệu từ bộ nhớ chính, trước tiên nó sẽ kiểm tra cache dữ liệu L1.

Cache hợp nhất có hai ưu điểm:

- Đối với một kích thước cache xác định, Cache hợp nhất có tỷ lệ truy cập thành công cao hơn cache phân chia vì nó tự động cân bằng tải giữa các truy xuất lệnh và dữ liệu. Tức là, nếu một mẫu thực thi có nhiều lượt truy xuất lệnh hơn so với dữ liệu thì bộ nhớ cache sẽ có xu hướng chứa đầy các lệnh và nếu một mẫu thực thi có tương đối nhiều lượt truy xuất dữ liệu hơn, điều ngược lại sẽ xảy ra.
- Chỉ cần thiết kế và thực hiện một bộ nhớ cache.

Xu hướng sử dụng cache là cache chia sẻ tại cấp L1 và Cache hợp nhất cho các cấp cao hơn, đặc biệt đối với các máy siêu vô hướng chú trọng việc thực hiện lệnh song song và truy xuất trước các lệnh dự đoán được sử dụng trong tương lai. Ưu điểm chính của việc thiết kế bộ nhớ cache phân chia là loại trừ sự tranh chấp cache giữa khối truy xuất/giải mã lệnh và khối thi hành. Điều này rất quan trọng trong các thiết kế có sử dụng pipeline lệnh.

Thông thường, bộ xử lý sẽ truy xuất trước các lệnh và điền chúng vào một bộ đệm, hoặc pipeline. Giả sử có một bộ nhớ cache lệnh/dữ liệu thống nhất. Khi khối thi hành truy cập bộ nhớ để tải và lưu trữ dữ liệu, yêu cầu được gửi đến Cache hợp nhất. Nếu cùng lúc đó, khối truy xuất lệnh đưa ra yêu cầu đọc một lệnh cho bộ nhớ cache, yêu cầu đó sẽ tạm thời bị chặn để cache có thể phục vụ khối thi hành trước, để hoàn thành lệnh hiện đang thi hành. Sự tranh chấp cache này có thể làm giảm hiệu suất. Cấu trúc cache phân chia khắc phục khó khăn này.

3.4. BỘ NHỚ NGOÀI

Phần này xem xét một loạt các thiết bị và hệ thống bộ nhớ ngoài. Chúng ta bắt đầu với thiết bị quan trọng nhất, đĩa từ. Đĩa từ là nền tảng của bộ nhớ ngoài trong hầu hết các hệ thống máy tính. Phần tiếp theo sẽ tìm hiểu việc sử dụng mảng đĩa để đạt được hiệu quả cao hơn, đặc biệt là hệ thống RAID (Mảng dư thừa các đĩa độc lập). Một thành phần ngày càng trở nên quan trọng trong nhiều hệ thống máy tính là đĩa bán dẫn, sẽ được thảo luận tiếp theo. Sau đó, bộ nhớ quang học bên ngoài và băng từ cũng được trình bày.

3.4.1. Đĩa từ

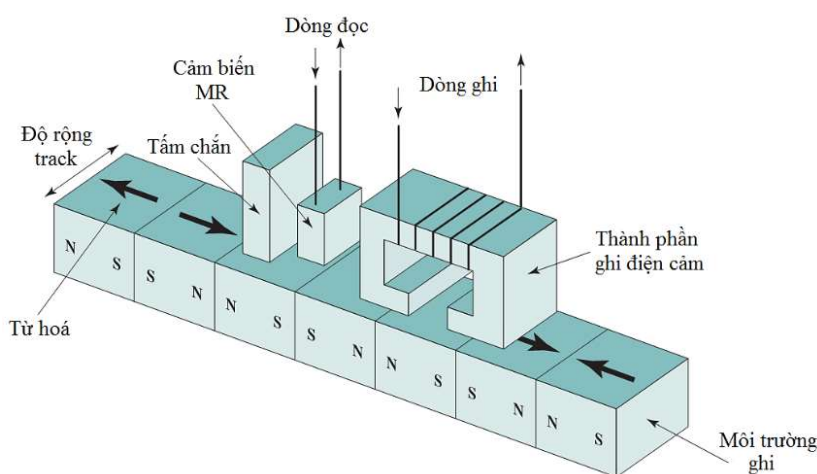
Đĩa từ là một tấm platter tròn chế tạo bằng vật liệu không từ tính, được gọi là chất nền (substrate), được phủ bởi một lớp vật liệu có từ tính. Thông thường, chất nền thường là vật liệu

nhôm hoặc hợp kim nhôm. Gần đây, chất nền thủy tinh đã được giới thiệu. Ưu điểm của chất nền thủy tinh là:

- Cải thiện tính đồng nhất của bề mặt phim từ để tăng độ tin cậy của đĩa
- Giảm đáng kể các khiếm khuyết bề mặt để giúp giảm thiểu lỗi đọc-ghi
- Hỗ trợ khoảng cách giữa đầu đọc/ghi với bề mặt đĩa rất nhỏ (giải thích ở phần sau)
- Độ cứng tốt hơn nên giảm động lực đĩa
- Khả năng chống sốc và hư hỏng lớn hơn

Cơ chế đọc và ghi từ

Dữ liệu được ghi vào đĩa và lấy ra từ đĩa thông qua một cuộn dây dẫn được gọi là đầu; các hệ thống thường có 2 đầu: đầu đọc và đầu ghi. Trong quá trình đọc hoặc ghi, đầu đứng yên trong khi đĩa xoay tròn bên dưới.



Hình 3.24. Đầu đọc điện từ/ghi điện cảm

Cơ chế ghi lợi dụng tính chất: dòng điện chạy qua cuộn dây tạo ra từ trường. Các xung điện được gửi đến đầu ghi và các mẫu từ trường sinh ra được ghi vào bề mặt bên dưới. Các mẫu từ khác nhau thể hiện dòng điện dương và âm khác nhau. Đầu ghi được làm bằng vật liệu từ hoá, dạng hình chữ nhật rỗng với khe hở dọc một cạnh và vài vòng dây dẫn cuốn dọc cạnh đối diện (Hình 3.24). Dòng điện chạy trong dây cảm ứng một từ trường qua khe, làm từ hoá một vùng nhỏ của môi trường ghi. Đảo chiều dòng điện sẽ làm đảo chiều từ hóa trên môi trường ghi.

Cơ chế đọc truyền thống lợi dụng tính chất: từ trường chuyển động quanh cuộn dây tạo ra dòng điện trong cuộn dây. Khi bề mặt đĩa di chuyển qua đầu, nó tạo ra một dòng điện cùng cực với dòng đã ghi trên đĩa. Cấu trúc của đầu đọc cơ bản giống đầu ghi, do đó cùng một đầu có thể được sử dụng cho cả đọc và ghi. Kiểu đầu đơn này được sử dụng trong hệ thống đĩa mềm và trong các hệ thống đĩa cứng trước đây.

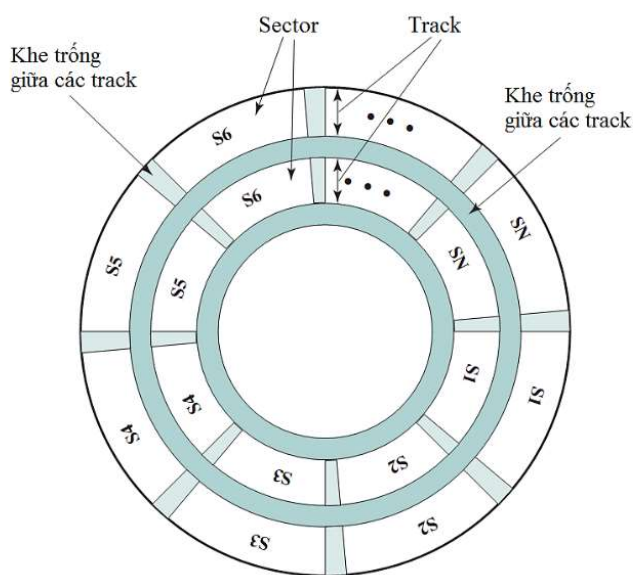
Các hệ thống đĩa cứng hiện đại sử dụng cơ chế đọc khác, đòi hỏi phải có đầu đọc, đầu ghi riêng biệt. Đầu đọc gồm một bộ cảm biến điện từ (MR – magnetoresistive) được che một phần. Vật liệu MR có điện trở phụ thuộc vào hướng từ hóa của môi trường di chuyển dưới nó. Khi cho một dòng điện chạy qua cảm biến MR, sự thay đổi điện trở được nhận biết dưới dạng tín hiệu điện áp. Thiết kế của MR cho phép mật độ lưu trữ lớn hơn và tốc độ vận hành cao hơn.

Tổ chức dữ liệu

Đầu là một thiết bị tương đối nhỏ có khả năng đọc hoặc ghi từng phần nhỏ của đĩa xoay bên dưới nó. Điều này dẫn đến việc dữ liệu trên đĩa được tổ chức thành một tập hợp các vành tròn, gọi là track. Mỗi track có độ rộng bằng độ rộng đầu. Một bề mặt có hàng ngàn track.

Hình 3.25 mô tả bố cục dữ liệu này. Các track liền kề nhau được ngăn cách nhau bằng **khe trống** (gap). Khe trống giúp giảm thiểu sai sót do sai lệch đầu hoặc nhiễu từ trường.

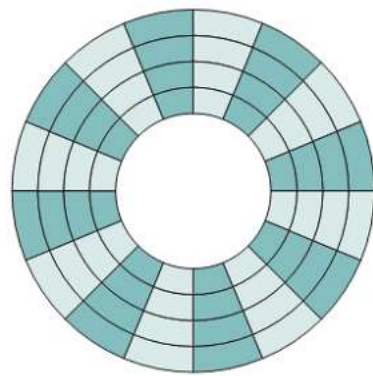
Dữ liệu được truyền ra khỏi đĩa/vào đĩa theo từng sector (Hình 3.25). Có hàng trăm sector trong mỗi track và các sector có thể có độ dài cố định hoặc biến đổi. Trong hầu hết các hệ thống hiện đại, sector có độ dài cố định, phổ biến là 512 byte. Tương tự, các sector liền kề được ngăn cách bởi khe trống.



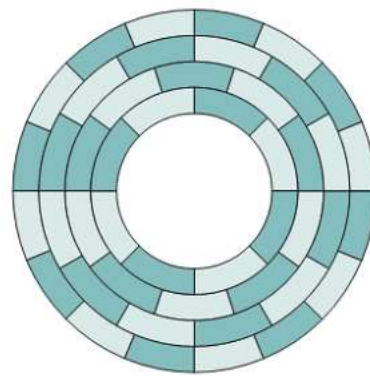
Hình 3.25. Bố trí dữ liệu trên đĩa

Khi đĩa quay tròn, một bit gần tâm đĩa di chuyển qua một điểm xác định (dưới đầu đọc/ghi) chậm hơn một bit phía ngoài vành đĩa. Do đó, cần có phương pháp bù lại độ biến động tốc độ sao cho đầu có thể đọc các bit ở cùng tốc độ. Điều này có thể được thực hiện bằng cách tăng khoảng cách giữa các bit thông tin trong các đoạn khác nhau trên đĩa. Khi đó thông tin được quét ở cùng tốc độ khi đĩa quay, được gọi là **vận tốc góc không đổi** (CAV). Hình 3.26a cho thấy bố cục của một đĩa sử dụng CAV. Đĩa chia thành một số sector hình bánh pie và một chuỗi track đồng tâm. Ưu điểm của CAV là có thể đánh địa chỉ trực tiếp cho từng khối dữ liệu theo track và sector. Để di chuyển đầu từ vị trí hiện tại đến một địa chỉ cụ thể, đầu chỉ cần di chuyển một đoạn ngắn đến một track cụ thể và chờ một khoảng thời gian ngắn để sector thích hợp quay đến bên dưới đầu. Nhược điểm của CAV là số lượng dữ liệu có thể lưu trữ trên track dài hơn phía ngoài chỉ bằng lượng dữ liệu lưu trữ trên các track ngắn gần trung tâm đĩa.

Do mật độ, số bit trên một đơn vị chiều dài, tăng dần nếu tính từ track ngoài cùng tới track trong cùng, nên dung lượng lưu trữ của đĩa trong trường hợp CAV bị hạn chế bởi mật độ ghi tối đa của track trong cùng. Để tăng mật độ, hệ thống đĩa cứng hiện đại sử dụng kỹ thuật **ghi nhiều vùng**, theo đó bề mặt được chia thành nhiều vùng đồng tâm (thường là 16 vùng). Trong cùng một vùng, các track có số lượng bit như nhau. Các vùng xa tâm đĩa hơn chứa nhiều bit hơn (nhiều sector hơn) so với vùng gần tâm. Điều này cho phép tổng dung lượng lưu trữ lớn hơn. Tuy nhiên, mạch điện phức tạp hơn. Khi đầu đĩa di chuyển từ vùng này sang vùng khác, chiều dài (đọc track) của từng bit sẽ thay đổi, làm thay đổi thời gian cho việc đọc và ghi. Hình 3.26b cho thấy bản chất của việc ghi nhiều vùng; trong hình minh họa này, mỗi vùng chỉ rộng một track.



(a) Vận tốc góc không đổi



(b) Ghi nhiều vùng

Hình 3.26. So sánh các phương pháp bố trí đĩa

Đặc tính vật lý

Bảng 3.7. Đặc tính vật lý của hệ thống đĩa

Chuyển động đầu	Tầm platter
Đầu cố định	Đơn tầm
Đầu di chuyển	Đa tầm
Tính di động của đĩa	Cơ chế của đầu
Đĩa không tháo được	Tiếp xúc (đĩa mềm)
Đĩa tháo được	Khe cố định
Mặt	Khe khí động học (Winchester)
Một mặt	
Hai mặt	

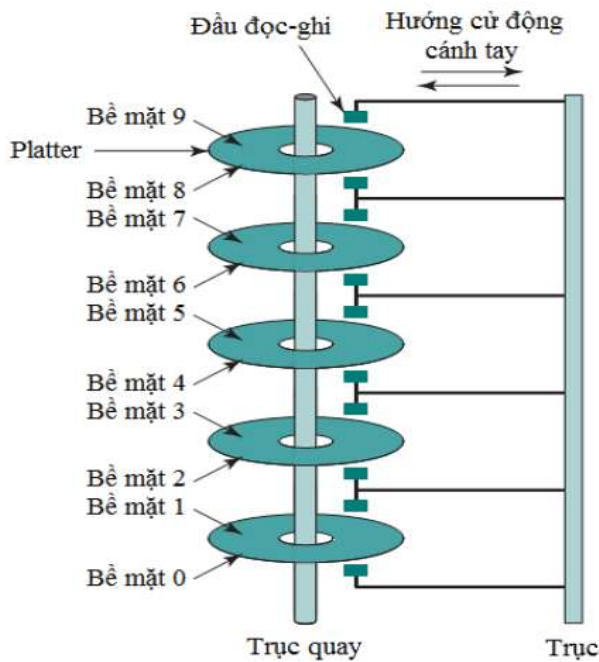
Bảng 3.7 liệt kê các đặc tính chính phân biệt các loại đĩa từ. Thứ nhất, đầu có thể cố định hoặc di chuyển được theo chiều hướng tâm của đĩa. Trong ổ đĩa có đầu cố định, có một đầu đọc-ghi dành riêng cho mỗi track. Tất cả các đầu được gắn trên một cánh tay cố định kéo dài trên toàn bộ track; tuy nhiên ngày nay rất hiếm các hệ thống như vậy. Trong ổ đĩa có đầu di chuyển, chỉ có một đầu đọc-ghi. Đầu này cũng được gắn trên một cánh tay. Để đầu có thể được đặt trên bất kỳ track nào, cánh tay phải có thể kéo dài hoặc rút ngắn được.

Một ổ đĩa bao gồm cánh tay, trục quay đĩa, và các thiết bị điện tử cần thiết để nhập và xuất dữ liệu nhị phân. **Đĩa không tháo được** được gắn vĩnh viễn vào ổ đĩa; đĩa cứng trong máy tính cá nhân là một ví dụ về đĩa không tháo được. **Đĩa tháo được** có thể được gỡ ra và thay thế bằng một đĩa khác. Ưu điểm của loại thứ hai là hệ thống đĩa hữu hạn nhưng sẵn sàng chứa lượng dữ liệu vô hạn. Hơn nữa, đĩa kiểu này có thể di chuyển được từ máy tính này sang máy tính khác. Đĩa mềm và đĩa cartridge ZIP là những ví dụ về đĩa tháo được.

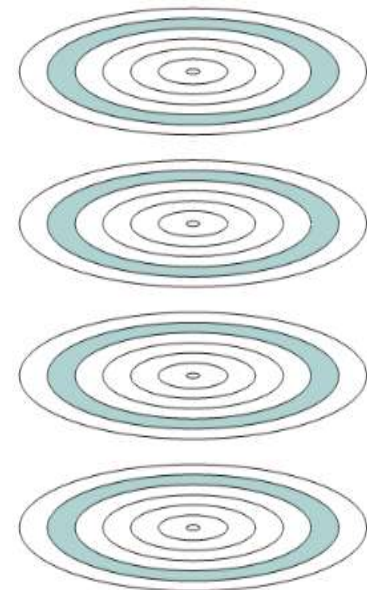
Lớp phủ từ tính thường được phủ lên cả hai mặt của đĩa, chúng được gọi là **đĩa hai mặt**. Một số hệ thống đĩa rẻ hơn sử dụng **đĩa một mặt**.

Một số ổ đĩa chứa nhiều tấm đĩa xếp chồng lên nhau theo chiều dọc cách nhau vài milimet. Ổ đĩa như vậy sử dụng nhiều cánh tay (Hình 3.27). **Ổ đĩa đa tầm** sử dụng đầu di chuyển, với một đầu đọc-ghi cho mỗi bề mặt đĩa. Tất cả các đầu được gắn cố định sao cho tất cả đều cách tâm đĩa một khoảng như nhau và di chuyển cùng nhau. Do đó, bất cứ lúc nào, tất cả các đầu đều được đặt trên các track có khoảng cách tới tâm đĩa bằng nhau. Tập hợp tất cả các track ở cùng vị trí tương

đôi trên đĩa được gọi là **cylinder**. Ví dụ, tất cả các track được tô đậm trong hình 3.28 là một phần của cylinder.



Hình 3.27. Các thành phần của ổ đĩa cứng



Hình 3.28. Track và cylinder

Cuối cùng, dựa theo cơ chế của đầu, có thể phân loại đĩa thành ba loại. Loại truyền thống, đầu đọc-ghi được đặt cách tấm đĩa một khoảng cố định, ở giữa là một khe không khí. Một loại khác hoàn toàn là cơ chế đầu thực sự tiếp xúc vật lý với môi trường đĩa trong quá trình đọc hoặc ghi. Cơ chế này được áp dụng với đĩa mềm (một loại đĩa nhỏ, mềm dẻo) và loại đĩa cứng rẻ nhất.

Trước khi tìm hiểu loại đĩa thứ ba, ta cần phải bàn về mối quan hệ giữa mật độ dữ liệu và độ rộng khe không khí. Đầu phải tạo ra hoặc cảm nhận một trường điện từ đủ lớn để ghi và đọc đúng. Đầu càng hẹp thì càng cần được đặt gần bề mặt đĩa. Đầu hẹp hơn nghĩa là các track hẹp hơn và do đó mật độ dữ liệu lớn hơn, đây là điều ta mong muốn. Tuy nhiên, đầu càng gần đĩa thì nguy cơ lỗi do tạp chất hoặc do không hoàn hảo càng cao.

Bảng 3.8. Các thông số ổ đĩa điển hình

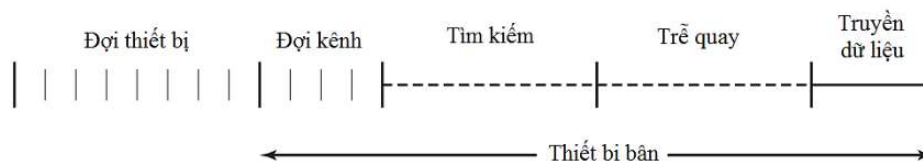
Đặc tính	Constellation ES.2	Seagate Barracuda XT	Cheetah NS	Momentum
Ứng dụng	Doanh nghiệp	Máy tính để bàn	Server ứng dụng	Máy tính xách tay
Dung lượng	3 TB	3 TB	400 GB	640 GB
Thời gian tìm kiếm trung bình	8.5 ms đọc 9.5 ms ghi	N/A	3.9 ms đọc 4.2 ms ghi	13 ms
Tốc độ quay trục	7200 rpm	7200 rpm	10, 075 rpm	5400 rpm
Độ trễ trung bình	4.16 ms	4.16 ms	2.98	5.6 ms
Tốc độ truyền tối đa	155 MB/s	149 MB/s	97 MB/s	300 MB/s
Byte/sector	512	512	512	4096
Track/cylinder (số lượng bề mặt platter)	8	10	8	4
Cache	64 MB	64 MB	16 MB	8 MB

Xét ví dụ về đĩa Winchester. Đầu Winchester được sử dụng trong các cụm ổ kín mà hầu như không có tạp chất. Nó được thiết kế để hoạt động gần bề mặt đĩa hơn so với các đầu đĩa cứng thông thường, do đó cho phép mật độ dữ liệu lớn hơn. Đầu là một tấm giấy foil khí động học đặt nhẹ trên bề mặt đĩa khi đĩa không di chuyển. Áp suất không khí tạo ra khi đĩa quay là đủ lớn để làm cho tấm foil bay lên khỏi bề mặt. Hệ thống không tiếp xúc này có thể được thiết kế để cho phép đầu đọc/ghi hoạt động gần bề mặt đĩa hơn so với các loại đầu đĩa cứng cùng thời. Bảng 3.8 cung cấp các thông số của các loại ổ đĩa hiệu suất cao điển hình.

Thông số hiệu suất đĩa

Chi tiết thực tế về hoạt động vào/ra của đĩa phụ thuộc vào hệ thống máy tính, hệ điều hành, tính chất của kênh vào/ra và phần cứng bộ điều khiển đĩa. Một sơ đồ thời gian tổng quát của quá trình truyền vào/ra của đĩa được thể hiện trong hình 3.29.

Khi ổ đĩa đang hoạt động, đĩa quay với tốc độ không đổi. Để đọc hoặc ghi, đầu phải được đặt ở track mong muốn và ở sector đầu tiên của track đó. Việc chọn track bao gồm việc di chuyển đầu (trong hệ thống đầu di chuyển) hoặc lựa chọn một đầu (trong hệ thống đầu cố định). Trong hệ thống đầu di chuyển, thời gian cần để đặt đầu vào track mong muốn được gọi là **thời gian tìm kiếm**. Khi đã chọn được track, bộ điều khiển đĩa sẽ đợi cho đến khi sector thích hợp quay tới thẳng hàng với đầu. Thời gian cần thiết để điểm bắt đầu của sector đến dưới đầu được gọi là **trễ quay**. Tổng cộng thời gian tìm kiếm và trễ quay là **thời gian truy cập**, đó là thời gian cần thiết để đầu vào vị trí để đọc hoặc ghi. Một khi đầu vào vị trí, hoạt động đọc hoặc ghi được thực hiện khi sector di chuyển dưới đầu; đây là giai đoạn truyền dữ liệu; thời gian cần thiết để truyền là **thời gian truyền**.



Hình 3.29. Thời gian truyền vào/ra của đĩa

Ngoài thời gian truy cập và thời gian truyền, hoạt động vào/ra của đĩa còn có một vài khoảng trễ hàng đợi khác. Khi tiến trình đưa ra một yêu cầu vào/ra, trước hết nó phải đợi trong hàng đợi cho tới khi thiết bị sẵn sàng. Lúc này, thiết bị đó được gán cho tiến trình này. Nếu thiết bị chia sẻ một kênh vào/ra hoặc một tập hợp các kênh vào/ra với các ổ đĩa khác, thì có thể nó phải chờ đợi thêm cho tới khi kênh này sẵn sàng. Tại thời điểm này, việc tìm kiếm được thực hiện để bắt đầu truy cập đĩa.

Trong một số hệ thống máy chủ cao cấp, kỹ thuật RPS (Rotational Positional Sensing) được sử dụng. RPS hoạt động như sau: Khi lệnh tìm kiếm đã được ban hành, kênh được giải phóng để xử lý các hoạt động vào/ra khác. Khi hoàn thành tìm kiếm, thiết bị sẽ xác định khi nào thì dữ liệu xoay tới dưới đầu. Khi sector đến dưới đầu, thiết bị thiết lập lại đường truyền nối tới host. Nếu khối điều khiển hoặc kênh đang bận với một hoạt động vào/ra khác, thì lần kết nối lại này không thành công và thiết bị phải quay tròn một vòng trước khi nó có thể kết nối lại. Đây là một RPS miss. Nó là một khoảng trễ khác phải được bổ sung vào đồ thị thời gian ở hình 3.29.

3.4.2. Ổ đĩa bán dẫn SSD

Một phát triển rất quan trọng trong kiến trúc máy tính những năm gần đây là việc sử dụng ngày càng phổ biến các ổ đĩa bán dẫn (SSD) để bổ sung hoặc thậm chí thay thế cho ổ đĩa cứng, cả ở bộ nhớ bên trong lẫn bên ngoài. Thuật ngữ “solid state” chỉ mạch điện tử được chế tạo bằng chất

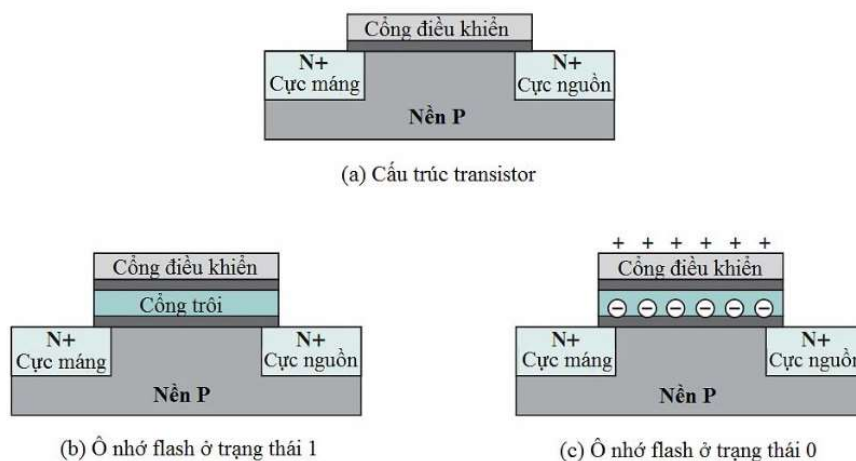
bán dẫn. Ổ đĩa bán dẫn là thiết bị nhớ được chế tạo bằng các linh kiện bán dẫn có thể được sử dụng thay thế cho ổ đĩa cứng. Các ổ SSD hiện đang có mặt trên thị trường sử dụng một loại bộ nhớ bán dẫn được gọi là bộ nhớ flash.

Trong mục này, trước tiên ta tìm hiểu về bộ nhớ flash, rồi sau đó xem xét việc sử dụng nó trong SSD.

Bộ nhớ flash

Bộ nhớ flash là một loại bộ nhớ bán dẫn đã xuất hiện khá lâu và được sử dụng trong nhiều sản phẩm điện tử tiêu dùng, bao gồm điện thoại thông minh, thiết bị GPS, máy nghe nhạc MP3, máy ảnh kỹ thuật số và USB. Trong những năm gần đây, chi phí và hiệu suất của bộ nhớ flash đã phát triển đến mức ổ đĩa với bộ nhớ flash hoàn toàn có thể được sử dụng thay thế ổ cứng.

Hình 3.30 mô tả hoạt động cơ bản của bộ nhớ flash. Hình 3.30a mô tả hoạt động của transistor. Transistor lợi dụng tính chất của chất bán dẫn là đặt một điện áp nhỏ vào cực cổng có thể điều khiển một dòng điện lớn chạy giữa cực nguồn và cực cổng.



Hình 3.30. Hoạt động của bộ nhớ flash

Trong một ô nhớ flash, một cực cổng thứ hai được thêm vào transistor. Nó được gọi là cổng trôi do được cách điện bởi lớp oxit mỏng. Ban đầu, cổng trôi không can thiệp vào hoạt động của transistor (Hình 3.30b). Trong trường hợp này, ô nhớ được xem là đang biểu diễn nhị phân 1. Đặt một điện áp lớn trên lớp oxit làm cho các electron đi qua nó và bị mắc kẹt lại ở cổng trôi, và chúng vẫn nằm nguyên ở đó ngay cả khi đã ngắt điện (Hình 3.30c).

Trong trạng thái này, ô nhớ được xem là đang biểu diễn nhị phân 0. Ta có thể đọc trạng thái của ô nhớ bằng cách sử dụng các mạch bên ngoài để kiểm tra xem transistor có đang hoạt động hay không. Đặt một điện áp lớn theo chiều ngược lại sẽ làm mất đi các electron ở cổng trôi, ô nhớ quay trở lại trạng thái nhị phân 1.

Có hai loại bộ nhớ flash đặc biệt là NOR và NAND. Trong bộ nhớ flash NOR, đơn vị truy cập cơ bản là 1 bit, và tổ chức logic giống với thiết bị logic NOR. Đối với bộ nhớ flash NAND, đơn vị cơ bản là 16 hoặc 32 bit, và tổ chức logic giống với thiết bị NAND.

Bộ nhớ flash NOR cung cấp khả năng truy cập ngẫu nhiên tốc độ cao. Nó có thể đọc và ghi dữ liệu ở các vị trí cụ thể, có thể tham chiếu và lấy ra một byte đơn. Bộ nhớ flash NOR được sử dụng để lưu trữ mã hệ điều hành cho điện thoại di động và chương trình BIOS trên máy tính Windows. Bộ nhớ flash NAND đọc và ghi trong các khối nhỏ. Nó được sử dụng trong ổ đĩa flash USB, thẻ nhớ (trong máy ảnh kỹ thuật số, máy nghe nhạc MP3, vv), và trong ổ SSD. NAND cung

cấp mật độ bit cao hơn NOR và tốc độ ghi lớn hơn. Bộ nhớ flash NAND không cung cấp bus địa chỉ bên ngoài cho việc truy cập ngẫu nhiên nên dữ liệu phải được đọc theo khối, trong đó mỗi khối chứa hàng trăm đến hàng nghìn bit.

SSD so với HDD

Khi giá thành các ổ SSD dựa trên bộ nhớ flash giảm còn hiệu suất và mật độ bit tăng lên thì SSD ngày càng trở nên cạnh tranh với HDD. Bảng 3.9 đưa ra những so sánh điển hình giữa SSD và HDD.

SSD có những điểm vượt trội hơn HDD như sau:

- Số lần xử lý vào/ra trên giây (IOPS) cao: Tăng đáng kể hiệu suất các hệ thống con vào/ra.
- Độ bền: Ít bị ảnh hưởng bởi xóc và rung động.
- Tuổi thọ dài hơn: SSD không dễ bị mài mòn cơ học.
- Tiêu thụ điện năng ít hơn: Mỗi ổ SSD chỉ sử dụng 2.1W, ít hơn đáng kể so với các ổ cứng dung lượng tương đương.
- Khả năng chạy êm hơn và mát hơn: Yêu cầu ít diện tích sàn, chi phí năng lượng thấp hơn, và thân thiện với môi trường hơn.
- Thời gian truy cập và trễ ngắn hơn: Nhanh gấp hơn 10 lần so với đĩa quay trong HDD.

Hiện tại, HDD có ưu điểm về chi phí cho mỗi bit tốt hơn và dung lượng lớn hơn, nhưng những khác biệt này đang dần thu hẹp lại.

Bảng 3.9. So sánh ổ đĩa bán dẫn SSD và ổ cứng HDD

	Ổ flash NAND	Ổ đĩa cứng
Xử lý vào/ra trên giây	Đọc: 45000 Ghi: 15000	300
Thông lượng (MB/s)	Đọc: 200+ Ghi: 100+	Lên tới 80
Thời gian truy cập ngẫu nhiên (ms)	0.1	4 – 10
Dung lượng	Lên tới 256 GB	Lên tới 4 TB

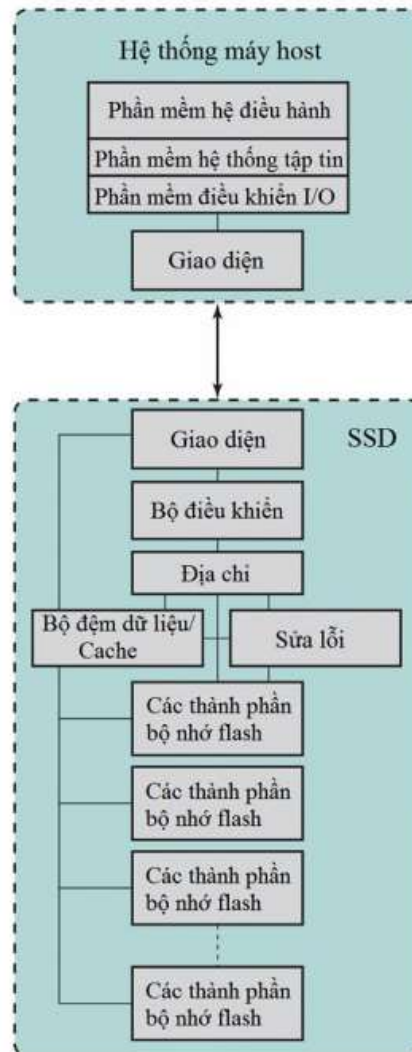
Tổ chức SSD

Hình 3.31 là hình vẽ mô tả các thành phần trong hệ thống kiến trúc tổng quát gắn với bất kỳ hệ thống SSD nào. Trên hệ thống máy host, hệ điều hành yêu cầu phần mềm hệ thống tập tin truy cập dữ liệu trên đĩa. Sau đó, hệ thống tập tin gọi phần mềm điều khiển I/O. Phần mềm điều khiển I/O cung cấp truy cập máy host cho sản phẩm SSD cụ thể. Thành phần giao diện trong hình 3.31 là giao diện vật lý (giao diện điện) giữa bộ xử lý của máy host và thiết bị ngoại vi SSD. Nếu thiết bị này là một ổ cứng gắn bên trong, giao diện thường gặp là PCIe. Đối với thiết bị bên ngoài, giao diện thường gặp là USB.

Ngoài thành phần giao diện với hệ thống máy host, SSD còn có các thành phần sau:

- Bộ điều khiển: Cung cấp giao diện và sự thi hành phân sụn (firmware) ở cấp thiết bị SSD.
- Địa chỉ: Logic thực hiện chức năng lựa chọn trên các thành phần bộ nhớ flash.
- Bộ đệm dữ liệu/bộ nhớ cache: Các thành phần bộ nhớ RAM tốc độ cao được sử dụng để phối hợp tốc độ và tăng thông lượng dữ liệu.
- Sửa lỗi: Logic để phát hiện và sửa lỗi.

- Các thành phần bộ nhớ flash: Các chip flash NAND riêng lẻ.



Hình 3.31. Kiến trúc ổ đĩa SSD

3.4.3. Một số biện pháp an toàn dữ liệu trong việc lưu trữ thông tin trong đĩa từ

Người ta thường chú trọng đến sự an toàn trong lưu giữ thông tin ở đĩa từ hơn là sự an toàn của thông tin trong bộ xử lý. Bộ xử lý có thể hư mà không làm tổn hại đến thông tin. Ổ đĩa của máy tính bị hư có thể gây ra các thiệt hại rất to lớn.

Một phương pháp giúp tăng cường độ an toàn của thông tin trên đĩa từ là dùng một mảng đĩa từ. Mảng đĩa từ này được gọi là Hệ thống đĩa dự phòng (RAID). Cách lưu trữ dư thông tin làm tăng giá tiền và sự an toàn (ngoại trừ RAID 0). Cơ chế RAID có các đặc tính sau:

1. RAID là một tập hợp các ổ đĩa cứng (vật lý) được thiết lập theo một kỹ thuật mà hệ điều hành chỉ “nhìn thấy” chỉ là một ổ đĩa (logic) duy nhất.
2. Với cơ chế đọc/ghi thông tin diễn ra trên nhiều đĩa (ghi đan chéo hay soi gương).
3. Trong mảng đĩa có lưu các thông tin kiểm tra lỗi dữ liệu; do đó, dữ liệu có thể được phục hồi nếu có một đĩa trong mảng đĩa bị hư hỏng.

Tuỳ theo kỹ thuật thiết lập, RAID có thể có các mức sau:

- a. **RAID 0:** Thực ra, kỹ thuật này không nằm trong số các kỹ thuật có cơ chế an toàn dữ liệu. Khi mảng được thiết lập theo RAID 0, ổ đĩa logic có được (mà hệ điều hành nhận

biết) có dung lượng bằng tổng dung lượng của các ổ đĩa thành viên. Điều này giúp cho người dùng có thể có một ổ đĩa logic có dung lượng lớn hơn rất nhiều so với dung lượng thật của ổ đĩa vật lý cùng thời điểm. Dữ liệu được ghi phân tán trên tất cả các đĩa trong mảng. Đây chính là sự khác biệt so với việc ghi dữ liệu trên các đĩa riêng lẻ bình thường bởi vì thời gian đọc-ghi dữ liệu trên đĩa tỉ lệ nghịch với số đĩa có trong tập hợp (số đĩa trong tập hợp càng nhiều, thời gian đọc – ghi dữ liệu càng nhanh). Tính chất này của RAID 0 thật sự hữu ích trong các ứng dụng yêu cầu nhiều thâm nhập đĩa với dung lượng lớn, tốc độ cao (đa phương tiện, đồ hoạ,...). Tuy nhiên, như đã nói ở trên, kỹ thuật này không có cơ chế an toàn dữ liệu, nên khi có bất kỳ một hư hỏng nào trên một đĩa thành viên trong mảng cũng sẽ dẫn đến việc mất dữ liệu toàn bộ trong mảng đĩa. Xác suất hư hỏng đĩa tỉ lệ thuận với số lượng đĩa được thiết lập trong RAID 0. RAID 0 có thể được thiết lập bằng phần cứng (RAID controller) hay phần mềm (Striped Applications).

- b. **RAID 1** (Mirror - Đĩa gương): Phương cách thông thường tránh mất thông tin khi ổ đĩa bị hư là dùng đĩa gương, tức là dùng 2 đĩa. Khi thông tin được viết vào một đĩa, thì nó cũng được viết vào đĩa gương và như vậy luôn có một bản sao của thông tin. Trong cơ chế này, nếu một trong hai đĩa bị hư thì đĩa còn lại được dùng bình thường. Việc thay thế một đĩa mới (cung thông số kỹ thuật với đĩa hư hỏng) và phục hồi dữ liệu trên đĩa đơn giản. Căn cứ vào dữ liệu trên đĩa còn lại, sau một khoảng thời gian, dữ liệu sẽ được tái tạo trên đĩa mới (rebuild). RAID 1 cũng có thể được thiết lập bằng phần cứng (RAID controller) hay phần mềm (Mirror Applications) với chi phí khá lớn, hiệu suất sử dụng đĩa không cao (50%).
- c. **RAID 2**: Dùng kỹ thuật truy cập đĩa song song, tất cả các đĩa thành viên trong RAID đều được đọc khi có một yêu cầu từ ngoại vi. Một mã sửa lỗi (ECC) được tính toán dựa vào các dữ liệu được ghi trên đĩa lưu dữ liệu, các bit được mã hoá được lưu trong các đĩa dùng làm đĩa kiểm tra. Khi có một yêu cầu dữ liệu, tất cả các đĩa được truy cập đồng thời. Khi phát hiện có lỗi, bộ điều khiển nhận dạng và sửa lỗi ngay mà không làm giảm thời gian truy cập đĩa. Với một thao tác ghi dữ liệu lên một đĩa, tất cả các đĩa dữ liệu và đĩa sửa lỗi đều được truy cập để tiến hành thao tác ghi. Thông thường, RAID 2 dùng mã Hamming để thiết lập cơ chế mã hoá, theo đó, để mã hoá dữ liệu được ghi, người ta dùng một bit sửa lỗi và hai bit phát hiện lỗi. RAID 2 thích hợp cho hệ thống yêu cầu giảm thiểu được khả năng xảy ra nhiều đĩa hư hỏng cùng lúc.
- d. **RAID 3**: Dùng kỹ thuật ghi song song, trong kỹ thuật này, mảng được thiết lập với yêu cầu tối thiểu là 3 đĩa có các thông số kỹ thuật giống nhau, chỉ một đĩa trong mảng được dùng để lưu các thông tin kiểm tra lỗi (parity bit). Như vậy, khi thiết lập RAID 3, hệ điều hành nhận biết được một đĩa logic có dung lượng $n-1/n$ (n : số đĩa trong mảng). Dữ liệu được chia nhỏ và ghi đồng thời trên $n-1$ đĩa và bit kiểm tra chẵn lẻ được ghi trên đĩa dùng làm đĩa chứa bit parity – chẵn lẻ đan chéo ở mức độ bit. Bit chẵn lẻ là một bit mà người ta thêm vào một tập hợp các bit làm cho số bit có trị số 1 (hoặc 0) là chẵn (hay lẻ). Thay vì có một bản sao hoàn chỉnh của thông tin gốc trên mỗi đĩa, người ta chỉ cần có đủ thông tin để phục hồi thông tin đã mất trong trường hợp có hỏng ổ đĩa. Khi một đĩa bất kỳ trong mảng bị hư, hệ thống vẫn hoạt động bình thường. Khi thay thế một đĩa mới vào mảng, căn cứ vào dữ liệu trên các đĩa còn lại, hệ thống tái tạo thông tin. Hiệu suất sử dụng đĩa cho cách thiết lập này là $n-1/n$. RAID 3 chỉ có thể được thiết lập bằng phần cứng (RAID controller).
- e. **RAID 4**: từ RAID 4 đến RAID 6 dùng kỹ thuật truy cập các đĩa trong mảng độc lập. Trong một mảng truy cập độc lập, mỗi đĩa thành viên được truy xuất độc lập, do đó mảng có thể đáp ứng được các yêu cầu song song của ngoại vi. Kỹ thuật này thích hợp với các ứng dụng yêu cầu nhiều ngoại vi là các ứng dụng yêu cầu tốc độ truyền dữ liệu

cao. Trong RAID 4, một đĩa dùng để chứa các bit kiểm tra được tính toán từ dữ liệu được lưu trên các đĩa dữ liệu. Khuyết điểm lớn nhất của RAID 4 là bị nghẽn cổ chai tại đĩa kiểm tra khi có nhiều yêu cầu đồng thời từ các ngoại vi.

Bảng 3.10. So sánh các cấp RAID

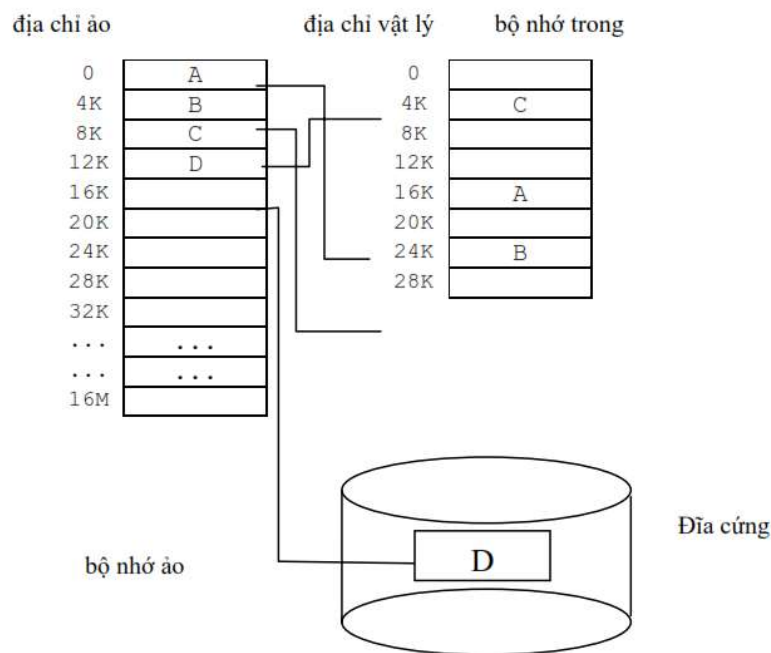
Cấp	Ưu điểm	Nhược điểm	Ứng dụng
0	Hiệu suất truyền vào/ra cải thiện nhiều nhờ dàn tải I/O trên nhiều kênh và ổ đĩa Không cần tính bit chẵn lẻ Thiết kế rất đơn giản Dễ triển khai	Hỏng 1 ổ đĩa thì mất tất cả dữ liệu trong mảng	Sản xuất, biên tập video Biên tập ảnh Ứng dụng yêu cầu băng thông cao
1	Độ dư thừa 100% Không cần tái tạo lại dữ liệu nếu có một ổ đĩa bị lỗi	Không hiệu quả Chi phí ổ đĩa lớn nhất (100%)	Kế toán Tài chính Ứng dụng đòi hỏi độ tin cậy cao
2	Tốc độ truyền dữ liệu cực cao Thiết kế điều khiển đơn giản hơn RAID 3, 4, 5	Tốc độ truyền dữ liệu càng cao, tỉ lệ ổ mã sửa lỗi/ổ dữ liệu rất cao nếu kích thước từ nhỏ - kém hiệu quả Đắt	Không sử dụng trong thực tế
3	Tốc độ đọc/ghi dữ liệu rất cao Đĩa hồng không ảnh hưởng nhiều đến thông lượng Tỉ lệ ổ mã sửa lỗi/ổ dữ liệu thấp – hiệu quả cao	Tốc độ giao dịch bằng tốc độ của 1 ổ đĩa đơn Thiết kế điều khiển khá phức tạp	Sản xuất video Live streaming Biên tập ảnh Ứng dụng yêu cầu thông lượng cao
4	Tốc độ giao dịch dữ liệu đọc cao Tỉ lệ ổ mã sửa lỗi/ổ dữ liệu thấp – hiệu quả cao	Thiết kế điều khiển khá phức tạp Tốc độ giao dịch ghi thấp nhất Tái tạo dữ liệu khó khăn, kém hiệu quả	Không sử dụng trong thực tế
5	Tốc độ giao dịch dữ liệu đọc cao nhất	Thiết kế điều khiển phức tạp nhất Tái tạo dữ liệu khó khăn, kém hiệu quả	Server ứng dụng, Server database Server web, e-mail Server intranet
6	Chịu được lỗi dữ liệu cực lớn Chịu được lỗi ổ đĩa đồng thời	Thiết kế điều khiển phức tạp hơn Overhead để tính địa chỉ parity cực cao	Giải pháp hoàn hảo cho ứng dụng quan trọng

- f. **RAID 5:** yêu cầu thiết lập giống như RAID 4, dữ liệu được ghi từng khối trên các đĩa thành viên, các bit chẵn lẻ được tính toán mức độ khối được ghi trải đều lên trên tất cả các ổ đĩa trong mảng. Tương tự RAID 4, khi một đĩa bất kỳ trong mảng bị hư hỏng, hệ thống vẫn hoạt động bình thường. Khi thay thế một đĩa mới vào mảng, căn cứ vào dữ liệu trên các đĩa còn lại, hệ thống tái tạo thông tin. Hiệu suất sử dụng đĩa cho cách thiết lập này là $n-1/n$. RAID 5 chỉ có thể được thiết lập bằng phần cứng (RAID controller). Cơ chế này khắc phục được khuyết điểm đã nêu trong cơ chế RAID 4.
- g. **RAID 6:** Trong kỹ thuật này, cần có $n+2$ đĩa trong mảng. Trong đó, n đĩa dữ liệu và 2 đĩa riêng biệt để lưu các khối kiểm tra. Một trong hai đĩa kiểm tra dùng cơ chế kiểm tra như trong RAID 4&5, đĩa còn lại kiểm tra độc lập theo một giải thuật kiểm tra. Qua đó, nó có thể phục hồi được dữ liệu ngay cả khi có hai đĩa dữ liệu trong mảng bị hư hỏng.

Hiện nay, RAID 0,1,5 được dùng nhiều trong các hệ thống. Các giải pháp RAID trên đây (trừ RAID 6) chỉ đảm bảo an toàn dữ liệu khi có một đĩa trong mảng bị hư hỏng. So sánh các cấp RAID được liệt kê chi tiết trong Bảng 3.10.

3.5. BỘ NHỚ ẢO

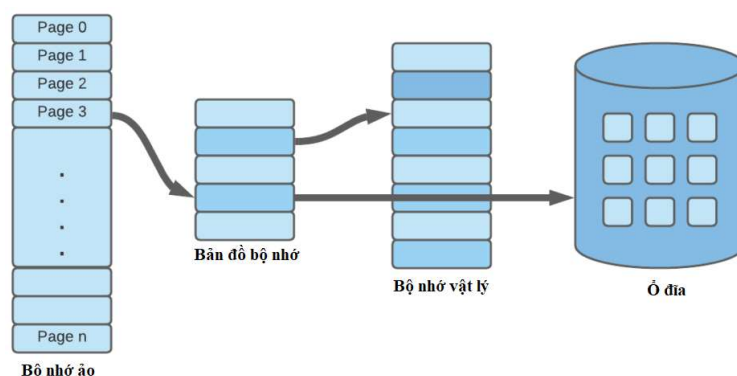
Bộ nhớ ảo xác định một cơ chế vận chuyển tự động dữ liệu giữa bộ nhớ trong và bộ nhớ ngoài (đĩa từ). Trước đây, khi độ dài của chương trình vượt quá giới hạn dung lượng bộ nhớ thì người lập trình phải phân chia chương trình của mình thành từng phần tự loại bỏ nhau (overlays) và phải tự quản lý việc trao đổi thông tin giữa bộ nhớ và đĩa từ. Bộ nhớ ảo làm nhẹ trách nhiệm của các nhà lập trình bằng cách làm cho việc trao đổi thông tin này được thực hiện một cách tự động.



Hình 3.32. Minh họa chương trình gồm 4 trang A,B,C,D trong đó trang D nằm trong ổ đĩa

Trong các bộ xử lý hiện đại, bộ nhớ ảo được dùng để cho phép thực hiện cùng lúc nhiều tiến trình (process), mỗi tiến trình có một không gian địa chỉ riêng. Nếu tất cả các không gian địa chỉ này đều thuộc không gian địa chỉ bộ nhớ trong thì rất tốn kém. Bộ nhớ ảo bao gồm bộ nhớ trong và bộ nhớ ngoài được phân tích thành khối để có thể cung cấp cho mỗi chương trình một số khối cần thiết cho việc thực hiện chương trình đó. Hình 3.32 cho thấy một chương trình chứa trong bộ nhớ ảo gồm 4 khối, 3 trong 4 khối nằm ở bộ nhớ trong, khối thứ tư nằm trên đĩa.

Ngoài việc phân chia không gian bộ nhớ, cần bảo vệ và quản lý tự động các cấp bộ nhớ, bộ nhớ ảo đơn giản hoá việc nạp chương trình vào bộ nhớ để thi hành nhờ một cơ chế được gọi là sự tái định địa chỉ (address relocation). Cơ chế này cho phép một chương trình có thể được thi hành khi nó nằm ở bất cứ vị trí nào trong bộ nhớ.



Hình 3.33. Ảnh xạ các trang nhớ của bộ nhớ ảo vào bộ nhớ vật lý

Bảng 3.11 cho thấy, so với bộ nhớ cache thì các tham số của bộ nhớ ảo tăng từ 10 đến 100.000 lần. Ngoài ra, một số khác biệt khác giữa bộ nhớ cache và bộ nhớ ảo có thể kể đến là:

- Khi tìm kiếm khối nhớ trong cache thất bại, việc thay thế khối nhớ trong cache được điều khiển bằng phần cứng, trong khi sự thay thế trong bộ nhớ ảo là chủ yếu do hệ điều hành.
- Không gian địa chỉ mà bộ xử lý quản lý là không gian địa chỉ của bộ nhớ ảo, trong khi đó thì dung lượng bộ nhớ cache không tùy thuộc vào không gian địa chỉ quản lý của bộ xử lý.
- Bộ nhớ ngoài còn được dùng để lưu trữ tập tin ngoài nhiệm vụ hỗ trợ cho bộ nhớ trong (trong các cấp bộ nhớ).

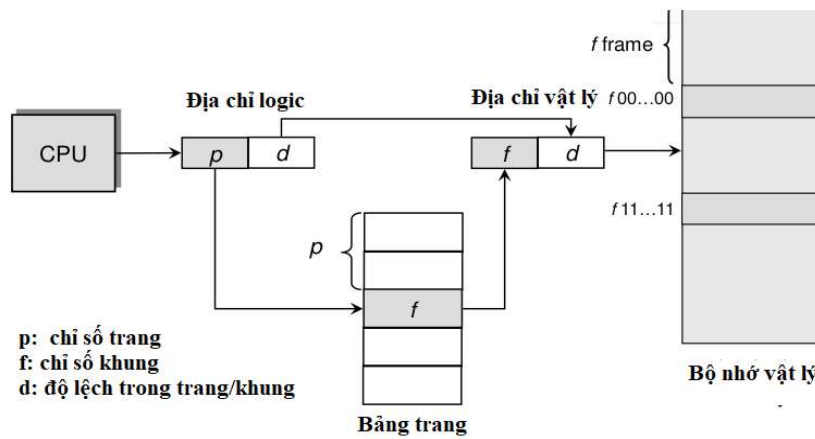
Bảng 3.11. Một số tham số điển hình cho bộ nhớ cache và bộ nhớ ảo.

Tham số	Cache	Bộ nhớ ảo
Chiều dài mỗi khối (trang)	16 - 128 byte	4096 - 65536 bytes
Thời gian thâm nhập thành công	1 - 2 xung nhịp	40 - 100 xung nhịp
Trùng phạt khi thất bại (Thời gian thâm nhập)	8 - 100 xung nhịp	700.000 - 6 triệu xung
(Di chuyển số liệu)	6 - 60 xung	500.000 - 4 triệu xung
	2 - 40 xung	200.000 - 2 triệu xung
Tỉ số thất bại	0,5% - 10%	0,00001% - 0,001%
Dung lượng	8 KB - 8MB	16 MB - 8GB

Bộ nhớ ảo cũng được thiết kế bằng nhiều kỹ thuật đặc thù cho chính nó.

Các hệ thống bộ nhớ ảo có thể được chia thành 2 loại: (1) Bộ nhớ ảo gồm các khối có dung lượng cố định gọi là trang, và (2) Bộ nhớ ảo gồm các khối có chiều dài thay đổi gọi là đoạn. Định vị trang sẽ xác định một địa chỉ trong trang, giống như định vị trong cache. Còn định vị đoạn sẽ cần 2 trường thông tin: một trường chứa số thứ tự đoạn và một trường chứa độ dời trong đoạn. Chương trình dịch sẽ gặp nhiều khó khăn hơn trong định vị đoạn.

Do việc thay thế các đoạn, ngày nay ít máy tính dùng định vị đoạn thuần túy. Một vài máy sử dụng cách hỗn hợp gọi là đoạn trang. Trong đó mỗi đoạn chứa một số nguyên các trang.



Hình 3.34. Minh họa sự ánh xạ địa chỉ giữa bộ nhớ ảo và bộ nhớ vật lý trong định vị trang

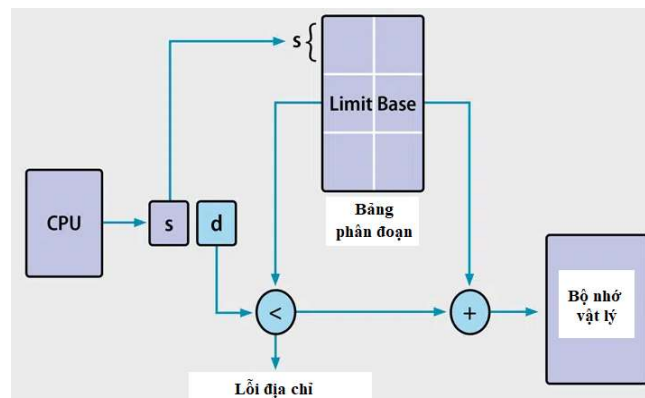
Tiếp theo là một số câu hỏi trao đổi đặt ra trong các cấp bộ nhớ cho bộ nhớ ảo:

- Câu hỏi 1: Một khối được đặt tại đâu trong bộ nhớ trong?

Việc các trang (đoạn) không có trong bộ nhớ vật lý, tương ứng với việc phải truy nhập vào ổ đĩa. Việc truy nhập này rất chậm nên người ta chọn phương án phối hợp trong đó các khối (trang) có thể nằm ở bất kỳ vị trí nào trong bộ nhớ trong. Cách này cho tỉ lệ thất bại thấp.

- Câu hỏi 2: Làm thế nào để tìm một khối khi nó đang nằm trong bộ nhớ trong?

Định vị trang và định vị đoạn đều dựa vào một cấu trúc dữ liệu trong đó số thứ tự trang hoặc số thứ tự đoạn được có chỉ số. Cho định vị trang, dựa vào bảng trang, địa chỉ trong bộ nhớ vật lý được xác lập cuối cùng là việc đặt kề nhau số thứ của trang vật lý với địa chỉ trong trang (Hình 3.34). Cho định vị đoạn, dựa vào thông tin trên bảng đoạn, việc kiểm tra tính hợp lệ của địa chỉ được tiến hành. Địa chỉ vật cuối cùng được xác lập bằng cách cộng địa chỉ đoạn và địa chỉ trong đoạn (độ dời trong đoạn) (Hình 3.35).



Hình 3.35. Ánh xạ địa chỉ giữa bộ nhớ ảo và bộ nhớ vật lý trong cách định vị đoạn

- Câu hỏi 3: Khối nào phải được thay thế khi có thất bại trang?

Hầu hết các hệ điều hành đều cố gắng thay thế khối ít dùng gần đây nhất (LRU: Least Recent Utilized) vì đây là khối ít cần nhất.

- Câu hỏi 4: Việc gì xảy ra khi cần ghi số liệu?

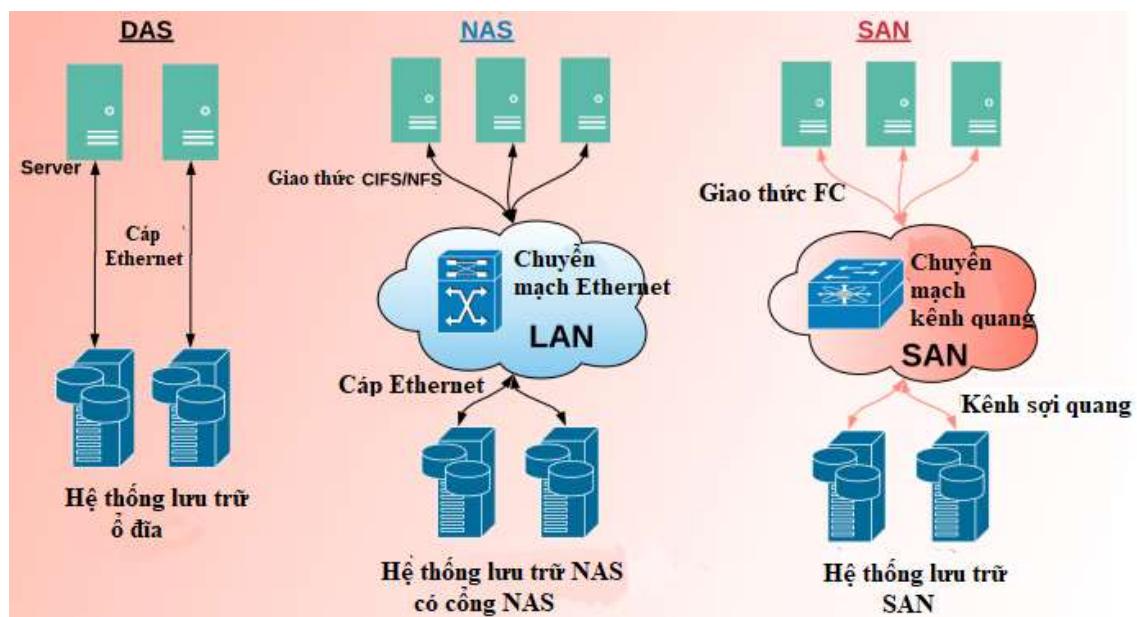
Chiến thuật ghi luôn là một sự ghi lại nghĩa là thông tin chỉ được viết vào trong khối của bộ nhớ trong. Khối có thay đổi thông tin, được chép vào đĩa từ nếu khối này bị thay thế.

3.6. CÁC CÔNG NGHỆ LƯU TRỮ DỮ LIỆU

Để lưu trữ dữ liệu, chúng ta có thể dùng nhiều thiết bị, công nghệ khác nhau. Các kho dữ liệu có thể là dùng đĩa cứng, dùng băng từ, dùng đĩa quang... Tùy theo yêu cầu cụ thể của bài toán đặt ra mà lựa chọn công nghệ và thiết bị cho phù hợp. Theo cơ chế lưu trữ, hiện nay có một số loại hình lưu trữ dữ liệu cơ bản như:

- DAS: lưu trữ dữ liệu qua các thiết bị gắn trực tiếp
- NAS: lưu trữ dữ liệu vào thiết bị lưu trữ thông qua mạng IP
- SAN: lưu trữ dữ liệu qua mạng lưu trữ chuyên dụng riêng.
- Lưu trữ trên server vật lý
- Lưu trữ dữ liệu trên đám mây – Điện toán đám mây

Mỗi loại hình lưu trữ dữ liệu có những ưu nhược điểm riêng và được dùng cho những mục đích nhất định. Phần dưới đây sẽ trình bày chi tiết về một số mô hình lưu trữ dữ liệu ở trên.



Hình 3.36. Giải pháp lưu trữ dữ liệu DAS, NAS, SAN

Mô hình lưu trữ truyền thống DAS

DAS là cơ chế lưu trữ với thiết bị gắn trực tiếp vào máy chủ. Đây được coi là công nghệ lưu trữ truyền thống được nhiều doanh nghiệp sử dụng. Với cơ chế DAS, mỗi máy chủ sẽ có một hệ thống lưu trữ và phần mềm quản lý lưu trữ riêng biệt. Ưu điểm của DAS là khả năng dễ lắp đặt, chi phí thấp và hiệu năng cao.

Nhược điểm của DAS là khả năng mở rộng hạn chế. Thực tế DAS làm việc rất tốt với một server nhưng khi dữ liệu tăng, số lượng máy chủ cũng tăng sẽ tạo nên những vùng dữ liệu phân tán và gián đoạn. Khi đó, nhà quản trị sẽ phải bổ sung hay thiết lập lại dung lượng, và công việc bảo trì sẽ phải thực hiện trên từng server. Điều đó sẽ làm tăng chi phí lưu trữ tổng thể cho doanh nghiệp và sẽ càng khó khăn hơn khi muốn sao lưu hay bảo vệ một hệ thống kho lưu trữ dữ liệu đang nằm rải rác và phân tán như vậy.

Mô hình lưu trữ theo công nghệ NAS

NAS là phương pháp lưu trữ dữ liệu sử dụng các thiết bị lưu trữ đặc biệt gắn trực tiếp vào mạng LAN như một thiết bị mạng bình thường (tương tự máy tính, switch hay router). Các thiết bị NAS cũng được gán các địa chỉ IP cố định và được người dùng truy nhập thông qua sự điều

khiển của máy chủ. Trong một số trường hợp, NAS có thể được truy cập trực tiếp không cần có sự quản lý của máy chủ.

Trong môi trường đa hệ điều hành với nhiều máy chủ khác nhau, việc lưu trữ dữ liệu, sao lưu và phục hồi dữ liệu, quản lý hay áp dụng các chính sách bảo mật đều được thực hiện tập trung.

Ưu điểm của NAS:

- Khả năng mở rộng: khi người dùng cần thêm dung lượng lưu trữ, các thiết bị lưu trữ NAS mới có thể được bổ sung và lắp đặt vào mạng.
- NAS cũng tăng cường khả năng chống lại sự cố cho mạng. Trong môi trường DAS, khi một máy chủ chứa dữ liệu không hoạt động thì toàn bộ dữ liệu đó không thể sử dụng được. Trong môi trường NAS, dữ liệu vẫn hoàn toàn có thể được truy nhập bởi người dùng. Các biện pháp chống lỗi và dự phòng tiên tiến được áp dụng để đảm bảo NAS luôn sẵn sàng cung cấp dữ liệu cho người sử dụng.

Nhược điểm của NAS:

- Với việc sử dụng chung hạ tầng mạng với các ứng dụng khác, việc lưu trữ dữ liệu có thể ảnh hưởng đến hiệu năng của toàn hệ thống (làm chậm tốc độ của LAN), điều này đặc biệt đáng quan tâm khi cần lưu trữ thường xuyên một lượng lớn dữ liệu.
- Trong môi trường có các hệ cơ sở dữ liệu thì NAS không phải là giải pháp tốt vì các hệ quản trị cơ sở dữ liệu thường lưu dữ liệu dưới dạng khối chứ không phải dưới dạng file nên sử dụng NAS sẽ không cho hiệu năng tốt.

Mô hình lưu trữ SAN

SAN là một mạng riêng tốc độ cao dùng cho việc truyền dữ liệu giữa các máy chủ tham gia vào hệ thống lưu trữ cũng như giữa các thiết bị lưu trữ với nhau. SAN cho phép thực hiện quản lý tập trung và cung cấp khả năng chia sẻ dữ liệu và tài nguyên lưu trữ. Hầu hết mạng SAN hiện nay dựa trên công nghệ kênh cáp quang, cung cấp cho người sử dụng khả năng mở rộng, hiệu năng và tính sẵn sàng cao.

Hệ thống SAN được chia làm hai mức: mức vật lý và logic

- Mức vật lý: mô tả sự liên kết các thành phần của mạng tạo ra một hệ thống lưu trữ đồng nhất và có thể sử dụng đồng thời cho nhiều ứng dụng và người dùng.
- Mức logic: bao gồm các ứng dụng, các công cụ quản lý và dịch vụ được xây dựng trên nền tảng của các thiết bị lớp vật lý, cung cấp khả năng quản lý hệ thống SAN.

Ưu điểm của hệ thống SAN

- Có khả năng sao lưu dữ liệu với dung lượng lớn và thường xuyên mà không làm ảnh hưởng đến lưu lượng thông tin trên mạng.
- SAN đặc biệt thích hợp với các ứng dụng cần tốc độ và độ trễ nhỏ ví dụ như các ứng dụng xử lý giao dịch trong ngành ngân hàng, tài chính.
- Dữ liệu luôn ở mức độ sẵn sàng cao.
- Dữ liệu được lưu trữ thống nhất, tập trung và có khả năng quản lý cao. Có khả năng khôi phục dữ liệu nếu có xảy ra sự cố.
- Hỗ trợ nhiều giao thức, chuẩn lưu trữ khác nhau như: iSCSI, FCIP, DWDM...
- Có khả năng mở rộng tốt trên cả phương diện số lượng thiết bị, dung lượng hệ thống cũng như khoảng cách vật lý.

- Mức độ an toàn cao do thực hiện quản lý tập trung cũng như sử dụng các công cụ hỗ trợ quản lý SAN. Do đó, SAN thường được sử dụng ở những trung tâm dữ liệu lớn vì mang một số đặc điểm nổi bật như: Giảm thiểu rủi ro cho dữ liệu, khả năng chia sẻ tài nguyên rất cao, khả năng phát triển dễ dàng, thông lượng lớn, hỗ trợ nhiều loại thiết bị, hỗ trợ và quản lý việc truyền dữ liệu lớn và tính an ninh dữ liệu cao. Hơn nữa, SAN tăng cường hiệu quả hoạt động của hệ thống bằng việc hỗ trợ đồng thời nhiều hệ điều hành, máy chủ và các ứng dụng, có khả năng đáp ứng nhanh chóng với những thay đổi về yêu cầu hoạt động của một tổ chức cũng như yêu cầu kỹ thuật của hệ thống mạng.

Mô hình lưu trữ trên server vật lý

Là giải pháp mà người quản trị dùng 1 máy chủ vật lý, cài đặt hệ điều hành để lưu trữ dữ liệu. Ứng dụng thường dùng để lưu trữ dữ liệu trên máy chủ là File Sharing Server.

- File Sharing Server là một hệ thống quản lý dữ liệu tập trung, cho phép doanh nghiệp lưu trữ và quản lý dữ liệu tập trung trên server đó. Từ server người quản trị phân quyền truy cập và sử dụng dữ liệu cho từng nhân viên, từng phòng ban. Hệ thống File Sharing Server thường được dùng phổ biến trong các doanh nghiệp.

Ưu điểm của lưu trữ trên server vật lý:

- Dễ lắp đặt, chi phí thấp, hiệu năng cao.
- Tận dụng được thiết bị máy chủ vật lý và các linh kiện hiện có
- Dễ quản trị và phân quyền.

Nhược điểm của lưu trữ trên server vật lý:

- Hư hỏng linh kiện máy chủ vật lý nói chung, ví dụ hư mainboard, hỏng RAM, hỏng nguồn. Đặc biệt là hư hỏng đĩa cứng: đây là vấn đề quan trọng nhất của an toàn dữ liệu. Nếu các linh kiện khác có thể thay thế, thì đĩa cứng là nơi lưu trữ dữ liệu lại không thể thay thế. Vì vậy, trong các máy chủ vật lý, cơ chế RAID được sinh ra để dữ liệu được lưu trữ đồng thời trên nhiều đĩa cứng vật lý của một máy chủ, phòng trừ trường hợp hỏng 1 hoặc vài đĩa cứng, dữ liệu vẫn bảo đảm an toàn.
- Khả năng mở rộng hạn chế do giới hạn số lượng khe cắm ổ nếu nhu cầu mở rộng lớn.
- Việc truy cập vào ổ đĩa sẽ bị chậm khi hệ điều hành quá tải.

Mô hình lưu trữ dữ liệu trên đám mây – Điện toán đám mây

Đây là một hình thức sử dụng dịch vụ của một nhà cung cấp nào đó cho phép người dùng dịch vụ của họ có thể lưu giữ, quản lý, chia sẻ và backup dữ liệu của họ từ xa. Dịch vụ này cho phép người dùng có thể truy cập tập tin ấy bất cứ lúc nào, miễn là có truy cập internet. Dữ liệu của chúng ta được lưu trên server của nhà cung cấp đó và bạn sẽ sử dụng một ứng dụng desktop hoặc ứng dụng web online của họ để truy xuất đến dữ liệu của mình.

Hiện nay, các nhà cung cấp hạ tầng đám mây có 2 giải pháp dịch vụ lưu trữ dữ liệu trên đám mây là Lưu trữ đám mây và Sao lưu đám mây.

- Lưu trữ đám mây: được truy cập qua internet, lưu trữ dữ liệu trong một trung tâm ngoại vi từ xa. Phần lớn các dịch vụ có giao diện web để bạn tải tệp lên, điều này không an toàn vì dữ liệu trong lúc truyền không được mã hóa. Một số nhà cung cấp cũng không có công cụ đồng bộ hóa các tệp giữa máy tính của bạn và dịch vụ của họ. Hơn nữa, lưu trữ đám mây chỉ các tệp và thư mục có thể được lưu trữ, do đó không có dữ liệu ứng

dụng. Một số nhà cung cấp dịch vụ này như: Dropbox, Google Drive, iCloud và Onedrive... Lưu trữ đám mây là một cách thức để chia sẻ tài liệu. Tuy nhiên, đây không phải là nơi để sao lưu các ứng dụng và tài liệu quan trọng trong kinh doanh.

- Sao lưu đám mây: là một ứng dụng dựa trên đám mây, cung cấp cho chúng ta khả năng tự động sao lưu các tệp, ứng dụng, máy ảo, máy chủ và lưu trữ. Sao lưu đám mây thường thường chạy theo lịch trình nhiều lần hàng ngày hoặc tự động. Đồng bộ đám mây rất hữu ích trong việc cho phép chia sẻ file trên nhiều thiết bị với nhiều người dùng, rất phù hợp cho làm việc theo nhóm và chia sẻ dự án. Ngoài ra, dữ liệu lưu trữ được mã hóa bằng các chuẩn mã hóa mới nhất AES 256, DES 64, RC2 128,... đảm bảo an toàn bảo mật thông tin.

Ưu điểm của lưu trữ trên đám mây: Có thể khẳng định đây là giải pháp lưu trữ liệu có tính sẵn sàng cao, khả năng mở rộng và an toàn dữ liệu ở mức cao nhất, nhờ các ứng dụng ưu điểm công nghệ điện toán đám mây.

- Tiết kiệm chi phí: Doanh nghiệp chỉ cần chi trả cho những không gian lưu trữ mình sử dụng. Đồng thời, chúng ta cũng không cần bỏ nhiều chi phí để mua ổ cứng, phần cứng,...
- Người dùng có thể dễ dàng kéo và thả các tệp trong bộ nhớ đám mây. Chúng ta dễ dàng lưu tất cả các tệp và dữ liệu trong đám mây mà không cần phải có kiến thức chuyên môn cao về công nghệ. Các tệp tin được lưu trữ có thể dễ dàng truy cập từ bất cứ nơi nào trên thế giới chỉ với một vài cú nhấp chuột và kết nối internet.
- Dữ liệu được lưu trữ trên đám mây sẽ được bảo vệ an toàn trước mọi loại lỗi phần cứng. Các dịch vụ lưu trữ đám mây cũng cung cấp khả năng backup để đảm bảo dữ liệu của chúng ta an toàn.
- Lưu trữ đám mây cho phép mở rộng theo cả chiều ngang và chiều dọc, chúng ta chỉ cần chi trả cho các tài nguyên đã sử dụng là được. Chúng ta có thể mở rộng quy mô môi trường lưu trữ đám mây của mình bất cứ khi nào có nhu cầu, xác định các thuộc tính, cấu hình của quy mô đám mây theo mong muốn. Điều này đảm bảo được không gian lưu trữ lớn với tính linh hoạt cao.
- Khi sử dụng lưu trữ trên máy chủ vật lý, bạn chỉ có thể truy cập dữ liệu từ một vị trí nhất định. Với lưu trữ đám mây, dữ liệu có thể được truy cập từ tất cả các thiết bị như PC hoặc điện thoại thông minh. Chúng ta có thể truy cập tệp và đồng bộ hóa dữ liệu dễ dàng với bất kỳ thiết bị nào thông qua kết nối internet. Do đó, chúng ta không cần phải lo lắng về việc chuyển các tài liệu từ thiết bị này sang thiết bị khác một cách thủ công và phức tạp nữa. Ngoài ra, các tệp được lưu trữ trên đám mây vẫn được giữ nguyên trên tất cả các thiết bị và chúng sẽ tự động cập nhật nếu có bất kỳ thay đổi nào.
- Lưu trữ trên đám mây cũng là một nền tảng lý tưởng cho mục đích hợp tác. Môi trường đám mây cho phép nhiều người truy cập, chỉnh sửa và cộng tác trên một tệp hoặc tài liệu. Mọi người có thể truy cập vào môi trường đám mây từ bất cứ nơi nào trên thế giới và cộng tác trong thời gian thực. Giúp chúng ta quản lý thông tin dễ dàng với các tính năng phân quyền, chia sẻ dữ liệu nội bộ, cá nhân, quản trị thành viên.
- Ngay cả khi chúng ta lưu trữ dữ liệu trên các thiết bị có thể di chuyển được như ổ cứng ngoài hoặc ổ flash, thì chúng ta vẫn phải thao tác thủ công và đối mặt với những vấn đề hỏng hóc vật lý. Dữ liệu được lưu trữ trên đám mây được sao lưu trực tuyến và được truy cập từ bất cứ đâu. Dữ liệu luôn được lưu tự động khi được tải lên đám mây. Chúng ta không cần phải lưu, gắn nhãn hoặc theo dõi các dữ liệu này. Sự tiện lợi của lưu trữ đám mây trực tuyến cho phép chúng ta hoàn toàn tập trung vào công việc khác mà không phải lo lắng về việc mất dữ liệu.

Nhược điểm của lưu trữ trên đám mây:

- Nếu mất mạng Internet tại thiết bị sử dụng thì người dùng không đồng bộ được dữ liệu lên máy chủ đám mây.
- Trong nhiều trường hợp, chúng ta có thể cài đặt cả hai hệ thống để đáp ứng các hoạt động khác nhau của doanh nghiệp. Ví dụ, chúng ta cần NAS để lưu trữ và truy cập tệp, còn SAN để ảo hóa hoặc cho các trường hợp sử dụng khác. Từ đó sẽ tận dụng được lợi thế của mỗi loại. Tích hợp cả NAS và SAN để tạo ra một bộ lưu trữ hợp nhất. Bộ nhớ này sử dụng nhiều giao thức như NFS, SMB, kênh sợi quang và SCSI để đảm bảo tính linh hoạt cao, khả năng mở rộng và khả năng thích ứng. Bằng cách này, cùng một bộ nhớ vật lý có thể phân bổ cho cả SAN hoặc NAS tùy theo yêu cầu sử dụng. Tuy nhiên, giải pháp này sẽ phù hợp cho các doanh nghiệp, tổ chức có quy mô lưu trữ lớn, dự trù kinh phí triển khai, mở rộng, nhất là các doanh nghiệp đòi hỏi cao về mặt xử lý dữ liệu như truyền hình, ngân hàng,... Còn với lưu trữ đám mây, nhìn chung sẽ mang lại lợi ích nhiều hơn.

Câu hỏi ôn tập chương 3

1. Hãy phân loại các loại bộ nhớ máy tính.
2. Phân biệt ba kỹ thuật: truy cập tuần tự, truy cập trực tiếp, và truy cập ngẫu nhiên?
3. Mối quan hệ giữa thời gian truy cập, giá cả bộ nhớ, và dung lượng là gì?
4. Nêu cấu trúc phân cấp của hệ thống bộ nhớ máy tính. Tại sao cấu trúc phân cấp của hệ thống bộ nhớ có thể giúp tăng hiệu năng và giảm giá thành sản xuất máy tính?
5. Phân biệt bộ nhớ RAM tĩnh và RAM động. Tại sao bộ nhớ RAM động cần quá trình làm tươi và RAM động thường rẻ hơn RAM tĩnh?
6. Nguyên lý cục bộ của tham chiếu ảnh hưởng thế nào đến việc sử dụng nhiều cấp độ bộ nhớ?
 1. Nêu phương thức trao đổi dữ liệu giữa CPU, bộ nhớ chính với cache.
 2. Phân biệt ba kỹ thuật: ánh xạ trực tiếp, ánh xạ kết hợp, và ánh xạ tập kết hợp là gì?
 3. Đối với cache ánh xạ trực tiếp, một địa chỉ bộ nhớ chính gồm bao nhiêu trường? Là những trường gì?
 4. Đối với cache kết hợp, một địa chỉ bộ nhớ chính gồm bao nhiêu trường? Là những trường gì?
 5. Đối với cache tập kết hợp, một địa chỉ bộ nhớ chính gồm bao nhiêu trường? Là những trường gì?
6. Cho máy tính có dung lượng bộ nhớ chính: 512MB, cache: 128KB, line: 64byte, độ dài ngăn nhớ: 4byte. Trong trường hợp kỹ thuật ánh xạ trực tiếp, xác định dạng địa chỉ phát ra để truy nhập cache.
7. Cho máy tính có dung lượng bộ nhớ chính: 256MB, cache: 64KB, line: 16byte, độ dài ngăn nhớ: 4byte. Trong trường hợp kỹ thuật Ánh xạ kết hợp toàn phần, xác định dạng địa chỉ phát ra để truy nhập cache.
8. Cho máy tính có dung lượng bộ nhớ chính : 256MB, cache : 128KB, line : 128byte, độ dài ngăn nhớ : 4byte, set : 8 line. Trong trường hợp kỹ thuật ánh xạ liên kết tập hợp, xác định dạng địa chỉ phát ra để truy nhập cache.
9. Nêu các phương pháp đọc ghi và các chính sách thay thế dòng cache. Tại sao thay thế dòng cache sử dụng phương pháp LRU có khả năng cho hệ số đoán trúng (hit) cao nhất ?
10. Phân biệt write through và write back?
11. Ưu và nhược điểm của cache phân chia là gì?
12. Trình bày kỹ thuật nạp trang theo nhu cầu dùng cho bộ nhớ ảo, bao gồm : khái niệm, ví dụ minh họa, quá trình thực hiện ngắt thiếu trang. Nạp trang hoàn toàn theo nhu cầu khác với nạp trang trước như thế nào?
13. Trình bày các bước thực hiện khi đổi trang?

14. Giả sử tiến trình được cấp 5 khung nhớ vật lý, các trang của tiến trình được truy cập theo thứ tự sau : 1, 2, 3, 4, 5, 3, 4, 1, 6, 7, 8, 7, 8, 9, 7, 8, 9, 5. Hãy xác định thứ tự nạp và đổi trang nếu sử dụng hai thuật toán FIFO và LRU.
15. Giả sử trong quá trình quản lý bộ nhớ dạng phân đoạn, hệ điều hành sử dụng bảng phân đoạn có thông tin được cho dưới đây:

Chỉ số đoạn	Địa chỉ cơ sở	Kích thước giới hạn đoạn
0	200	600
1	2500	50
2	96	100
3	1300	580
4	1952	150

Hãy tính địa chỉ vật lý cho những địa chỉ lô-gic sau: (0, 450), (1, 010), (2, 500), (3, 400) và (4, 192).

16. Giả sử không gian nhớ lô-gic của tiến trình gồm 4 trang, mỗi trang kích thước 2048 Byte, bộ nhớ vật lý gồm 32 khung. Bảng trang (bảng chứa chỉ số khung trang vật lý tương ứng với trang logic của tiến trình) được cho dưới đây:

Chỉ số trang	Chỉ số khung
0	3
1	0
2	
3	5

Hãy cho biết :

- Để biểu diễn địa chỉ lô gic trong trường hợp này cần bao nhiêu bit?
 - Tính địa chỉ vật lý cho những địa chỉ lô gic sau : 1000, 2500, 5000.
17. Hệ thống máy tính được trang bị cache, bộ nhớ chính và sử dụng một phần ổ đĩa làm bộ nhớ ảo. Giả sử để đọc một từ vào CPU thì thời gian cần thiết là: 1) 10 ns nếu từ đó nằm trong cache, 2) 60 ns nếu từ đó nằm trong bộ nhớ chính và 3) 10 ms để tải từ đó từ bộ nhớ ảo vào bộ nhớ chính cùng với thời gian để tải từ đó từ bộ nhớ chính (đã bao gồm cả thời gian tìm kiếm và kiểm tra xác định vị trí của từ đó). Biết rằng tỉ lệ hit của cache là 0,92 và tỉ lệ hit của bộ nhớ chính là 0.65, hãy tính thời gian truy cập trung bình để đọc một từ vào CPU trong hệ thống này.
18. Dữ liệu được ghi lên đĩa từ như thế nào?
19. Dữ liệu được đọc ra từ đĩa từ như thế nào?
20. So sánh hai kỹ thuật CAV vận tốc góc không đổi và ghi nhiều vùng?
21. Định nghĩa track, cylinder và sector?
22. Kích thước diện tích của sector là bao nhiêu?
23. Định nghĩa các thuật ngữ: thời gian tìm kiếm, trễ quay, thời gian truy cập, và thời gian truyền?
24. Những đặc điểm chung của các cấp độ RAID là gì?
25. Phân biệt các cấp RAID?

CHƯƠNG 4. XỬ LÝ SONG SONG VÀ ĐA LỖI

Nhiều lĩnh vực mới như đồ họa máy tính, trí tuệ nhân tạo, phân tích số,... đòi hỏi phải xử lý một khối lượng dữ liệu rất lớn do đó cần phải có những hệ thống máy tính mạnh để thực hiện được những yêu cầu trong thực tế. Những vấn đề về xử lý ngôn ngữ tự nhiên, nhận dạng, xử lý ảnh ba chiều, dự báo thời tiết, mô hình và mô phỏng những hệ thống lớn,... đều yêu cầu phải xử lý dữ liệu với tốc độ rất cao, cùng khối lượng dữ liệu lớn. Hầu hết với những bài toán này thì những máy tính xử lý tuần tự kiểu Von Neumann là không đáp ứng được.

Mặc dù tốc độ xử lý của các bộ xử lý tăng nhưng khả năng tính toán của chúng không thể tăng mãi được do giới hạn về vật lý. Điều này dẫn tới là muốn tăng được khả năng tính toán của các hệ thống máy tính thì mục tiêu là phải khai thác được khả năng xử lý song song của chúng.

Xử lý song song là quá trình xử lý gồm nhiều tiến trình được kích hoạt đồng thời và cùng tham gia giải quyết một vấn đề, nói chung là thực hiện trên những hệ thống đa bộ xử lý.

Ba yếu tố chính cho phép thực hiện xử lý song song:

1. Hiện nay giá thành của phần cứng (CPU) giảm mạnh, tạo điều kiện để xây dựng những hệ thống có nhiều bộ xử lý (đa CPU) với giá thành hợp lý.
2. Sự phát triển của công nghệ mạch tích hợp VLSI cho phép tạo ra những hệ phức hợp có hàng triệu transistor trên một chip.
3. Tốc độ xử lý của các bộ xử lý (BXL) theo kiểu von Neumann đã dần tiến tới giới hạn, không thể cải tiến thêm được do vậy dẫn tới đòi hỏi phải thực hiện xử lý song song.

Vấn đề xử lý song song liên quan trực tiếp đến kiến trúc máy tính, phần mềm hệ thống (hệ điều hành), thuật toán và ngôn ngữ lập trình, v.v. Các máy tính song song có thể phân thành nhiều loại dựa vào các đặc trưng của các kiến trúc và thể thực thao tác khác nhau. Cụ thể là có thể dựa vào các chỉ tiêu về kiểu và số lượng các BXL, sự kết nối giữa chúng, dựa vào sơ đồ truyền thông và các thao tác vào/ra, v.v. Phần lớn các hệ điều hành ngày nay đều đã hỗ trợ đa xử lý/đa nhiệm và cho phép nghiên cứu, khai thác các phương pháp lập trình song song. Vấn đề là chúng ta phải có nhiều BXL (các đơn vị tính toán độc lập) cùng hoạt động. Nhưng điều quan trọng là chúng phải tham gia "cùng giải một bài toán". Nói cách khác, những tiến trình thực hiện trên mỗi BXL phải kết hợp, trao đổi với nhau để giải quyết một bài toán cho trước.

4.1. MÔ HÌNH TỔ CHỨC ĐA CPU

Định nghĩa: Mô hình tổ chức đa CPU là mô hình gồm một tập các BXL, thường là cùng một loại, kết nối với nhau theo một cách nào đó để có thể hợp tác với nhau trong hoạt động và trao đổi dữ liệu được với nhau.

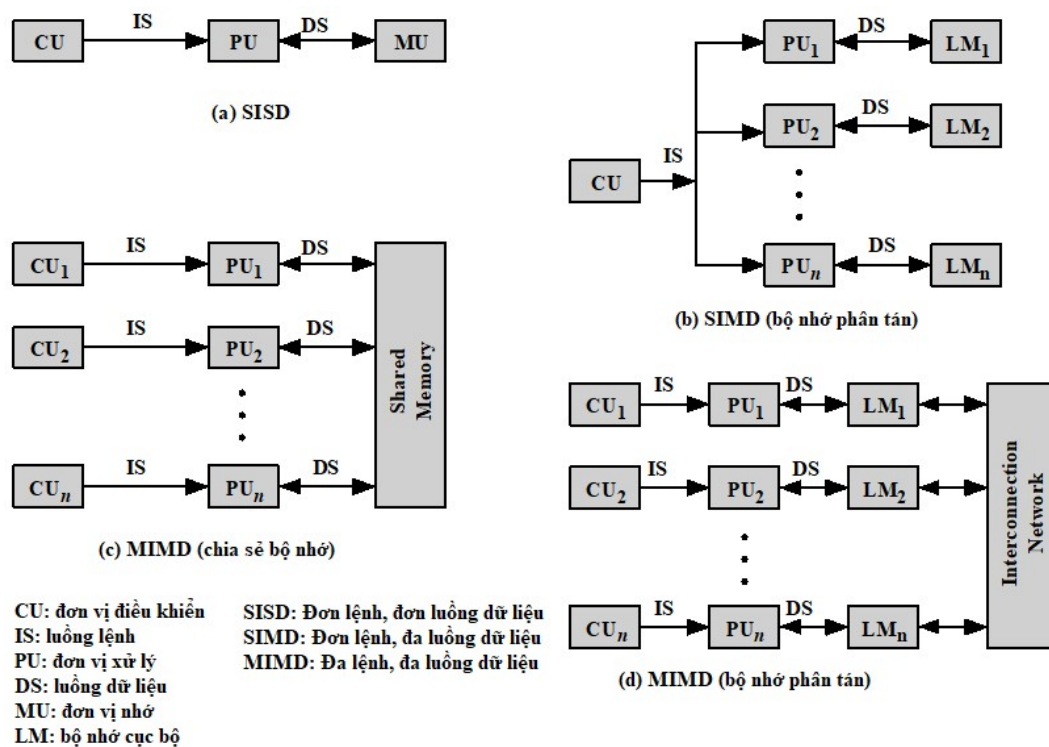
Dựa vào các đặc tính về số lượng BXL, số chương trình thực hiện, cấu trúc bộ nhớ, v.v., Michael Flynn (1966) đã đưa ra cách phân loại nổi tiếng được nhiều người chấp nhận (Bảng 4.1):

1. Mô hình Đơn dòng lệnh, đơn dòng dữ liệu (SISD: Single Instruction, Single Data Stream). Đây thực chất chính là kiến trúc Von Neumann.
2. Mô hình Đơn dòng lệnh, đa dòng dữ liệu (SIMD: Single Instruction, Multiple Data Stream). Một lệnh máy duy nhất điều khiển việc thực thi đồng thời một số phần tử xử lý trên cơ sở Lockstep. Mỗi phần tử xử lý có một bộ nhớ dữ liệu được liên kết, do đó, các lệnh được thực thi trên các tập dữ liệu khác nhau bởi các bộ xử lý khác nhau. Các bộ xử lý kiểu vectơ và mảng đều thuộc loại mô hình này.

3. Mô hình Đa dòng lệnh, đơn dòng dữ liệu (MISD: Multiple Instruction, Single Data Stream). Một chuỗi dữ liệu được truyền đến một tập các bộ xử lý, mỗi bộ xử lý thực hiện một chuỗi lệnh khác nhau. Cấu trúc này không được triển khai thương mại.
4. Mô hình Đa dòng lệnh, đa dòng dữ liệu (MIMD: Multiple Instruction, Multiple Data Stream). Một tập các bộ xử lý thực hiện đồng thời các chuỗi lệnh khác nhau trên các tập dữ liệu khác nhau. Các hệ thống SMP (đa xử lý đối xứng), cụm và NUMA (truy nhập bộ nhớ không đồng nhất) phù hợp với kiểu mô hình này.

Bảng 4.1. Bảng phân loại một số mô hình tổ chức đa CPU

		Luồng dữ liệu	
		Đơn	Đa
Luồng lệnh	Đơn	SISD: Intel Pentium 4	SIMD: các lệnh SSE của x86
	Đa	MISD: chưa có	MIMI: Intel Core i7



Hình 4.1. Một số mô hình tổ chức đa CPU

Sau đây chúng ta nghiên cứu chi tiết các mô hình trên:

Mô hình Đơn luồng lệnh, đơn luồng dữ liệu (SISD)

Máy tính loại SISD (Hình 4.1a) chỉ có một đơn vị xử lý (PU), ở mỗi thời điểm thực hiện một chỉ lệnh và chỉ đọc, ghi một mục dữ liệu. Tất cả các máy tính SISD chỉ có một thanh ghi register được gọi là bộ đếm chương trình (program counter) được sử dụng để nạp địa chỉ của lệnh tiếp theo khi xử lý tuần tự và kết quả là thực hiện theo một thứ tự xác định của các câu lệnh.

Mô hình SISD còn được gọi là SPSD, đơn chương trình và đơn luồng dữ liệu. Đây chính là mô hình máy tính truyền thống kiểu Von Neumann.

Mô hình Đơn luồng lệnh, đa luồng lệnh (SIMD)

Máy tính loại SIMD (Hình 4.1b), còn được gọi là SPMD (đơn chương trình, đa luồng dữ liệu), có một đơn vị điều khiển để điều khiển nhiều đơn vị xử lý thực hiện theo một luồng các câu

lệnh. CPU phát sinh tín hiệu điều khiển tới tất cả các phần tử xử lý, những bộ xử lý này cùng thực hiện một phép toán trên các mục dữ liệu khác nhau, nghĩa là mỗi BXL có luồng dữ liệu riêng. Máy tính SIMD có thể hỗ trợ xử lý kiểu vector, trong đó có thể gán các phần tử của vector cho các phần tử xử lý để tính toán đồng thời. Máy tính vector và các BXL mảng là mô hình chủ yếu thuộc loại này.

Mô hình Đa luồng lệnh, đơn luồng dữ liệu (MISD)

Máy tính loại MISD (Hình 4.1c) là ngược lại với SIMD. Máy tính MISD có thể thực hiện nhiều chương trình (nhiều lệnh) trên cùng một mục dữ liệu, nên còn được gọi là MPSD (đa chương trình, đơn luồng dữ liệu). Kiến trúc kiểu này có thể chia thành hai nhóm:

Lớp các máy tính yêu cầu những đơn vị xử lý khác nhau có thể nhận được những chỉ lệnh khác nhau để thực hiện trên cùng một mục dữ liệu. Đây là kiến trúc khó và hiện nay chưa có loại máy tính nào được sản xuất theo loại này.

Lớp các máy tính có các luồng dữ liệu được chuyển tuần tự theo dãy các CPU liên tiếp. Đây là loại kiến trúc hình ống thực hiện xử lý theo vector thông qua một dãy các bước, trong đó mỗi bước thực hiện một chức năng và sau đó chuyển kết quả cho đơn vị xử lý thực hiện bước tiếp theo.

Mô hình Đa luồng lệnh, đa luồng dữ liệu (MIMD)

Máy tính loại MIMD (Hình 4.1d) còn gọi là đa BXL, trong đó mỗi BXL có thể thực hiện những luồng lệnh khác nhau trên các luồng dữ liệu riêng.

Hầu hết các hệ thống MIMD đều có bộ nhớ riêng và cũng có thể truy cập vào được bộ nhớ chung khi cần, do vậy giảm thiểu được sự trao đổi giữa các BXL trong hệ thống.

Đây là kiến trúc phức tạp nhất, nhưng nó là mô hình hỗ trợ xử lý song song cao nhất và đã có nhiều máy tính được sản xuất theo kiến trúc này, ví dụ như BBN Butterfly, Alliant FX, isPC của Intel,...

4.2. ĐA LUỒNG ĐỒNG THỜI

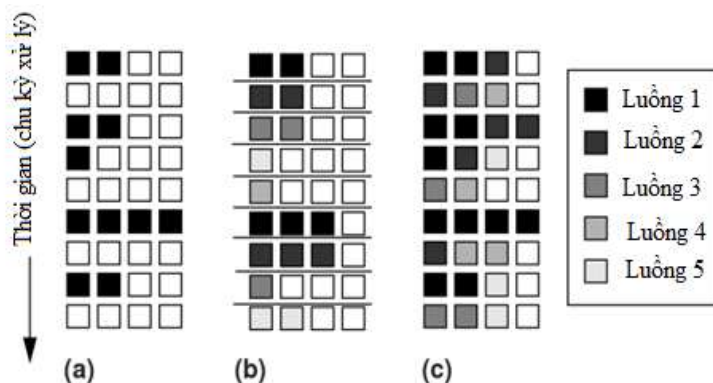
Nghiên cứu hiện tại về công nghệ bộ xử lý và kiến trúc máy tính được thúc đẩy chủ yếu bởi nhu cầu về hiệu suất cao hơn. Trong bối cảnh này, người ta hiểu rõ rằng hiệu suất thu được từ việc chỉ cải thiện hệ thống bộ nhớ bị hạn chế và việc sử dụng tích hợp mức hệ thống (chẳng hạn như hỗ trợ đồ họa/âm thanh trên chip) chỉ có thể dẫn đến những lợi ích hiệu suất cận biên. Lợi ích đáng kể nhất có thể đạt được bằng cách tăng tính song song trong thực thi.

Có hai loại song song trong khối lượng công việc lập trình điển hình: Song song mức lệnh (ILP) và Song song mức luồng (TLP). Các kiến trúc siêu vô hướng (superscalar) hiện đại được thiết kế để nắm bắt ILP trong các chương trình, trong khi các hệ thống đa luồng và đa xử lý được thiết kế để nắm bắt TLP hoặc tính song song giữa các luồng/ các tiến trình. Giải pháp tốt hơn sau đó sẽ là khai thác cả ILP và TLP; TLP từ các chương trình song song đa luồng hoặc từ khối lượng công việc đa chương trình và ILP từ mỗi luồng.

Cả superscalar và đa xử lý (MP) đều không thể nắm bắt toàn bộ ILP và TLP và chúng vốn dĩ không có khả năng thích ứng với các mức động của ILP và TLP. Đây là động lực chính cho một kiến trúc mới của bộ xử lý được gọi là Đa luồng đồng thời (SMT).

Trong phần này, chúng ta sẽ tìm hiểu một số đặc điểm chính của kiến trúc SMT và một số yêu cầu thiết kế có thể tạo điều kiện thuận lợi cho việc triển khai SMT trên một kiến trúc siêu vô hướng thông thường. Các đặc điểm của bộ xử lý SMT là:

1. Kế thừa từ superscalar: cấp phát nhiều lệnh mỗi chu kỳ.
2. Từ đa luồng: duy trì trạng thái phần cứng cho nhiều luồng.



Hình 4.2. Các kiến trúc phân chia các khe cấp phát: (a) siêu vô hướng; (b) siêu vô hướng đa luồng; (c) SMT.

Sự khác biệt giữa siêu vô hướng, đa luồng và đa luồng đồng thời được mô tả trong Hình 1, cho thấy các trình tự thực thi của ba kiến trúc. Mỗi hàng biểu diễn cho các khe cấp phát cho một chu kỳ thực thi đơn lẻ: mỗi ô màu cho thấy bộ xử lý đã tìm thấy một lệnh để thực thi trong khe cấp phát của chu kỳ đó; còn ô trắng biểu thị khe không sử dụng. Các khe không sử dụng này được gọi là sự lãng phí.

Trong Hình 4.2, chúng ta có thể thấy rằng có một lượng lãng phí đáng kể các khe cấp phát trong hệ thống siêu vô hướng và hệ thống đa luồng. Về cơ bản có hai loại lãng phí: lãng phí (toàn bộ chu kỳ không được sử dụng) và lãng phí ngang (trong một chu kỳ, các khe cấp phát không được sử dụng). Bộ xử lý siêu vô hướng xem xét nhiều lệnh từ cùng một tiến trình và có cả lãng phí ngang (do không đủ ILP) và lãng phí dọc (do phụ thuộc dữ liệu và hoạt động có độ trễ dài). Hệ thống SMT giảm thiểu lãng phí theo chiều dọc vì nó có thể xem xét nhiều luồng để tìm nạp trong mỗi chu kỳ và do đó nó có thể chịu được các hoạt động có độ trễ dài trong mỗi luồng. Kết quả là, mỗi chu kỳ, bộ xử lý SMT có thể cấp phát nhiều lệnh từ nhiều luồng.

Hình 4.2a cho thấy một trình tự từ một bộ xử lý siêu vô hướng thông thường. Như trong tất cả các superscalars, nó đang thực thi một chương trình hoặc một luồng, từ đó nó cố gắng tìm nhiều lệnh để cấp phát mỗi chu kỳ. Khi nó không thể, các khe cấp phát sẽ không được sử dụng và gây ra cả lãng phí ngang và dọc.

Hình 4.2b cho thấy một trình tự từ một bộ xử lý đa luồng, chẳng hạn như Tera. Bộ xử lý đa luồng chứa trạng thái phần cứng (bộ đếm chương trình và các thanh ghi) cho một số luồng. Trong bất kỳ chu kỳ nhất định nào, bộ xử lý sẽ thực hiện các lệnh từ một trong các luồng. Trong chu kỳ tiếp theo, nó chuyển sang ngữ cảnh luồng khác và thực hiện các lệnh từ luồng mới. Kết quả cho thấy, ưu điểm chính của bộ xử lý đa luồng là chúng chịu đựng tốt hơn các hoạt động kéo dài, loại bỏ hiệu quả sự lãng phí dọc.

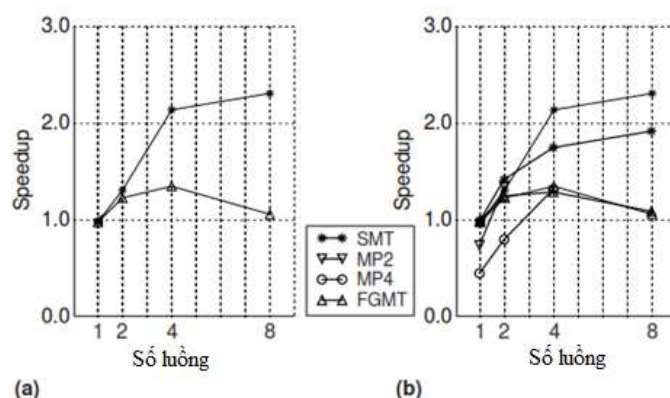
Tuy nhiên, chúng không thể loại bỏ lãng phí ngang. Do đó, khi độ rộng cấp phát lệnh tiếp tục tăng lên, các kiến trúc đa luồng cuối cùng cũng giống như siêu vô hướng: chúng sẽ bị giới hạn bởi tính song song mức lệnh trong luồng đơn.

Hình 4.2c cho thấy cách thức mỗi chu kỳ mà bộ xử lý SMT chọn các lệnh để thực thi từ tất cả các luồng. Nó khai thác song song mức lệnh bằng cách chọn các lệnh từ bất kỳ luồng nào có thể tiềm ẩn sự cố. Sau đó, bộ xử lý sẽ tự động lập lịch tài nguyên máy trong số các lệnh, mang lại cơ hội lớn nhất cho việc sử dụng phần cứng cao nhất. Nếu một luồng có tính song song mức lệnh cao, thì tính song song đó có thể được đáp ứng; nếu nhiều luồng, mỗi luồng có mức song song mức lệnh thấp, chúng có thể được thực thi cùng nhau để bù lại. Bằng cách này, SMT có thể khôi phục các khe cấp phát bị mất cho cả lãng phí ngang và dọc.

Một mô hình SMT có thể được xây dựng từ kiến trúc siêu vô hướng, hiệu suất cao, không theo thứ tự, có lỗi lập lịch động tương tự như của Mips R10000. Trong mỗi chu kỳ, bộ xử lý nạp tám lệnh từ bộ đệm lệnh. Sau khi giải mã lệnh, logic đổi tên thanh ghi ánh xạ các thanh ghi kiến trúc đến thanh ghi đổi tên phần cứng để loại bỏ sự phụ thuộc. Sau đó, các lệnh được đưa đến các hàng đợi.

Khi các toán hạng khả dụng, các lệnh được lấy ra từ các hàng đợi và được chuyển đến các đơn vị chức năng tương ứng. Để hỗ trợ thực thi không theo thứ tự, bộ xử lý theo dõi lệnh và phụ thuộc toán hạng để có thể xác định lệnh nào nó có thể đưa ra và lệnh nào phải đợi các lệnh đã cấp trước đó kết thúc. Sau khi các lệnh hoàn thành thực thi, bộ xử lý sẽ gỡ bỏ chúng theo thứ tự và giải phóng các thanh ghi phần cứng không còn cần thiết nữa.

Một số kết quả mô phỏng đánh giá hệ số cải tiến (speedup) của kiến trúc đa xử lý, đa luồng và đa luồng đồng thời được thể hiện ở Hình 4.3.



Hình 4.3. Speedup với tải làm việc đa chương trình (a) và tải làm việc song song (b)

4.3. ĐA XỬ LÝ ĐỐI XỨNG (SMP) VÀ CỤM (CLUSTER)

4.3.1. Đa xử lý đối xứng

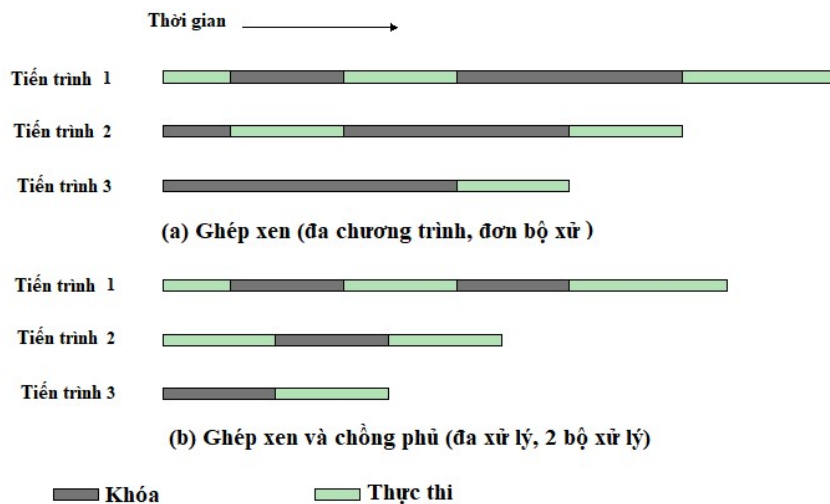
Cho đến gần đây, hầu như tất cả các máy tính cá nhân của một người dùng và hầu hết các máy trạm đều chứa một bộ vi xử lý đa dụng duy nhất. Khi nhu cầu về hiệu suất tăng lên và chi phí của bộ vi xử lý tiếp tục giảm, các nhà cung cấp đã giới thiệu hệ thống đa xử lý đối xứng (SMP). Thuật ngữ SMP đề cập đến kiến trúc phần cứng máy tính và cũng là hành vi của hệ điều hành phản ánh kiến trúc đó. SMP có thể được định nghĩa là một hệ thống máy tính độc lập với các đặc điểm sau:

1. Có hai hoặc nhiều bộ xử lý giống nhau có khả năng tương đương.
2. Các bộ xử lý này chia sẻ cùng một bộ nhớ chính và các thiết bị I/O và được kết nối với nhau bằng một bus hoặc sơ đồ kết nối nội bộ khác, sao cho thời gian truy cập bộ nhớ xấp xỉ giống nhau đối với mỗi bộ xử lý.

3. Tất cả các bộ xử lý chia sẻ quyền truy cập vào các thiết bị I/O, thông qua các kênh giống nhau hoặc thông qua các kênh khác nhau cung cấp đường dẫn đến cùng một thiết bị.
4. Tất cả các bộ xử lý có thể thực hiện các chức năng giống nhau (do đó có thuật ngữ đối xứng).
5. Hệ thống được điều khiển bởi một hệ điều hành tích hợp cung cấp sự tương tác giữa các bộ xử lý và chương trình của chúng ở cấp độ công việc, tác vụ, tệp và phần tử dữ liệu.

Trong SMP, các phần tử dữ liệu riêng lẻ có thể tạo thành mức độ tương tác và có thể có mức độ hợp tác cao giữa các tiến trình. Hệ điều hành của SMP lập lịch cho các tiến trình hoặc luồng trên tất cả các bộ xử lý. SMP có một số ưu điểm hơn so với một tổ chức đơn xử lý như sau:

- Hiệu suất: Nếu công việc được thực hiện bởi một máy tính được tổ chức sao cho một số phần công việc có thể được thực hiện song song, thì hệ thống có nhiều bộ xử lý sẽ mang lại hiệu suất cao hơn hệ thống có một bộ xử lý cùng loại (Hình 4.4).



Hình 4.4. Đa chương trình và đa xử lý

- Tính khả dụng: Trong bộ đa xử lý đối xứng, bởi vì tất cả các bộ xử lý đều có thể thực hiện các chức năng giống nhau, nên sự cố của một bộ xử lý không làm máy dừng hoạt động. Thay vào đó, hệ thống có thể tiếp tục hoạt động với hiệu suất giảm hơn.
- Cải thiện hiệu suất: Người dùng có thể nâng cao hiệu suất của hệ thống bằng cách bổ sung thêm bộ xử lý.
- Tính mở rộng: Các nhà cung cấp có thể cung cấp một loạt các sản phẩm với các đặc điểm về giá cả và hiệu suất khác nhau dựa trên số lượng bộ xử lý được cấu hình trong hệ thống.

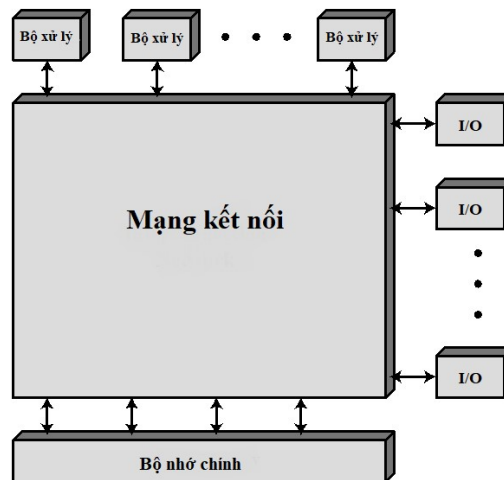
Tổ chức

Hình 4.5 mô tả tổng quát về tổ chức của một hệ thống đa xử lý. Có hai hoặc nhiều bộ xử lý. Mỗi bộ xử lý là độc lập, bao gồm một đơn vị điều khiển, ALU, các thanh ghi, và một hoặc nhiều mức bộ nhớ đệm. Mỗi bộ xử lý có quyền truy cập vào bộ nhớ chính dùng chung và các thiết bị I / O thông qua một số dạng cơ chế kết nối. Các bộ vi xử lý có thể giao tiếp với nhau thông qua bộ nhớ (các thông báo và thông tin trạng thái trong các vùng dữ liệu chung).

Bộ xử lý cũng có thể trao đổi tín hiệu trực tiếp. Bộ nhớ thường được tổ chức để có thể thực hiện nhiều truy cập đồng thời vào các khối bộ nhớ riêng biệt. Trong một số cấu hình, mỗi bộ xử lý cũng có thể có bộ nhớ chính và các kênh I/O riêng, ngoài các tài nguyên được chia sẻ.

Tổ chức phổ biến nhất cho máy tính cá nhân, máy trạm và máy chủ là bus chia sẻ thời gian. Bus chia sẻ thời gian là cơ chế đơn giản nhất để xây dựng một hệ thống đa xử lý (Hình 4.6). Cấu

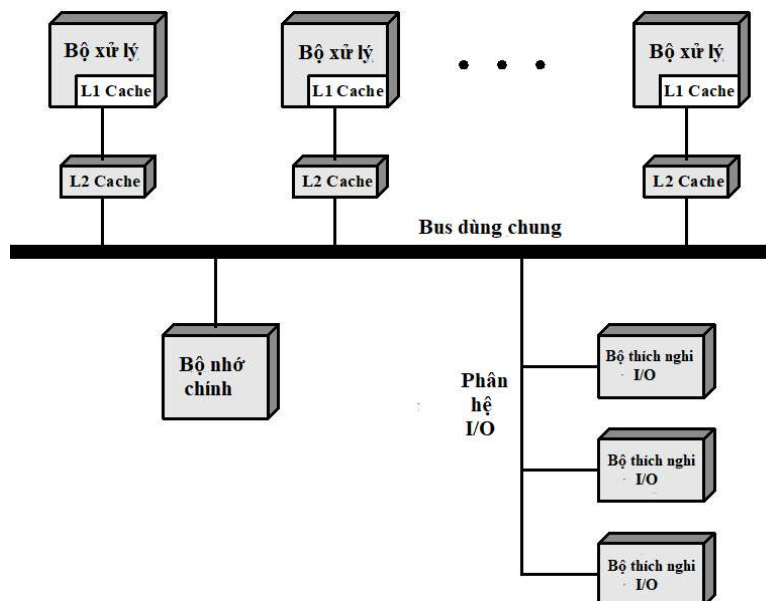
trúc và giao diện về cơ bản giống như đối với hệ thống đơn xử lý sử dụng kết nối bus. Bus bao gồm các đường điều khiển, địa chỉ và dữ liệu.



Hình 4.5. Sơ đồ khối của bộ đa xử lý

Để tạo điều kiện thuận lợi cho việc truyền DMA từ các phân hệ I/O sang bộ xử lý, các tính năng sau được cung cấp:

- Định địa chỉ: Phải có khả năng phân biệt các mô-đun trên bus để xác định nguồn và đích của dữ liệu.
- Phân xử: Bất kỳ mô-đun I/O nào cũng có thể tạm thời hoạt động với vai trò “chính”. Một cơ chế được cung cấp để phân xử các yêu cầu cạnh tranh về điều khiển bus, sử dụng một số loại ưu tiên.
- Chia sẻ thời gian: Khi một mô-đun đang điều khiển bus, các mô-đun khác sẽ bị khóa và nếu cần, phải tạm dừng hoạt động cho đến khi đạt được quyền truy cập vào bus.
- Các tính năng này của bộ xử lý đơn có thể được sử dụng trực tiếp trong tổ chức SMP. Với SMP (hình 4.6), có nhiều bộ xử lý cũng như nhiều bộ xử lý I/O đều cố gắng truy cập vào một hoặc nhiều mô-đun bộ nhớ thông qua bus.



Hình 4.6. Tổ chức bộ đa xử lý đối xứng SMP

Khi đó, tổ chức bus cần đảm bảo:

- Tính đơn giản: Đây là cách tiếp cận đơn giản nhất để tổ chức nhiều bộ xử lý. Giao diện vật lý và logic địa chỉ, phân xử và logic chia sẻ thời gian của mỗi bộ xử lý vẫn giống như trong hệ thống một bộ xử lý.
- Tính linh hoạt: Nhìn chung có thể dễ dàng mở rộng hệ thống bằng cách gắn nhiều bộ xử lý hơn vào bus.
- Độ tin cậy: Bus về cơ bản là một phương tiện thụ động, và sự cố của bất kỳ thiết bị nào gắn kèm với bus sẽ không gây ra hỏng hóc cho toàn bộ hệ thống.

Tuy nhiên, hạn chế chính đối với tổ chức bus là hiệu suất. Tất cả các tham chiếu bộ nhớ đều đi qua bus chung. Do đó, thời gian chu kỳ bus giới hạn tốc độ của hệ thống. Để cải thiện hiệu suất, cần trang bị cho mỗi bộ vi xử lý một bộ nhớ đệm. Điều này sẽ làm giảm đáng kể số lượng truy cập bus. Thông thường, các SMP của máy trạm và PC có hai cấp bộ đệm, với bộ đệm L1 bên trong (cùng một chip với bộ xử lý) và bộ đệm L2 bên trong hoặc bên ngoài. Một số bộ xử lý hiện cũng sử dụng bộ nhớ đệm L3.

4.3.2. Cụm máy tính

Một bước phát triển quan trọng gần đây trong thiết kế hệ thống máy tính là tạo cụm máy tính. Tạo cụm là một giải pháp thay thế cho đa xử lý đối xứng, như là cách tiếp cận để cung cấp hệ thống máy tính hiệu suất cao và tính sẵn sàng cao, và đặc biệt đối với các ứng dụng máy chủ. Cụm máy tính là một nhóm các máy tính được kết nối với nhau và toàn bộ các máy tính này hoạt động cùng nhau như một tài nguyên tính toán thống nhất. Mỗi máy tính trong một cụm thường được gọi là một nút.

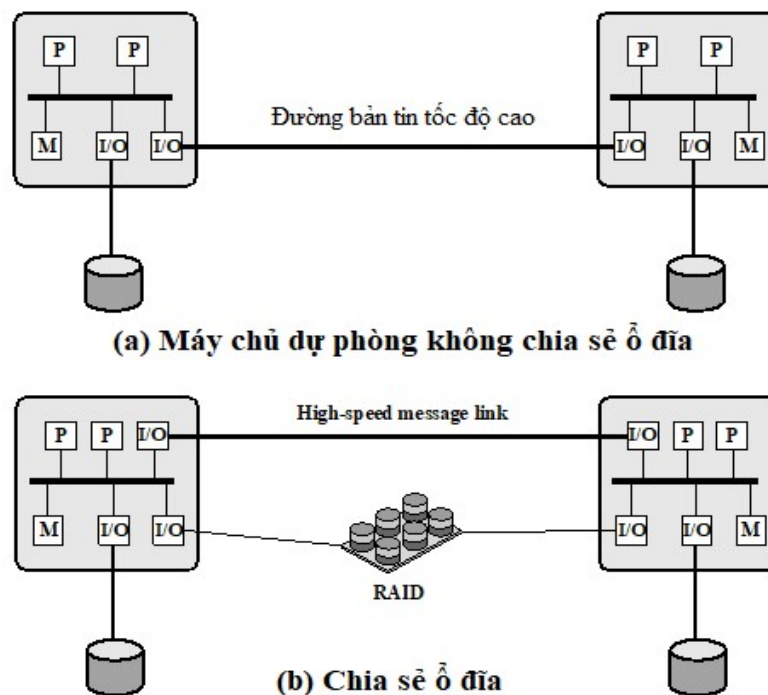
Bốn lợi ích có thể đạt được khi tạo cụm máy tính. Đây cũng có thể được coi là mục tiêu hoặc yêu cầu thiết kế:

- Khả năng mở rộng tuyệt đối: Có thể tạo ra các cụm máy tính lớn vượt xa sức mạnh của những máy tính độc lập. Một cụm có thể có hàng chục, hàng trăm hoặc thậm chí hàng nghìn máy tính, mỗi máy tính trong số đó là một bộ xử lý đa.
- Khả năng mở rộng tăng dần: Một cụm máy tính được cấu hình theo cách có thể thêm các hệ thống mới vào cụm theo từng bước. Do đó, người dùng có thể bắt đầu với một hệ thống vừa phải và mở rộng nó khi nhu cầu phát triển mà không cần phải trải qua một đợt nâng cấp lớn, trong đó một hệ thống nhỏ hiện có sẽ được thay thế bằng một hệ thống lớn hơn.
- Tính sẵn sàng cao: Bởi vì mỗi nút trong một cụm là một máy tính độc lập, nên sự cố của một nút không có nghĩa là mất dịch vụ. Trong nhiều sản phẩm, khả năng chịu lỗi được xử lý tự động bằng phần mềm.
- Giá thành/hiệu suất vượt trội: có thể tạo các cụm máy tính với khả năng tính toán bằng hoặc lớn hơn một máy lớn đơn lẻ, với chi phí thấp hơn nhiều.

Cấu hình cụm máy tính

Các cụm máy tính được phân loại theo một số cách khác nhau. Cách phân loại đơn giản nhất dựa trên việc các máy tính trong một cụm có chia sẻ quyền truy cập vào các đĩa giống nhau hay không. Hình 4.7a cho thấy một cụm máy tính hai nút trong đó kết nối duy nhất là bằng một liên kết tốc độ cao có thể được sử dụng để trao đổi thông điệp nhằm điều phối hoạt động của cụm. Liên kết có thể là một mạng LAN được chia sẻ với các máy tính khác không thuộc cụm, hoặc liên kết có thể là một phương tiện kết nối chuyên dụng. Trong trường hợp sau, một hoặc nhiều máy tính trong cụm sẽ có liên kết đến mạng LAN hoặc WAN để có kết nối giữa cụm máy chủ và các hệ thống máy khách từ xa.

Lưu ý rằng trong hình, mỗi máy tính được mô tả là một bộ xử lý đa năng. Điều này là không cần thiết nhưng sẽ nâng cao cả hiệu suất và tính khả dụng.



Hình 4.7. Các cấu hình cụm máy tính

Phân loại được mô tả trong hình 4.7 là một giải pháp tạo cụm máy tính dùng chung ổ đĩa. Trong trường hợp này, thường vẫn có một liên kết thông báo giữa các nút. Ngoài ra, có một phân hệ ổ đĩa được liên kết trực tiếp với nhiều máy tính trong cụm. Trong hình này, phân hệ ổ đĩa dùng chung là hệ thống RAID. Việc sử dụng RAID hoặc một số công nghệ đĩa dự phòng tương tự là phổ biến trong các cụm sao cho đạt được tính khả dụng cao.

4.4. MÔ HÌNH TỔ CHỨC HỆ THỐNG MÁY TÍNH ĐA LỖI

Máy tính đa lỗi hay Bộ xử lý đa lỗi, là sự kết hợp hai hoặc nhiều đơn vị bộ xử lý (được gọi là lỗi) trên một miếng silicon duy nhất (được gọi là khuôn).

Thông thường, mỗi lỗi bao gồm tất cả các thành phần của một bộ xử lý độc lập, chẳng hạn như thanh ghi, ALU, phần cứng đường ống lệnh và đơn vị điều khiển, cộng với các bộ nhớ đệm lệnh và dữ liệu mức L1. Ngoài nhiều lỗi, các chip đa lỗi hiện đại cũng bao gồm bộ nhớ đệm mức L2 và ngày càng có nhiều bộ nhớ đệm mức L3. Các bộ xử lý đa lỗi được tích hợp cao nhất, được gọi là hệ thống trên chip (SoC), cũng bao gồm bộ nhớ và bộ điều khiển ngoại vi.

Phần này chúng ta xem xét tổ chức đa lỗi cùng một số ví dụ về các sản phẩm đa lỗi, bao gồm máy tính cá nhân và máy trạm (Intel), hệ thống nhúng (ARM) và máy tính lớn (IBM).

4.4.1. Tổ chức đa lỗi

Ở cấp độ mô tả cao nhất, các yếu tố chính trong một tổ chức đa lỗi như sau:

- Số lượng bộ xử lý lỗi trên chip
- Số lượng cấp bộ nhớ đệm
- Cách bộ nhớ cache được chia sẻ giữa các lỗi
- Đa luồng đồng thời (SMT) có được sử dụng hay không
- Các loại lỗi

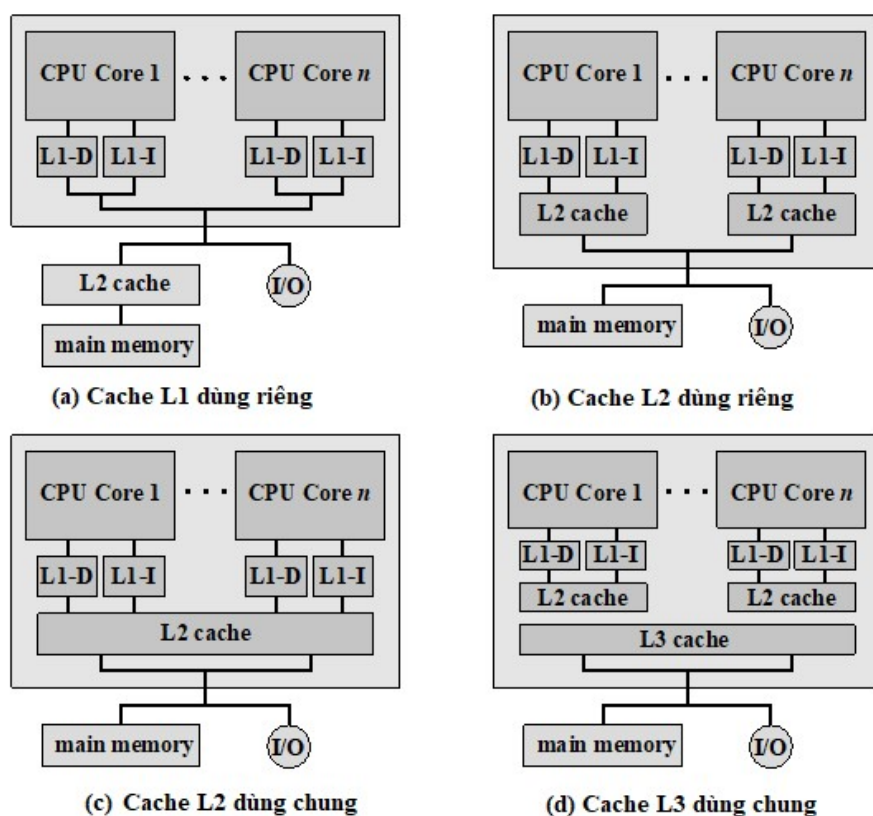
Về các cấp độ của bộ nhớ cache

Hình 4.8 cho thấy bốn tổ chức chung cho các hệ thống đa lõi. Hình 4.8a là một tổ chức được tìm thấy trong một số chip máy tính đa lõi trước đó, và vẫn được thấy trong một số chip nhúng. Trong tổ chức này, bộ nhớ đệm trên chip duy nhất là bộ đệm mức L1, với mỗi lõi có bộ đệm L1 riêng. Các bộ nhớ đệm L1 được chia thành các bộ nhớ đệm lệnh và dữ liệu vì lý do hiệu suất, trong khi bộ nhớ đệm L2 và cấp cao hơn thì hợp nhất. Một ví dụ về tổ chức này là ARM11 MPCore.

Tổ chức của Hình 4.8b cũng là một tổ chức trong đó không có chia sẻ bộ nhớ đệm trên chip và có đủ diện tích có sẵn trên chip để cho phép bộ đệm L2. Một ví dụ về tổ chức này là AMD Opteron. Hình 4.8c cho thấy sự phân bổ tương tự không gian chip cho bộ nhớ, nhưng với việc sử dụng bộ đệm L2 dùng chung. Intel Core Duo có tổ chức này. Cuối cùng, khi dung lượng bộ nhớ đệm có sẵn trên chip tiếp tục tăng lên, các cân nhắc về hiệu suất sẽ dẫn đến việc tách bộ nhớ đệm L3 dùng chung, riêng biệt (Hình 4.8d), với các bộ nhớ đệm L1 và L2 dành riêng cho mỗi bộ xử lý lõi. Intel Core i7 là một ví dụ về tổ chức này.

Việc sử dụng bộ nhớ đệm cấp cao hơn được chia sẻ trên chip có một số lợi thế so với việc phụ thuộc hoàn toàn vào các bộ nhớ đệm chuyên dụng:

1. Sự can thiệp mang tính xây dựng có thể làm giảm tỷ lệ miss tổng thể. Có nghĩa là, nếu một luồng trên một lõi truy cập vào một vị trí bộ nhớ chính, điều này sẽ đưa dòng chứa vị trí được tham chiếu vào bộ nhớ đệm được chia sẻ. Nếu một luồng trên lõi khác ngay sau đó truy cập vào cùng một khối bộ nhớ, các vị trí bộ nhớ sẽ có sẵn trong bộ nhớ cache trên chip được chia sẻ.
2. Một lợi thế liên quan là dữ liệu được chia sẻ bởi nhiều lõi không được sao chép ở mức bộ nhớ cache dùng chung.



Hình 4.8. Các giải pháp tổ chức đa lõi

3. Với các thuật toán thay thế dòng thích hợp, dung lượng bộ nhớ cache dùng chung được phân bổ cho mỗi lõi là động, do đó các luồng có ít cục bộ hơn (tập làm việc lớn hơn) có thể sử dụng nhiều bộ nhớ cache hơn.
4. Giao tiếp giữa các lõi rất dễ thực hiện, thông qua các vị trí bộ nhớ dùng chung.
5. Việc sử dụng bộ đệm cấp cao hơn dùng chung sẽ hạn chế vấn đề liên lạc bộ đệm ở các cấp bộ đệm thấp hơn, điều này có thể cung cấp thêm một số lợi thế về hiệu suất.

Một lợi thế tiềm năng khi chỉ có bộ nhớ đệm L2 chuyên dụng trên chip là mỗi lõi được truy cập nhanh hơn vào bộ đệm L2 riêng của nó. Điều này là thuận lợi cho các luồng có tính cục bộ. Khi cả số lượng bộ nhớ có sẵn và số lượng lõi đều tăng lên, việc sử dụng bộ đệm L3 dùng chung kết hợp với bộ đệm L2 dành riêng cho mỗi lõi trong Hình 4.8d dường như có khả năng cung cấp hiệu suất tốt hơn trường hợp bộ đệm L2 lớn dùng chung hoặc bộ đệm L2 lớn chuyên dụng với bộ nhớ đệm L3 không tích hợp trên chip.

4.4.2. Tổ chức đa lõi không đồng nhất

Khi các nhà thiết kế phải cân bằng nhiều yếu tố thiết kế nhằm tối đa hóa hiệu suất và giảm thiểu tiêu thụ điện năng, một số cách tiếp cận như vậy bao gồm:

1. Tăng phần trăm chip dành cho bộ nhớ đệm.
2. Tăng số lượng cấp bộ nhớ đệm.
3. Thay đổi độ dài (tăng hoặc giảm) và các thành phần chức năng của đường ống lệnh.
4. Sử dụng đa luồng đồng thời.
5. Sử dụng nhiều lõi.

Một trường hợp điển hình cho việc sử dụng nhiều lõi là một con chip có nhiều lõi giống hệt nhau, được gọi là tổ chức đa lõi đồng nhất. Để đạt được kết quả tốt hơn, về mặt hiệu suất và/hoặc điện năng tiêu thụ, một lựa chọn thiết kế khác ngày càng phổ biến là tổ chức đa lõi không đồng nhất, đề cập đến một chip xử lý bao gồm nhiều loại lõi khác nhau. Trong phần này, chúng ta xem xét hai cách tiếp cận đối với tổ chức đa lõi không đồng nhất: Kiến trúc tập lệnh khác nhau và giống nhau.

Các kiến trúc tập lệnh khác nhau

Cách tiếp cận nhận được nhiều sự chú ý nhất là sử dụng các lõi có ISA riêng biệt.

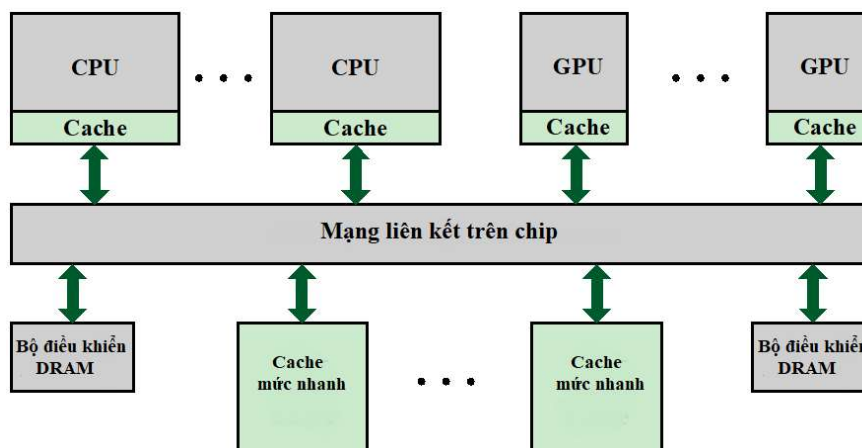
Thông thường, điều này liên quan đến việc ghép các lõi thông thường (CPU) với các lõi chuyên biệt được tối ưu hóa cho một số loại dữ liệu hoặc ứng dụng nhất định. Thông thường, các lõi bổ sung được tối ưu hóa để xử lý dữ liệu vector và ma trận.

CPU/GPU đa lõi

Xu hướng nổi bật nhất về thiết kế đa lõi không đồng nhất là sử dụng cả CPU và đơn vị xử lý đồ họa (GPU) trên cùng một con chip. Tóm lại, GPU được đặc trưng bởi khả năng hỗ trợ hàng nghìn luồng thực thi song song. Do đó, GPU rất phù hợp với các ứng dụng xử lý lượng lớn dữ liệu vector và ma trận. Ban đầu nhằm mục đích cải thiện hiệu suất của các ứng dụng đồ họa, nhờ các mô hình lập trình dễ áp dụng như CUDA (Kiến trúc thiết bị hợp nhất tính toán), các bộ xử lý mới này ngày càng được áp dụng để cải thiện hiệu suất của các ứng dụng khoa học và mục đích chung liên quan đến số lượng các hoạt động lặp lại trên dữ liệu có cấu trúc. Tuy nhiên, kiểu thiết kế này cũng đặt ra nhiều vấn đề về phối hợp và tính đúng đắn.

Hình 4.9 là một tổ chức bộ xử lý đa lõi điển hình. Nhiều CPU và GPU chia sẻ tài nguyên trên chip, chẳng hạn như bộ nhớ đệm cấp cuối (LLC), mạng kết nối và bộ điều khiển bộ nhớ. Điều

quan trọng nhất là cách mà các chính sách quản lý bộ nhớ cache cung cấp việc chia sẻ LLC một cách hiệu quả. Sự khác biệt về bộ nhớ cache và tốc độ truy cập bộ nhớ giữa CPU và GPU tạo ra những thách thức đáng kể đối với việc chia sẻ hiệu quả LLC.

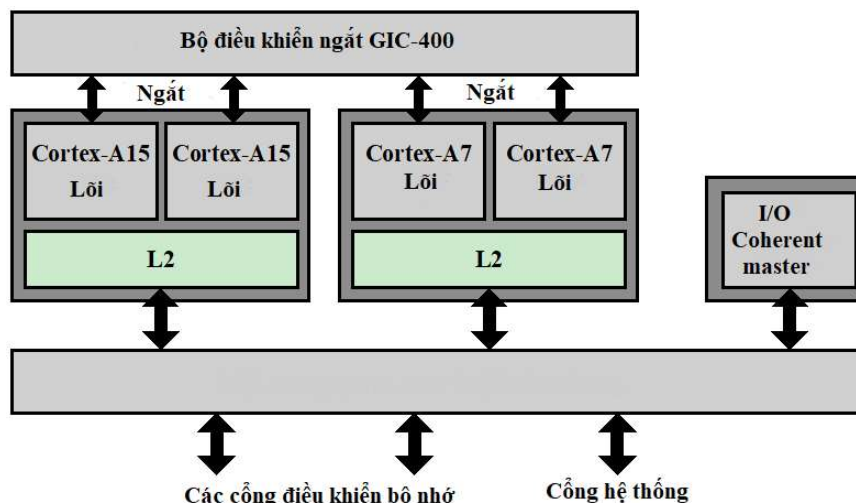


Hình 4.9. Các thành phần chip đa lõi không đồng nhất

Bảng 4.2. Các thông số hoạt động của Bộ xử lý đa lõi không đồng nhất AMD 5100K

(FLOPS = phép toán dấu phẩy động mỗi giây; FLOPS /core = số phép toán dấu phẩy động song song có thể được thực hiện)

	CPU	GPU
Clock frequency (GHz)	3.8	0.8
Cores	4	384
FLOPS/core	8	2
GFLOPS	121.6	614.4



Hình 4.10. Các thành phần chip big.Little

Cho dù đó là các ứng dụng khoa học hay xử lý đồ họa truyền thống, chìa khóa để tận dụng các bộ xử lý GPU được bổ sung là xem xét thời gian cần thiết để chuyển một khối dữ liệu sang GPU, xử lý nó, sau đó trả kết quả về luồng ứng dụng chính. Trong các triển khai trước đó của các chip tích hợp GPU, bộ nhớ vật lý được phân vùng giữa CPU và GPU. Nếu một luồng ứng dụng đang chạy trên CPU yêu cầu xử lý GPU, CPU sẽ sao chép dữ liệu một cách rõ ràng vào bộ nhớ GPU. GPU hoàn thành quá trình tính toán và sau đó sao chép kết quả trở lại bộ nhớ CPU. Các vấn đề về tính nhất quán của bộ đệm giữa các bộ nhớ đệm CPU và GPU không phát sinh do bộ nhớ

được phân vùng. Mặt khác, việc chuyển giao dữ liệu qua lại vật lý cũng làm ảnh hưởng đến hiệu suất.

Kiến trúc tập lệnh tương đương

Một cách tiếp cận khác gần đây đối với tổ chức đa lõi không đồng nhất là việc sử dụng nhiều lõi có ISA tương đương nhưng khác nhau về hiệu suất hoặc hiệu suất năng lượng. Hình 4.10 minh họa kiến trúc big.Little của ARM. Hình vẽ cho thấy một chip xử lý đa lõi chứa hai lõi Cortex-A15 hiệu suất cao và hai lõi Cortex-A7 hiệu suất thấp hơn, tiêu thụ điện năng thấp hơn. Các lõi A7 xử lý các tác vụ ít đòi hỏi tính toán hơn, chẳng hạn như xử lý nền, phát nhạc, gửi văn bản và gọi điện. Các lõi A15 được sử dụng cho các tác vụ cường độ cao, chẳng hạn như video, chơi game và điều hướng.

Câu hỏi ôn tập chương 4

1. Đặc điểm cơ bản của cách tiếp cận siêu vô hướng đối với thiết kế bộ xử lý là gì?
2. Sự khác biệt giữa cách tiếp cận siêu vô hướng và siêu đường ống là gì?
3. Tính song song ở mức lệnh là gì?
4. Định nghĩa ngắn gọn các thuật ngữ sau: Phụ thuộc dữ liệu, phụ thuộc cấu trúc, Xung đột tài nguyên, Phụ thuộc đầu ra, Chống phụ thuộc?
5. Sự khác biệt giữa song song mức lệnh và song song máy là gì?
6. Đổi tên thanh ghi là gì và mục đích của nó là gì?
7. Các yếu tố chính của tổ chức bộ xử lý siêu vô hướng là gì?
8. Liệt kê và định nghĩa ngắn gọn ba kiểu tổ chức hệ thống máy tính.
9. Các đặc điểm chính của SMP là gì?
10. Một số lợi thế tiềm năng của SMP so với bộ xử lý đơn là gì?
11. Một số vấn đề thiết kế hệ điều hành cho SMP là gì?
12. Sự khác biệt giữa sơ đồ kết hợp bộ đệm phần mềm và phần cứng là gì?
13. Ý nghĩa của từng trạng thái trong bốn trạng thái của giao thức MESI là gì?
14. Một số lợi ích chính của việc phân cụm là gì?
15. Sự khác biệt giữa UMA, NUMA và CC-NUMA là gì?

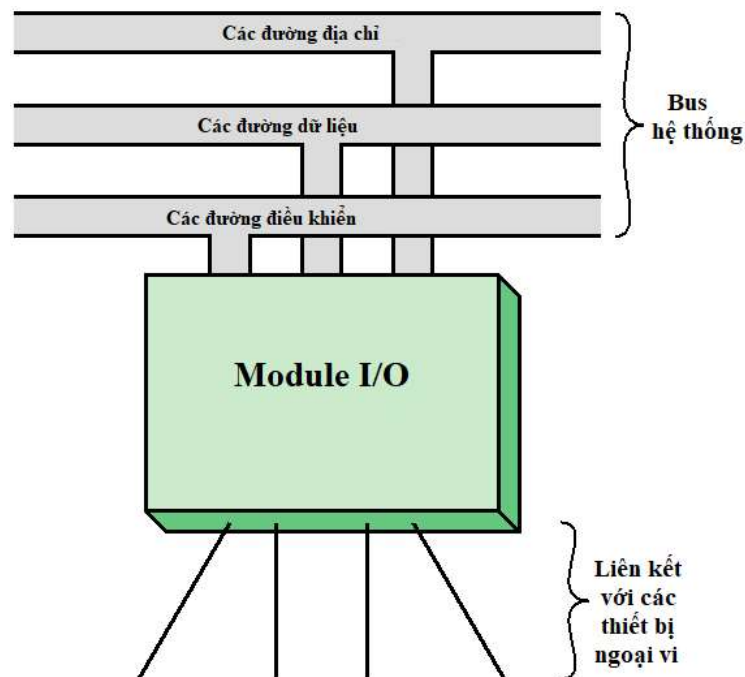
CHƯƠNG 5. HỆ THỐNG VÀO RA

5.1. TỔNG QUAN VỀ HỆ THỐNG VÀO RA

Ngoài bộ vi xử lý và bộ nhớ, thành phần quan trọng thứ ba của một hệ thống máy tính là các module I/O. Mỗi module giao tiếp với bus hệ thống hoặc bộ chuyển mạch trung tâm và điều khiển một hoặc nhiều thiết bị ngoại vi. Một module I/O không đơn giản chỉ kết nối cơ học các thiết bị với bus hệ thống mà còn chứa các logic thực hiện chức năng truyền thông tin giữa thiết bị ngoại vi và bus.

Vậy tại sao ta không kết nối trực tiếp thiết bị ngoại vi bus hệ thống. Có một số lý do như sau:

- Có rất nhiều thiết bị ngoại vi với nhiều phương thức hoạt động khác nhau. Nếu bộ xử lý kết nối trực tiếp với thiết bị ngoại vi nó sẽ cần phải được trang bị các logic để điều khiển các thiết bị đó, điều này không cần thiết và gây lãng phí tài nguyên của bộ xử lý.
- Tốc độ truyền dữ liệu của thiết bị ngoại vi thường chậm hơn nhiều so với bộ nhớ hoặc bộ vi xử lý. Do đó, sẽ là không thực tế nếu sử dụng bus tốc độ cao để giao tiếp trực tiếp với thiết bị ngoại vi.
- Mặt khác, tốc độ truyền dữ liệu của một số thiết bị ngoại vi nhanh hơn bộ nhớ hoặc bộ vi xử lý. Một lần nữa, sự chênh lệch này sẽ dẫn đến sự không hiệu quả trong hiệu năng hệ thống nếu ta không có cơ chế quản lý phù hợp.
- Thiết bị ngoại vi thường sử dụng các định dạng dữ liệu và kích thước từ khác so với máy tính mà chúng được gắn vào.



Hình 5.1. Mô hình cơ bản của các module vào/ra

Do đó, việc sử dụng một module I/O là cần thiết. Module này có hai chức năng chính (Hình 5.1):

- Giao tiếp với bộ xử lý và bộ nhớ thông qua bus hệ thống hoặc một chuyển mạch trung tâm
- Giao tiếp với một hoặc nhiều thiết bị ngoại vi bằng các liên kết dữ liệu phù hợp

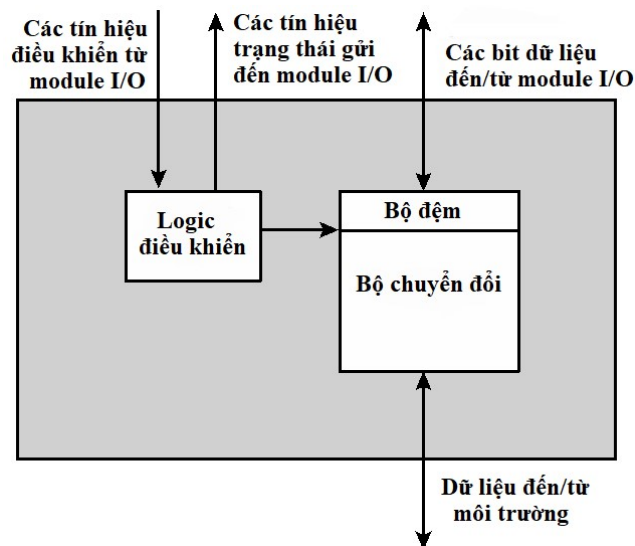
5.1.1. Thiết bị ngoại vi

Các hoạt động I/O được thực hiện thông qua một loạt các thiết bị ngoại vi cung cấp một phương tiện trao đổi dữ liệu giữa môi trường bên ngoài và máy tính. Thiết bị ngoại vi kết nối với máy tính thông qua một liên kết tới module I/O (Hình 5.1). Liên kết này được sử dụng để chuyển tiếp điều khiển, trạng thái và dữ liệu giữa module I/O và thiết bị ngoại vi.

Chúng ta có thể phân loại các thiết bị ngoại vi thành ba loại như sau:

- Con người có thể đọc được: Thích hợp để giao tiếp với người sử dụng máy tính
- Máy có thể đọc được: Thích hợp để giao tiếp với thiết bị
- Truyền thông tin: Thích hợp để giao tiếp với các thiết bị từ xa

Ví dụ về thiết bị con người có thể đọc được đó là màn hình và máy in. Các thiết bị máy có thể đọc được là các hệ thống đĩa và băng từ, các cảm biến và bộ truyền động được sử dụng trong ứng dụng robot. Có một điểm cần chú ý là ta đang xem các hệ thống đĩa và băng từ là các thiết bị I/O trong khi ở chương 3, ta xem chúng như các thiết bị nhớ. Về mặt chức năng, các thiết bị này là một phần của hệ thống phân cấp bộ nhớ, tuy nhiên về mặt cấu trúc, các thiết bị này được điều khiển bởi các module I/O và do đó phải được xét đến trong chương này. Thiết bị truyền thông tin cho phép máy tính trao đổi dữ liệu với thiết bị từ xa.



Hình 5.2. Sơ đồ khối của các thiết bị ngoại vi

Hình 5.2 là sơ đồ khối chung của các thiết bị ngoại vi. Giao diện với module I/O thông qua các tín hiệu điều khiển, dữ liệu và trạng thái. Các tín hiệu điều khiển xác định chức năng mà thiết bị sẽ thực hiện, chẳng hạn như gửi dữ liệu đến module I/O (INPUT hoặc ĐỌC), tiếp nhận dữ liệu từ module I/O (OUTPUT hoặc GHI), báo cáo trạng thái hoặc thực hiện một số chức năng điều khiển đặc biệt đối với thiết bị (ví dụ: đặt đầu đọc/ghi của đĩa từ vào một vị trí nào đó). Dữ liệu ở dạng một tập bit được gửi tới hoặc nhận từ module I/O.

Các tín hiệu trạng thái cho biết trạng thái của thiết bị. Ví dụ là READY/NOT-READY (sẵn sàng hay chưa sẵn sàng) để thông báo thiết bị có sẵn sàng cho việc truyền dữ liệu hay không.

Logic điều khiển tiếp nhận tín hiệu điều khiển từ module I/O và thực hiện việc điều khiển hoạt động thiết bị. Bộ chuyển đổi thực hiện việc chuyển đổi dữ liệu từ dạng tín hiệu điện sang các dạng biểu diễn khác đối với các thiết bị ra và từ các dạng tín hiệu khác nhau thành dữ liệu dạng điện với các thiết bị vào. Bộ chuyển đổi thường đi kèm với một bộ nhớ đệm để lưu trữ dữ liệu tạm

thời trong quá trình trao đổi dữ liệu giữa module I/O và môi trường bên ngoài; một kích thước bộ đệm phổ biến từ 8 đến 16 bit.

5.1.2. Các module I/O

5.1.2.1. Chức năng của module

Các chức năng chính module I/O như sau:

- Điều khiển và định thời
- Giao tiếp với bộ vi xử lý
- Giao tiếp với thiết bị
- Đệm dữ liệu
- Phát hiện lỗi

Trong bất kỳ thời điểm nào, bộ xử lý có thể giao tiếp với một hoặc nhiều thiết bị ngoài theo nhiều cách khác nhau tùy thuộc vào yêu cầu vào/ra dữ liệu của chương trình. Các tài nguyên bên trong, ví dụ như bộ nhớ chính và hệ thống bus sẽ bị chia sẻ với nhiều hoạt động, trong đó gồm cả các hoạt động I/O. Vì vậy, chức năng I/O bao gồm cả các yêu cầu điều khiển và định thời để phối hợp luồng lưu lượng giữa các tài nguyên bên trong và các thiết bị ngoài. Ví dụ quá trình điều khiển truyền dữ liệu từ thiết bị ngoại vi đến bộ xử lý gồm các bước sau:

1. Bộ xử lý yêu cầu module I/O kiểm tra trạng thái của thiết bị.
2. Module I/O trả về trạng thái thiết bị.
3. Nếu thiết bị đang hoạt động và sẵn sàng truyền, bộ xử lý yêu cầu truyền dữ liệu bằng cách gửi lệnh cho module I/O.
4. Module I/O nhận một đơn vị dữ liệu (ví dụ, 8 hoặc 16 bit) từ thiết bị ngoài.
5. Dữ liệu được module I/O chuyển sang cho bộ xử lý.

Nếu hệ thống chỉ sử dụng một bus thì tương tác giữa bộ vi xử lý và module I/O sẽ phải sử dụng các cơ chế phân xử bus. Trong đó, module I/O phải thực hiện việc giao tiếp với bộ vi xử lý và thiết bị ngoại vi. Giao tiếp với bộ xử lý như sau:

- Giải mã lệnh: Module I/O nhận lệnh từ bộ xử lý (thường được gửi dưới dạng các tín hiệu trên bus điều khiển). Ví dụ, một module I/O cho ổ đĩa có thể nhận các lệnh sau: READ SECTOR (đọc một sector), WRITE SECTOR (ghi một sector), SEEK track number (tìm kiếm một track), và SCAN record ID (thăm dò một bản ghi). Hai lệnh sau được gửi kèm với một tham số qua bus dữ liệu.
- Dữ liệu: Dữ liệu được trao đổi giữa bộ xử lý và module I/O qua bus dữ liệu.
- Báo cáo trạng thái: Vì các thiết bị ngoại vi thường rất chậm nên bộ xử lý thường phải biết tình trạng của module I/O để yêu cầu thực hiện các hoạt động tiếp theo. Ví dụ, nếu một module I/O được yêu cầu gửi dữ liệu đến bộ xử lý (READ), có thể module này chưa sẵn sàng để thực hiện yêu cầu vì nó vẫn đang thực hiện yêu cầu I/O trước. Điều này cần được báo cáo với bộ xử lý thông qua một tín hiệu trạng thái. Tín hiệu trạng thái thường sử dụng BUSY và READY. Ngoài ra, cũng có thể có một số tín hiệu báo cáo khác với các điều kiện lỗi khác nhau.
- Nhận dạng địa chỉ: Giống như mỗi từ nhớ có một địa chỉ, các thiết bị I/O cũng vậy. Một module I/O phải nhận ra một địa chỉ duy nhất cho mỗi thiết bị ngoại vi mà nó điều khiển.

Mặt khác, module I/O phải thực hiện việc giao tiếp với các thiết bị. Giao tiếp này gồm có các lệnh, thông tin trạng thái và dữ liệu (Hình 5.2).

Một nhiệm vụ thiết yếu nữa của module I/O là đệm dữ liệu. Chức năng này hết sức cần thiết do tốc độ trao đổi dữ liệu của bộ nhớ chính và bộ xử lý khá cao so với hầu hết các thiết bị ngoại vi. Dữ liệu từ bộ nhớ chính được gửi đến một module I/O với tốc độ nhanh và cần được đệm trong module I/O sau đó gửi đến thiết bị ngoại vi với tốc độ của thiết bị.

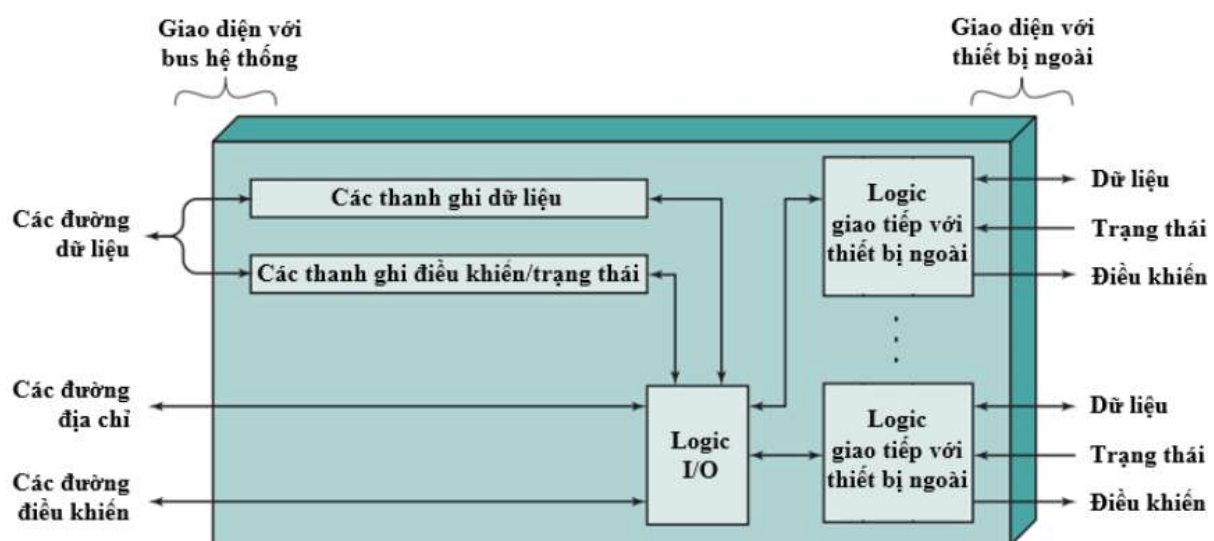
Ngược lại, dữ liệu được đệm lại để không làm mất thời gian của bộ nhớ nhận truyền chậm. Do đó, module I/O phải hoạt động ở cả tốc độ của bộ nhớ và thiết bị ngoại vi. Tương tự trong trường hợp nếu thiết bị I/O có tốc độ cao hơn tốc độ truy cập bộ nhớ, module I/O sẽ phải thực hiện hoạt động đệm dữ liệu cần thiết.

Cuối cùng, một module I/O thường chịu trách nhiệm phát hiện lỗi và sau đó báo lỗi cho bộ vi xử lý. Một số loại bao gồm: hỏng về điện hoặc cơ học (ví dụ: kẹt giấy, một track trong ổ cứng). Một loại lỗi khác có thể do mẫu dữ liệu truyền từ thiết bị đến module I/O vô tình bị thay đổi. Người ta sử dụng một loại mã phát hiện lỗi để phát hiện lỗi truyền tải. Ví dụ như việc truyền mã ký tự IRA, mã này chiếm 7 bit của một byte, vậy bit thứ tám được thiết lập để tổng các bit 1 trong byte là chẵn (kiểm tra chẵn) hoặc lẻ (kiểm tra lẻ). Khi module I/O nhận được byte, nó sẽ kiểm tra bit chẵn lẻ này để xác định xem có lỗi xảy ra hay không.

5.1.2.2. Cấu trúc Module I/O

Các module I/O khác nhau đáng kể về độ phức tạp và số lượng thiết bị bên ngoài mà chúng điều khiển. Tại đây ta sẽ cố gắng chỉ mô tả một cấu trúc chung nhất của các module này. Sơ đồ khối chung của module I/O được minh họa trong hình 5.3. Module này kết nối với phần còn lại của máy tính thông qua một tập hợp các đường tín hiệu (ví dụ: bus hệ thống). Dữ liệu được chuyển đến hoặc từ module được lưu đệm trong các thanh ghi dữ liệu. Các thanh ghi trạng thái cung cấp thông tin về trạng thái hiện tại của thiết bị. Thanh ghi trạng thái cũng có chức năng như một thanh ghi điều khiển: nhận các thông tin điều khiển từ bộ vi xử lý. Logic I/O tương tác với bộ vi xử lý thông qua một tập các đường điều khiển. Bộ vi xử lý sử dụng các đường điều khiển để chuyển các lệnh cho module I/O. Một số đường điều khiển có thể được sử dụng bởi module I/O (ví dụ các tín hiệu phân xử và trạng thái bus).

Module này cũng phải có khả năng nhận dạng và sinh ra các địa chỉ liên kết với các thiết bị mà nó điều khiển. Mỗi module I/O có một (nếu chỉ nối với một TBNV) hoặc một tập địa chỉ (nếu module nối với nhiều TBNV). Cuối cùng, module I/O chứa các logic giao tiếp với từng thiết bị nối vào nó.



Hình 5.3. Sơ đồ khối Module I/O

Với hoạt động của module I/O, bộ xử lý có thể nhìn nhận một loạt các thiết bị theo một cách đơn giản. Module I/O có thể ẩn các thông tin về định thời, định dạng, các vấn đề về điện-cơ của thiết bị ngoại vi, bộ xử lý chỉ cần điều khiển hoạt động thông qua các lệnh đọc và ghi đơn giản hoặc có thể là các lệnh đóng, mở tập tin. Tuy nhiên, cũng có những module I/O hoạt động khá đơn giản và vì vậy để lại phần lớn công việc điều khiển thiết bị cho bộ xử lý.

Những module I/O có khả năng xử lý cao, hỗ trợ nhiều cho bộ xử lý thường được gọi là kênh I/O (I/O channel) hoặc bộ xử lý I/O (I/O processor). Những module I/O đơn giản hơn và cần có sự điều khiển chi tiết hơn từ bộ xử lý thường được gọi là bộ điều khiển I/O (I/O controller) hoặc bộ điều khiển thiết bị (device controller). Bộ điều khiển I/O thường được sử dụng trong các máy vi tính, còn kênh I/O thường được sử dụng trên các dòng máy tính lớn (mainframe).

5.2. CÁC PHƯƠNG PHÁP ĐIỀU KHIỂN VÀO RA

Có ba phương pháp để thực hiện các hoạt động vào/ra (I/O).

- I/O chương trình: Dữ liệu được trao đổi giữa bộ vi xử lý và module I/O. Bộ xử lý thực thi một chương trình cho phép nó trực tiếp điều khiển hoạt động vào/ra, bao gồm cảm nhận tình trạng thiết bị, gửi lệnh đọc hoặc ghi, truyền dữ liệu. Khi bộ vi xử lý ra lệnh cho module I/O, nó phải đợi cho đến khi hoạt động I/O hoàn thành. Nếu bộ xử lý nhanh hơn module I/O thì việc chờ đợi này gây lãng phí thời gian của bộ xử lý.
- I/O điều khiển ngắt: bộ xử lý đưa ra một lệnh I/O (I/O command) sau đó tiếp tục thực hiện các lệnh (instruction) khác trong chương trình, khi nào module I/O hoàn thành công việc của mình nó sẽ gửi yêu cầu ngắt tới bộ xử lý để bộ xử lý tiếp tục điều khiển hoạt động vào/ra.
- Với cả I/O chương trình và I/O điều khiển ngắt, bộ vi xử lý có trách nhiệm lấy dữ liệu từ bộ nhớ chính rồi chuyển cho thiết bị ngoại vi (hoạt động Ghi) hoặc lưu dữ liệu từ thiết bị ngoại vào bộ nhớ chính (hoạt động Đọc). Vì vậy, một cơ chế thay thế được đưa ra là cơ chế DMA (direct memory access – truy cập bộ nhớ trực tiếp) để giảm thiểu sự tham gia của bộ xử lý vào quá trình trao đổi dữ liệu giữa bộ nhớ và thiết bị ngoại vi, tăng hiệu năng của hệ thống. Bảng 7.1 chỉ ra mối quan hệ giữa ba phương pháp này.

Bảng 5.1. Các phương pháp vào/ra

	Không có ngắt	Sử dụng ngắt
Truyền dữ liệu giữa bộ nhớ và thiết bị ngoại vi thông qua bộ xử lý	I/O chương trình	I/O điều khiển ngắt
Truyền dữ liệu trực tiếp giữa bộ nhớ và thiết bị ngoại vi		Truy cập bộ nhớ trực tiếp (DMA)

5.2.1. I/O chương trình

5.2.1.1. Tổng quan

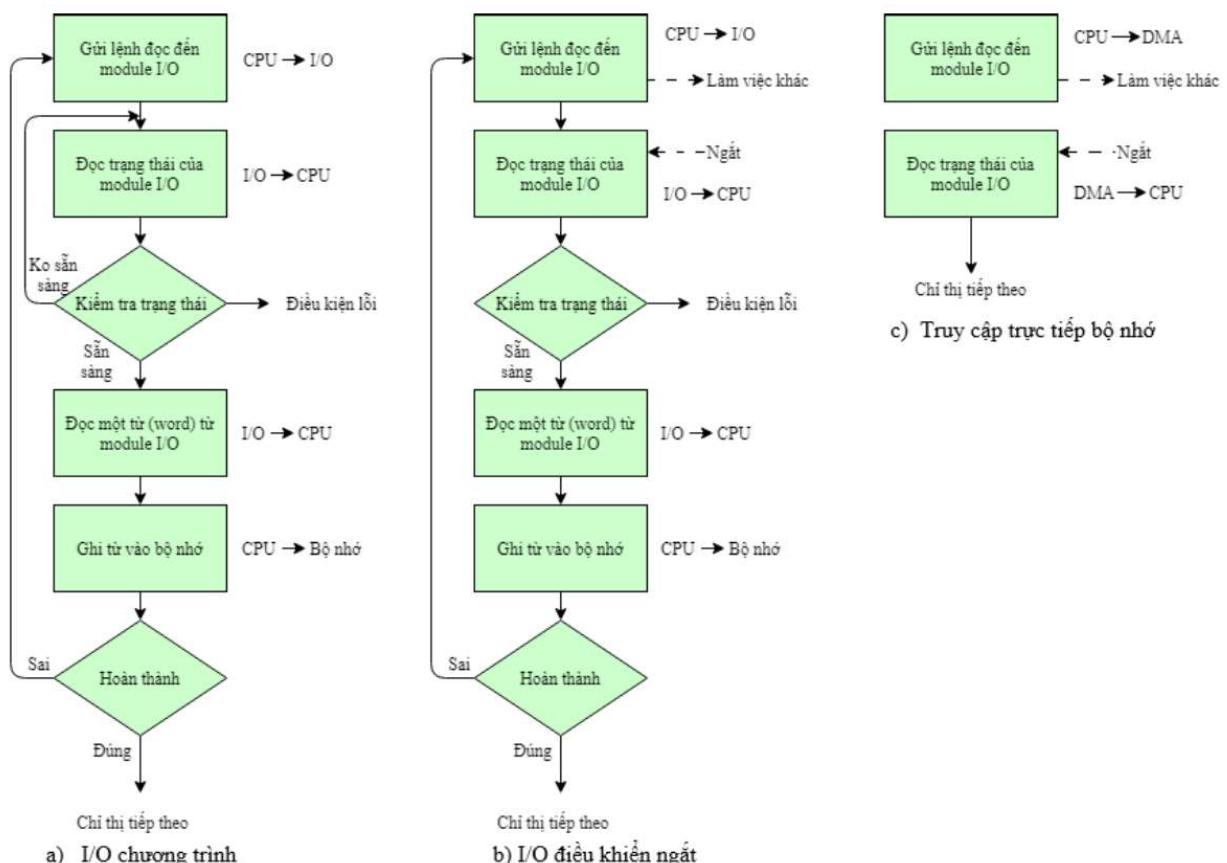
Khi bộ vi xử lý đang thực hiện một chương trình và gặp một chỉ thị (instruction) liên quan đến vào/ra dữ liệu, nó thực hiện chỉ thị đó bằng cách phát ra lệnh (command) cho module I/O thích hợp. Với I/O chương trình, module I/O thực hiện hành động yêu cầu và sau đó thiết lập các bit thích hợp trong thanh ghi trạng thái I/O (Hình 5.3). Module I/O không có tác vụ nào khác để báo cáo về bộ vi xử lý. Do đó, để biết được công việc đã hoàn thành hay chưa, bộ xử lý phải định kỳ để kiểm tra trạng thái của module I/O cho đến khi hoạt động đã hoàn thành.

Để giải thích cho kỹ thuật I/O chương trình, chúng ta xem xét nó từ cách nhìn của các lệnh I/O do bộ vi xử lý gửi đến module I/O và sau đó từ cách nhìn của các chỉ thị được thực hiện bởi bộ vi xử lý.

5.2.1.2. Các lệnh I/O

Để thực hiện chỉ thị liên quan đến vào/ra, bộ xử lý sẽ đưa ra một địa chỉ xác định module I/O và thiết bị bên ngoài cụ thể và đưa ra một lệnh I/O. Có bốn loại lệnh I/O như sau:

- Điều khiển: Được sử dụng để kích hoạt một thiết bị ngoại vi và ra lệnh cho nó phải làm gì. Ví dụ, một ổ đĩa được điều khiển để quay lại hoặc tới trước một bản ghi. Các lệnh điều khiển được thiết kế riêng cho từng loại thiết bị ngoại vi.
- Kiểm tra: Được sử dụng để kiểm tra các điều kiện trạng thái của module I/O và thiết bị ngoại vi. Bộ xử lý cần phải biết liệu các thiết bị ngoại vi có đang bật nguồn và sẵn sàng sử dụng. Nó cũng cần phải biết hoạt động I/O gần nhất đã hoàn thành chưa và có lỗi nào xảy ra không.
- Đọc: Yêu cầu module I/O lấy dữ liệu từ thiết bị ngoại vi và đặt nó vào bộ đệm (thanh ghi dữ liệu trong Hình 5.3). Bộ xử lý sau đó có thể lấy dữ liệu bằng cách yêu cầu module I/O đặt dữ liệu lên bus dữ liệu.
- Ghi: Yêu cầu module I/O chuyển dữ liệu (1 byte hoặc 1 từ) từ bus dữ liệu đến thiết bị ngoại vi.



Hình 5.4. Ba kỹ thuật Đọc một khối dữ liệu vào

Hình 5.4a minh họa một ví dụ về việc sử dụng I/O chương trình để đọc một khối dữ liệu từ thiết bị ngoại vi (ví dụ: một bản ghi từ băng từ) vào bộ nhớ. Dữ liệu được đọc mỗi lần một từ (16 bit). Đối với mỗi từ được đọc, bộ xử lý phải kiểm tra trạng thái cho đến khi nó xác định được từ đó đã được đọc vào thanh ghi dữ liệu của module I/O. Từ sơ đồ này ta thấy rõ nhược điểm chính

của kỹ thuật này là một quá trình lặp đi lặp lại việc kiểm tra trạng thái làm cho bộ xử lý thêm bận rộn.

5.2.1.3. Chỉ thị vào/ra

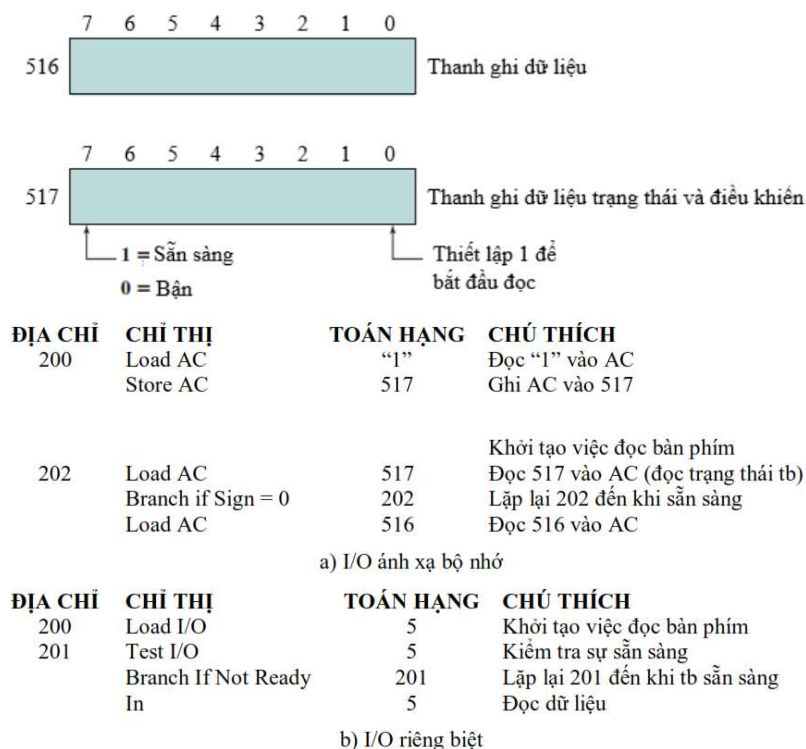
Với I/O chương trình, có sự tương ứng chặt chẽ giữa các chỉ thị (lệnh) liên quan đến vào/ra mà bộ vi xử lý truy xuất từ bộ nhớ và các lệnh I/O mà bộ xử lý gửi đến module I/O để thực hiện chỉ thị đó. Do đó, các chỉ thị cần được ánh xạ thành các lệnh I/O và chúng thường có quan hệ một-một đơn giản. Dạng của chỉ thị phụ thuộc vào cách các thiết bị ngoại được định địa chỉ.

Thông thường, có nhiều thiết bị I/O được kết nối với hệ thống thông qua các module I/O. Mỗi thiết bị có một số nhận dạng hoặc địa chỉ duy nhất. Khi bộ xử lý đưa lệnh I/O đến thiết bị thì trong lệnh phải chứa thông tin địa chỉ của thiết bị. Sau đó, các module I/O phải dịch các dòng địa chỉ để xác định xem liệu lệnh này có phải gửi cho nó.

Khi bộ xử lý, bộ nhớ và module I/O chia sẻ một bus chung, hai chế độ định địa chỉ có thể được thực hiện như sau: I/O ánh xạ bộ nhớ (memory-mapped I/O) và I/O riêng biệt (isolated I/O). I/O ánh xạ bộ nhớ sử dụng một không gian địa chỉ chung cho bộ nhớ và các thiết bị ngoại vi. Bộ xử lý coi thanh ghi trạng thái và dữ liệu của các module I/O giống như vị trí bộ nhớ và sử dụng cùng các lệnh máy để truy cập cả bộ nhớ và các thiết bị ngoại vi.

Ví dụ, với 10 đường địa chỉ ta có một không gian địa chỉ gồm $2^{10} = 1024$ địa chỉ vị trí bộ nhớ.

Với kỹ thuật I/O riêng biệt, bus được trang bị thêm một đường command line, đường này sẽ cho biết địa chỉ trên bus là địa chỉ của vị trí bộ nhớ hay thiết bị I/O. Như vậy, ta có thể sử dụng toàn bộ không gian địa chỉ cho cả bộ nhớ và thiết bị ngoại vi. Như vậy, với 10 đường địa chỉ, hệ thống có thể hỗ trợ 1024 vị trí bộ nhớ và 1024 địa chỉ I/O.



Hình 5.5. So sánh I/O ánh xạ bộ nhớ và I/O riêng biệt

Hình 5.5 so sánh hai kỹ thuật I/O chương trình. Với Hình 5.5a, bàn phím được kết nối với hệ thống theo kỹ thuật I/O ánh xạ bộ nhớ. Giả sử hệ thống có 10-bit địa chỉ, trong đó 512 giá trị

dành cho bộ nhớ (từ 0-511) và 512 dành cho I/O (từ 512-1023). Hai địa chỉ được dành riêng cho việc giao tiếp với bàn phím. Địa chỉ 516 được gán cho thanh ghi dữ liệu và 517 được gán cho thanh ghi trạng thái. Thanh ghi trạng thái hoạt động như một thanh ghi điều khiển để nhận các lệnh của bộ vi xử lý. Chương trình trong hình có mục đích đọc 1 byte dữ liệu từ bàn phím vào bộ thanh ghi AC trong bộ xử lý.

Với I/O riêng biệt (Hình 5.5b), các cổng vào/ra chỉ có thể truy cập bằng các lệnh I/O đặc biệt. Các lệnh này sẽ kích hoạt các đường command line trên bus.

Hầu hết các bộ xử lý đều có một tập khá lớn các chỉ thị khác nhau để tham chiếu bộ nhớ. Nếu sử dụng I/O riêng biệt, số lượng chỉ thị vào/ra sẽ khá ít. Do đó, ưu điểm của I/O ánh xạ bộ nhớ là số lượng các chỉ thị tham chiếu bộ nhớ cũng có thể được sử dụng trong tham chiếu thiết bị ngoại vi, vì vậy cho phép việc lập trình hiệu quả hơn. Tuy nhiên, nhược điểm của nó là không gian địa chỉ phải chia sẻ với thiết bị ngoại vi. Cả hai phương pháp I/O này đều được sử dụng phổ biến.

5.2.2. I/O điều khiển ngắt

Một vấn đề của I/O chương trình là bộ xử lý phải đợi một thời gian dài cho đến khi module I/O sẵn sàng tiếp nhận hoặc truyền dữ liệu. Trong khoảng thời gian đó, bộ xử lý phải liên tục kiểm tra trạng thái của module I/O. Kết quả là hiệu năng của toàn bộ hệ thống suy giảm nghiêm trọng.

Để giải quyết vấn đề trên, người ta đưa ra một giải pháp như sau: bộ vi xử lý gửi lệnh I/O đến module và sau đó thực hiện các hoạt động khác. Khi nào module I/O đã sẵn sàng để trao đổi dữ liệu nó sẽ ngắt bộ xử lý để yêu cầu phục vụ. Bộ xử lý thực thi việc truyền dữ liệu rồi quay trở lại công việc của nó.

Chúng ta sẽ xem xét cách hoạt động này từ hai phía: bộ xử lý và module I/O.

Từ phía module I/O, việc đọc dữ liệu vào như sau: module I/O nhận lệnh READ từ bộ xử lý. Sau đó, nó tiến hành đọc dữ liệu từ một thiết bị ngoại vi được yêu cầu. Khi dữ liệu đã được đọc vào thanh ghi dữ liệu của module, module gửi một báo hiệu ngắt tới bộ xử lý qua một đường điều khiển. Module chờ đợi cho đến khi bộ xử lý gửi yêu cầu dữ liệu. Khi có yêu cầu, module đặt dữ liệu vào bus dữ liệu và được giải phóng, sẵn sàng cho một hoạt động vào/ra khác.

Từ phía bộ xử lý, hoạt động Đọc dữ liệu vào như sau: bộ xử lý ra lệnh READ, sau đó nó sẽ thực hiện một công việc khác (ví dụ: bộ xử lý có thể đang chạy trên nhiều chương trình cùng một lúc). Vào cuối mỗi chu kỳ lệnh, bộ xử lý sẽ kiểm tra các ngắt (Hình 1.25).

Khi có ngắt gửi từ module I/O, bộ xử lý lưu lại ngữ cảnh (ví dụ: thanh ghi PC và các thanh ghi khác của bộ xử lý) của chương trình hiện tại và thực hiện việc xử lý ngắt. Khi xử lý ngắt, bộ xử lý đọc dữ liệu từ module I/O và lưu trữ nó vào bộ nhớ. Sau đó nó khôi phục lại ngữ cảnh trước và tiếp tục thực thi công việc.

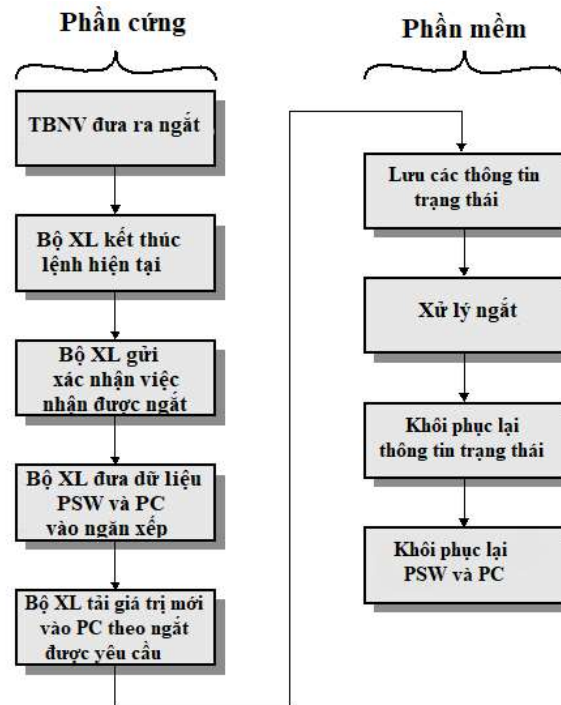
Hình 5.4b minh họa việc sử dụng I/O điều khiển ngắt để đọc trong một khối dữ liệu. So với Hình 5.4a, rõ ràng việc sử dụng ngắt hiệu quả hơn vì nó giúp loại bỏ khoảng thời gian chờ không cần thiết của bộ xử lý. Tuy nhiên, việc truyền dữ liệu giữa bộ nhớ và module I/O vẫn phải có sự tham gia của bộ xử lý.

5.2.2.1. Xử lý ngắt

Ta hãy xem xét chi tiết hơn vai trò của bộ xử lý trong I/O điều khiển ngắt. Sự xuất hiện của một ngắt gây ra một số sự kiện của cả phần cứng và phần mềm (Hình 5.6). Khi một thiết bị vào/ra thực hiện một hoạt động vào/ra, một chuỗi các hoạt động phần cứng sau đây sẽ xảy ra.

1. Thiết bị phát tín hiệu ngắt cho bộ xử lý.

2. Bộ xử lý hoàn thành lệnh hiện tại trước khi trả lời ngắt (Hình 1.25).
3. Bộ xử lý kiểm tra xem có ngắt hay không, nếu có một ngắt, và gửi một tín hiệu báo đã nhận (tín hiệu ACK) đến thiết bị đã gửi ngắt. Khi nhận được ACK, thiết bị loại bỏ tín hiệu ngắt.
4. Bộ xử lý cần phải chuyển điều khiển sang chế độ ngắt. Đầu tiên, nó cần phải lưu lại các thông tin của chương trình hiện tại để có thể khôi phục lại công việc sau khi hoàn thành xong việc xử lý ngắt. Các thông tin tối thiểu bắt buộc là (a) trạng thái của bộ xử lý được lưu trữ trong một thanh ghi PSW (thanh ghi trạng thái chương trình), và (b) địa chỉ lệnh tiếp theo sẽ được thực hiện (nội dung thanh ghi PC). Chúng được đẩy lên vùng bộ nhớ ngăn xếp của hệ thống.



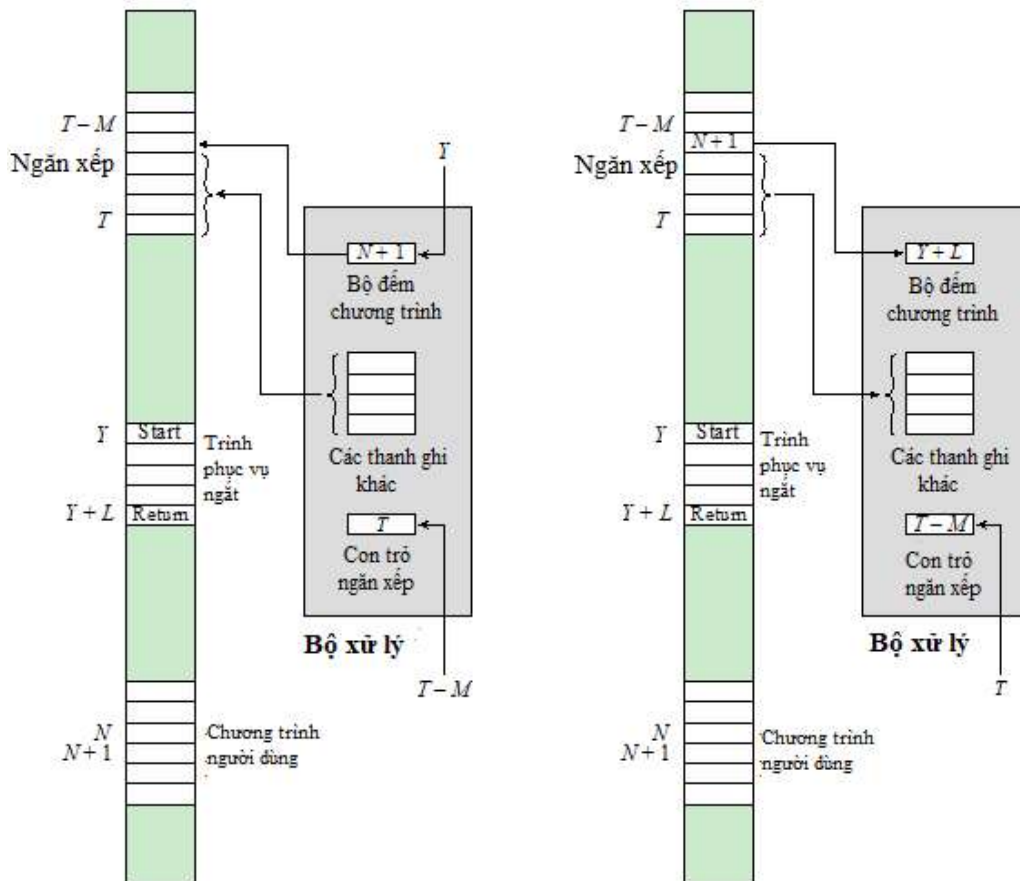
Hình 5.6. Quá trình xử lý ngắt đơn giản

5. Sau đó, bộ xử lý sẽ tải vị trí đầu tiên của chương trình xử lý ngắt vào thanh ghi PC. Tùy thuộc vào kiến trúc hệ thống và thiết kế hệ điều hành, trình xử lý ngắt có thể là một chương trình; một chương trình cho mỗi loại ngắt hoặc một chương trình cho mỗi thiết bị và mỗi loại ngắt. Nếu có nhiều hơn một trình xử lý ngắt, bộ xử lý phải xác định xem ngắt được gửi đến từ đâu. Thông tin đó có thể đã được chứa trong tín hiệu ngắt gửi đến hoặc bộ xử lý phải gửi đến thiết bị gửi ngắt để yêu cầu phản hồi các thông tin cần thiết. Sau khi thanh ghi PC đã được nạp, bộ xử lý bắt đầu thực thi các lệnh trong chương trình xử lý ngắt.

Việc thực hiện chương trình này dẫn đến các hoạt động sau:

6. Tại thời điểm này, nội dung các thanh ghi PC và PSW của chương trình bị ngắt đã được lưu trên vùng nhớ ngăn xếp của hệ thống. Tuy nhiên, vẫn còn một số thông tin khác liên quan đến trạng thái chương trình cũng cần phải được lưu lại, đặc biệt là nội dung các thanh ghi trong bộ xử lý vì các thanh ghi này có thể được sử dụng bởi trình xử lý ngắt. Do đó, thông thường, trình xử lý ngắt sẽ bắt đầu bằng việc lưu nội dung của tất cả các thanh ghi vào ngăn xếp. Một ví dụ đơn giản được minh họa trong Hình 5.7a. Trong trường hợp này, một chương trình bị ngắt tại vị trí N . Nội dung của tất cả các thanh ghi cộng với địa chỉ của lệnh tiếp theo ($N+1$) được đẩy lên ngăn xếp. Thanh ghi SP (con

- trở ngăn xếp) trở đến đỉnh mới của ngăn xếp, và thanh ghi PC trở đến lệnh đầu tiên của trình phục vụ ngắt.
- Lúc này, ngắt được xử lý bao gồm việc kiểm tra thông tin trạng thái liên quan đến hoạt động I/O hoặc các sự kiện khác gây ra ngắt. Nó cũng có thể thực hiện việc gửi thêm lệnh tiếp theo của một hoạt động I/O đang được thực hiện hoặc gửi tín hiệu ACK đến thiết bị ngoại vi.
 - Khi quá trình xử lý ngắt hoàn tất, giá trị các thanh ghi đã lưu trên ngăn xếp sẽ được lấy ra và khôi phục lại vào các thanh ghi (ví dụ Hình 5.7b).
 - Hoạt động cuối cùng là khôi phục giá trị PSW và PC từ ngăn xếp. Nhờ đó, chương trình bị ngắt sẽ được khôi phục lại tiếp tục được thực thi.



Hình 5.7. Sự thay đổi bộ nhớ và thanh ghi khi có ngắt

5.2.2.2. Vấn đề thiết kế

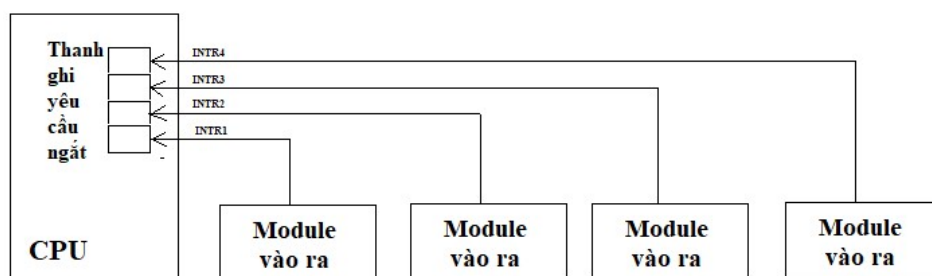
Thực hiện ngắt I/O phát sinh hai vấn đề như sau. Thứ nhất, bởi vì trong hệ thống thường có nhiều module I/O, vậy bộ xử lý xác định thiết bị gửi tín hiệu ngắt như thế nào?

Thứ hai, nếu nhiều yêu cầu ngắt được cùng gửi đến, bộ xử lý sẽ quyết định phục vụ ngắt nào trước ngắt nào sau?

Với vấn đề đầu tiên, có bốn kỹ thuật thường được sử dụng để xác định thiết bị gửi ngắt như sau:

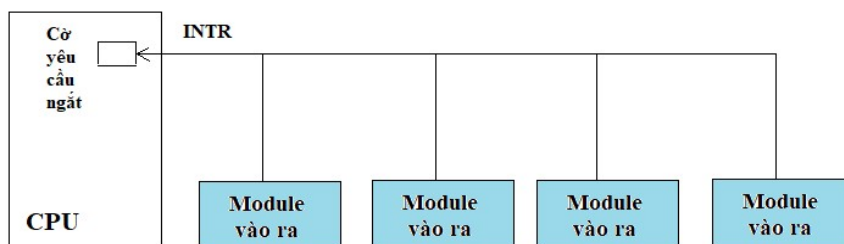
- Sử dụng nhiều đường ngắt
- Thăm dò phần mềm
- Chuỗi Daisy (thăm dò phần cứng, vector)
- Phân xử bus (vector)

Phương pháp đơn giản nhất để giải quyết vấn đề này là ta có thể **sử dụng nhiều đường truyền yêu cầu ngắt** giữa bộ xử lý và các module I/O. Tuy nhiên, việc sử dụng các đường bus và các chân của bộ xử lý để làm các đường ngắt là không thực tế. Do đó, ngay cả khi sử dụng kỹ thuật này, có thể mỗi đường ngắt vẫn sẽ nối với nhiều module I/O và vẫn phải sử dụng một trong ba kỹ thuật còn lại cho mỗi đường.



Hình 5.8. Kỹ thuật nhiều đường ngắt

Với **kỹ thuật thăm dò phần mềm**: khi bộ xử lý phát hiện ra một ngắt, nó rẽ nhánh sang một trình phục vụ ngắt chung có nhiệm vụ thăm dò từng module I/O để xác định module nào phát ra ngắt. Tín hiệu thăm dò có thể dưới dạng một đường lệnh riêng (ví dụ: TEST I/O). Bộ xử lý phát ra tín hiệu TEST I/O và đặt địa chỉ của từng module vào các đường địa chỉ. Module I/O gửi yêu cầu ngắt sẽ trả lời khi nhận được thăm dò. Một cách thăm dò khác là bộ xử lý sẽ đọc các thanh ghi trạng thái của từng module để xác định module nào gửi ngắt. Sau khi xác định được module ngắt, bộ xử lý sẽ chuyển điều khiển sang trình phục vụ ngắt dành riêng cho thiết bị đó. Tuy nhiên, nhược điểm của phương pháp này là tốn thời gian.



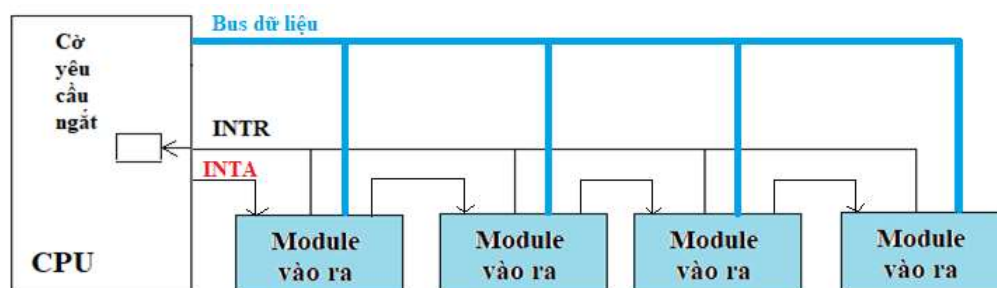
Hình 5.9. Kỹ thuật thăm dò phần mềm

Kỹ thuật chuỗi Daisy thực hiện thăm dò bằng phần cứng. Kỹ thuật này sử dụng một đường ngắt chung (INTR) cho tất cả các module. Khi nhận được yêu cầu ngắt, bộ xử lý gửi một tín hiệu ACK thừa nhận ngắt, tín hiệu này được truyền trên một đường INTA đi qua tất cả các module (nối chuỗi) (Hình 7.9) cho đến khi gặp được module gửi ngắt, module này sẽ gửi lại một tín hiệu trả lời bằng cách đặt một từ lên bus dữ liệu, từ này là có thể là địa chỉ hoặc thông tin nhận diện của module đó, được gọi là vector. Khi đó, bộ xử lý sẽ sử dụng vector để trở tới trình phục vụ ngắt của module đó. Với kỹ thuật này, bộ xử lý không cần phải rẽ nhánh đến trình phục vụ ngắt chung như trường hợp trên.

Một kỹ thuật khác cũng sử dụng vector ngắt là kỹ thuật **phân xử bus**. Một module I/O muốn gửi yêu cầu ngắt thì cần phải có quyền chiếm bus trước. Vì vậy, tại một thời điểm chỉ có một module có thể sử dụng bus. Khi bộ xử lý phát hiện ra ngắt, nó trả lời bằng đường ACK. Khi nhận được ACK, module yêu cầu đặt các vector của nó vào các đường dữ liệu và bộ xử lý cũng thực hiện việc xử lý ngắt tương tự như với kỹ thuật chuỗi Daisy.

Với các kỹ thuật trên, ta cũng có thể giải quyết luôn vấn đề xác định thứ tự ưu tiên trong trường hợp có nhiều ngắt gửi đến bộ xử lý cùng một thời điểm. Với kỹ thuật nhiều đường ngắt, mỗi đường ngắt sẽ được gán một độ ưu tiên nhất định, khi đó, bộ xử lý chỉ cần chọn đường ngắt

có độ ưu tiên cao nhất. Với thăm dò phần mềm, bộ xử lý sẽ thực hiện việc thăm dò các module lần lượt theo thứ tự ưu tiên nó. Tương tự, thứ tự các module được nối chuỗi daisy sẽ được xác định theo độ ưu tiên của chúng. Cuối cùng, trong kỹ thuật phân xử bus, module có độ ưu tiên cao hơn sẽ được cho phép chiếm bus trước, vì vậy cũng đã giải quyết được vấn đề này.



Hình 5.10. Kỹ thuật chuỗi Daisy

5.2.3. Cơ chế truy cập bộ nhớ trực tiếp

5.2.3.1. Nhược điểm của các phương pháp I/O chương trình và I/O điều khiển ngắt

I/O điều khiển ngắt mặc dù đã hiệu quả hơn rất nhiều so với I/O chương trình, tuy nhiên nó vẫn cần có sự tham gia của bộ xử lý vào việc trao đổi dữ liệu giữa bộ nhớ và các module I/O, bất cứ dữ liệu nào cũng phải đi qua bộ xử lý. Do đó các phương pháp này có hai nhược điểm như sau:

1. Tốc độ trao đổi dữ liệu vào/ra bị hạn chế bởi tốc độ bộ xử lý thực hiện các thao tác kiểm tra và phục vụ thiết bị.
2. Bộ xử lý phải quản lý việc truyền I/O; với mỗi thao tác vào/ra dữ liệu, bộ xử lý phải thực thi một số chỉ thị (ví dụ, Hình 5.5).

Có sự cân bằng giữa hai nhược điểm này. Khi xét đến việc trao đổi khối dữ liệu.

Nếu sử dụng I/O chương trình, bộ xử lý được dành hoàn toàn cho việc vào/ra dữ liệu nên việc trao đổi này diễn ra với tốc độ khá cao, tuy nhiên bộ xử lý lại không làm được việc gì khác. Với I/O điều khiển ngắt, bộ xử lý được giải phóng nhưng thời gian thực hiện một hoạt động truyền dữ liệu lại nhiều hơn. Cả hai phương pháp đều có ảnh hưởng nhất định đến hoạt động của bộ xử lý và tốc độ truyền I/O.

Khi cần trao đổi một khối lượng lớn dữ liệu, người ta đưa ra một kỹ thuật hiệu quả hơn, đó là: truy cập bộ nhớ trực tiếp (DMA).

5.2.3.2. Chức năng DMA

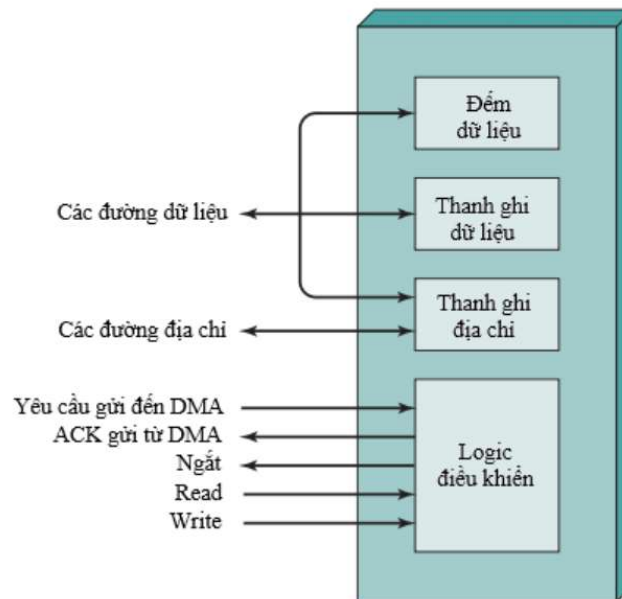
DMA thường là một module bổ sung trên hệ thống bus. Module DMA (Hình 5.11) có khả năng bắt chước bộ xử lý, tức là tiếp nhận việc điều khiển hệ thống từ bộ xử lý. Nó cần phải thực hiện điều này để trao đổi dữ liệu với bộ nhớ qua bus hệ thống. Với mục đích này, module DMA chỉ sử dụng bus khi bộ xử lý không cần đến nó hoặc DMA phải buộc bộ xử lý phải tạm ngừng hoạt động (gọi là trộm chu kỳ). Kỹ thuật trộm chu kỳ thường phổ biến hơn.

Khi bộ xử lý muốn đọc hoặc ghi một khối (khối) dữ liệu, nó sẽ đưa ra một lệnh cho module DMA bằng cách gửi tới module DMA các thông tin sau:

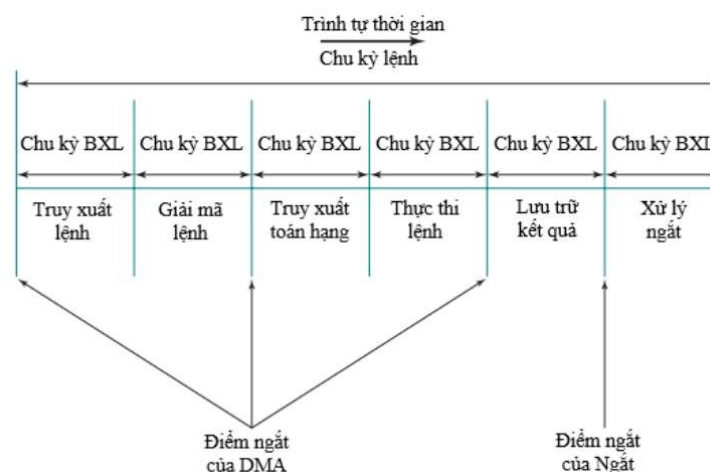
- Yêu cầu Đọc hoặc Ghi bằng cách sử dụng các đường điều khiển READ hoặc WRITE bộ xử lý và module DMA.
- Địa chỉ của thiết bị I/O liên quan, gửi qua các đường dữ liệu

- Địa chỉ vị trí đầu tiên trong bộ nhớ để đọc hoặc ghi lên, truyền qua các đường dữ liệu và được lưu trữ lại bởi module DMA trong Thanh ghi địa chỉ.
- Số lượng từ cần đọc hoặc ghi, truyền qua các đường dữ liệu và được lưu trữ trong thanh ghi Đếm dữ liệu.

Sau đó, bộ xử lý tiếp tục thực hiện các công việc khác. Hoạt động vào/ra được ủy quyền hoàn toàn cho module DMA. Module DMA chuyển toàn bộ khối dữ liệu giữa bộ nhớ và thiết bị ngoại vi mà không cần sự tham gia của bộ xử lý. Khi quá trình trao đổi hoàn tất, module DMA gửi một tín hiệu ngắt tới bộ xử lý. Do đó, bộ xử lý chỉ tham gia vào thời điểm bắt đầu và kết thúc quá trình trao đổi (Hình 5.4c).



Hình 5.11. Sơ đồ khối DMA

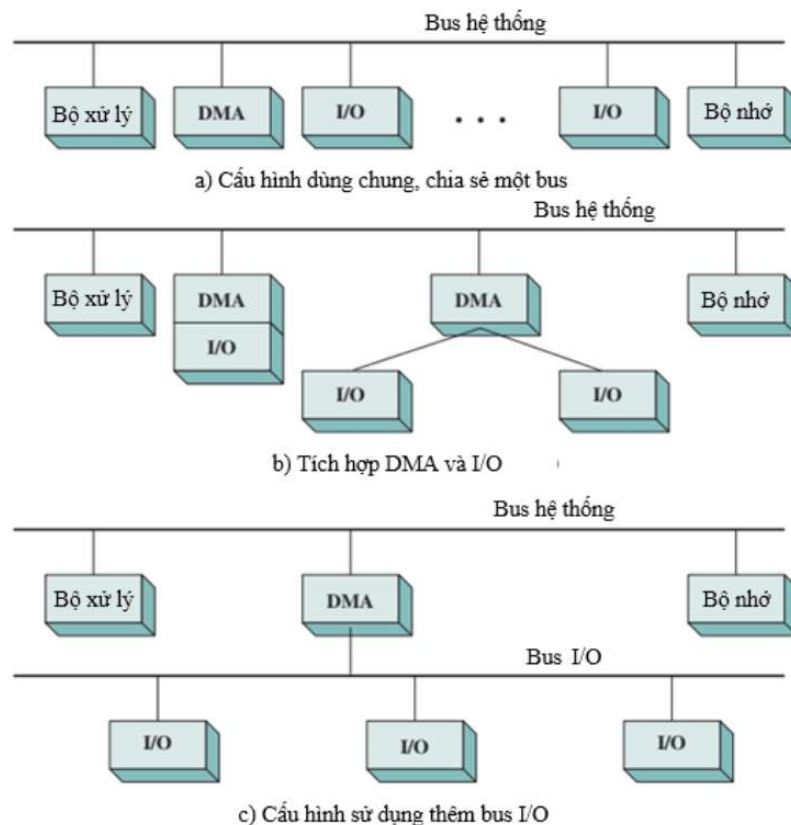


Hình 5.12. Điểm ngắt của DMA và của ngắt trong chu kỳ lệnh

Như đã nói ở trên, khi DMA cần chiếm bus để truyền, nó sẽ trộm chu kỳ bus. Hình 5.12 chỉ ra các thời điểm trong chu kỳ lệnh mà bộ xử lý bị treo (trong lúc DMA sử dụng bus). Trong mỗi trường hợp, bộ xử lý bị treo ngay trước khi nó cần sử dụng bus. Module DMA chuyển một từ sau đó trả lại điều khiển cho bộ xử lý. Lưu ý rằng đây không phải là ngắt; bộ xử lý không cần lưu trữ ngữ cảnh hoặc bất cứ thông tin gì. Đơn giản là bộ xử lý chỉ dừng lại một chu kỳ bus. Về mặt

hiệu năng của bộ xử lý thì nó sẽ hoạt động chậm hơn. Tuy nhiên, với việc trao đổi vào/ra nhiều từ, cơ chế DMA hiệu quả hơn rất nhiều so với I/O điều khiển ngắt hoặc I/O chương trình.

Cơ chế DMA có thể được cấu hình theo nhiều cách khác nhau. Hình 5.13 là sơ đồ một số cấu hình DMA. Hình 5.13a, tất cả các module chia sẻ chung hệ thống bus. Module DMA đại diện cho bộ xử lý sử dụng I/O chương trình để trao đổi dữ liệu giữa bộ nhớ và module I/O. Cấu hình này mặc dù không tốn kém nhưng rõ ràng là không hiệu quả. Giống như I/O chương trình do bộ xử lý điều khiển, mỗi lần truyền một từ sẽ tiêu tốn hai chu kỳ bus.



Hình 5.13. Các cấu hình DMA

Số lượng chu kỳ bus có thể được giảm bớt bằng cách tích hợp DMA và các module I/O. Như trong Hình 5.13b, module DMA kết nối trực tiếp với một hoặc nhiều module I/O mà không đi qua bus hệ thống. Logic DMA có thể là một phần của module I/O hoặc có thể là một module riêng biệt điều khiển một hoặc nhiều module I/O. Phương thức này có thể được cải tiến thêm một bước nữa bằng cách kết nối các module I/O với module DMA qua một bus I/O (Hình 5.13c). Cấu hình này giảm số giao diện ngoại vi của module DMA thành một và cho phép khả năng mở rộng (tăng thêm số lượng module I/O thì chỉ cần kết nối với bus I/O). Trong cả hai trường hợp Hình 5.13b và c, module DMA chỉ chiếm bus hệ thống khi cần trao đổi dữ liệu với bộ nhớ, như vậy chỉ mất một chu kỳ bus cho mỗi từ được truyền.

5.3. NỐI GHÉP THIẾT BỊ I/O

5.3.1. Kết nối có dây

Các thiết bị I/O thường kết nối có dây với máy tính thông qua các cổng giao tiếp. Mỗi cổng giao tiếp được gán một địa chỉ và có tập tham số làm việc riêng thông qua các chuẩn.

a. Cổng nối tiếp

Cổng nối tiếp (chuẩn Com) là cổng có 9 chân cắm nhô ra và chiều dài cáp không quá 15m. Cổng này dùng để cắm thiết bị ngoại vi như máy in, máy quét,... Nhưng hiện nay rất ít thiết bị dùng cổng COM.

b. Cổng song song

Cổng song song (chuẩn LPT) truyền được nhiều bit tại một thời điểm. Cổng này thường được dùng để kết nối máy in, máy vẽ, máy quét. Cổng này sử dụng chủ yếu loại đầu nối 25 chân và chiều dài cáp không quá 2m.

c. Chuẩn giao tiếp PS/2

Chuẩn truyền thông nối tiếp dùng cho chuột và máy tính. Cổng PS/2 có 6 chân. Màu xanh lá là cổng cắm chuột còn màu xanh tím là dây bàn phím. Truyền tín hiệu trên một dây và không cho phép cắm nóng.

Bảng 5.3. Tốc độ của một số kiểu kết nối

Kết nối có dây hoặc không dây	Tốc độ tối đa	Chiều dài cáp tối đa hoặc không dây
eSATA Version 3 (eSATA-600)	6.0 Gbps	2m
SuperSpeed USB (USB 3.0)	5.0 Gbps	3m
eSATA Version 2 (eSATA-300)	3.0 Gbps	2m
eSATA Version 1 (eSATA-150)	1.5 Gbps or 1500 Mbps	2m
FireWire 800 (hoặc 1394b)	1.2 Gbps or 800 Mbps	100 m
Wi-Fi 802.11n tần số 2,4 GHz hoặc 5,0 GHz	Lên tới 500 Mbps	Trong phạm vi 70m
Hi-Speed USB (USB 2.0)	480 Mbps	5m
FireWire 400 (hoặc 1394a)	400 Mbps	4,5m
Original USB (USB 1.1)	12 Mbps or 1.5 Mbps	3m
Wi-Fi 802.11g tần số 2,4 GHz	Lên tới 54 Mbps	Trong phạm vi 100m
Wi-Fi 802.11a tần số 5,0 GHz	Lên tới 54 Mbps	Trong phạm vi 50m
Wi-Fi 802.11b tần số 2,4 GHz	Lên tới 11 Mbps	Trong phạm vi 100m
Không dây Bluetooth tần số 2,4 GHz	Lên tới 3 Mbps	Trong phạm vi 10m

d. Cổng USB

USB là chuẩn truyền thông nối tiếp phổ biến cho phép kết nối đồng thời đến các thiết bị ngoại vi với khả năng tự nhận dạng cho OS hỗ trợ. Kết nối sử dụng đầu kết nối 4 chân và chiều dài cáp không quá 25m.

Các chuẩn USB: USB 1.0 tốc độ truyền 1.5 Mbps, USB 2.0 tốc độ truyền 480 Mbps, USB 3.0 tốc độ truyền 5 Mbps.

Một số thùng máy có cổng USB phía trước, muốn dùng được USB này phải nối dây nối từ thùng máy vào chân cắm dành cho nó có ký hiệu USB trên mainboard.

e. Chuẩn IEEE 1394

IEEE 1394 là cổng truyền thông tốc độ cao thường dùng để trao đổi dữ liệu giữa các máy tính với các thiết bị số như máy quay, chụp hình, ghi âm...

Kết nối chuẩn sử dụng đầu 4 chân hoặc 6 chân và chiều dài lên đến 1000m. Tốc độ truyền tải dữ liệu 400 Mbps – 3.2 Gbps.

f. Một số chuẩn giao tiếp khác

Ngoài ra còn một số chuẩn giao tiếp khác dùng trên Laptop như ThunderBolt 10Gbps, PCMLA, Express Card.

5.3.2. Kết nối không dây

Ngoài hình thức kết nối có dây ở trên thì các thiết bị I/O cũng có thể kết nối với máy tính thông qua các kết nối không dây là mạng wi-fi hoặc ghép nối qua Bluetooth - bàn phím, chuột, máy in, máy quét,... Điều này cho phép giảm đáng kể số lượng dây kết nối, tuy nhiên chúng lại cần pin và sẽ cần thay thế pin/sạc lại định kỳ.

Chính vì vậy, việc lựa chọn hình thức kết nối thiết bị I/O có thể cần có sự cân nhắc, chẳng hạn như giữa kết nối máy in USB 2.0 và kết nối không dây Bluetooth,... Khi đó tốc độ cũng được xem là yếu tố quyết định cho việc lựa chọn này. Bảng 5.3 cho thấy tốc độ của những kiểu kết nối này.

Câu hỏi ôn tập chương 5

1. Các chức năng chính của mô-đun I/O là gì?
2. Liệt kê và định nghĩa ngắn gọn ba kỹ thuật để thực hiện I/O.
3. Sự khác biệt giữa I/O ánh xạ bộ nhớ và I/O biệt lập là gì?
4. Khi xảy ra ngắt thiết bị, làm thế nào để bộ xử lý xác định thiết bị nào đã gây ra ngắt?
5. Khi một mô-đun DMA nắm quyền điều khiển một bus và trong khi nó vẫn giữ quyền kiểm soát bus, thì bộ xử lý sẽ làm gì?
6. Xét bộ vi xử lý (Z8000) có lệnh truyền khối I/O. Sau lần thực hiện đầu tiên, một lệnh như vậy sẽ mất năm chu kỳ xung nhịp để thực hiện lại. Tuy nhiên, nếu sử dụng lệnh I/O không chặn, thì phải mất tổng cộng 20 chu kỳ xung nhịp để tìm nạp và thực thi. Tính tốc độ tăng lên với lệnh I/O khối khi truyền các khối 128 byte.
7. Một hệ thống dựa trên bộ vi xử lý 8 bit và có hai thiết bị I/O. Bộ điều khiển I/O cho hệ thống này sử dụng các thanh ghi trạng thái và điều khiển riêng biệt. Cả hai thiết bị đều xử lý dữ liệu trên cơ sở 1 byte tại một thời điểm. Thiết bị đầu tiên có hai dòng trạng thái và ba dòng điều khiển. Thiết bị thứ hai có ba dòng trạng thái và bốn dòng điều khiển.
 - a. Hỏi cần bao nhiêu thanh ghi mô-đun điều khiển I/O 8-bit để đọc trạng thái và điều khiển từng thiết bị?
 - b. Tổng số thanh ghi mô-đun điều khiển cần thiết là bao nhiêu nếu thiết bị đầu tiên là thiết bị chỉ có đầu ra?
 - c. Cần bao nhiêu địa chỉ riêng biệt để điều khiển hai thiết bị?
8. Đối với I/O theo chương trình, bộ xử lý có thể bị mắc kẹt trong vòng lặp chờ thực hiện kiểm tra trạng thái của thiết bị I/O. Để tăng hiệu quả, phần mềm I/O có thể được viết để bộ xử lý kiểm tra định kỳ trạng thái của thiết bị. Nếu thiết bị chưa sẵn sàng, bộ xử lý có thể chuyển sang các tác vụ khác. Sau một khoảng thời gian nhất định, bộ xử lý quay lại để kiểm tra lại trạng thái.

- a. Nếu việc xuất dữ liệu mỗi lần một ký tự sang máy in hoạt động ở tốc độ 10 ký tự mỗi giây (cps) thì điều gì sẽ xảy ra nếu trạng thái của nó được quét cứ sau 200 ms?
- b. Xét một bàn phím có bộ đệm một ký tự. Trung bình, các ký tự được nhập với tốc độ 10 cps. Tuy nhiên, khoảng thời gian giữa hai lần nhấn phím liên tiếp có thể chỉ mất 60 ms. Hỏi bàn phím nên được quét bởi chương trình I/O ở tần số nào?
9. Xét một hệ thống trong đó chu kỳ bus mất 500 ns. Chuyển điều khiển bus theo cả hai hướng, từ bộ xử lý sang thiết bị I/O hoặc ngược lại, mất 250 ns. Một trong các thiết bị I/O có tốc độ truyền dữ liệu là 50 KB/s và sử dụng DMA. Dữ liệu được truyền 1 byte tại một thời điểm.
- a. Giả sử sử dụng DMA ở chế độ liên tục. Nghĩa là, giao diện DMA giành được quyền kiểm soát bus trước khi bắt đầu chuyển khối và duy trì quyền kiểm soát bus cho đến khi toàn bộ khối được chuyển. Thiết bị sẽ kết nối bus trong bao lâu khi truyền một khối 128 byte?
- b. Lập lại phép tính cho chế độ đánh cắp chu kỳ (cycle-stealing mode).
10. Máy tính 32 bit có hai kênh bộ chọn và một kênh ghép kênh. Mỗi kênh bộ chọn hỗ trợ hai đĩa từ và hai băng từ. Kênh ghép kênh có hai máy in dòng, hai đầu đọc thẻ và 10 thiết bị đầu cuối VDT được kết nối với nó. Giả sử các tỷ lệ truyền như sau:

Disk drive	800 Kbytes/s
Magnetic tape drive	200 Kbytes/s
Line printer	6.6 Kbytes/s
Card reader	1.2 Kbytes/s
VDT	1 Kbyte/s

Ước tính tổng tốc độ truyền I/O tối đa trong hệ thống này.

TÀI LIỆU THAM KHẢO

- [1]. William Stallings, Computer Organization and Architecture (11th edition), Pearson, ISBN 978-0-13-499719-3, 2019.
- [2]. David A. Patterson and John L. Hennesy, Computer Organization and Architecture: Hardware and Software Interface, 5th Edition, Prentice – Hall 2014.
- [3]. John L. Hennessy, David A. Patterson, Computer architecture: a quantitative approach, Morgan Kaufmann, ISBN: 978-0-12-811905-1, 2019.
- [4]. Nguyễn Đình Việt, Kiến trúc máy tính, NXB ĐHQGHN, 2005.
- [5]. Hoàng Xuân Dậu, Bài giảng kiến trúc máy tính, Học viện Công nghệ Bưu chính Viễn thông, 2010.